

Server Manage 운영 위키

DECS 서버 관리 코드의 기능, 경계와 설계 의도

문서 기준: 2026-08-10 저장소 상태

소스: /home/hyrrn/admin_wiki

목차

1. 전체 구조
2. server-state
3. container-images
4. monitoring
5. remote-operations
6. kerberos-nfs
7. Backend (Admin BE)
8. infra
9. kdc-setup

System

원본: md/system/index.md

System 영역은 FARM/LAB GPU 서버를 일관된 기준으로 운영하기 위한 관리 기능을 제공한다. 서버의 공통 상태와 구축 기준, 사용자 GPU 컨테이너 이미지, 상태 관측, 원격 부팅, AD Kerberos 기반 NFS 접근을 다섯 모듈로 나누어 관리한다. 운영 관리자는 이 페이지에서 필요한 기능의 담당 모듈과 세부 문서를 확인할 수 있다.

전체 구조

`server-state` 는 전체 서버 운영 기준을 확인하는 시작점이고, 나머지 네 모듈은 이미지, 관측, 원격 기동, 인증·스토리지라는 개별 기능을 소유한다. 아래 순서는 문서를 이해하기 위한 배치이며 모듈 간 실행 순서나 의존 순서를 뜻하지 않는다. 각 모듈의 역할과 담당 범위는 다음과 같다.

구성요소	핵심 역할	담당 범위
<code>server-state</code>	FARM/LAB 서버의 공통 상태와 구축·점검 기준 정의	서버 profile, Docker/NVIDIA/Kubernetes/network/NFS 전제조건, 신규 서버 bootstrap 순서, 기존 서버 drift 점검과 remediation 계획
<code>container-images</code>	사용자 GPU 컨테이너 이미지와 시작 환경 관리	CUDA/TensorFlow image variant, Dockerfile, entrypoint, UID/GID·VNC·Kerberos ccache 런타임 계약, 이미지 테스트·배포
<code>monitoring</code>	서버와 서비스의 현재 상태를 지속적으로 관측	exporter, Prometheus, Grafana, Alertmanager, GPU/container/NFS 상태, 경보, forensics와 제한된 안전 복구
<code>remote-operations</code>	서버 원격 부팅과 부팅 시점의 일회성 준비 작업 수행	Wake-on-LAN, 부팅 시 1회 health gate, 필수 mount·GPU·SSH 확인, 기존 stopped container 시작과 사후 점검
<code>kerberos-nfs</code>	AD Kerberos 인증과 NFS 공유 스토리지 접근 기준 관리	AD Kerberos, UID/GID 일치, keytab·ccache, RPCSEC_GSS, FARM/LAB NFS 기준 문서, 명시적 repair·rotation·mount 절차

PDF와 세부 문서

문서	상세 문서	PDF
전체 통합 매뉴얼	-	PDF 열기
전체 구조	현재 페이지	PDF 열기
<code>server-state/</code>	문서 열기	PDF 열기
<code>container-images/</code>	문서 열기	PDF 열기
<code>monitoring/</code>	문서 열기	PDF 열기
<code>remote-operations/</code>	문서 열기	PDF 열기

문서	상세 문서	PDF
kerberos-nfs/	문서 열기	PDF 열기

server-state 개요

원본: md/system/server-state/index.md, md/system/server-state/design.md, md/system/server-state/operations.md

server-state 는 FARM/LAB GPU 서버의 공통 운영 기준을 정의하고, 각 서버가 해당 기준에 맞게 설정되어 있는지 점검하는 모듈이다. 구성요소별 설정 작업은 신규 서버를 같은 운영 기준에 맞게 구성할 때도 사용한다.

서버 상태는 OS 공통 패키지, Docker, NVIDIA, Kubernetes, storage network, Kerberos/NFS와 monitoring으로 나뉜다. 각 구성요소에는 목표 상태, 점검 방법, 설정 방법, 실행 형태와 안전 수준이 연결되어 있다.

문서	내용
설계	목표, 구조, 구성요소별 상태·점검·설정, 실행 형태와 안전 수준, 설정값과 코드 위치
운영	서버 등록, 환경 설정 변경, 구성요소 확장, 실행 제어 변경, 점검·설정과 검증 절차

server-state 설계

개요 · 운영

GitHub 코드 링크: `admin_infra_server` 는 비공개 저장소다. 링크를 누르면 GitHub 로그인 화면을 거쳐 해당 파일로 이동한다. 조직 저장소에 접근 권한이 있는 계정으로 로그인해야 한다.

1. 개요

server-state 의 주요 목표는 FARM/LAB GPU 서버가 공통 운영 기준에 맞게 설정되어 있는지 일관된 방법으로 점검하는 것이다. 운영 기준은 OS 공통 패키지와 Docker, NVIDIA driver/runtime, Kubernetes, storage network, Kerberos/NFS, monitoring의 목표 상태와 점검 순서를 정의한다.

구성요소(component)는 운영 기준의 단위로, 독립적으로 점검하고 설정할 수 있다. 예를 들어 `docker-engine` 은 Docker 설치, service, daemon과 cgroup driver 상태를 하나의 구성요소로 관리한다.

server-state 는 이미 등록된 서버가 기준에서 벗어났는지 점검하고 교정하는 데에도, 신규 서버를 처음부터 같은 기준에 맞게 구성하는 데에도 쓸 수 있다. 두 경우 모두 정책에 정의된 같은 구성요소 순서를 그대로 따른다.

2. 설계 구조

Ansible playbook은 서버에서 실행할 작업 순서를 YAML로 정의한 파일이다. **server-state** 는 구성요소별 점검·설정을 이 playbook으로 수행한다.

구분	저장 위치	역할
정책	<code>policy/standard-gpu-server.yml</code>	구성요소와 실행 순서를 기록한다.
구성요소 정의	<code>components/*.yml</code>	목표 상태, 점검 방법, 설정 방법과 승인 수준을 기록한다.
서버·환경 정보	<code>servers.jsonl</code> , <code>config/environments.yml</code>	서버 접속·network 정보와 FARM/LAB별 설정값을 제공한다.
명령 구성	<code>server_state/</code>	대상 서버에 맞는 playbook, tag와 변수를 조합한다.
Ansible 작업	<code>ansible/roles/</code> 와 playbook	구성요소별 점검과 서버 설정을 수행한다.
실행 파일	<code>bin/server-state</code>	<code>describe</code> , <code>audit</code> , <code>plan</code> , <code>apply</code> 명령을 시작한다.

관리자가 `bin/server-state` 로 `describe`, `audit`, `plan`, `apply` 중 하나를 실행하면, 각 구성요소들은 다음 순서로 함께 동작한다. 네 가지 명령 모두 순서를 공통으로 따르며, 명령별로 달라지는 세부 동작은 4절에서 설명한다.

1. `--hosts` 로 대상 서버를 선택한다.
2. `--component` 로 구성요소를 선택한다.
3. 서버·환경 정보(`servers.jsonl`, `config/environments.yml`)와 FARM/LAB 설정을 결합해 실행값을 만든다.
4. 정책과 구성요소 정의에서 해당 구성요소에 연결된 `audit` 또는 `converge` playbook을 선택한다.
5. Ansible 명령을 화면에 보여주거나 실행한다.

3. 구성요소

`standard-gpu-server.yml` 은 10개 구성요소의 실행 순서를 정의한다. 모든 구성요소는 `server-state` 정책과 CLI에서 관리한다. 각 구성요소는 서버의 현재 상태를 점검하는 작업과, 서버를 목표 상태에 맞게 설정하는 작업을 정의한다.

각 구성요소는 아래 네 항목으로 설명된다.

항목	의미
목표 상태	서버가 도달해야 하는 상태
점검 (<code>audit</code>)	목표 상태 충족 여부를 확인하는 방법. 서버 상태를 읽기만 하고 아무것도 변경하지 않는다.
설정 (<code>converge</code>)	서버를 목표 상태로 만드는 방법. 서버를 실제로 바꿔서 목표 상태로 만든다.
설정 실행	설정 작업의 실행 형태와 안전 수준

실행 형태는 설정 작업을 어떻게 수행하는지, 안전 수준은 실행 전 어떤 확인·승인이 필요한지를 나타낸다. 두 값이 실제 명령에서 어떻게 쓰이는지는 4절에서 설명한다.

실행 형태	동작
<code>ansible-playbook</code>	설정 작업을 Ansible playbook으로 자동 실행한다.
<code>manual</code>	CLI가 수행할 절차를 안내하고, 관리자가 그 절차를 직접 수행한다.

안전 수준	실행 조건
safe	별도 확인 없이 바로 실행할 수 있다.
gated	운영 영향을 확인한 뒤 관리자 승인이 있어야 실행된다.
risky	작업 영향과 운영 일정까지 확인한 뒤 관리자 승인이 있어야 실행된다.

3.1 baseline-access

목표 상태: 서버가 inventory(`servers.jsonl`)와 Ansible inventory(`monitoring/ansible_playbook/inventory.ini`)에 등록되어 있고, 설정된 hostname으로 SSH 접속과 비대화형 sudo를 사용할 수 있다.

점검: Ansible ping으로 접속을 확인하고, `sudo -n true` 실행과 실제 hostname-inventory hostname 일치 여부를 확인한다.

설정: IP, hostname, SSH key, 관리 계정과 sudo 권한을 준비하고 두 inventory에 서버를 등록한다. 등록 절차는 [운영 문서](#)의 "2. 신규 서버 추가"를 참고한다.

설정 실행: 실행 형태 `manual`, 안전 수준 `gated`. 관리자가 준비 절차를 수행한다. 이 구성요소는 이후 Ansible 작업을 실행하기 위한 선행 조건이다.

관련 코드: `baseline-access.yml`, `baseline_access/tasks/audit.yml`

3.2 os-common

목표 상태: 서버가 정책에 지정된 Ubuntu release를 사용하고, 다른 구성요소에 필요한 공통 package가 설치되어 있다. 주요 package는 `curl`, `gnupg`, `ethtool`, `krb5-user`, `nfs-common`, `keyutils`, `adcli` 와 `python3` 이다.

점검: OS 종류와 version을 확인하고 package facts에서 필수 package를 하나씩 확인한다.

설정: apt cache를 갱신하고 공통 package 목록을 설치한다.

설정 실행: 실행 형태 `ansible-playbook`, 안전 수준 `safe`.

관련 코드: `os-common.yml`, `os_common/tasks/audit.yml`, `os_common/tasks/main.yml`

3.3 docker-engine

목표 상태: Docker Engine package가 설치되고 service가 활성화·실행되며, daemon이 응답하고 `systemd` cgroup driver를 사용한다. `daemon.json` 에는 `overlay2`, `json-file` 과 log size 설정이 포함된다.

점검: Docker service의 enabled·active 상태, `docker info` 응답과 cgroup driver 값을 확인한다.

설정: Docker apt key와 repository를 등록하고 Engine package를 설치한다. 기존 `daemon.json` 에 필요한 값을 병합하고 변경된 경우 Docker service를 재시작한다.

설정 실행: 실행 형태 `ansible-playbook`, 안전 수준 `gated`. package source와 daemon 설정이 변경되고 service가 재시작될 수 있어 운영 영향 확인이 필요하다.

관련 코드: `docker-engine.yml`, `docker_engine/tasks/audit.yml`, `docker_engine/tasks/main.yml`

3.4 nvidia-driver

목표 상태: 정책에 지정된 NVIDIA driver package가 설치되고 GPU를 정상 인식하며, 해당 package가 apt hold 상태로 유지된다.

점검: `nvidia-smi` 로 GPU 이름과 driver version을 읽고 `apt-mark showhold` 에서 NVIDIA driver package를 확인한다.

설정: 지정 driver package를 설치하고 apt hold를 설정한다.

설정 실행: 실행 형태 `ansible-playbook`, 안전 수준 `risky`. driver 변경은 GPU workload와 reboot 일정에 영향을 줄 수 있어 작업 영향과 운영 일정을 확인해야 한다.

관련 코드: `nvidia-driver.yml`, `nvidia_driver/tasks/audit.yml`, `nvidia_driver/tasks/main.yml`

3.5 nvidia-runtime

목표 상태: NVIDIA Container Toolkit이 설치되고 Docker와 containerd가 NVIDIA runtime을 사용할 수 있다.

점검: `nvidia-ctk` 설치, Docker runtime 목록과 containerd 설정의 NVIDIA runtime 항목을 확인한다.

설정: NVIDIA repository와 toolkit package를 설치하고 `nvidia-ctk runtime configure` 를 Docker와 containerd에 실행한다. 설정 변경 후 관련 service를 재시작한다.

설정 실행: 실행 형태 `ansible-playbook`, 안전 수준 `gated`. container runtime 설정이 변경되고 관련 service가 재시작될 수 있어 운영 영향 확인이 필요하다.

관련 코드: `nvidia-runtime.yml`, `nvidia_runtime/tasks/audit.yml`, `nvidia_runtime/tasks/main.yml`

3.6 kubernetes-packages

목표 상태: 정책에 지정된 version의 `kubeadm`, `kubelet`, `kubectrl` package가 설치·hold되고 kubelet service가 활성화되어 있다.

점검: 세 package의 설치 여부와 kubelet enabled 상태를 확인한다.

설정: Kubernetes apt key와 repository를 등록하고 세 package를 설치·hold한 뒤 kubelet을 활성화한다.

설정 실행: 실행 형태 `ansible-playbook`, 안전 수준 `gated`. Kubernetes package version이 바뀌고 kubelet에 영향을 줄 수 있어 운영 영향 확인이 필요하다.

관련 코드: `kubernetes-packages.yml`, `kubernetes_packages/tasks/audit.yml`, `kubernetes_packages/tasks/main.yml`

3.7 kubernetes-membership

목표 상태: 서버가 FARM 또는 LAB의 지정 Kubernetes cluster에 join되고, node에 해당 domain label이 설정되어 있다.

점검: 서버의 kubelet credential 파일을 확인하고 controller에서 node와 domain label을 조회한다.

설정: 관리자가 cluster, join token과 node 정보를 확인하고 검토를 마친 뒤 `kubeadm join` 명령을 실행한다.

설정 실행: 실행 형태 `manual`, 안전 수준 `risky`. join token의 유효 시간과 cluster 선택을 관리자가 확인한다.

관련 코드: `kubernetes-membership.yml`, `kubernetes_membership/tasks/audit.yml`,
`kubernetes_membership/tasks/main.yml`

3.8 storage-network

목표 상태: `servers.jsonl`에 storage interface가 기록되고, 해당 NIC의 RX queue를 4096으로 설정하는 systemd service가 활성화되어 있다.

점검: storage interface 값, 현재 RX queue 크기와 `decs-rx-queue.service` enabled 상태를 확인한다.

설정: RX queue를 설정하는 oneshot systemd unit을 설치하고 활성화한다.

설정 실행: 실행 형태 `ansible-playbook`, 안전 수준 `safe`.

관련 코드: `storage-network.yml`, `storage_network/tasks/audit.yml`, `storage_network/tasks/main.yml`

3.9 kerberos-nfs

목표 상태: 서버가 FARM/LAB Kerberos 설정과 host keytab을 사용해 machine principal 인증과 NFS service ticket 발급에 성공한다. `rpc-gssd`가 실행되고 공유 경로가 지정 source와 `sec=krb5` option으로 mount된다.

점검: `/etc/krb5.conf`, keytab, `kinit -k`, `kvno`, `rpc-gssd`와 mount source·option을 순서대로 확인한다. 자세한 Kerberos/NFS 인증·mount 구조는 [Kerberos/NFS 설계](#) 참고.

설정: domain Kerberos 설정을 설치하고 keytab과 principal을 검증한다. NFS GSS readiness·recovery unit을 설치하고 fstab에 mount를 기록한다.

설정 실행: 실행 형태 `ansible-playbook`, 안전 수준 `gated`. Kerberos 설정, fstab과 mount recovery에 영향을 줄 수 있어 운영 영향 확인이 필요하다. 실제 mount 실행은 별도 `server_state_mount_now` 값으로 제어한다.

관련 코드: `kerberos-nfs.yml`, `audit_client.yml`, `kerberos_nfs_client/`

3.10 monitoring

목표 상태: `cluster-monitor-exporter`와 `gpu-user-exporter` service가 활성화·실행되고, 두 metrics endpoint와 public health endpoint가 HTTP 200으로 응답한다.

점검: 두 exporter service의 enabled·active 상태와 로컬 metrics·health endpoint 응답을 확인한다.

설정: exporter binary를 build하고 설정 파일과 systemd unit을 설치한다. service를 시작한 뒤 metrics 응답을 검증한다.

설정 실행: 실행 형태 `ansible-playbook`, 안전 수준 `safe`.

관련 코드: `monitoring.yml`, `audit_exporters.yml`, `deploy_exporters.yml`

4. 명령별 동작

4.1 describe

`describe` 는 서버에 접속하지 않고, 정책과 구성요소 정의에 기록된 목표 상태, 점검·설정의 실행 형태와 안전 수준을 보여준다. `--component` 를 생략하면 정책에 포함된 전체 구성요소를 대상으로 한다.

4.2 audit

`audit` 은 구성요소의 읽기 전용 점검 playbook을 실행한다. 각 실행 결과는 서버와 구성요소 단위로 `OK` 또는 `FAILED` 로 표시된다. `--show-command` 는 같은 playbook과 변수를 사용한 명령을 화면에 출력한다.

4.3 plan

`plan` 은 구성요소의 설정 playbook을 Ansible `--check --diff` 로 실행한다. `package`, 파일과 `service`의 예상 변경을 확인할 수 있다. `manual` 구성요소는 실행 명령 대신 운영 절차 reference를 표시한다.

4.4 apply

`apply` 는 모든 선택 항목의 실행 형태와 안전 수준을 먼저 확인한다. `ansible-playbook` 구성요소는 `--execute` 로 실행하며, `gated` 는 `--approve-gated` , `risky` 는 `--approve-risky` 승인을 함께 사용한다. `manual` 구성요소는 운영 절차 reference를 표시한다.

5. 설정 구조

5.1 정책과 구성요소 정의

정책 파일은 구성요소 ID와 순서를 기록한다. 각 `components/<id>.yaml` 은 다음 정보를 기록한다.

필드	의미
<code>id</code>	<code>--component</code> 에서 사용하는 이름
<code>desired_state</code>	점검과 설정의 기준이 되는 서버 상태
<code>audit</code>	점검 playbook, tag와 <code>safe</code> 수준
<code>converge</code>	설정 playbook 또는 수동 절차와 안전 수준

`docker-engine.yaml` 은 다음과 같이 구성된다.

```

id: docker-engine
desired_state: Docker Engine is installed, enabled, responsive, and configured with the systemd cgroup driver.
audit:
  kind: ansible-playbook
  playbook: server-state/ansible/playbooks/audit.yml
  tags: [docker-engine]
  safety: safe
converge:
  kind: ansible-playbook
  playbook: server-state/ansible/playbooks/converge.yml
  tags: [docker-engine, cgroups]
  safety: gated

```

`audit` 명령은 audit playbook에 `docker-engine` tag를 전달한다. `plan` 과 `apply` 는 converge playbook에 `docker-engine,cgroups` tag를 전달한다. 실제 점검 task는 role의 `tasks/audit.yml`, 설정 task는 `tasks/main.yml`에 있다.

5.2 서버와 FARM/LAB 설정

`servers.jsonl` 은 host, server ID, domain, SSH 주소·port·계정과 management/storage interface 정보를 제공한다. `config/environments.yml` 은 FARM/LAB별 realm, Kerberos config, storage host·mount와 Kubernetes context를 제공한다.

`inventory.py` 는 두 정보를 결합해 Kerberos principal, NFS mount target, Kubernetes context와 public health port를 만든다.

5.3 명령 구성

`planner.py` 는 대상 서버와 구성요소를 결합해 inventory, playbook, host, tag와 extra vars가 포함된 Ansible 명령을 만든다.

`commands.py` 는 `audit`, `plan`, `apply` 를 실행하고 결과를 text 또는 JSON으로 출력한다.

6. 코드 위치

파일·디렉터리	역할
<code>policy/standard-gpu-server.yml</code>	구성요소와 실행 순서
<code>components/</code>	구성요소별 목표 상태, 점검·설정 진입점과 승인 수준
<code>server_state/catalog.py</code>	정책과 구성요소 로딩·검증
<code>server_state/inventory.py</code>	서버 선택과 FARM/LAB 실행값 생성
<code>server_state/planner.py</code>	Ansible 명령 생성

파일·디렉터리	역할
<code>server_state/commands.py</code>	CLI 명령과 결과 처리
<code>ansible/playbooks/audit.yml</code>	공통 구성요소 audit 순서
<code>ansible/playbooks/converge.yml</code>	공통 구성요소 설정 순서
<code>ansible/roles/</code>	구성요소별 점검·설정 task
<code>kerberos-nfs/ansible/</code>	Kerberos/NFS 점검·설정 task
<code>monitoring/ansible_playbook/</code>	exporter 점검·배포 playbook
<code>tests/</code>	정책, 서버 정보, 명령 구성과 승인 test

server-state 운영

개요 · 설계

1. 개요

이 문서의 목표는 `server-state` 를 운영하면서 서버와 구성요소를 추가하고, 환경 설정과 실행 제어를 변경하며, 서버 점검과 설정 작업을 수행하는 절차를 제공하는 것이다. 설계 문서에서 정의한 구조를 기준으로 다음 작업을 다룬다.

작업	변경 위치	확인 방법
서버 추가	<code>servers.jsonl</code> , Ansible inventory	<code>list-hosts</code> , <code>baseline-access</code> 점검
FARM/LAB 설정 변경	<code>server-state/config/environments.yml</code>	<code>plan --show-command</code> 의 extra vars 확인
구성요소 추가·수정	<code>server-state/components/</code> , Ansible role·playbook	<code>describe</code> , <code>audit</code> , <code>plan</code>
실행 형태·안전 수준 변경	구성요소 정의와 <code>server_state/</code> Python package	단위 테스트와 승인 동작 확인
서버 점검·설정	<code>bin/server-state</code> CLI	실행 결과와 종료 코드 확인

명령은 `admin_infra_server` 저장소를 clone한 루트 디렉터리(예: `/home/jy/server_manage`)에서 실행한다.

```
cd /home/jy/server_manage
./server-state/bin/server-state --help
```

2. 신규 서버 추가

신규 서버 추가는 서버 정보 등록, Ansible 접속 대상 등록, 접속 점검, 표준 구성 순서로 진행한다.

2.1 등록 정보 준비

정보	용도
<code>host</code>	CLI의 <code>--hosts</code> 에서 사용하는 소문자 서버 이름
<code>server_id</code>	Kerberos principal과 서버 식별에 사용하는 대문자 ID
<code>domain</code>	FARM 또는 LAB 환경 선택
<code>server_no</code>	공유 경로와 public health port 계산
<code>ansible_host</code> , <code>ansible_port</code> , <code>ansible_user</code>	SSH 접속
management interface-IPv4	관리 network 정보
storage interface-IPv4	storage network 설정과 점검

`host` 와 `server_id` 는 기존 서버와 겹치지 않는 값을 쓰고, `server_no` 는 같은 domain 안에서 기존 서버와 겹치지 않는 값을 쓴다. IP, hostname, SSH key와 비대화형 sudo도 이 단계에서 준비한다.

2.2 서버 정보 등록

공용 `servers.jsonl` 에 서버 한 대를 한 줄의 JSON으로 추가한다. 다음 예시는 `server-state` 가 사용하는 핵심 필드를 보여준다.

```
{"host": "farm10", "server_id": "FARM10", "domain": "FARM", "server_no": 10, "inventory":  
{"group": "FARM", "ansible_host": "192.168.2.20", "ansible_port": 8090, "ansible_user": "jy"}, "networks":  
{"management": {"name": "eno1", "ipv4": "192.168.2.20"}, "storage": {"name": "eno2", "ipv4": "100.100.100.110"}}
```

같은 서버를 `monitoring/ansible_playbook/inventory.ini` 의 FARM 또는 LAB group에도 등록한다. host 이름과 SSH 주소·port는 `servers.jsonl` 의 값과 맞춘다.

```
[FARM]  
farm10 ansible_host=192.168.2.20 ansible_port=8090
```

group과 다른 SSH 계정을 사용하는 서버는 해당 host 행에 `ansible_user` 를 함께 지정한다.

2.3 등록 결과 확인

CLI가 새 서버의 ID, domain, SSH 주소와 계산된 public health port를 읽는지 확인한다.

```
./server-state/bin/server-state list-hosts --hosts farm10
```

Ansible 접속, hostname과 sudo 조건은 `baseline-access` 점검으로 확인한다.

```
./server-state/bin/server-state audit \  
--hosts farm10 \  
--component baseline-access
```

2.4 표준 구성 진행

전체 구성요소의 예상 변경과 수동 절차를 먼저 확인한다.

```
./server-state/bin/server-state plan --hosts farm10
```

`safe`, `gated`, `risky` 순서로 설정 작업을 나누어 실행하고 각 단계가 끝날 때 동일한 구성요소를 다시 점검한다. `baseline-access`와 `kubernetes-membership`은 출력된 운영 절차에 따라 진행한다. 실행 명령과 승인 옵션은 아래의 **서버 설정** 절에서 설명한다.

3. FARM/LAB 환경 설정 변경

`server-state/config/environments.yml`은 여러 서버가 공유하는 환경 값을 관리한다.

설정	사용하는 작업
<code>kerberos_realm</code> , <code>kerberos_config_source</code>	Kerberos principal과 client 설정 구성
<code>storage_host</code> , <code>mount_source</code> , <code>mount_options</code>	NFS mount와 인증 점검
<code>kubernetes_cluster</code>	서버가 참여할 cluster 선택
<code>mount_target_template</code>	서버별 공유 경로 계산
public health port 계산값	monitoring health endpoint port 계산

환경 값을 변경할 때는 다음 순서를 사용한다.

1. FARM 또는 LAB 항목의 값을 수정한다.
2. 해당 domain의 서버 한 대를 선택해 생성될 Ansible 명령과 extra vars를 확인한다.
3. 관련 구성요소를 `audit` 하고 `plan` 으로 예상 변경을 확인한다.
4. 한 서버에 설정한 뒤 같은 domain의 나머지 서버로 범위를 넓힌다.

```
./server-state/bin/server-state plan \  
--hosts farm8 \  
--component kerberos-nfs \  
--show-command
```

public health port 계산값은 다음 명령으로 확인한다.

```
./server-state/bin/server-state list-hosts --hosts FARM
```

4. 구성요소 추가·수정

구성요소는 하나의 목표 상태와 그 상태를 확인하는 점검 작업, 목표 상태를 만드는 설정 작업을 묶는다.

4.1 점검과 설정 작업 작성

점검 작업은 서버 상태를 읽어 목표 상태 충족 여부를 판단한다. 설정 작업은 여러 번 실행해도 같은 목표 상태에 도달하도록 Ansible task를 작성한다. 공통 playbook에서 tag로 role을 선택하거나 구성요소 전용 playbook을 연결할 수 있다.

- 점검 role은 `tasks/audit.yml`에 둔다.
- 설정 role은 `tasks/main.yml`과 handler에 둔다.
- 점검 action의 `safety`는 `safe`로 지정한다.
- service 재시작, package 교체와 cluster 변경 같은 영향은 설정 action의 안전 수준에 반영한다.

4.2 구성요소 정의 추가

`server-state/components/<id>.yml`을 추가한다. 파일명과 `id`는 같아야 한다.

```
id: example-component  
desired_state: The host has the expected example service and configuration.  
audit:  
  kind: ansible-playbook  
  playbook: server-state/ansible/playbooks/audit.yml  
  tags: [example-component]  
  safety: safe  
converge:  
  kind: ansible-playbook  
  playbook: server-state/ansible/playbooks/converge.yml  
  tags: [example-component]  
  safety: gated
```

필드	작성 기준
<code>id</code>	CLI에서 선택할 고유한 이름
<code>desired_state</code>	점검과 설정이 공통으로 사용하는 목표 상태
<code>kind</code>	Ansible 실행은 <code>ansible-playbook</code> , 운영 절차 실행은 <code>manual</code>
<code>playbook</code>	저장소 루트 기준 playbook 경로
<code>tags</code>	playbook에서 이 구성요소의 task를 선택할 tag
<code>host_variable</code>	playbook이 대상 host를 받는 변수 이름. 기본값은 <code>server_state_hosts</code>
<code>reference</code>	<code>manual</code> 작업에서 표시할 운영 절차
<code>safety</code>	<code>safe</code> , <code>gated</code> , <code>risky</code> 중 설정 영향에 맞는 수준

4.3 정책 순서에 추가

전체 서버가 사용하는 구성요소는 `server-state/policy/standard-gpu-server.yml`의 `components`에 추가한다. 필요한 package, runtime, network와 인증 조건을 기준으로 선행 구성요소 뒤에 배치한다. `describe`, `audit`, `plan`과 `apply`는 이 순서를 사용한다.

4.4 기존 구성요소 수정

변경 내용	수정 위치
목표 상태 문구	<code>components/<id>.yml</code> 의 <code>desired_state</code>
점검 기준	<code>audit role·playbook</code>
서버 설정 내용	<code>converge role·playbook</code> 과 <code>handler</code>
<code>playbook·tag·실행 형태·안전 수준</code>	<code>components/<id>.yml</code> 의 <code>audit</code> , <code>converge</code>
구성요소 실행 순서	<code>policy/standard-gpu-server.yml</code>

변경 후에는 한 서버에서 `audit`, `plan`, `apply`, `audit` 순서로 결과를 확인한 뒤 domain 단위로 실행 범위를 넓힌다.

4.5 연결 확인


```
./server-state/bin/server-state describe \
--component example-component

./server-state/bin/server-state audit \
--hosts farm8 \
--component example-component \
--show-command

./server-state/bin/server-state plan \
--hosts farm8 \
--component example-component \
--show-command
```

5. 실행 형태와 안전 수준 변경

설정 작업의 실행 제어는 `converge.kind` 와 `converge.safety` 로 나뉜다.

5.1 실행 형태

kind	동작
ansible-playbook	plan 에서 check/diff를 실행하고 apply 에서 설정 playbook을 실행한다.
manual	CLI가 reference 에 기록된 운영 절차를 표시한다.

manual 구성요소도 작업 영향을 나타내는 safety 를 기록한다. apply 는 해당 구성요소를 MANUAL 로 표시하며(상태값 설명은 9절 참고) 관리자가 reference의 절차를 수행한다.

5.2 안전 수준

safety	설정 실행 조건
safe	apply --execute
gated	apply --execute --approve-gated
risky	apply --execute --approve-risky

audit 은 읽기 전용 작업이므로 항상 safe 를 사용한다. 기존 구성요소의 실행 형태나 안전 수준을 바꿀 때는 components/<id>.yaml 의 converge 를 수정하고 describe , plan 과 승인 동작을 확인한다.

5.3 새 실행 형태 추가

새 kind 는 YAML 값과 해당 값을 처리하는 실행 코드를 함께 추가한다.

1. `catalog.py`의 `ACTION_KINDS`, `Action` 필드와 로딩 검증을 확장한다.
2. `planner.py`에서 새 실행 형태에 필요한 `PlanItem`을 만든다.
3. `commands.py`에서 실행 결과와 오류를 표시한다.
4. `catalog`, `planner`와 `command` 단위 테스트에 로딩·계획·실행 사례를 추가한다.
5. 구성요소 하나에 새 값을 연결해 `describe`, `plan`, `apply`를 확인한다.

5.4 새 안전 수준 추가

새 `safety`는 승인 조건과 CLI 옵션을 함께 설계한다.

1. `catalog.py`의 `SAFETY_LEVELS`와 검증 사례를 확장한다.
2. `commands.py`의 `apply` option과 승인 판정을 추가한다.
3. 승인 전 `BLOCKED`, 승인 후 실행되는 `command test`를 추가한다.
4. 설계 문서와 운영 문서에 영향 범위와 실행 명령을 기록한다.
5. 한 서버에서 차단·승인·실행·재점검 흐름을 확인한다.

6. 대상과 정책 확인

관리 대상과 파생된 `public health port`를 확인한다.

```
./server-state/bin/server-state list-hosts --hosts all
./server-state/bin/server-state list-hosts --hosts farm
./server-state/bin/server-state list-hosts --hosts farm8,lab10
```

`--hosts`는 `all`, `FARM/LAB domain`, `host 이름`, `server ID`와 쉼표·공백으로 구분한 조합을 지원한다.

전체 정책이나 선택한 구성요소의 목표 상태, 점검·설정 실행 형태와 안전 수준을 확인한다.

```
./server-state/bin/server-state describe
./server-state/bin/server-state describe --component docker-engine
./server-state/bin/server-state describe \
  --component docker-engine,kerberos-nfs,monitoring
```

`--component`를 생략하면 전체 정책을 선택한다. 여러 번 사용하거나 쉼표로 묶을 수 있으며, 실행 순서는 정책의 구성요소 순서를 따른다.

7. 운영 서버 점검

7.1 실행 명령 확인

`--show-command`는 실제로 사용할 Ansible inventory, playbook, tag와 extra vars가 포함된 명령을 화면에 출력한다.

```
./server-state/bin/server-state audit \  
--hosts farm8 \  
--component docker-engine \  
--show-command
```

7.2 점검 실행

`--show-command` 를 제외하면 선택한 서버에 접속해 읽기 전용 audit task를 실행한다.

```
# 한 서버의 전체 구성요소 점검  
./server-state/bin/server-state audit --hosts farm8  
  
# FARM 전체의 Docker와 NVIDIA runtime 점검  
./server-state/bin/server-state audit \  
--hosts farm \  
--component docker-engine \  
--component nvidia-runtime
```

각 구성요소는 별도 Ansible 실행으로 처리된다. 결과에는 서버와 구성요소가 함께 표시되며 하나 이상의 실행이 실패하면 종료 코드는 1이다.

8. 서버 설정

8.1 예상 변경 확인

`plan` 은 설정 playbook을 Ansible `--check --diff` 로 실행한다. package, 파일과 service의 예상 변경을 확인하며, `manual` 구성요소는 운영 절차 reference를 표시한다.

```
./server-state/bin/server-state plan \  
--hosts farm8 \  
--component os-common \  
--component docker-engine \  
--component storage-network
```

명령 구성만 확인할 때는 `--show-command` 를 추가한다.

8.2 `safe` 설정 실행

```
./server-state/bin/server-state apply \  
--hosts farm8 \  
--component os-common \  
--component storage-network \  
--execute
```

8.3 **gated** 설정 승인과 실행

예상 diff와 service 재시작, package·설정 변경 영향을 확인한 뒤 승인한다.

```
./server-state/bin/server-state apply \  
--hosts farm8 \  
--component docker-engine \  
--component nvidia-runtime \  
--component kubernetes-packages \  
--execute \  
--approve-gated
```

8.4 **risky** 설정 승인과 실행

GPU workload와 reboot 일정까지 확인한 뒤 승인한다.

```
./server-state/bin/server-state apply \  
--hosts farm8 \  
--component nvidia-driver \  
--execute \  
--approve-risky
```

`--approve-risky` 는 `gated` 승인도 포함한다. 필요한 승인이 빠지면 Ansible 실행 전에 종료 코드 2로 끝난다. `manual` 구성요소는 Ansible로 실행할 수 없으므로 `apply` 요청에 포함돼도 같은 방식으로 종료 코드 2가 된다. 검토가 끝난 구성요소를 안전 수준별로 선택해 실행한다.

8.5 설정 후 점검

설정에 사용한 서버와 구성요소를 같은 조건으로 다시 점검한다.

```
./server-state/bin/server-state audit \  
--hosts farm8 \  
--component docker-engine \  
--component nvidia-runtime
```

9. 결과 해석

상태	의미
COMMAND	<code>--show-command</code> 로 실행 예정 명령을 출력했다.
OK	해당 Ansible 실행이 종료 코드 0으로 끝났다.
FAILED	Ansible 실행이 실패했다. <code>detail</code> 의 stdout/stderr를 확인한다.
MANUAL	표시된 운영 절차 reference를 사용한다.
BLOCKED	선택한 안전 수준에 필요한 승인을 추가한다.

자동 처리용 JSON은 전역 옵션인 `--format`을 하위 명령 앞에 둔다.

```
./server-state/bin/server-state --format json audit \  
--hosts farm8 \  
--component docker-engine \  
--show-command
```

종료 코드는 전체 성공 0, Ansible 실행 실패 1, 설정 오류 또는 승인 차단 2다.

10. 변경 검증

Python 단위 테스트:

```
cd /home/jy/server_manage/server-state  
python3 -m unittest discover -s tests -v
```

주요 Ansible 구문 검사:

```
cd /home/jy/server_manage
ANSIBLE_CONFIG="$PWD/monitoring/ansible_playbook/ansible.cfg" \
ANSIBLE_ROLES_PATH="$PWD/server-state/ansible/roles:$PWD/kerberos-nfs/ansible/roles" \
ansible-playbook --syntax-check \
  -i monitoring/ansible_playbook/inventory.ini \
  server-state/ansible/playbooks/audit.yml

ANSIBLE_CONFIG="$PWD/monitoring/ansible_playbook/ansible.cfg" \
ANSIBLE_ROLES_PATH="$PWD/server-state/ansible/roles:$PWD/kerberos-nfs/ansible/roles" \
ansible-playbook --syntax-check \
  -i monitoring/ansible_playbook/inventory.ini \
  server-state/ansible/playbooks/converge.yml
```

CLI 연결 확인:

```
./server-state/bin/server-state audit \
  --hosts farm8 \
  --component docker-engine \
  --show-command

./server-state/bin/server-state plan \
  --hosts lab10 \
  --component kerberos-nfs \
  --show-command
```

운영 반영은 한 서버에서 **audit**, **plan**, **apply**, **audit** 순서로 검증한 뒤 같은 domain의 서버로 범위를 넓힌다.

container-images 개요

원본: md/system/container-images/index.md, md/system/container-images/design.md, md/system/container-images/operations.md

`container-images` 는 GPU container가 시작된 뒤 사용자가 바로 작업할 수 있도록 계정, 공유 홈, SSH, Jupyter, VNC 와 Kerberos 환경을 구성하고 CUDA/TensorFlow 조합별 image를 제공한다. **설계**는 공통 entrypoint의 런타임 구성 과 이미지 버전을 설명하고, **운영**은 빌드·배포에 사용하는 명령, 테스트 전략과 변경 절차를 설명한다.

문서 구성

문서	핵심 내용
현재 페이지	책임 범위와 문서 안내
설계	개요, 계정·홈·SSH·Jupyter·VNC·Kerberos 구성, CUDA/TensorFlow 조합별 이미지, 디렉터리 지도
운영	빌드/배포 흐름, 테스트 전략, 변경 가이드, 운영 안전 수칙

처음 보는 사람은 **설계 문서**에서 이미지가 무엇을 책임지고 무엇을 책임지지 않는지 먼저 확인하고, 실제로 빌드하거나 새 이미지 버전을 추가해야 할 때 **운영 문서**로 이동한다.

container-images 설계

개요 · 운영

GitHub 코드 링크: `admin_infra_server` 는 비공개 저장소다. 코드 링크를 누르면 GitHub 로그인 화면을 거쳐 원래 파일과 line으로 이동한다. 조직 저장소에 접근 권한이 있는 계정으로 로그인해야 한다.

1. 개요

`container-images` 의 목표는 GPU container가 시작된 직후 사용자가 자신의 계정과 공유 홈으로 접속하고, 선택한 GPU software 환경에서 바로 작업을 시작할 수 있게 하는 것이다.

`container-images` 는 GPU container용 image(`Dockerfile` 과 `image-variants.json`)와, 그 image에 담겨 있다가 container가 시작될 때 자동 실행되는 `entrypoint.sh` 를 관리하는 모듈이다. 실제로 container를 만들고 실행하는 행위(`docker run`)는 `infra` 라는 별도 시스템이 수행하며, `container-images` 의 역할은 그 실행이 시작된 뒤 image에 포함된 `entrypoint.sh` 가 넘겨받은 입력으로 사용자 환경을 구성하는 지점부터다.

모든 container는 전달받은 UID/GID로 계정과 그룹을 구성한다. 홈 디렉터리는 컨테이너 밖의 공유 스토리지이므로, 그 안에 이미 있는 `.bashrc` 같은 사용자 설정 파일이나 `vnc_password.txt` 같은 서비스별 password 파일이 있으면 새로 만들지 않고 그대로 이어서 사용한다. SSH와 Jupyter를 기본으로 제공하고, 필요한 경우 VNC와 Kerberos ccache 환경도 함께 구성한다. UID/GID와 홈이 항상 같은 값으로 전달되므로, container를 재시작하거나 다른 이미지 버전을 선택해도 같은 사용자(identity)와 작업 환경을 유지한다.

계정, supplemental group, 홈 디렉터리와 Kerberos 정보처럼 사용자와 host마다 달라지는 값은 공통 `entrypoint.sh`가 시작 시 반영한다. CUDA, TensorFlow, Python과 Ubuntu 조합은 각각의 이미지 버전으로 제공한다. 따라서 사용자는 필요한 GPU software 조합을 선택하면서도 모든 이미지 버전에서 동일한 방식으로 SSH, Jupyter, VNC와 공유 홈을 사용할 수 있다.

2. 사용자 실행 환경

`Dockerfile`은 모든 이미지 버전에 필요한 프로그램과 `entrypoint.sh`를 image에 포함한다. 실제 사용자 정보와 실행 옵션은 container를 생성하는 외부 시스템이 전달하며, `entrypoint`가 이를 검증하고 런타임 환경을 구성한다.

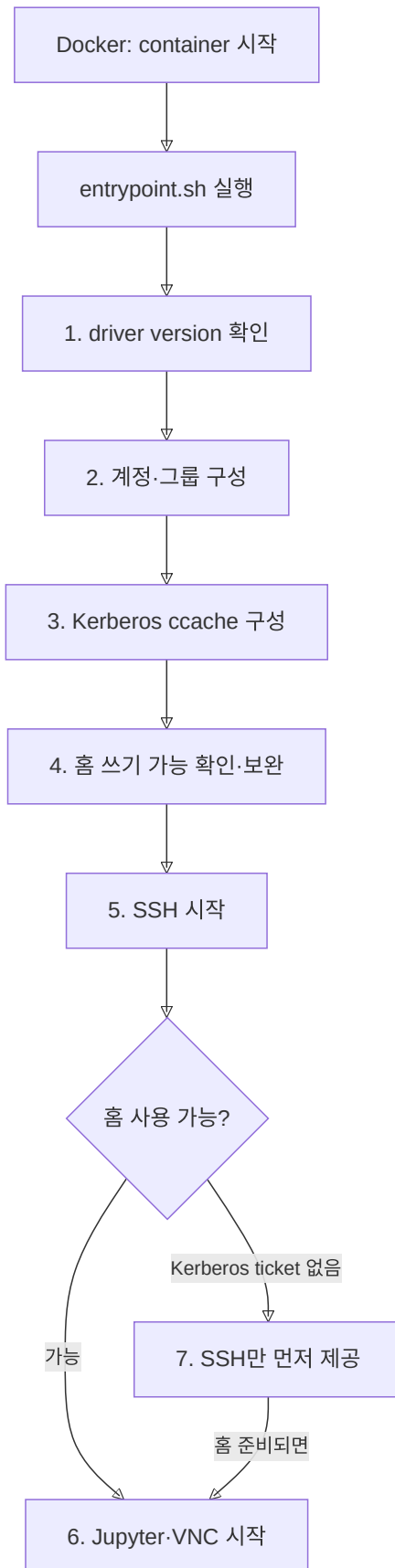
`container-images`는 사용자 identity, group membership, host mount, port와 Kerberos ticket을 결정하지 않는다. 즉 image에는 특정 사용자의 계정이나 홈을 미리 넣지 않는다 — 이 값들은 container 생성 전에 외부 시스템이 정해 전달하는 runtime 입력이다. 이 문서에서는 그 입력이 만들어지는 과정이 아니라, image와 `entrypoint.sh`가 입력을 받아 container 내부 환경을 구성하는 동작을 설명한다.

구분	이 문서에서 다루는 범위
외부에서 전달되는 입력	UID/GID와 그룹 목록, <code>/home</code> mount, Kerberos ccache, port mapping과 실행 option
<code>container-images</code> 가 수행하는 작업	image에 필요한 프로그램을 포함하고, 전달된 입력을 container 내부 계정·파일·service 설정에 반영한다.

같은 image를 사용하더라도, 외부 시스템이 container를 생성할 때 넘겨주는 identity, mount와 실행 옵션에 따라 `entrypoint`가 각 사용자의 환경을 구성한다.

2.1 컨테이너 시작 흐름

Docker가 `entrypoint.sh`를 container의 시작 프로세스로 실행한다(image에 Docker `ENTRYPOINT`로 지정되어 있다). 아래 7단계는 전부 이 하나의 스크립트 안에서 순서대로 호출되는 함수이며, 별도의 `.py`나 다른 프로세스가 아니다. 다음 순서는 `container-images` 내부에서 수행하는 작업만 나타낸다.



1. image에 기록된 CUDA/TensorFlow version을 출력하고 host NVIDIA driver가 최소 version을 충족하는지 확인한다.
2. 전달받은 UID/GID로 사용자, primary group과 supplemental group을 구성한다.
3. 전달받은 Kerberos ccache를 shell과 사용자 process에서 사용할 수 있게 구성한다.

4. mount되어 있는 홈 디렉터리의 쓰기 가능 여부를 사용자 권한으로 확인하고 초기 설정을 보완한다.
5. SSH 로그인, audit와 login message를 구성하고 SSH service를 시작한다.
6. 홈을 사용할 수 있으면 Jupyter와 선택적인 VNC/noVNC를 시작한다.
7. Kerberos ticket이 없어 홈을 쓸 수 없다면 SSH만 먼저 제공하고, 홈이 준비된 뒤 사용자 서비스를 시작한다.

이 순서를 하나의 entrypoint에서 관리하므로 이미지 버전마다 사용자 환경을 별도로 구현하지 않는다.

관련 코드

- **entrypoint의 전체 실행 순서**: 계정, Kerberos, 홈, SSH와 사용자 service를 순서대로 호출한다.
- **start_user_apps**: 사용자 shell 환경을 보완한 뒤 Jupyter와 VNC/noVNC를 시작한다.

2.2 계정, 그룹과 권한

사용자 계정과 group membership은 외부에서 결정되고, 공유 스토리지는 그 결과인 UID/GID를 기준으로 파일 권한을 판단한다. container 안에서 실행되는 process도 같은 권한을 사용하려면 Linux 사용자와 그룹이 동일한 UID/GID를 가져야 한다.

따라서 entrypoint는 전달된 값으로 container 내부 사용자와 primary group을 만들고, 전달된 supplemental group만 같은 GID로 추가한다. 사용자가 어떤 그룹에 속해야 하는지는 판단하지 않는다. 같은 이름의 사용자나 그룹이 다른 UID/GID로 이미 존재하면 외부 권한과 다르게 동작할 수 있으므로 container 시작을 중단한다.

계정과 그룹을 만드는 것 외에, entrypoint는 사용자가 나중에 직접 쓸 수 있는 그룹 공유 도구도 하나 설치해 둔다. Kerberos 공유 스토리지에서는 사용자가 자신의 홈 directory를 배정된 그룹과 공유할 수 있도록 **group-dir-share** 명령을 제공한다. **Dockerfile**은 ACL 도구를 포함하고, entrypoint는 Kerberos 환경을 구성할 때 이 명령을 **/usr/local/bin**에 생성해 둔다. 이 명령을 실행하는 주체는 entrypoint가 아니라 로그인한 사용자다.

```
group-dir-share ~/project vision
```

이 명령은 사용자가 해당 그룹에 속하는지와 경로가 사용자 홈 안에 있는지를 확인한 뒤 group owner, **2770** mode와 ACL을 설정한다. 별도의 권한을 부여하는 명령이 아니라 실행한 사용자가 이미 가진 group membership 범위에서만 동작한다.

계정·그룹 구성과 별개로, entrypoint는 sudo 권한도 함께 제어한다. 기본 sudo mode인 **restricted**는 package 설치에 필요한 명령은 허용하지만 사용자 전환, mount, 권한 변경, root shell과 우회 가능한 interpreter 실행은 막는다. 기존 사용자의 password는 재시작할 때 변경하지 않으며, **USER_PW**는 사용자를 처음 생성할 때만 적용한다.

entrypoint가 입력으로 받는 주요 값은 다음과 같다.

값	의미
USER_ID	container에 생성하거나 확인할 사용자 이름
UID , GID	DB·AD·NFS와 일치하는 숫자 identity
USER_GROUP	primary group 이름
DECS_SUPPLEMENTAL_GROUPS	추가할 group:GID 목록

값	의미
DECS_USER_SUDO_MODE	disabled, restricted, allowed 중 하나

TARGET_UID 나 TARGET_GID 같은 두 번째 identity 체계를 만들지 않는다. 하나의 UID / GID 만 사용해야 container, 공유 홈과 DB의 소유권이 서로 달라지는 오류를 시작 단계에서 찾을 수 있다.

관련 코드

- ensure_group_and_user : primary group과 사용자를 생성·검증하고 sudo mode를 적용한다.
- ensure_supplemental_groups : 추가 그룹의 이름과 GID가 전달값과 일치하는지 확인한다.
- Dockerfile 의 ACL package 설치: group-dir-share 가 ACL을 구성할 때 사용하는 setfacl 을 image에 포함한다.
- install_kerberos_share_helper : /usr/local/bin/group-dir-share script의 검증과 권한 설정 동작을 정의하고 container 시작 시 파일을 생성한다.
- ensure_kerberos_runtime : Kerberos ccache가 전달된 경우에만 그룹 공유 명령을 생성한다.
- write_restricted_sudoers : 제한적 sudo에서 허용하지 않을 권한 상승 경로를 정의한다.

2.3 홈 디렉터리와 공유 스토리지

공유 스토리지를 host에 mount하고 Docker bind mount로 container에 연결하는 작업은 container 시작 전에 외부에서 수행한다. container-images 는 NFS를 mount하거나 mount source를 선택하지 않는다. entrypoint가 동작할 때는 다음 경로가 이미 container에 연결되어 있다고 가정한다.

전달되는 mount	entrypoint에서 사용하는 위치	사용 목적
사용자 홈의 상위 공유 경로	/home	/home/<USER_ID> 를 사용자 홈으로 사용한다.
사용자 Kerberos ccache directory	host와 같은 경로	사용자 process에 KRB5CCNAME 으로 전달한다.
krb5.conf	/etc/krb5.conf (read-only)	Kerberos client 설정으로 사용한다.

entrypoint가 담당하는 작업은 mount된 /home/<USER_ID> 를 실제 사용자 권한으로 검증하고 container 실행에 필요한 최소 설정을 보완하는 것이다. 홈이 없으면 생성을 시도하고, 전달받은 UID/GID의 사용자로 파일을 직접 써 보아 사용 가능 여부를 확인한다. 공유 홈 전체의 소유권을 변경하지 않으며 .profile, .bashrc, .bash_logout 은 없는 경우에만 추가하며 기존 사용자 파일을 덮어쓰지 않는다.

NFS가 root_squash 를 사용하면 container root도 홈의 owner를 임의로 바꿀 수 없다. 이는 외부 스토리지의 조건이며 entrypoint는 이를 우회하지 않는다. Kerberos ccache가 전달되었지만 아직 홈을 쓸 수 없다면 SSH만 먼저 시작하고, 홈이 쓰기 가능해진 뒤 초기 설정과 Jupyter·VNC 시작을 이어서 수행한다.

관련 코드

- ensure_user_home : 홈 생성, 쓰기 권한, 기본 shell 파일과 history 저장 설정을 처리한다.
- start_kerberos_home_watcher : Kerberos ticket을 기다리면서 홈이 쓰기 가능해지는 시점을 확인한다.

2.4 SSH, Jupyter와 VNC

host port mapping, firewall과 `ENABLE_VNC` 같은 실행 option은 container 시작 전에 외부에서 정한다. `container-images` 는 host port를 할당하거나 network 접근 정책을 설정하지 않고, container 내부 service만 구성한다.

entrypoint는 SSH가 사용자와 관리 계정만 로그인할 수 있도록 `AllowUsers` 를 구성하고, PAM·command audit·login message를 적용한 뒤 SSH service를 재시작한다. 홈이 아직 Kerberos 인증을 기다리는 상태여도 SSH는 먼저 사용할 수 있으므로 사용자가 로그인하여 ticket 상태를 확인할 수 있다.

Jupyter 설정은 사용자 홈의 `.jupyter` 에 둔다. 관리하는 network·notebook 경로 항목만 갱신하고 나머지 사용자 설정은 유지한다. 시작할 때 임의 token을 만들고 권한이 0600인 `jupyter_token.txt` 에 저장한 뒤 사용자 권한으로 JupyterLab을 실행한다.

VNC/noVNC는 `ENABLE_VNC=true` 일 때만 시작한다. TigerVNC는 container 내부의 localhost에 bind하고, noVNC가 지정된 web port를 통해 연결한다. 기존 홈에 `vnc_password.txt` 가 있으면 재사용하므로 container 재시작만으로 password가 바뀌지 않는다.

관련 코드

- `configure_system_login`: SSH 접근 사용자, PAM, audit와 login message를 구성한다.
- `ensure_jupyter_config`: 사용자 설정을 보존하면서 Jupyter의 공통 항목을 갱신한다.
- `start_jupyter`: 접근 token을 저장하고 사용자 권한으로 JupyterLab을 시작한다.
- `start_novnc`: VNC password를 준비하고 TigerVNC와 noVNC proxy를 시작한다.

2.5 Kerberos 사용자 환경

Kerberos principal, ticket 발급·갱신, ccache 생성과 host NFS 인증은 `container-images` 밖에서 준비한다. host의 machine keytab도 container에 전달하지 않는다.

`container-images` 가 담당하는 작업은 전달받은 ccache를 container의 사용자 process에서 사용할 수 있게 만드는 것이다. entrypoint는 `KRB5CCNAME` 이 있을 때 ccache directory의 접근 권한, shell 환경 변수와 `decs-kerberos-status` 명령을 구성하고 Jupyter와 VNC에도 같은 환경 변수를 전달한다. 사용자는 `decs-kerberos-status` 로 ticket 상태를 확인할 수 있지만, entrypoint가 ticket을 직접 발급하거나 갱신하지는 않는다.

Kerberos keytab과 ccache를 분리하는 이유와 credential 경계는 Kerberos/NFS의 keytab과 ccache 모델을 따른다.

관련 코드

- `ensure_kerberos_runtime`: ccache 경로의 권한과 Kerberos 환경 변수, ticket 확인 helper를 구성한다.

3. 이미지 구성

사용자 실행 환경은 모든 image에서 같지만 GPU software stack은 workload에 따라 달라진다. `container-images` 는 공통 image 구성과 CUDA/TensorFlow 조합만 분리해 관리한다.

3.1 모든 이미지 버전의 공통 build 구성

모든 이미지 버전은 하나의 `Dockerfile` 을 사용하고 다음 프로그램을 공통으로 포함한다.

영역	공통 구성
기본 도구	ACL, audit, network, certificate와 package repository 도구
원격 접근	OpenSSH server와 Kerberos client
개발 환경	Miniforge, conda, pip, JupyterLab과 Notebook
GUI	Chrome, Xfce, TigerVNC, noVNC와 websockify
사용자 환경	한국어 글꼴과 입력기, 공통 entrypoint와 Jupyter config 위치

이미지 버전별 Dockerfile을 복제하지 않는 이유는 보안 patch와 공통 package 변경을 한 번만 적용하기 위해서다. 실제 차이는 build argument로 전달하며 공통 설치와 시작 흐름은 모든 이미지 버전에서 동일하게 유지한다.

3.2 CUDA/TensorFlow 조합별 이미지

이미지 버전 간 차이는 `image-variants.json`에 정의한다.

manifest 값	이미지 버전마다 달라지는 내용
<code>base_image</code>	CUDA, cuDNN과 Ubuntu가 포함된 NVIDIA base image
<code>cuda_version</code>	image가 제공하는 CUDA major/minor version
<code>tensorflow_version</code> , <code>tensorflow_package</code>	CUDA 조합에 맞춰 설치할 TensorFlow version과 package 형태
<code>python_version</code> , <code>conda_packages</code>	Python version과 TensorFlow/Jupyter 호환을 위한 package 제약
<code>min_nvidia_driver</code>	해당 CUDA image를 실행할 host의 최소 NVIDIA driver
<code>support</code>	실장비 검증을 마친 <code>stable</code> 또는 검증 중인 <code>experimental</code> 상태
<code>aliases</code>	version alias와 <code>stable</code> , <code>latest</code> 같은 권장 tag

현재 Python은 모두 3.10, Ubuntu는 모두 22.04를 사용한다. CUDA 조합에 따라 TensorFlow package, conda 제약과 최소 host driver가 달라진다.

CUDA	TensorFlow	주요 package 차이	최소 NVIDIA driver	지원 상태와 alias
11.8	2.13.0	구버전 Jupyter·IPython과 <code>typing_extensions=4.5</code> 제약	520.61.05	<code>stable</code> , <code>legacy</code>
12.2	2.15.0	기본 TensorFlow package와 공통 Jupyter package	535.104.05	<code>stable</code>
12.3	2.16.1	<code>tensorflow[and-cuda]</code> package 사용	545.23.08	<code>stable</code>
12.5	2.20.0	TensorFlow 2.20 기준 이미지	555.42.06	<code>stable</code> , <code>latest</code>

CUDA	TensorFlow	주요 package 차이	최소 NVIDIA driver	지원 상태와 alias
12.8	2.20.0	H200 대상 실험 이미지	570.124.06	experimental, h200-experimental

로컬 빌드 도구와 GitHub Actions는 같은 manifest를 읽어 build argument와 tag를 만든다. 따라서 지원 조합을 한 곳에서 검토하고 로컬과 CI가 서로 다른 image를 생성하는 것을 막는다.

3.3 GPU 호환성 처리

image는 시작할 때 build에 기록된 최소 NVIDIA driver와 `nvidia-smi` 결과를 출력한다. 기본값은 경고 중심이며 `STRICT_CUDA_COMPAT=true` 일 때만 최소 driver보다 낮은 host에서 시작을 실패시킨다.

기본을 강제 실패로 두지 않은 이유는 GPU가 없는 검증 환경이나 긴급 진단 container까지 막지 않기 위해서다. 운영 배포에서 호환성을 반드시 보장해야 하면 strict mode를 명시한다.

관련 코드

- `Dockerfile`: 공통 package를 설치하고 manifest 값을 build argument로 받는다.
- `image-variants.json`: 지원하는 이미지 버전별 CUDA/TensorFlow 조합, 최소 driver와 tag를 정의한다.
- `print_image_runtime_info`: image의 요구 driver와 실제 host driver를 비교한다.

4. 디렉터리 지도

경로	핵심 기능
<code>Dockerfile</code>	모든 이미지 버전이 공유하는 단일 image 정의
<code>image-variants.json</code>	base image, CUDA/TensorFlow, 최소 driver와 alias 목록
<code>entrypoint.sh</code>	시작 시 계정·그룹·홈·SSH·Jupyter·VNC·Kerberos 환경 구성
<code>scripts/variant_matrix.py</code>	manifest를 Docker/GitHub Actions build matrix로 변환
<code>scripts/build_variants.py</code>	로컬 build/push 명령 생성과 실행
<code>scripts/test_image_variants.py</code>	image metadata, TensorFlow와 선택적 GPU smoke test
<code>scripts/test_uid_create_container.py</code>	infra가 전달하는 container 생성 입력값의 dry-run 계약 검증
<code>tests/</code>	root_squash, Kerberos, sudo와 build 설정 회귀 test
<code>.github/workflows/docker-publish.yml</code>	PR merge 또는 수동 실행 시 matrix build/push

container-images 운영

개요 · 설계

GitHub 코드 링크: `admin_infra_server` 는 비공개 저장소다. 코드 링크를 누르면 GitHub 로그인 화면을 거쳐 원래 파일과 line으로 이동한다. 조직 저장소에 접근 권한이 있는 계정으로 로그인해야 한다.

1. 개요

이 문서의 목표는 `container-images` 설계에서 설명한 image와 `entrypoint.sh` 를 실제로 변경·검증하여 Docker registry에 배포한 뒤, 운영 container에서 사용할 수 있게 만드는 전체 절차를 제공하는 것이다. 새 CUDA/TensorFlow 조합을 추가하는 경우뿐 아니라 공통 package, 사용자 실행 환경, tag 정책이나 build 도구를 변경할 때 확인해야 할 범위도 함께 다룬다.

하나의 변경은 다음 순서로 운영한다.

1. 변경 목적에 맞는 파일을 선택한다.
2. manifest(`image-variants.json`)와 shell 회귀 test(`tests/*.sh` , 이번 변경이 기존 동작을 깨지 않았는지 image build 없이 먼저 확인하는 test)로 build 입력과 runtime 계약(entrypoint가 특정 입력에 대해 정해진 대로 동작하는지)을 확인한다.
3. 새로운 날짜 tag로 image를 build한다.
4. CPU smoke test(핵심 기능만 빠르게 도는지 확인하는 test)와 대상 GPU host test를 수행한다.
5. 검증한 image를 registry에 push한다.
6. 날짜 tag를 지정하여 신규 container를 생성하거나 기존 container를 교체한다.

`stable` 과 `latest` 는 고정된 release가 아니라 다른 image를 가리킬 수 있는 alias다. build, 배포, 장애 분석 기록에는 `cuda12.5-tf2.20-ubuntu22.04-260706` 처럼 날짜가 포함된 전체 tag를 사용한다.

2. 목적별 수정 위치

변경하려는 목적을 먼저 정한 뒤 그 목적을 소유하는 파일을 수정한다. 하나의 변경이 image 구성, 시작 동작과 검증에 함께 영향을 주면 표의 관련 파일도 같이 변경한다.

변경 목적	주로 수정할 위치	함께 확인하거나 수정할 위치
새로운 CUDA·TensorFlow·Python·Ubuntu 조합을 제공한다.	<code>image-variants.json</code>	<code>README.md</code> 의 image·alias 목록, matrix 출력과 GPU smoke test
모든 이미지 버전에 package나 공통 파일을 추가·갱신한다.	<code>Dockerfile</code>	<code>test_image_build_config.sh</code> , 전체 이미지 버전 build

변경 목적	주로 수정할 위치	함께 확인하거나 수정할 위치
container 시작 시 계정·그룹·홈·Kerberos·SSH·Jupyter·VNC를 구성하는 동작을 바꾼다.	<code>entrypoint.sh</code>	<code>test_entrypoint_root_squash.sh</code> , 기존 홈과 재시작 동작
image repository, 날짜 tag, alias 또는 지원 상태를 바꾼다.	<code>image-variants.json</code> , <code>variant_matrix.py</code>	<code>build_variants.py</code> 의 dry-run tag
로컬 build 명령어나 build argument 전달 방식을 바꾼다.	<code>build_variants.py</code>	<code>variant_matrix.py</code> , <code>Dockerfile</code> 의 ARG, 배포 workflow의 build argument
image에서 확인할 package·TensorFlow·GPU 항목을 추가한다.	<code>test_image_variants.py</code>	CPU test와 대상 GPU host test 결과
Docker Hub build/push를 자동화하거나 trigger를 변경한다.	<code>docker-publish.yml</code>	workflow 위치, build context, Docker Hub secret과 release tag

특정 사용자의 UID/GID, group membership, 홈 mount나 port를 변경하는 것은 `container-images` 변경 목적이 아니다. 외부에서 전달된 값을 container 내부에서 처리하는 방식 자체를 바꿀 때만 `entrypoint.sh`를 수정한다.

3. 새 이미지 버전 추가

3.1 조합 결정

새 이미지 버전은 CUDA 숫자만 추가하는 작업이 아니다. 다음 항목을 하나의 호환 조합으로 결정해야 한다.

- NVIDIA base image의 CUDA, cuDNN과 Ubuntu version
- TensorFlow version과 설치 package 표현
- Python version과 conda package 제약
- 해당 CUDA를 실행할 host의 최소 NVIDIA driver
- 실장비 검증 여부를 나타내는 `support`
- 날짜 tag 외에 제공할 alias

검증을 시작하는 단계에서는 `support`를 `experimental`로 두고 `stable`이나 `latest` alias를 부여하지 않는다. CPU와 대상 GPU test를 통과한 뒤에 지원 상태와 권장 alias를 변경한다.

3.2 manifest 항목 추가

`image-variants.json`의 `variants`에 다음 형태의 항목을 추가한다.


```
{
  "id": "cudaX.Y-tfA.B-ubuntu22.04",
  "base_image": "nvidia/cuda:<CUDA-cuDNN-Ubuntu tag>",
  "cuda_version": "X.Y",
  "tensorflow_version": "A.B.C",
  "tensorflow_package": "tensorflow==A.B.C",
  "conda_packages": "ipywidgets jupyterlab micromamba notebook pip",
  "python_version": "3.10",
  "ubuntu_version": "22.04",
  "min_nvidia_driver": "<minimum driver>",
  "support": "experimental",
  "aliases": [
    "cudaX.Y-tfA.B-experimental"
  ]
}
```

`id`에는 날짜를 넣지 않는다. `build_variants.py`가 `default_date_tag` 또는 `--date-tag` 값을 붙여 실제 배포 tag를 만든다. `aliases`에는 날짜 없이 계속 사용할 이름만 둔다. 기존 `stable`이나 `latest`를 새 항목으로 옮기면 다음 push부터 그 alias가 새 image를 가리키므로 별도의 승격 변경으로 검토한다.

정적 목록을 제공하는 `container-images/README.md`의 이미지 표와 alias 표도 함께 갱신한다. 그다음 `variant_matrix.py`가 manifest에서 만들어내는 matrix와 tag를 확인한다.

```
cd /home/jy/server_manage/container-images
python3 scripts/variant_matrix.py \
  --variant cudaX.Y-tfA.B-ubuntu22.04 \
  --date-tag YYMMDD
```

출력의 `base_image`, build argument와 `docker_tags`가 추가한 항목과 일치해야 한다.

4. 기존 구성 변경

4.1 공통 package와 build 환경 변경

apt package, Miniforge·conda, Python package 설치 순서와 image에 포함할 공통 파일은 `Dockerfile`에서 변경한다. 모든 이미지 버전이 하나의 Dockerfile을 공유하므로 이 변경은 특정 CUDA 조합에만 국한되지 않는다.

변경 후에는 최소한 현재 권장 이미지와 package 제약이 가장 강한 legacy 이미지를 각각 build한다. release 전에는 전체 이미지 버전의 build와 smoke test를 확인한다. build 순서나 필수 package 제약이 달라졌다면 `test_image_build_config.sh`도 같이 수정한다.

4.2 사용자 실행 환경 변경

계정과 그룹 생성, 홈 초기화, Kerberos credential, SSH, Jupyter와 VNC 동작은 `entrypoint.sh` 에서 변경한다. 다음 조건을 유지해야 한다.

- 같은 입력으로 container를 재시작해도 계정과 설정이 중복되거나 손상되지 않는다.
- 기존 공유 홈의 `.bashrc`, Jupyter 설정과 password 파일을 임의로 덮어쓰지 않는다.
- NFS `root_squash` 환경에서 공유 홈 전체를 `chown` 하지 않는다.
- 홈과 사용자 application에 쓰는 파일은 실제 사용자 UID/GID로 생성한다.
- Kerberos keytab을 container에 전달하지 않고 ccache만 사용한다.
- restricted sudo의 identity 전환, mount와 권한 변경 차단을 유지한다.

`entrypoint.sh` 는 image 안에 복사되므로 source만 수정하거나 실행 중인 container를 재시작해도 변경 내용이 적용되지 않는다. 새 날짜 tag로 image를 다시 build·push하고, 운영 container도 그 tag로 다시 생성해야 한다.

4.3 tag, alias와 지원 상태 변경

지원 상태와 alias는 `image-variants.json` 에서 변경한다. `experimental` 을 `stable` 로 승격할 때는 해당 날짜 image의 CPU·GPU smoke 결과와 대상 server의 최소 driver 충족 여부를 먼저 확인한다. `stable` 과 `latest` 는 한 이미지 버전에만 두어 운영자가 어떤 조합이 권장 버전인지 판단할 수 있게 한다.

이미 배포 기록에 사용한 날짜 tag는 같은 이름으로 다시 build하지 않는다. source가 달라졌다면 새로운 `YYMMDD` tag를 사용한다. 같은 tag를 덮어쓰면 target host에 남아 있는 image cache와 registry image가 서로 달라질 수 있다.

4.4 build와 test 도구 변경

manifest field를 추가하거나 tag 규칙을 바꾸면 `variant_matrix.py`, `build_variants.py`, `test_image_variants.py` 가 같은 field와 tag를 사용하는지 함께 확인한다. 로컬 build와 배포 workflow가 서로 다른 image를 만들지 않도록 한쪽에만 build argument를 추가하지 않는다.

5. 로컬 build

모든 명령은 `container-images` directory에서 실행한다.

```
cd /home/jy/server_manage/container-images
```

먼저 실제 build 없이 Docker 명령과 생성될 tag를 확인한다.

```
python3 scripts/build_variants.py \  
  --variant cuda12.5-tf2.20-ubuntu22.04 \  
  --date-tag YYMMDD \  
  --dry-run
```

확인한 이미지 버전을 build한다.

```
python3 scripts/build_variants.py \  
  --variant cuda12.5-tf2.20-ubuntu22.04 \  
  --date-tag YYMMDD
```

`--variant` 를 생략하면 manifest의 모든 이미지 버전을 순서대로 build한다. Docker cache 때문에 package 갱신이 반영되지 않았는지 확인해야 할 때만 `--no-cache` 를 추가한다.

6. 검증

6.1 source 회귀 test

```
bash tests/test_image_build_config.sh  
bash tests/test_entrypoint_root_squash.sh
```

첫 번째 test는 Dockerfile의 version pin, 필수 package와 설치 순서를 확인한다. 두 번째 test는 entrypoint 문법과 UID/GID, 그룹, sudo, NFS 홈, Kerberos, Jupyter와 VNC의 runtime 계약을 확인한다.

6.2 image smoke test

build한 날짜와 동일한 tag를 지정해 CPU 환경을 확인한다.

```
python3 scripts/test_image_variants.py \  
  --variant cuda12.5-tf2.20-ubuntu22.04 \  
  --date-tag YYMMDD
```

이 test는 image에 기록된 TensorFlow version, Python, Jupyter, micromamba와 entrypoint 포함 여부를 확인한다. 실제 NVIDIA runtime과 TensorFlow GPU 인식은 대상 GPU host에서 `--gpu` 를 추가하여 확인한다.

```
python3 scripts/test_image_variants.py \  
  --variant cuda12.5-tf2.20-ubuntu22.04 \  
  --date-tag YYMMDD \  
  --gpu
```

6.3 container 생성 계약 확인

image tag가 `infra` 의 container 생성 입력으로 올바르게 전달되는지 실제 DB나 container를 변경하지 않고 확인한다.

```
python3 scripts/test_uid_create_container.py \  
  --variant cuda12.5-tf2.20-ubuntu22.04 \  
  --date-tag YYMMDD \  
  --print-only
```

출력된 명령에서 repository 이름과 날짜가 포함된 version tag를 확인한다.

7. registry 배포

7.1 로컬 build와 push

Docker Hub 로그인에 사용할 관리자 계정은 [관리자 계정 통합관리문서](#)에서 확인한다. 인증과 test가 완료된 환경에서 `--push` 를 추가한다.

```
docker login  
python3 scripts/build_variants.py \  
  --variant cuda12.5-tf2.20-ubuntu22.04 \  
  --date-tag YYMMDD \  
  --push
```

이 명령은 날짜 tag와 해당 이미지 버전에 선언된 alias를 모두 push한다. 따라서 `stable` 이나 `latest` 가 포함된 항목은 alias 변경까지 승인된 뒤 실행한다. push 후 대상 GPU host에서 날짜 tag를 pull하고 6.2의 GPU smoke test를 다시 수행한다.

7.2 GitHub Actions 상태

build/push workflow 정의에는 `main` 대상 PR merge와 수동 `workflow_dispatch` 가 trigger로 작성되어 있다. 그러나 현재 파일은 모노레포 최상위 `.github/workflows/` 가 아니라 `container-images/.github/workflows/docker-publish.yml` 에 있으므로 GitHub가 자동 workflow로 등록하지 않는다.

workflow를 최상위 위치로 옮기고 working directory, Docker build context와 `DOCKERHUB_USERNAME · DOCKERHUB_TOKEN` secret을 확인하기 전까지는 PR merge를 image 배포 완료로 간주하지 않는다. 현재 배포 기준은 7.1의 로컬 build/push다.

8. 운영 container 실행과 교체

8.1 신규 container 실행

운영 container는 긴 `docker run` 명령을 직접 작성하지 않고 `infra` 의 생성 명령을 사용한다. 그래야 UID/GID, 그룹 membership, port, GPU, 공유 홈과 Kerberos ccache가 같은 기준으로 준비된다.

먼저 날짜 tag를 넣고 `--dry-run` 으로 계획과 Docker 명령을 확인한다.

```
uidctl create-container \
  --name "사용자 이름" \
  --username username \
  --server-id LAB10 \
  --expiration-date YYYY-MM-DD \
  --image decs \
  --version cuda12.5-tf2.20-ubuntu22.04-YYMMDD \
  --created-by operator \
  --email user@example.com \
  --phone 000-0000-0000 \
  --dry-run
```

계획의 target server, GPU, host port, mount source와 최종 image tag를 검토한 뒤 같은 명령에서 `--dry-run` 만 제거하여 실행한다. Kerberos 또는 VNC가 필요한 경우 각각 `--enable-kerberos`, `--enable-vnc` 를 추가한다.

`infra`의 `container` 생성 구현은 target host에 image가 없으면 pull하고, storage와 credential을 준비한 뒤 Docker container와 DB record를 생성한다.

8.2 기존 container에 새 image 적용

`docker restart` 는 기존 container가 참조하는 image를 그대로 다시 실행하므로 새로 push한 image를 적용하지 않는다. 공유 홈을 보존한 상태에서 새 날짜 tag를 지정해 container를 교체해야 한다.

교체 전에는 기존 container의 사용자, server, GPU, port, 만료일, VNC·Kerberos option을 기록하고 새 생성 계획과 비교한다. DB와 port record가 함께 관리되므로 운영 container를 `docker rm` 만으로 직접 교체하지 않는다. 새 container에서 SSH, Jupyter, 선택적인 VNC, 홈 쓰기와 GPU 인식을 확인한 뒤 이전 container를 정리한다.

9. 배포 확인과 rollback

배포가 끝나면 다음을 확인한다.

- registry와 target host가 동일한 날짜 tag를 사용하는가?
- 시작 log의 image 버전, CUDA/TensorFlow와 최소 driver가 manifest와 일치하는가?
- TensorFlow가 할당된 GPU를 인식하는가?
- 사용자 UID/GID와 primary·supplemental group이 공유 스토리지 권한과 일치하는가?
- 공유 홈에서 읽기·쓰기, SSH와 Jupyter가 동작하는가?
- Kerberos 사용 시 ccache가 보이고 keytab은 container에 없는가?

문제가 발견되면 `stable` 이나 `latest` 의 과거 의미를 추측하지 않고, 마지막으로 검증한 날짜 tag를 지정하여 container를 다시 생성한다. 문제가 있는 날짜 tag는 덮어쓰지 않고 manifest의 `support` 와 alias를 수정하여 신규 사용을 막은 뒤 원인을 수정한 새 날짜 tag를 배포한다.

10. 운영 안전 수칙

- 실제 사용자 password를 image layer나 manifest에 넣지 않는다.
- keytab을 image 또는 container mount로 제공하지 않는다.
- source 변경, local image, registry image와 실행 중인 container를 구분해 진단한다.
- 실제 push 전에는 `--dry-run`의 전체 tag 목록을 확인한다.
- `experimental` 이미지 버전을 `stable`로 승격하기 전에 대상 GPU test를 수행한다.
- 기존 공유 홈을 사용하는 test는 파일과 권한을 변경할 수 있으므로 별도 test 사용자와 경로에서 수행한다.

monitoring 개요

원본: md/system/monitoring/index.md, md/system/monitoring/design.md, md/system/monitoring/operations.md

monitoring은 FARM/LAB 서버의 자원, GPU, container와 주요 service 상태를 지속적으로 수집하고 metric, dashboard와 alert로 제공한다. 서버에서 수집한 상태는 Prometheus에 저장되며, Grafana에서 조회하고 Alertmanager를 통해 운영 알림으로 전달한다.

문서	내용
설계	monitoring의 목표와 구조, 구성요소별 역할·입력·처리·출력, 주요 설정과 코드 위치
운영	exporter와 monitoring stack 배포, endpoint 확인, 장애 진단과 변경 검증 절차

monitoring 설계

개요 · 운영

GitHub 코드 링크: `admin_infra_server`는 비공개 저장소다. 링크를 누르면 GitHub 로그인 화면을 거쳐 해당 파일로 이동한다. 조직 저장소에 접근 권한이 있는 계정으로 로그인해야 한다.

1. 개요

monitoring의 목표는 FARM/LAB 서버와 주요 service의 현재 상태와 변화 추이를 지속적으로 관측하고, 운영자가 dashboard와 alert를 통해 이상 상태를 확인할 수 있게 하는 것이다. 관측 범위에는 CPU, memory, filesystem, network, GPU, container, Docker와 외부 연결 상태가 포함된다.

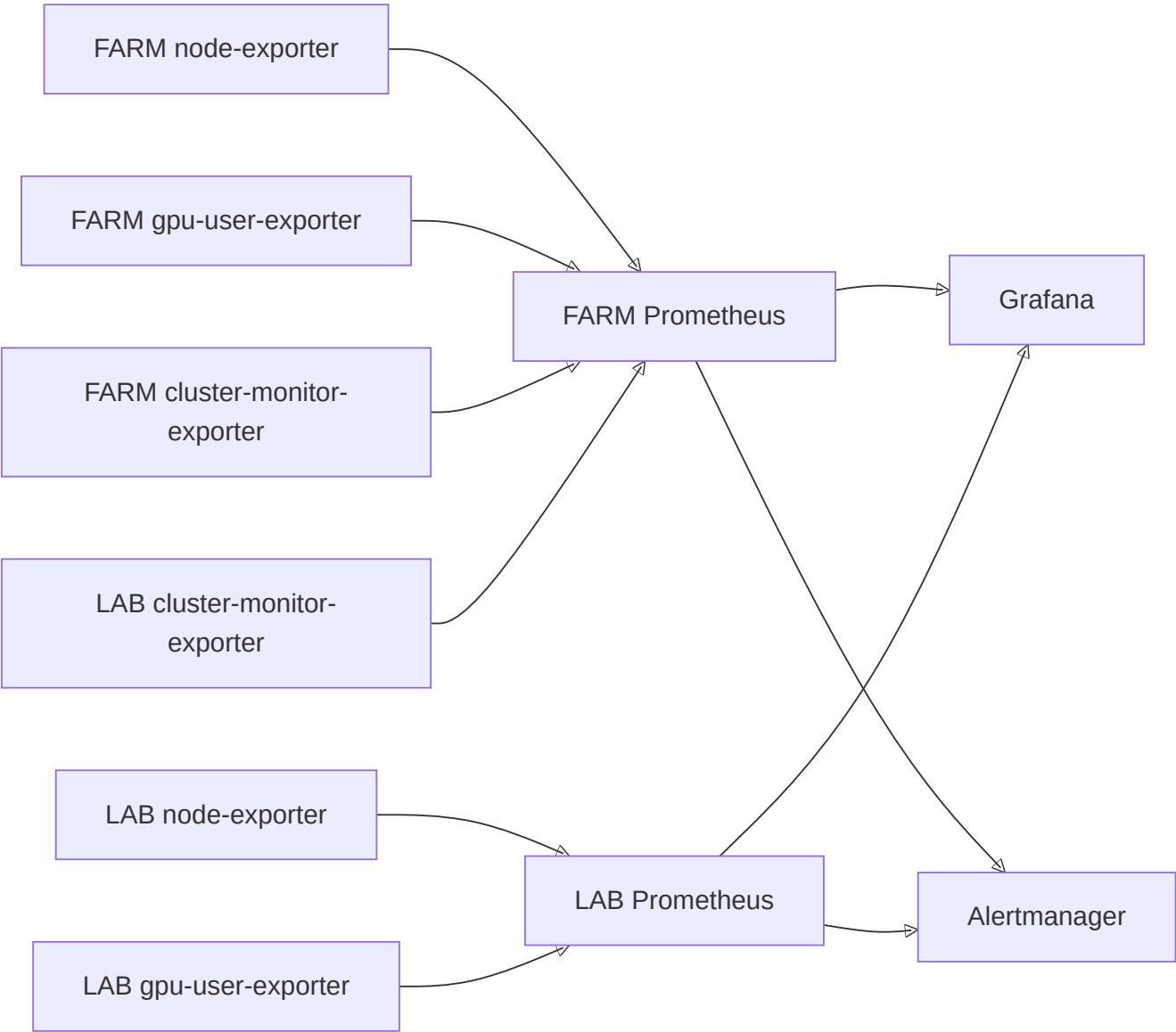
monitoring은 Prometheus·Grafana·Alertmanager 조합을 쓴다. 서버에서는 exporter가 상태를 수집해 Prometheus metric으로 제공한다. Prometheus는 이 metric을 주기적으로 가져와 시계열로 저장하고, alert rule을 주기적으로 계산해 alert를 만든다. Alertmanager는 여러 Prometheus가 만든 alert를 모아 중복을 정리하고 전달하며, Grafana는 저장된 시계열을 그래프로 보여주는 dashboard를 제공한다.

2. 설계 구조

monitoring은 서버에서 상태를 만드는 구성요소와 metric을 저장·조회·전달하는 control plane으로 구성된다.

구분	구성요소	구현 형태	역할
서버 자원 수집	node-exporter	Prometheus 기본 exporter	CPU, memory, filesystem, disk와 network 상태를 metric 으로 제공한다.
GPU 사용량 수집	gpu-user-exporter	커스텀 Go exporter	GPU process를 container와 사용자 정보에 연결한다.
운영 상태 수집	cluster-monitor-exporter	커스텀 Go exporter	Docker, container, GPU와 연결 상태를 수집한다.
Metric 저장·평가	FARM/LAB Prometheus	kube-prometheus-stack	환경별 metric을 저장하고 alert rule을 평가한다.
시각화	Grafana	kube-prometheus-stack	두 Prometheus의 시계열을 하나의 dashboard 환경에서 표시한다.
Alert 처리	Alertmanager	kube-prometheus-stack	두 Prometheus가 생성한 alert를 한곳에서 묶고 routing한다.

구성요소 사이의 주요 데이터 흐름은 다음과 같다.



FARM Prometheus는 FARM 서버의 자원·GPU metric과 FARM/LAB 전체의 `cluster-monitor-exporter` metric을 수집한다. 공통 service alert rule은 FARM Prometheus가 평가하고 Alertmanager로 전달한다.

LAB Prometheus는 LAB 서버의 node·GPU metric을 별도로 저장한다. 현재 LAB node·GPU metric은 Grafana 조회에 사용되고, LAB의 Docker·container 같은 운영 상태 경보는 FARM Prometheus가 LAB `cluster-monitor-exporter`를 수집해 처리한다. Grafana는 두 Prometheus를 datasource로 사용하며, 두 Prometheus는 같은 Alertmanager로 alert를 보낸다. Grafana와 Alertmanager는 각각 하나의 인스턴스로 운영한다.

3. 서버 metric 수집

3.1 `node-exporter`

구현 형태: Prometheus 커뮤니티에서 제공하는 기본 `node_exporter`다. `kube-prometheus-stack`의 `prometheus-node-exporter` chart가 FARM/LAB node에 DaemonSet으로 배포한다. 이 저장소는 exporter 구현 코드 대신 환경별 port, collector와 scrape 설정을 관리한다.

역할: 운영체제와 hardware의 기본 자원 사용량을 제공한다.

입력: Linux kernel이 제공하는 `/proc`, `/sys`, filesystem과 network interface 정보를 읽는다.

처리: upstream `node-exporter` collector가 CPU, memory, load, filesystem, disk I/O와 network 통계를 Prometheus 형식으로 변환한다. FARM과 LAB의 `kube-prometheus-stack`이 DaemonSet으로 각 node에 배포한다.

출력: 각 서버의 `:30070/metrics`에서 `node_cpu_*`, `node_memory_*`, `node_filesystem_*`, `node_disk_*`, `node_network_*` metric을 제공한다.

주요 설정: service port, collector option과 Prometheus 연결은 FARM/LAB Prometheus values의 `prometheus-node-exporter` 항목에서 관리한다.

관련 코드: `prometheus-farm-values.yaml`, `prometheus-lab-values.yaml`

3.2 `gpu-user-exporter`

구현 형태: GPU process, Docker container와 UID DB 사용자 정보를 연결하기 위해 만든 커스텀 Go exporter다. Prometheus Go client의 custom collector를 구현해 scrape 요청마다 GPU·container·사용자 정보를 수집하고 metric을 생성한다.

역할: 서버의 GPU 사용량을 실제 사용자와 container 단위로 제공한다.

입력: `nvidia-smi`의 GPU·PID·memory·utilization, `/proc/<pid>/cgroup`의 container ID, Docker inspect 결과와 UID DB의 container 사용자 정보를 사용한다.

처리: GPU process의 PID를 container에 연결하고, container record에서 사용자 정보를 조회한 뒤 GPU별 사용량을 집계한다. 실행 중인 DB 등록 container는 현재 GPU process가 없는 경우에도 값이 0인 시계열을 유지한다. 사용자 정보를 찾을 수 있는 process와 별도 관리가 필요한 process는 metric에서 구분한다.

출력: `:30072/metrics`와 `:30072/-/healthy`를 제공한다. 대표 metric은 다음과 같다.

- `docker_gpu_user_memory_used_bytes`

- `docker_gpu_user_sm_utilization_percent`
- `docker_gpu_user_process_count`
- `docker_gpu_device_utilization_percent`
- `docker_gpu_exporter_ignored_process_count`

주요 설정: server ID, DB 접속값, DB cache refresh 주기, command timeout, `nvidia-smi` 경로와 `/proc` 경로를 systemd 환경 파일로 전달한다. exporter는 host PID와 Docker 정보를 읽을 수 있는 권한으로 실행된다.

관련 코드: `main.go`, `gpu-user-exporter README`, `deploy_exporters.yml`

Go 코드 구조

`gpu-user-exporter`의 Go 구현은 `main.go`에 있다. 먼저 전체 실행 흐름을 확인한 뒤 아래 코드 표를 따라가면, 한 번의 scrape가 원천 데이터를 수집하고 사용자별 metric을 만드는 과정을 파악할 수 있다.

```
main -> config·DB·registry 초기화
      -> Prometheus scrape
      -> gpuUserCollector.Collect
      -> DB cache 갱신 + GPU/process/Docker 수집
      -> PID를 container와 사용자에게 연결
      -> 사용자·container·GPU 단위 집계
      -> Prometheus metric 출력
```

실행 흐름의 각 단계는 다음 코드가 담당한다.

코드	역할
<code>main</code>	DB 연결, Prometheus registry, custom collector와 HTTP endpoint를 초기화한다.
<code>config</code> , <code>loadConfig</code> , <code>resolveDSN</code>	실행 option과 환경 변수를 읽고 server ID에 맞는 DB 연결 정보를 만든다.
<code>gpuUserCollector</code> , <code>newGPUUserCollector</code>	설정과 DB cache를 보관하고 exporter가 제공할 metric을 정의한다.
<code>Collect</code>	한 번의 scrape에서 GPU, process, container와 사용자 정보를 결합해 metric을 출력한다.
<code>ownerCache</code> 와 <code>refreshIfNeeded</code>	UID DB의 active container 정보를 일정 시간 cache하고 container ID 조회를 제공한다.
<code>collectGPUInfo</code> , <code>collectGPUProcesses</code> , <code>collectPmon</code>	<code>nvidia-smi</code> 출력을 GPU·process·utilization 구조체로 변환한다.
<code>containerIDFromPID</code>	<code>/proc/<pid>/cgroup</code> 에서 GPU process가 속한 container ID를 찾는다.
<code>runningContainerZeroKeys</code> , <code>collectRunningDockerContainers</code>	실행 중인 container와 GPU 할당을 확인해 GPU process가 없는 사용자 container에도 값이 0인 시계열을 만든다.

3.3 `cluster-monitor-exporter`

구현 형태: FARM/LAB 서버의 운영 상태와 환경별 진단 항목을 수집하기 위해 만든 커스텀 Go exporter다. background collection, metric rendering과 HTTP endpoint를 직접 구현한다.

역할: 기본 자원 metric만으로 판단하기 어려운 서버와 service의 운영 상태를 수집한다.

입력: process·kernel 상태, `nvidia-smi`, Docker와 container 상태, systemd service와 외부 연결 상태를 읽는다.

처리: background collection loop가 각 점검을 실행해 마지막 결과를 memory에 보관한다. HTTP 요청은 이 결과를 Prometheus 형식으로 출력한다. 외부 명령에는 timeout을 적용하며, 마지막 수집 시각과 허용된 stale 시간을 함께 기록한다.

주요 관측 항목은 다음과 같다.

- host GPU와 Docker daemon 상태
- 대상 container의 실행, SSH와 GPU 상태
- 외부 network 연결 상태

출력: `:30074/metrics`, `:30074/healthz` 와 서버별 public `:N89/healthz` 를 제공한다. `/healthz` 는 HTTP process 응답과 최근 collection freshness를 함께 확인한다.

제한된 복구: 설정으로 활성화한 경우 대상 container 시작, container SSH service 시작과 NVML driver/library symlink 복구를 수행한다. 복구 시도와 결과는 metric으로 기록한다.

주요 설정: 수집 주기, stale 기준, command timeout, 대상 container image pattern, public health port와 복구 기능 활성화 여부를 `/etc/default/cluster-monitor-exporter` 에서 관리한다.

관련 코드: `main.go`, `환경 설정 예시`, `cluster-monitor-exporter README`

Go 코드 구조

`cluster-monitor-exporter` 는 `main` 이 설정과 HTTP server를 준비하고, background loop가 영역별 점검 결과를 metric text로 만든 뒤 HTTP handler가 마지막 결과를 제공한다.

```
main -> Config 로딩·검증
      -> Collector.run background loop
      -> collect가 영역별 collector 호출
      -> renderer가 Prometheus text 생성
      -> Collector가 마지막 결과와 수집 시각 보관
      -> /metrics와 /healthz가 저장된 결과 제공
```

실행 흐름의 각 단계는 다음 코드가 담당한다.

파일·코드	역할
<code>main</code>	<code>Collector</code> , background goroutine, metric·health endpoint와 public health HTTP server를 시작한다.
<code>Config</code> , <code>LoadConfig</code>	환경 파일과 환경 변수를 읽고 수집 주기, timeout, endpoint와 기능별 설정을 구성한다.

파일·코드	역할
<code>Collector.run</code> , <code>collectOnce</code>	설정된 주기로 전체 수집을 실행하고 완성된 metric text와 수집 시각을 교체한다.
<code>Collector.collect</code>	영역별 collector를 순서대로 호출하고 한 번의 수집 결과를 만든다.
<code>collectHostGPU</code> 부터 <code>collectContainers</code>	host 명령과 Docker 상태를 읽고 GPU, Docker와 container metric을 생성한다.
<code>renderer</code>	metric help, type, label과 값을 Prometheus text format으로 직렬화한다.
<code>handleMetrics</code> , <code>handleHealthz</code> , <code>collectionHealth</code>	마지막 수집 결과를 <code>/metrics</code> 로 제공하고 수집 시각을 기준으로 health 상태를 응답한다.

Collection freshness

`up` 은 Prometheus가 exporter의 HTTP endpoint에 접속했는지를 나타낸다. 실제 점검 loop의 진행 상태는 `cluster_monitor_exporter_last_collection_timestamp_seconds` 와 `cluster_monitor_exporter_metrics_stale_after_seconds` 를 함께 사용해 판단한다. 이 구조를 통해 HTTP listener와 background collection의 상태를 각각 확인할 수 있다.

Container alert 조건

container GPU 상태는 container 실행 상태와 함께 평가한다. 정지한 container는 `running=0`, `gpu_up=0` 으로 기록되고 stopped alert의 대상이 된다. 실행 중인 container에서 GPU 점검이 실패한 경우에만 GPU alert가 발생한다.

```
cluster_monitor_container_gpu_up{job="cluster-monitor-exporter"} == 0
and on (instance, server, container, image)
cluster_monitor_container_running{job="cluster-monitor-exporter"} == 1
```

`instance`, `server`, `container`, `image` label을 함께 사용해 같은 서버와 같은 container의 시계열을 결합한다.

4. Metric 저장, 시각화와 alert

4.1 Prometheus

역할: exporter metric을 주기적으로 수집해 시계열로 저장하고 alert rule을 평가한다.

FARM 구성: FARM node·GPU metric과 FARM/LAB 전체 `cluster-monitor-exporter` metric을 수집한다. 공통 service alert rule을 평가하고 Prometheus data를 FARM local PV에 저장한다.

LAB 구성: LAB node·GPU metric을 별도 Prometheus에 저장한다. LAB control plane과 storage가 FARM과 분리되어 있다. LAB node·GPU metric은 Grafana가 LAB datasource를 통해 조회하며, LAB Prometheus가 생성한 alert는 공용 Alertmanager로 전달한다.

Grafana와 Alertmanager는 FARM control plane에 각각 하나만 배포한다. FARM Prometheus는 cluster 내부 Service를 사용하고, LAB Prometheus는 Alertmanager의 NodePort endpoint를 사용한다.

주요 설정: scrape target, label, retention, storage, alert rule과 Helm release 설정은 환경별 values에서 관리한다. FARM values가 운영 alert rule의 기준 파일이다.

관련 코드: `prometheus-farm-values.yaml`, `prometheus-lab-values.yaml`, `deploy_prometheus.yaml`

4.2 Alertmanager

역할: FARM/LAB Prometheus가 생성한 alert를 한곳에서 받아 label과 상태를 기준으로 묶고 receiver로 routing한다.

입력: 두 Prometheus가 alert rule을 평가해 만든 firing·resolved alert와 각 alert의 label·annotation을 사용한다.

처리: 같은 alert를 group label에 따라 묶고 route의 matcher에 맞는 receiver를 선택한다. 상태가 resolved로 바뀌면 같은 alert group의 해제 상태도 처리한다.

출력: receiver별로 정리된 alert와 현재 firing·resolved 상태를 제공한다.

주요 설정: 공용 Alertmanager의 route, receiver, NodePort와 Secret 연결은 FARM Prometheus values에서 관리한다. LAB Prometheus의 공용 Alertmanager endpoint는 LAB Prometheus values에서 관리한다.

관련 코드: `Alertmanager 설정`, `LAB Prometheus 연결 설정`

4.3 Grafana

역할: FARM과 LAB의 시계열을 서버, GPU와 network 관점의 dashboard로 제공한다.

입력: 기본 FARM datasource와 UID가 `prometheus-lab` 인 LAB datasource를 사용한다.

처리: dashboard query가 datasource, cluster, server, instance, GPU와 network interface label을 기준으로 시계열을 선택하고 비교한다.

출력: GPU usage와 network traffic dashboard를 제공한다. Grafana service는 NodePort `30080` 을 사용한다.

주요 설정: datasource와 dashboard ConfigMap은 `monitoring/grafana/` 에서, Grafana service·persistence·Secret 연결은 FARM Prometheus values에서 관리한다.

관련 코드: `Grafana dashboards`, `LAB datasource`, `Grafana PV`

5. 배포와 설정 구조

위치	역할
<code>monitoring/ansible_playbook/inventory.ini</code>	FARM/LAB 배포 대상과 SSH 접속 정보
<code>monitoring/ansible_playbook/group_vars/exporters.yml</code>	두 custom exporter의 공통·서버별 설정
<code>monitoring/ansible_playbook/deploy_exporters.yml</code>	두 custom exporter build·설치·검증
<code>monitoring/ansible_playbook/deploy_prometheus.yml</code>	FARM/LAB monitoring stack 검증·배포
<code>monitoring/prometheus/config/</code>	Prometheus, Alertmanager, PV와 storage 설정
<code>monitoring/grafana/</code>	datasource와 dashboard 원본

Ansible은 Go exporter binary를 build하고 서버별 환경 파일과 systemd unit을 설치한다. Prometheus 배포는 대 상 Kubernetes context, node 상태, PV와 Secret을 확인한 뒤 환경별 Helm values를 적용한다. 구체적인 변경·배 포·점검 절차는 [운영 문서](#)에서 설명한다.

6. 주요 설계 기준

6.1 HTTP 응답과 collection 상태 분리

`cluster-monitor-exporter` 는 background loop의 결과를 endpoint에서 제공한다. Prometheus의 `up` , exporter의 마지막 collection 시각과 stale 기준을 함께 사용해 HTTP listener와 실제 점검 진행 상태를 각각 확인한다.

6.2 Alert label과 secret 관리

alert는 `instance` , `server` , `container` , `image` , `cluster` 처럼 진단에 필요한 label을 사용한다. credential과 password는 Kubernetes Secret과 systemd 환경 파일에서 제공하며 metric과 alert payload에는 상태와 식별 정 보만 기록한다.

monitoring 운영

이 문서는 monitoring 구성 요소를 배포하고 endpoint, metric, alert와 incident를 점검하는 방법을 설명한다.

운영 원칙

- `up=1` 과 실제 collection freshness를 구분한다(용어는 [설계 문서](#) 3.3·6.1절 참고). HTTP process가 응답해도 마 지막 성공 collection이 오래됐으면 정상으로 판단하지 않는다.
- metric 수집, 증거 보존과 상태 변경을 분리한다. exporter는 mount하지 않고, forensics는 service restart나 reboot를 수행하지 않는다.
- NFS caller가 D-state이면 `find` , `du` , `stat` , canary나 recovery를 반복 실행하지 않고 local state와 이미 수집 된 snapshot을 먼저 보존한다.
- 자동 복구 결과는 원래 정상 상태와 구분해 metric과 state에 남긴다. inhibit 또는 backoff가 설정된 worker를 endpoint 호출로 우회하지 않는다.
- 배포는 `monitoring/ansible_playbook/` 을 기준으로 하고 target host의 개별 install script를 운영 entry point로 사용하지 않는다.
- DB password, Grafana password, webhook과 Kerberos credential은 metric, alert label, Git values와 일 반 journal에 기록하지 않는다.

용어	의미
D-state	프로세스가 I/O 응답을 기다리며 멈춰서 강제 종료도 안 되는 리눅스 상태
forensics	문제를 직접 고치지 않고 로그·상태 스냅샷 같은 증거를 보존해두는 것

용어	의미
canary	실제 트래픽에 영향 없는 별도 경로로 주기적으로 살아있는지 찢러보는 점검
inhibit / backoff	반복 실패 시 복구 시도를 억제하거나 점점 간격을 늘려 재시도하는 안전장치

1. 배포 순서

새 host 또는 NFS GSS monitoring을 처음 적용할 때는 다음 순서를 권장한다.

1. inventory와 host FQDN/SPN/mount source 확인
2. NFS GSS readiness/canary/recovery worker 설치
3. NFS forensics 설치
4. exporter 배포
5. Prometheus target/rule 반영
6. Grafana datasource/dashboard 확인
7. alert delivery test

운영 배포 entry point는 exporter 내부 개별 shell script가 아니라 `monitoring/ansible_playbook/` 이다.

2. Exporter 배포

```
cd /home/jy/server_manage/monitoring
```

두 exporter를 한 host에 배포:

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \
ansible-playbook ansible_playbook/deploy_exporters.yml \
-e exporter_hosts=farm8
```

FARM 전체 또는 FARM/LAB 전체:

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \
ansible-playbook ansible_playbook/deploy_exporters.yml \
-e exporter_hosts=FARM
```

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \
ansible-playbook ansible_playbook/deploy_exporters.yml
```

한 exporter만 전체 재배포:

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \  
ansible-playbook ansible_playbook/deploy_cluster_monitor_exporter_all.yml
```

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \  
ansible-playbook ansible_playbook/deploy_gpu_user_exporter_all.yml
```

target에는 다음 systemd service와 endpoint가 생긴다.

```
cluster-monitor-exporter.service -> :30074/metrics, :30074/healthz, :N89/healthz  
gpu-user-exporter.service        -> :30072/metrics, :30072/-/healthy
```

배포 기준과 요구 조건은 [Ansible README](#), 실제 task는 [deploy_exporters.yml](#)에서 확인한다.

3. NFS GSS와 forensics 배포

이 절의 canary/GSS 개념은 [Kerberos/NFS 운영](#) 문서를 전제로 한다.

NFS GSS readiness/canary/recovery worker:

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \  
ansible-playbook ansible_playbook/deploy_nfs_gss_health.yml \  
-e nfs_gss_health_hosts=lab8
```

NFS forensics:

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \  
ansible-playbook ansible_playbook/deploy_nfs_forensics.yml
```

중요한 systemd unit과 state:


```
decs-kerberos-nfs-ready.service
decs-nfs-gss-canary-probe.service/.timer
decs-nfs-gss-mount-recovery.service/.timer
/run/decs-nfs-gss/ready.state
/var/lib/decs-nfs-gss/canary.state
/var/lib/decs-nfs-gss/recovery.state
```

```
decs-nfs-forensics-buffer.service
decs-nfs-forensics-watch.service/.timer
decs-nfs-forensics-snapshot.service
/run/decs-nfs-forensics/status.json
/var/lib/decs-nfs-forensics/
```

FARM은 전용 read-only canary export가 준비될 때까지 canary만 비활성화하며 keytab/kinit/kvno/rpc-gssd readiness와 guarded recovery는 유지한다. LAB canary는 `sec=krb5` 전용 read-only export를 사용한다.

4. Keytab checker 배포

FARM 읽기 전용 checker:

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \
ansible-playbook ansible_playbook/deploy_kerberos_nfs_keytab_check.yml
```

LAB을 상시 점검 대상으로 전환한 뒤에만 두 profile을 활성화한다.

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \
ansible-playbook ansible_playbook/deploy_kerberos_nfs_keytab_check.yml \
-e '{"keytab_check_profiles":["farm","lab"]}'
```

```
systemctl --user list-timers 'check-nfs-keytab@*.timer'
systemctl --user status check-nfs-keytab@farm.service --no-pager
```

checker는 drift를 고치지 않는다. KVNO history, repair와 rotation은 [Kerberos/NFS 운영](#)을 따른다.

5. Prometheus/Alertmanager 배포

먼저 server-side render(Helm이 실제로 적용하지 않고 렌더링 결과만 검증하는 것)와 cluster precondition만 검증한다.

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \
ansible-playbook ansible_playbook/deploy_prometheus.yml
```

변경 적용:

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \
ansible-playbook ansible_playbook/deploy_prometheus.yml \
-e prometheus_dry_run=false
```

한 환경만 배포하려면 `prometheus_farm_enabled=false` 또는 `prometheus_lab_enabled=false` 를 사용한다. playbook은 kubeconfig 대상 cluster, node Ready/DiskPressure, 필수 Secret 이름을 확인한 후 Helm을 실행한다.

FARM에서 필요한 Secret:

- `monitoring-grafana-admin`: `admin-user`, `admin-password`
- `cluster-monitor-slack-webhook-farm`: `url`

값은 Git values에 넣지 않는다. Alertmanager는 Slack webhook으로 직접 보내지 않고, 같은 host에서 도는 작은 중계 프로세스(localhost relay)를 거쳐 다른 모듈(예: remote-operations)도 함께 쓰는 사내 공용 알림 API(internal notify API)로 전달한다.

현재 control plane의 역할은 다음처럼 구분한다.

- FARM Prometheus는 FARM 자원 metric과 FARM/LAB의 `cluster-monitor-exporter` 를 수집하고 중앙 alert rule을 평가한다.
- Alertmanager는 하나만 실행하며 FARM/LAB Prometheus가 생성한 alert를 함께 처리한다. LAB Prometheus는 `192.168.2.199:30903` 으로 연결한다.
- LAB Prometheus는 LAB node/GPU metric을 독립 저장한다.
- Grafana는 하나만 실행하며 기본 FARM datasource와 `prometheus-lab` datasource를 함께 사용한다.

6. 배포 후 endpoint 확인

target host에서:

```
systemctl status cluster-monitor-exporter gpu-user-exporter --no-pager
journalctl -u cluster-monitor-exporter -n 100 --no-pager
journalctl -u gpu-user-exporter -n 100 --no-pager

curl -fsS http://127.0.0.1:30074/healthz
curl -fsS http://127.0.0.1:30074/metrics | head
curl -fsS http://127.0.0.1:30072/-/healthy
curl -fsS http://127.0.0.1:30072/metrics | head
curl -fsS http://127.0.0.1:30070/metrics | head
```

:30070의 `node-exporter`는 `kube-prometheus-stack`에서 배포하므로 앞의 두 custom exporter처럼 host systemd unit을 확인하지 않는다. endpoint가 없으면 그 cluster의 DaemonSet, Service와 node scheduling 상태를 확인한다.

public health port는 서버 번호 `N`에 대해 `9000 + N*100 - 11`이다. 예를 들어 FARM8은 `9789/healthz`다. 이 endpoint는 exporter process/collection freshness와 외부 NAT 도달 경로를 확인하는 용도이며, 모든 dependency가 준비됐음을 보장하지는 않는다.

NFS GSS host-only API:

```
curl -fsS http://127.0.0.1:30074/nfs-gss/ready
curl -fsS http://127.0.0.1:30074/nfs-gss/health
curl -fsS -X POST http://127.0.0.1:30074/nfs-gss/probe
curl -fsS -X POST http://127.0.0.1:30074/nfs-gss/recover
```

`probe`와 `recover`는 loopback 요청만 허용하며 worker를 queue한 뒤 즉시 응답한다. `recover`를 반복 호출하기 전에 recovery state와 inhibit reason을 확인한다.

7. 무엇을 어디에서 보는가

관측 대상	대표 metric/alert	구현/설정 링크
CPU, memory, filesystem, disk, network	<code>node_cpu_*</code> , <code>node_memory_*</code> , <code>node_filesystem_*</code> , <code>node_disk_*</code> , <code>node_network_*</code>	FARM values, LAB values
exporter process와 freshness	<code>up</code> , <code>cluster_monitor_exporter_last_collection_timestamp_seconds</code> , <code>ClusterMonitorExporter*</code>	collector, rules
required mount와 latency	<code>cluster_monitor_host_mount_up</code> , <code>..._responsive</code> , <code>..._probe_seconds</code>	mount collector, storage dashboards
Kerberos NFS readiness	<code>cluster_monitor_nfs_gss_ready</code> , <code>ClusterMonitorNFSGSSReadinessFailed</code>	readiness, GSS collector

관측 대상	대표 metric/alert	구현/설정 링크
canary/recovery	<code>cluster_monitor_nfs_gss_canary_*</code> , <code>..._recovery_*</code>	canary, recovery
D-state, lease, hung task	<code>cluster_monitor_host_dstate_*</code> , <code>cluster_monitor_nfs_*</code> , <code>ClusterMonitorNFS*</code>	forensics adapter, forensics scripts
host GPU/Docker/container	<code>cluster_monitor_host_gpu_up</code> , <code>...docker_daemon_up</code> , <code>...container_*</code>	cluster collector
사용자별 GPU	<code>docker_gpu_user_memory_used_bytes</code> , <code>...sm_utilization_percent</code> , <code>...process_count</code>	gpu-user-exporter, GPU dashboard
외부 연결	<code>cluster_monitor_external_connectivity_*</code> , public health	exporter, Apps Script
AD/NAS keytab	checker JSON/exit code	keytab checker

canonical alert rule은 [prometheus-farm-values.yaml](#)이다. exporter 내부 [prometheus/cluster_monitor_alerts.yml](#) 은 metric 이름을 보여 주는 reference이며 실제 FARM release와 값이 다를 수 있다.

8. Alert별 진단 순서

Exporter down 또는 stale

1. Prometheus target에서 connection error와 scrape timestamp를 구분한다.
2. target systemd status/journal을 확인한다.
3. `:30074/healthz` 와 `/metrics` 를 각각 확인한다.
4. process는 살아 있지만 stale이면 마지막 실행 command와 D-state를 확인한다.
5. 변경된 env/config와 binary 배포 시간을 확인한 뒤 exporter만 재배포한다.

Required mount down/slow/unresponsive

1. `findmnt` 로 mount 존재, FQDN source, filesystem과 `sec` 를 확인한다.
2. mount probe와 storage ping/peer diagnosis label을 확인한다.
3. GSS readiness에서 처음 실패한 stage를 확인한다.
4. D-state, lease expiry, hung-task와 local forensic snapshot을 확인한다.
5. mount가 없을 때만 recovery state를 확인한다.
6. mount가 이미 있거나 D-state caller가 있으면 강제 remount를 반복하지 않는다.

Kerberos source는 IP가 아니라 FQDN이어야 한다. 자세한 기준은 [Kerberos/NFS 운영의 mount 절](#)을 따른다.

GSS readiness/canary/recovery

```
systemctl status decs-kerberos-nfs-ready.service --no-pager
systemctl status decs-nfs-gss-canary-probe.timer --no-pager
systemctl status decs-nfs-gss-mount-recovery.timer --no-pager
journalctl -u decs-kerberos-nfs-ready.service -n 100 --no-pager
cat /run/decs-nfs-gss/ready.state
cat /var/lib/decs-nfs-gss/canary.state
cat /var/lib/decs-nfs-gss/recovery.state
```

- keytab/kinit stage: host machine credential 확인
- kvno stage: DNS/FQDN/SPN/KDC 확인
- rpc-gssd stage: service와 잘못된 `-n` override 확인
- canary stage: 전용 export와 user share를 혼동하지 않았는지 확인
- inhibited recovery: D-state 또는 mount timeout 증거를 보존하고 원인을 처리

GPU 사용자 mapping 이상

1. `nvidia-smi` 와 exporter scrape success를 확인한다.
2. ignored process count가 증가했는지 확인한다.
3. PID의 cgroup container ID와 Docker inspect를 비교한다.
4. DB cache refresh success와 cache age/entry를 확인한다.
5. UID DB의 container record와 실제 running container를 비교하고 DB 갱신 절차로 수정한다.

Container SSH/GPU alert

1. container가 running인지 먼저 확인한다.
2. exporter가 start/SSH/NVML recovery를 시도했는지 metric과 journal을 확인한다.
3. container image가 configured regex 대상인지 확인한다.
4. NVML mismatch가 아니라 application/GPU allocation 문제면 자동 symlink repair를 반복하지 않는다.

9. Grafana 점검

- LAB dashboard datasource UID가 `prometheus-lab` 인지 확인한다.
- dashboard query의 `cluster`, `server_id`, `job` label이 scrape config와 일치하는지 확인한다.
- storage dashboard에서 mount probe, responsive, D-state, blocked process, hung-task를 같은 시간축으로 비교한다.
- GPU dashboard에서 DB-known 사용자 metric과 device 전체 metric을 함께 본다.
- 새 server 추가 시 dashboard에 hard-coded server/GPU panel 누락이 없는지 확인한다.

dashboard 원본:

- [FARM storage latency](#)

- LAB storage latency
- GPU usage
- Network traffic

10. 테스트

```
cd /home/jy/server_manage/monitoring

(cd prometheus/exporters/cluster-monitor-exporter && go test ./...)
(cd prometheus/exporters/gpu-user-exporter && go test ./...)
bash nfs-forensics/tests/test_watch.sh
bash prometheus/exporters/cluster-monitor-exporter/tests/test_nfs_gss_ready.sh
bash prometheus/exporters/cluster-monitor-exporter/tests/test_nfs_gss_mount_recovery.sh
python3 prometheus/config/tests/test_slack_notify_relay.py
```

변경 위험에 맞게 최소한 다음을 함께 검증한다.

- metric 이름/type/help와 0/failure path
- recovery가 healthy mount를 건드리지 않는지
- D-state에서 mount attempt가 차단되지만 existing mount는 healthy로 처리되는지
- Alertmanager relay가 secret을 출력하지 않고 internal API payload로 변환하는지
- FARM/LAB dashboard datasource가 바뀌지 않았는지

11. 새 서버 추가 체크리스트

1. `ansible_playbook/inventory.ini` 에 `farmN` / `labN` 형식으로 추가
2. `group_vars/exporters.yml` 의 mount source, SPN과 환경 기본값 확인
3. host share target와 public `N89` port 계산 확인
4. GSS readiness/forensics/exporter 배포
5. Prometheus `:30070` , `:30072` , `:30074` scrape target 추가
6. 방화벽/NAT public health 확인
7. Grafana dashboard server/GPU panel 확인
8. alert rule label과 Alertmanager route 확인
9. endpoint와 실제 alert delivery 검증

12. 운영 문서에 계속 추가할 내용

설계와 운영을 분리한 뒤 다음 정보를 운영 문서에 누적하면 좋다.

- 서비스별 owner, endpoint, systemd/Kubernetes resource와 dependency
- metric별 정상 범위, alert **for** 와 runbook link
- 최근 배포 version/commit과 마지막 검증 시각
- FARM/LAB retention, PV 용량과 capacity threshold
- alert silence 승인/만료 기준과 false-positive 기록
- 자동 복구별 시도 횟수, inhibit 해제와 rollback 절차
- incident snapshot 보존 기간, 접근 권한과 개인정보 취급
- 새 server/metric/dashboard 추가 checklist

설계 문서에는 구성 요소의 관계와 안전 경계를 유지하고, host별 현재 값과 일회성 incident 결과는 canonical config/runbook 또는 별도 evidence에 기록한다.

remote-operations 개요

원본: md/system/remote-operations/index.md, md/system/remote-operations/design.md, md/system/remote-operations/operations.md, md/system/remote-operations/config.md

`remote-operations` 는 관리 서버에서 FARM/LAB GPU 서버에 Wake-on-LAN 패킷을 보내고, 부팅된 서버의 SSH·공유 스토리지·GPU를 확인한 뒤 기존 사용자 컨테이너를 시작한다. 부팅 신호 전송부터 컨테이너 내부 SSH와 GPU 확인까지를 하나의 부팅 작업으로 묶어, 서버가 사용자 작업을 받을 수 있는 상태인지 확인하는 것이 목표다.

NIC·네트워크, Ansible 원격 접근, 서버의 공유 스토리지 mount와 GPU driver, 사용자 컨테이너는 미리 준비된 상태를 입력으로 사용한다. 이 모듈은 부팅 시 그 상태를 확인하고 제한된 복구를 수행하며, 해결되지 않은 실패를 로그와 알림으로 남긴다.

문서 구성

문서	핵심 내용
현재 페이지	remote-operations의 목표와 실행 전제
설계	실행 구조, 대상 서버 지정 방식, 부팅 신호 전송·서버 준비 상태 확인·컨테이너 기동 후 점검과 systemd 설계
운영	수동 실행, 모의 실행, 배포·점검과 장애 대응 절차
설정	대상 서버, 네트워크, 제한 시간, 공유 스토리지, 로그와 알림 설정 준비

remote-operations 설계

개요 · 운영 · 설정

GitHub 코드 링크: `admin_infra_server` 는 비공개 저장소다. 코드 링크를 누르면 GitHub 로그인 화면을 거쳐 원래 파일과 line으로 이동한다. 조직 저장소에 접근 권한이 있는 계정으로 로그인해야 한다.

1. 개요

`remote-operations` 는 관리 서버에서 FARM/LAB GPU 서버를 원격으로 깨운 뒤, 각 서버가 운영 가능한 상태인지 확인하고 기존 사용자 컨테이너를 시작하는 일련의 부팅 작업을 제공한다.

Wake-on-LAN 패킷을 전송한 것만으로는 서버를 사용할 수 있다고 판단할 수 없다. OS가 부팅되어 SSH가 열리고, 공유 스토리지와 NVIDIA driver가 준비된 뒤, 기존 컨테이너 안의 SSH와 GPU까지 동작해야 사용자 작업을 재개할 수 있다. 이 모듈은 이 과정을 하나의 제한 시간 안에서 순서대로 수행하고, 복구되지 않은 실패를 로그와 알림으로 남긴다.

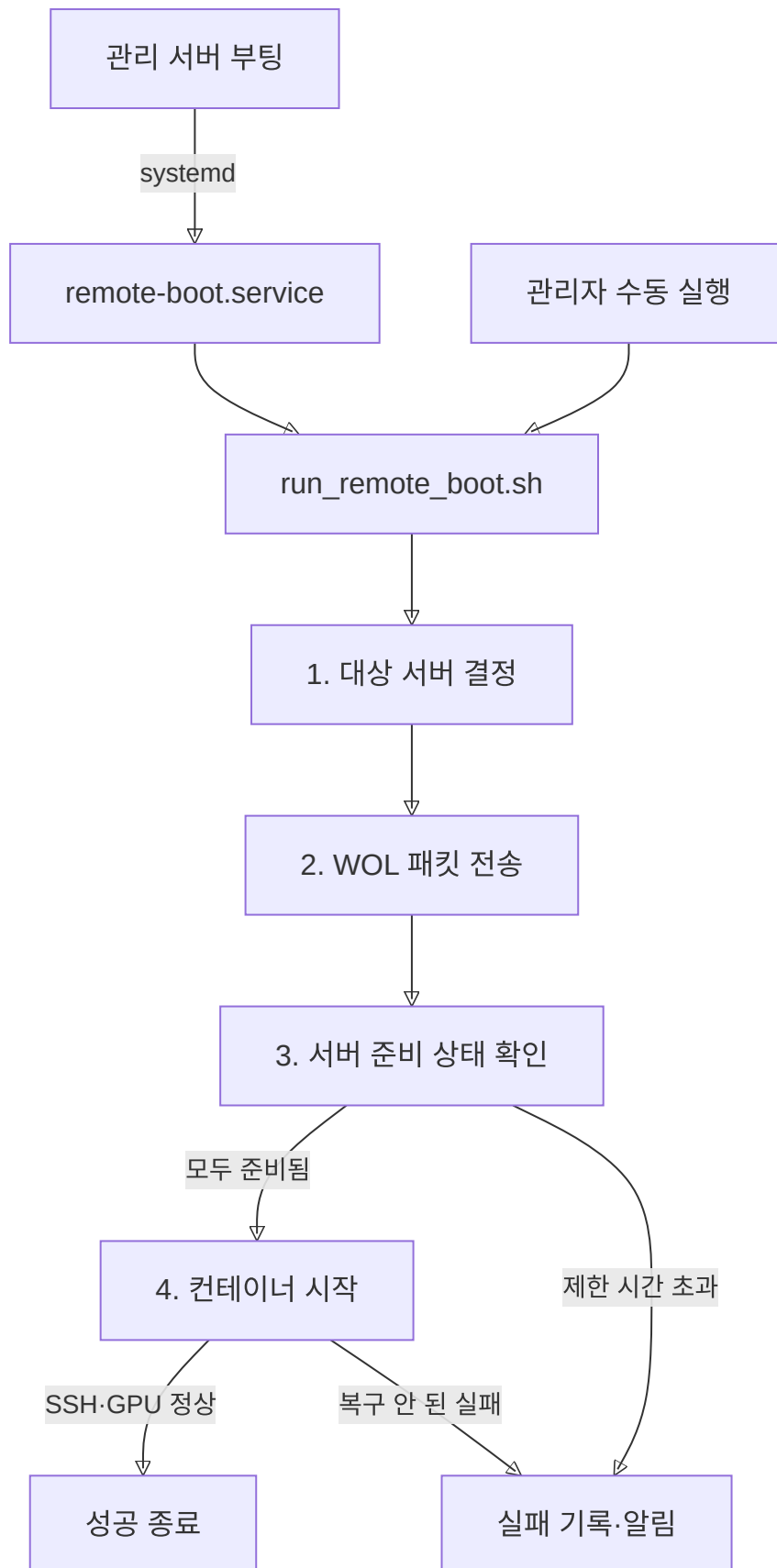
NIC·firmware의 Wake-on-LAN 설정, 네트워크 전달, Ansible inventory, 서버 mount와 GPU driver, 사용자 컨테이너 정의는 미리 준비되어 있어야 한다. `remote-operations` 는 이러한 설정들이 이미 되어 있다는 전제로, 부팅 대상으로 지정된 서버의 상태를 확인하고, 안전하게 제한할 수 있는 복구만 수행한다.

2. 실행 구조

`remote-operations` 의 모든 로직은 하나의 스크립트 `run_remote_boot.sh` 에 들어 있다. 이 스크립트를 실행하는 경로는 두 가지다.

- **자동:** 관리 서버가 부팅되면 systemd가 `remote-boot.service` 를 실행한다. 이 service가 하는 일은 `run_remote_boot.sh` 를 한 번 호출하는 것뿐이다 (service를 만드는 방법은 6절 참고).
- **수동:** 관리자가 같은 스크립트를 직접 실행한다.

`run_remote_boot.sh` 는 실행되면 아래 다섯 단계를 순서대로 진행한다.



각 단계를 담당하는 스크립트는 다음과 같다.

단계	담당 스크립트	수행하는 작업
대상 서버 결정	<code>run_remote_boot.sh</code> , <code>common.sh</code>	설정 또는 명령행 입력을 구체적인 서버 ID 목록으로 변환한다.
WOL 패킷 전송	<code>wake_targets.sh</code>	각 서버의 MAC과 broadcast IP로 Wake-on-LAN 패킷을 보낸다.
서버 준비 상태 확인	<code>wait_for_priority_servers.sh</code> , <code>check_server_boot_health.sh</code>	선택한 모든 서버에서 SSH·공유 스토리지·GPU가 준비될 때까지 다시 확인한다.
컨테이너 시작	<code>restart_all_remote_containers.sh</code>	대상 이미지의 기존 컨테이너를 시작하고 내부 SSH·GPU를 확인한다.
실패 기록·알림	<code>common.sh</code>	단계별 로그를 남기고 같은 내용의 반복 전송을 억제한 알림을 보낸다.

`common.sh`는 특정 단계 전용이 아니라 여러 단계가 공통으로 불러 쓰는 유틸리티 스크립트다. 대상 서버 변환(3절), Ansible 원격 실행과 로그·알림 (4.5절)에서 반복해서 쓰인다.

위 표가 각 단계의 담당 스크립트를 보여준다면, 아래는 그 단계들이 실행되는 시간 흐름과 조건이다. 실행 순서는 다음과 같다.

1. 설정된 사전 대기 시간이 지나면 대상 서버 전체에 WOL 패킷을 전송한다.
2. 서버 준비 상태 확인이 활성화되어 있으면 선택한 모든 서버의 SSH·공유 스토리지·GPU가 준비될 때까지 기다린다.
3. 하나라도 제한 시간 안에 준비되지 않으면 컨테이너 시작으로 넘어가지 않고 실패를 기록한다.
4. 모든 서버가 준비되면 선택한 서버에서 중지되어 있던 대상 컨테이너를 시작한다.
5. 각 대상 컨테이너의 SSH와 GPU를 확인하고, 복구되지 않은 실패가 있으면 전체 실행을 실패로 종료한다.

서버 준비 상태 확인과 컨테이너 시작 단계는 설정으로 각각 끌 수 있다. 기본 실행에서는 두 단계를 모두 사용하여 서버가 준비되기 전에 사용자 컨테이너가 시작되는 것을 막는다.

관련 코드

- `run_remote_boot.sh`: 설정 읽기, 대상 서버 확장, WOL, 서버 준비 상태 확인과 컨테이너 시작 단계를 조합한다.
- `install_remote_boot_service.sh`: 관리 서버 부팅 시 전체 부팅 작업을 호출하는 systemd unit을 만든다.

3. 대상 서버 지정과 원격 접근

실행 대상은 다음과 같이 지정한다.

입력	선택되는 서버
<code>FARM1</code> , <code>LAB10</code>	지정한 서버 한 대
<code>all-farm</code> , <code>all-lab</code>	설정에 등록된 FARM 또는 LAB 서버 전체
<code>all</code>	설정에 등록된 FARM과 LAB 서버 전체

입력	선택되는 서버
명령행 입력 생략	<code>REMOTE_BOOT_TARGETS</code> 에 설정한 기본 대상

`common.sh`는 입력된 서버 ID와 그룹 이름을 실제 서버 ID 목록으로 변환하고 중복을 제거한다. 이후의 부팅 신호 전송과 상태 확인은 이 목록을 기준으로 실행한다.

서버 ID는 두 종류의 연결 정보에 사용된다.

목적	사용하는 정보
WOL 패킷 전송	서버별 MAC address와 FARM/LAB broadcast IP
상태 확인과 원격 명령	서버 ID에서 변환한 Ansible inventory 별칭

부팅 신호를 보내는 네트워크 주소와 OS 부팅 후 접속하는 Ansible 주소를 분리했기 때문에 SSH 주소나 port가 바뀌어도 `FARM1`과 같은 운영 ID는 유지할 수 있다. 반대로 두 정보가 같은 물리 서버를 가리키도록 설정되어 있어야 한다.

최종 서버 ID 목록을 모든 단계가 공통으로 사용하므로 단일 서버, 한 구역 전체와 전체 서버 실행이 같은 절차를 따른다.

관련 코드

- `common.sh`의 대상 서버 목록 변환: 대상 문자열을 정리하고 그룹 이름을 구체적인 서버 목록으로 변환한다.
- `wake_targets.sh`의 주소 선택: 서버 ID에 해당하는 MAC과 구역별 broadcast IP를 선택한다.
- `common.sh`의 Ansible 실행: 서버 ID를 inventory 별칭으로 변환하고 원격 shell을 실행한다.

4. 부팅 단계별 설계

4.1 Wake-on-LAN

`wake_targets.sh`는 선택한 각 서버의 MAC address를 확인하고 FARM/LAB 네트워크에 맞는 broadcast IP로 magic packet을 전송한다. 서버별 상태 확인을 중간에 수행하지 않고 모든 패킷을 먼저 보낸 뒤 서버 준비 상태 확인 단계로 이동한다.

WOL 명령의 성공은 패킷을 보냈다는 의미다. 실제 전원 인가, firmware 부팅과 OS 준비는 확인하지 못하므로 다음 단계에서 먼저 SSH 접속 가능 여부를 확인한다. 서버의 MAC이 없거나 알 수 없는 ID가 들어오면 패킷을 보내지 않고 실패한다.

관련 코드

- `send_magic_packet`: 서버별 WOL 명령을 실행하거나 모의 실행 계획을 기록한다.

4.2 전체 서버 준비 상태 확인

이 단계는 선택한 모든 서버가 준비된 뒤에만 컨테이너 시작을 허용한다. 아직 준비되지 않은 서버만 목록에 남겨 설정된 주기마다 다시 확인하며, 한 번 준비가 확인된 서버는 같은 실행에서 다시 검사하지 않는다.

제한 시간은 서버마다 따로 적용되지 않고 선택한 서버 전체에 한 번 적용된다. 한 서버라도 제한 시간까지 준비되지 않으면 확인 단계가 실패하고 컨테이너 시작 단계는 실행되지 않는다. 이를 통해 일부 서버만 준비된 상태에서 사용자 컨테이너가 먼저 올라오는 것을 막는다.

`wait_for_priority_servers.sh` 라는 파일명은 과거 명칭이 남아 있는 것이며, 현재 구현은 우선순위 서버를 따로 선택하지 않고 전달된 모든 대상 서버를 동일하게 확인한다.

관련 코드

- `wait_for_priority_servers.sh`: 준비가 확인된 서버와 아직 확인 중인 서버를 구분하고 전체 제한 시간을 관리한다.

4.3 서버 상태 확인과 제한된 복구

`check_server_boot_health.sh` 는 각 서버의 상태를 다음 순서대로 확인한다.

확인 항목	확인 방법	실패했을 때 수행하는 복구
SSH	Ansible remote shell로 <code>true</code> 실행	설정된 횟수만큼 연결을 다시 시도한다.
공유 스토리지	지정한 source가 서버별 mount 경로에 연결됐는지 <code>findmnt</code> 로 확인	해당 경로의 mount 또는 <code>mount -a</code> 를 한 번 시도한 뒤 다시 확인한다.
서버 GPU	서버에서 <code>nvidia-smi</code> 실행	NVIDIA module load와 persistence service 재시작을 한 번 시도한 뒤 다시 확인한다.

mount와 GPU 복구는 한 번의 상태 확인 과정에서 각각 한 번만 수행한다. 그래도 실패하면 서버가 아직 준비되지 않은 것으로 처리하며, 전체 제한 시간이 남아 있으면 해당 서버의 상태를 다시 확인한다.

이 복구는 이미 준비된 mount 설정과 NVIDIA driver를 다시 활성화하는 범위다. fstab, NFS 인증이나 driver package를 새로 구성하지 않는다. 따라서 설정 자체가 잘못된 경우에는 재시도로 숨기지 않고 실패로 남겨 해당 운영 절차에서 수정할 수 있게 한다.

관련 코드

- `run_step_with_single_recovery`: mount와 GPU를 확인하고 한 번의 복구 후 재확인한다.
- `check_server_boot_health.sh` 의 실행 순서: 서버 ID를 mount·Ansible 정보로 변환하고 SSH, mount, GPU를 순서대로 확인한다.

4.4 기존 컨테이너 시작과 기동 후 점검

모든 서버의 준비가 확인되면 `restart_all_remote_containers.sh` 가 Docker 목록에서 설정한 이미지 정규식에 맞는 컨테이너만 선택한다. 기본값은 `decs` 또는 `dguailab/decs` 이미지를 대상으로 한다.

선택된 컨테이너 중 중지된 항목을 시작하고, 이미 실행 중인 항목도 포함해 다음 기동 후 점검을 수행한다.

1. 컨테이너가 실행 상태인지 확인한다.
2. 컨테이너 내부 SSH service를 확인하고, 준비되지 않았으면 service 시작을 시도한다.
3. 컨테이너 내부에서 `nvidia-smi` 를 실행한다.
4. GPU 확인이 처음 실패하면 컨테이너를 한 번 재시작하고 제한 시간 동안 다시 확인한다.

중지된 컨테이너의 일괄 시작이 실패하면 Docker service를 한 번 재시작한 뒤 다시 시도한다. 서버별 처리가 실패하면 전체 제한 시간이 남아 있는 동안 해당 서버를 다시 처리한다.

사용자 계정, mount, port와 Docker option은 기존 컨테이너에 이미 들어 있다. 이 모듈은 그 정보를 다시 결정하거나 컨테이너를 재생성하지 않고, 기존 컨테이너를 시작하고 준비 상태만 확인한다. 임시 GPU 컨테이너를 만드는 `create_test_container.sh`와 `delete_test_container.sh`는 독립 점검 도구이며 이 부팅 흐름에서는 호출하지 않는다.

관련 코드

- `restart_all_remote_containers.sh`의 대상 선택: 이미지 정규식과 Docker 목록으로 처리할 컨테이너를 고른다.
- `restart_remote_containers`: 중지된 컨테이너 시작과 SSH·GPU 기동 후 점검을 수행한다.
- 서버별 재시도와 제한 시간: 실패한 서버만 남겨 전체 제한 시간 안에서 재시도한다.

4.5 로그와 실패 알림

각 스크립트는 시각, 실행 구분값, 단계, 서버와 실패 원인을 포함한 로그를 남긴다. 서버 상태 확인과 컨테이너 기동 후 점검 로그는 실행별 파일로도 저장할 수 있어 systemd 전체 로그와 개별 실패 근거를 나누어 확인할 수 있다.

실패 알림은 내부 알림 API를 통해 Slack으로 전달한다. Slack이 설정되지 않았거나 전송이 실패하면 대체 로그에 남긴다. 같은 실패 내용을 기준으로 만든 상태 파일이 이미 있으면 반복 알림을 억제한다. 서버 상태 확인과 컨테이너 단계가 다시 성공하면 각각 관련 상태 파일을 지운다. 실행 로그와 알림 억제 상태를 분리하여 과거 로그를 지우지 않고도 중복 알림 상태만 초기화할 수 있다.

관련 코드

- `common.sh`의 알림 상태 관리: 실패 내용별 상태 파일을 만들고 성공한 단계의 상태를 정리한다.
- `send_slack_message`와 `notify_failure`: 내부 알림 API 전송, 대체 로그 기록과 중복 억제를 처리한다.

5. 설정 모델

실제 실행값은 Git에서 제외된 `config/remote_boot.local.env`에 두고, 저장소에는 변수와 기본 구조만 보여 주는 `config/remote_boot.example.env`를 유지한다. 설정은 다음 영역으로 나뉜다.

설정 영역	결정하는 내용
대상 서버	FARM/LAB 서버 목록과 기본 실행 대상
WOL 네트워크	서버별 MAC과 구역별 broadcast IP
서버 준비 상태 확인	확인 단계 사용 여부, 전체 제한 시간과 재확인 주기
서버 상태 기준	요구하는 NFS source와 서버별 mount 경로
컨테이너	시작 단계 사용 여부, 대상 이미지 정규식과 기동 후 점검 제한 시간
로그·알림	상태 확인 로그 위치, service 로그 보존, 알림 상태와 Slack 전달

스크립트는 설정값을 읽되 MAC, webhook 같은 실제 값은 예제나 source에 넣지 않는다. 설정 파일의 각 항목과 준비 방법은 [설정 문서](#)에서 설명한다.

관련 코드

- `remote_boot.example.env`: 실행 설정의 공개 가능한 구조와 기본값을 정의한다.

6. systemd 연동

설치 스크립트는 관리 서버에 `remote-boot.service` (systemd가 관리하는 서비스 정의 단위, 즉 unit)와 logrotate 설정을 만든다. logrotate는 systemd와 별개인 리눅스 표준 도구로, 로그 파일이 계속 커지지 않도록 주기적으로 나누고 오래된 로그를 정리한다. service는 `network-online.target` 이후 설치를 실행한 사용자 권한으로 `run_remote_boot.sh`를 호출하고, 출력은 service 로그에 추가한다.

이 unit의 `Type`은 `oneshot`이다. `Type`은 systemd가 서비스의 실행 방식을 구분하는 값이고, `oneshot`은 계속 떠 있는 daemon이 아니라 작업을 한 번 수행하고 종료되는 타입이다. 부팅 작업이 완료되면 process가 종료되며 상태를 계속 관찰하지 않는다. 따라서 제한 시간과 재시도는 이번 부팅 작업을 완료하기 위한 범위로 제한되고, 이후의 지속적인 서버 상태 관측과 알림은 `monitoring`에서 수행한다.

설치 스크립트를 다시 실행하면 생성할 unit과 현재 파일을 비교해 필요한 경우에만 갱신하고, `systemctl daemon-reload`와 `enable`를 적용한다. service 로그에는 daily logrotate와 보존 개수가 함께 설정된다.

관련 코드

- `install_remote_boot_service.sh`: oneshot unit, logrotate와 enable 동작을 구성한다.

7. 디렉터리 지도

경로	역할
<code>script/run_remote_boot.sh</code>	전체 부팅 작업의 시작점
<code>script/wake_targets.sh</code>	대상 서버별 Wake-on-LAN 패킷 전송
<code>script/wait_for_priority_servers.sh</code>	선택한 모든 서버의 준비 상태 확인
<code>script/check_server_boot_health.sh</code>	SSH·mount·서버 GPU 확인과 제한된 복구
<code>script/restart_all_remote_containers.sh</code>	기존 대상 컨테이너 시작과 SSH·GPU 기동 후 점검
<code>script/common.sh</code>	대상 서버, Ansible, 로그와 알림 공통 함수
<code>script/install_remote_boot_service.sh</code>	systemd unit과 logrotate 설치
<code>script/reset_remote_boot_alert_state.sh</code>	중복 알림 억제 상태 초기화
<code>script/create_test_container.sh</code> , <code>script/delete_test_container.sh</code>	부팅 흐름과 분리된 임시 GPU 컨테이너 점검 도구
<code>config/remote_boot.example.env</code>	공개 가능한 설정 구조와 기본값
<code>config/remote_boot.local.env</code>	실제 환경값을 두는 Git 제외 설정 파일
<code>test/dry_run_remote_boot.sh</code>	WOL, 서버 상태, 컨테이너와 전체 흐름 모의 실행

경로	역할
test/integration_smoke_test.sh	Ansible·Docker·GPU 통합 확인
test/test_slack_notification.sh	내부 알림 API와 Slack 알림 경로 확인

remote-operations 운영

개요 · 설계 · 설정

1. 개요

이 문서의 목표는 `remote-operations`를 관리 서버에 배포하고, 변경 내용을 실제 서버에 적용하기 전에 검증하며, WOL·서버 상태 확인·컨테이너 시작 단계를 목적에 맞게 실행하고 실패 원인을 확인하는 절차를 제공하는 것이다.

운영은 다음 순서를 기준으로 한다.

- 로컬 설정과 Ansible 원격 접근을 준비한다.
- 대상 서버와 실행 계획을 모의 실행으로 확인한다.
- WOL, 서버 상태 확인과 컨테이너 단계를 개별 서버에서 검증한다.
- 전체 흐름을 수동 실행해 로그와 알림을 확인한다.
- systemd unit을 설치하거나 갱신해 관리 서버 부팅에 연결한다.

모든 명령은 다음 directory에서 실행한다.

```
cd /home/jy/server_manage/remote-operations
```

2. 목적별 실행 위치

운영 목적	사용할 명령 또는 파일
사용 가능한 대상 서버를 확인한다.	<code>./script/wake_targets.sh --list-targets</code>
WOL 패킷만 보낸다.	<code>./script/wake_targets.sh TARGET...</code>
한 서버의 SSH·mount·GPU를 확인한다.	<code>./script/check_server_boot_health.sh --server-id SERVER_ID</code>
기존 대상 컨테이너를 시작하고 SSH·GPU를 확인한다.	<code>./script/restart_all_remote_containers.sh SERVER_ID...</code>
WOL부터 컨테이너 기동 후 점검까지 전체 흐름을 실행한다.	<code>./script/run_remote_boot.sh TARGET...</code>
관리 서버 부팅에 전체 흐름을 연결한다.	<code>./script/install_remote_boot_service.sh</code>

운영 목적	사용할 명령 또는 파일
Slack 알림 전달을 확인한다.	<code>./test/test_slack_notification.sh</code>
저장된 중복 알림 억제 상태를 초기화한다.	<code>./script/reset_remote_boot_alert_state.sh</code>

3. 실행 전 준비

실제 서버에 접근하기 전에 다음 조건을 확인한다.

- `config/remote_boot.local.env` 가 준비되어 있다.
- 대상 서버의 MAC과 broadcast IP가 실제 네트워크와 일치한다.
- 각 서버의 Wake-on-LAN이 firmware와 NIC에서 활성화되어 있다.
- 설정한 Ansible inventory로 대상 서버에 SSH 접속할 수 있다.
- mount와 GPU 복구에 필요한 원격 명령을 `sudo -n` 으로 실행할 수 있다.
- 서버의 NFS mount와 NVIDIA driver가 이미 구성되어 있다.
- 컨테이너 시작 대상 이미지 정규식이 실제 사용자 컨테이너와 일치한다.

설정 파일을 처음 만드는 절차와 변수 의미는 [설정 문서](#)를 따른다.

4. 변경 전 검증

4.1 대상 서버 확인

```
./script/wake_targets.sh --list-targets
./script/run_remote_boot.sh --list-targets
```

로컬 설정의 FARM/LAB 목록과 `all-farm`, `all-lab`, `all` 확장 결과를 확인한다. 새 서버를 추가했다면 이 목록에 먼저 나타나야 한다.

4.2 모의 실행

`dry_run_remote_boot.sh` 의 첫 번째 인자는 어느 단계를 모의 실행할지 고르는 모드다. 각각 [설계 문서](#) 2절의 단계에 대응한다: `wake` 는 WOL 패킷 전송, `health` 는 서버 준비 상태 확인, `containers` 는 컨테이너 시작, `full` 은 전체 부팅 흐름이다.

```
./test/dry_run_remote_boot.sh wake FARM1 LAB1
./test/dry_run_remote_boot.sh health FARM1
./test/dry_run_remote_boot.sh containers FARM1
./test/dry_run_remote_boot.sh full FARM1 LAB1
```

`wake`, `health`, `full` 모의 실행은 WOL 패킷, 대기와 상태 변경 명령을 실행하지 않고 계획을 출력한다. `containers` 모의 실행은 실제 컨테이너를 시작하거나 재시작하지 않지만 현재 Docker 목록을 읽어 처리 대상과 기동 후 점검 계획을 만든다. 따라서 컨테이너 모의 실행에도 Ansible inventory와 조회 목적의 원격 연결이 필요하다.

출력에서 다음을 확인한다.

- 입력한 그룹 이름이 의도한 구체적 서버 ID로 확장되는가?
- 각 ID의 MAC, broadcast IP와 Ansible 서버 별칭이 같은 장비를 가리키는가?
- 구역별 요구 mount source와 서버별 mount 경로가 올바른가?
- 대상 이미지 정규식이 시작해야 할 컨테이너만 선택하는가?
- 제한 시간과 재확인 주기가 선택한 서버 수와 실제 부팅 시간을 감당하는가?

4.3 실제 연결 확인

```
./test/integration_smoke_test.sh
```

이 명령은 선택된 서버에 Ansible ping, Docker daemon과 서버 GPU 확인을 실제로 수행한 뒤 서버 준비 상태를 확인한다. 이 과정은 mount와 GPU가 실패하면 제한된 복구도 수행하므로 단순 조회 명령이 아니다.

`--full-flow`를 추가하면 WOL부터 컨테이너 시작까지 실제 전체 흐름을 실행한다. 변경 검증의 첫 단계에서는 사용하지 않고, 개별 단계가 통과한 뒤 명시적으로 실행한다.

```
./test/integration_smoke_test.sh --full-flow
```

4.4 Slack 알림 확인

```
./test/test_slack_notification.sh --server-id FARM1
```

이 명령은 내부 알림 API를 통해 실제 Slack message를 전송한다. 확인 전에 `REMOTE_BOOT_SLACK_ENABLED`와 webhook 설정을 확인하고 운영 채널에 시험 메시지가 전송된다는 점을 공유한다.

5. 수동 실행

5.1 WOL만 실행

```
./script/wake_targets.sh FARM1
./script/wake_targets.sh all-farm
./script/wake_targets.sh FARM1 LAB1
```

명령 성공은 magic packet을 전송했다는 의미이며 서버 부팅 완료를 보장하지 않는다. 이후 준비 상태 확인으로 실제 SSH·mount·GPU를 확인한다.

5.2 한 서버의 준비 상태 확인

```
./script/check_server_boot_health.sh --server-id FARM1
```

SSH, 요구 mount와 서버의 `nvidia-smi` 를 순서대로 확인한다. mount 실패 시 `mount` 또는 `mount -a`, GPU 실패 시 `module load`와 `persistence service` 재시작을 시도하므로 실제 서버 상태가 바뀔 수 있다.

5.3 기존 컨테이너 시작과 기동 후 점검

```
./script/restart_all_remote_containers.sh FARM1
```

설정된 이미지 정규식에 맞는 기존 컨테이너만 처리한다. 중지된 컨테이너를 시작한 뒤 컨테이너 내부 SSH와 GPU를 확인하며, 실패 시 Docker service 또는 컨테이너 재시작이 발생할 수 있다.

5.4 전체 부팅 작업

```
./script/run_remote_boot.sh FARM1 LAB1  
./script/run_remote_boot.sh --targets "all-farm"
```

명령행에서 지정한 대상은 `REMOTE_BOOT_TARGETS` 기본값을 대체한다. 실제 전체 서버를 대상으로 실행하기 전에 한 서버, 한 구역 순서로 범위를 늘린다.

6. systemd 배포와 갱신

6.1 최초 설치

```
./script/install_remote_boot_service.sh
```

설치 스크립트는 다음 항목을 적용한다.

- `/etc/systemd/system/remote-boot.service`
- `/etc/logrotate.d/remote-boot`
- service 로그 파일과 소유권
- `systemctl daemon-reload` 와 service enable

설치 스크립트를 실행한 사용자와 group이 oneshot service를 실행한다. 해당 계정이 repository, 로컬 설정, Ansible inventory와 SSH key를 읽을 수 있어야 한다.

설치 직후 실행까지 확인하려면 `--start-now` 를 사용한다.

```
./script/install_remote_boot_service.sh --start-now
```

6.2 변경 후 재설치가 필요한 경우

`run_remote_boot.sh`를 비롯한 source 스크립트와 기존 로컬 설정값은 service가 다음 실행 때 같은 경로에서 다시 읽으므로 일반적으로 unit 재설치가 필요하지 않다. 다음 항목을 바꾸면 설치 스크립트를 다시 실행한다.

- 저장소 또는 실행 스크립트 경로
- service 실행 사용자와 group
- 설정 파일 경로
- service 로그 경로나 logrotate 보존 개수
- systemd unit의 dependency나 실행 option

unit 내용을 강제로 다시 쓰려면 `--force`를 사용한다.

```
./script/install_remote_boot_service.sh --force
```

6.3 배포 확인

```
systemctl is-enabled remote-boot.service
systemctl status remote-boot.service
journalctl -u remote-boot.service -b
tail -f /var/log/remote-boot.log
```

`Type=oneshot`이므로 실행을 완료한 뒤 계속 실행 중인 daemon process는 없다. exit status와 이번 boot의 log로 성공 여부를 판단한다.

7. 새 서버 추가

1. 서버의 Wake-on-LAN을 firmware와 NIC에서 활성화하고 실제 MAC을 확인한다.
2. 관리 서버의 Ansible inventory에 해당 서버 별칭을 추가하고 SSH 접속을 검증한다.
3. `REMOTE_BOOT_FARM_TARGETS` 또는 `REMOTE_BOOT_LAB_TARGETS`에 서버 ID를 추가한다.
4. `REMOTE_BOOT_MAC_<SERVER_ID>`에 MAC을 설정한다.
5. 구역별 broadcast IP와 요구 mount source, 서버별 mount 경로를 확인한다.
6. `--list-targets`와 `wake` 모의 실행으로 대상 해석과 WOL 명령을 확인한다.
7. 실제 WOL 후 해당 서버 하나의 준비 상태를 확인한다.
8. 기존 대상 컨테이너가 있는 경우 컨테이너 모의 실행과 기동 후 점검을 수행한다.
9. 마지막으로 전체 부팅 작업을 단일 서버 대상으로 모의 실행하고 실제 실행한다.

새 서버를 대상 목록에 추가하는 것만으로 Ansible 접근, mount, GPU driver나 컨테이너가 구성되지는 않는다. 각 영역의 준비가 완료된 뒤 이 모듈의 검증 절차를 연결한다.

8. 실패 진단

실패 단계 또는 원인	먼저 확인할 내용
wake_failed	MAC 누락, wakeonlan 설치, broadcast IP와 관리 네트워크
ssh_connection_failed	실제 전원 상태, inventory host, SSH key·사용자와 boot 시간
mount_unavailable	findmnt source/target, fstab, NFS·Kerberos 상태와 sudo -n
host_gpu_unavailable	서버의 nvidia-smi, module, persistence service와 driver 로그
docker_start_failed	Docker daemon, 대상 컨테이너 상태와 서버 resource
ssh_unavailable	컨테이너 로그, entrypoint와 컨테이너 내부 sshd
gpu_unavailable	컨테이너 image, NVIDIA runtime, 할당 GPU와 컨테이너 로그
Slack 전송 실패	내부 알림 API 도달성, webhook 설정과 대체 로그

서버 준비 상태 확인이 실패하면 제한 시간을 늘리기 전에 어느 단계에서 대기 중인지 확인한다. mount·GPU 설정 자체가 잘못되었거나 컨테이너 정의가 오래된 경우에는 해당 설정을 수정한 뒤 단일 서버부터 다시 검증한다.

9. 중복 알림 억제 상태 초기화

같은 실패가 이미 전송되어 새 알림이 억제되는 경우, 저장된 메시지를 확인한 뒤 필요한 범위만 초기화한다.

```
./script/reset_remote_boot_alert_state.sh --server-id FARM1
./script/reset_remote_boot_alert_state.sh --stage mount_check
./script/reset_remote_boot_alert_state.sh --reason mount_unavailable
```

여러 조건을 같이 주면 모두 일치하는 상태 파일만 지운다. 전체 상태 파일 삭제는 현재 실패 상태와 알림 재전송 영향을 확인한 뒤 실행한다.

```
./script/reset_remote_boot_alert_state.sh --all
```

이 명령은 실행 로그를 지우지 않고 중복 알림 억제를 상태 파일만 삭제한다.

10. 운영 안전 수칙

- 실제 실행 전에 같은 대상 서버로 모의 실행을 확인한다.

- `all` 실행보다 단일 서버와 구역 단위 검증을 먼저 수행한다.
- 로컬 설정의 MAC, SSH 경로와 webhook을 source나 로그에 복사하지 않는다.
- mount와 GPU 복구가 상태를 변경한다는 점을 고려해 maintenance 범위를 정한다.
- 실제 사용자 컨테이너를 임시 시험용 컨테이너로 대체하거나 재생성하지 않는다.
- 지속적으로 반복되는 장애는 부팅 재시도로 숨기지 않고 해당 모듈의 운영 절차에서 원인을 수정한다.

remote-operations 설정

개요 · 설계 · 운영

1. 개요

이 문서는 `remote-operations` 가 부팅 신호 전송, 서버 준비 상태 확인, 컨테이너 기동 후 점검과 알림에 사용하는 설정을 준비하는 방법을 설명한다. 실제 환경값은 Git에서 제외된 `config/remote_boot.local.env` 에 두고, 저장소의 `config/remote_boot.example.env` 에는 변수 구조와 공개 가능한 기본값만 유지한다.

로컬 설정 파일을 만든다.

```
cd /home/jy/server_manage/remote-operations
cp config/remote_boot.example.env config/remote_boot.local.env
chmod 600 config/remote_boot.local.env
```

실제 MAC, 개인 경로와 Slack webhook을 example 파일이나 Git commit에 넣지 않는다.

2. 대상 서버 설정

```
REMOTE_BOOT_FARM_TARGETS="FARM1 FARM2"
REMOTE_BOOT_LAB_TARGETS="LAB1 LAB2 LAB10"
REMOTE_BOOT_TARGETS="all"
```

변수	의미
<code>REMOTE_BOOT_FARM_TARGETS</code>	<code>all-farm</code> 이 나타내는 FARM 서버 ID 목록
<code>REMOTE_BOOT_LAB_TARGETS</code>	<code>all-lab</code> 이 나타내는 LAB 서버 ID 목록
<code>REMOTE_BOOT_TARGETS</code>	명령행에서 대상을 지정하지 않았을 때 사용할 기본 대상

서버 ID는 `FARM<number>` 또는 `LAB<number>` 형식을 사용한다. 목록에 추가된 ID는 WOL MAC과 Ansible inventory의 서버가 같은 물리 서버를 가리켜야 한다.

3. WOL 네트워크 설정

```
REMOTE_BOOT_FARM_BROADCAST_IP="192.168.2.255"
REMOTE_BOOT_LAB_BROADCAST_IP="192.168.1.255"

REMOTE_BOOT_MAC_FARM1="<FARM1 MAC>"
REMOTE_BOOT_MAC_LAB1="<LAB1 MAC>"
```

구역별 broadcast IP는 관리 서버에서 해당 네트워크로 WOL 패킷을 보낼 수 있는 주소여야 한다.

`REMOTE_BOOT_MAC_<SERVER_ID>` 에는 서버의 WOL 대상 NIC MAC을 넣는다. MAC은 장비 관리 기준 문서나 실제 서버에서 확인하며 임의로 추정하지 않는다.

설정 파일만으로 Wake-on-LAN이 활성화되지는 않는다. 서버 firmware, NIC와 네트워크의 broadcast 전달 설정은 별도로 준비되어 있어야 한다.

4. Ansible 원격 접근

`remote-operations` 는 서버 ID를 소문자 Ansible 별칭으로 바꾸어 사용한다. 예를 들어 `FARM1` 은 inventory의 `farm1` , `LAB10` 은 `lab10` 에 연결된다.

기본 Ansible 설정을 사용하지 않을 경우 로컬 설정에 개인 inventory를 지정한다.

```
REMOTE_BOOT_ANSIBLE_INVENTORY="/home/<관리자계정>/ansible/inventory.ini"
```

inventory 예시는 다음과 같다.

```
[farm]
farm1 ansible_host=<FARM1 address> ansible_user=<관리자계정>

[lab]
lab10 ansible_host=<LAB10 address> ansible_user=<관리자계정>
```

inventory는 접속 주소와 사용자를 정할 뿐 SSH 권한을 부여하지 않는다. 관리 서버의 해당 service 사용자 SSH key가 원격 계정의 `authorized_keys` 에 등록되어 있어야 한다. mount와 GPU 복구에는 대상 서버에서 password 입력 없이 실행할 수 있는 `sudo -n` 권한도 필요하다.

다음 명령으로 각 alias를 먼저 확인한다.

```
ansible farm1 -i /home/<관리자계정>/ansible/inventory.ini -m ping
ansible lab10 -i /home/<관리자계정>/ansible/inventory.ini -m ping
```

5. 서버 준비 상태 확인과 제한 시간

```
REMOTE_BOOT_PRE_DELAY_SECONDS=0
REMOTE_BOOT_ENABLE_GATE=true
REMOTE_BOOT_GATE_TIMEOUT_SECONDS=360
REMOTE_BOOT_GATE_POLL_SECONDS=20
```

변수	의미
REMOTE_BOOT_PRE_DELAY_SECONDS	관리 서버의 네트워크 준비 후 WOL을 보내기 전 대기 시간
REMOTE_BOOT_ENABLE_GATE	컨테이너 시작 전에 모든 서버의 준비 상태를 확인할지 결정
REMOTE_BOOT_GATE_TIMEOUT_SECONDS	선택된 서버 전체의 준비 상태를 확인할 제한 시간
REMOTE_BOOT_GATE_POLL_SECONDS	아직 준비되지 않은 서버를 다시 확인할 간격

제한 시간은 서버마다 새로 시작되지 않는다. 동시에 선택한 서버 수와 가장 느린 실제 부팅 시간을 기준으로 정한다. 원인을 확인하지 않은 채 제한 시간만 계속 늘리지 않는다.

6. 서버 상태 확인 기준

```
REMOTE_BOOT_FARM_REQUIRED_MOUNT="<FARM NFS source>"
REMOTE_BOOT_LAB_REQUIRED_MOUNT="<LAB NFS source>"
REMOTE_BOOT_HOST_SHARE_MOUNT_TEMPLATE="/home/tako%s/share"
```

REMOTE_BOOT_*_REQUIRED_MOUNT 는 findmnt 에서 일치해야 하는 source다. REMOTE_BOOT_HOST_SHARE_MOUNT_TEMPLATE 의 %s 는 서버 번호로 바뀐다. 예를 들어 FARM6 은 /home/tako6/share 를 검사한다.

이 값은 기존 서버 mount 설정을 확인하기 위한 기준이다. NFS export, fstab이나 Kerberos 인증을 생성하지 않는다. 실제 findmnt -rn -T <path> -o SOURCE,TARGET 출력과 동일한 source를 사용한다.

7. 컨테이너 시작과 기동 후 점검

```
REMOTE_BOOT_ENABLE_CONTAINER_RESTART=true
REMOTE_BOOT_CONTAINER_RESTART_TIMEOUT_SECONDS=600
REMOTE_BOOT_CONTAINER_RESTART_POLL_SECONDS=20
REMOTE_BOOT_CONTAINER_POST_RESTART_CHECK_TIMEOUT_SECONDS=60
REMOTE_BOOT_CONTAINER_POST_RESTART_CHECK_POLL_SECONDS=5
```

변수	의미
REMOTE_BOOT_ENABLE_CONTAINER_RESTART	서버 준비 상태 확인 후 기존 대상 컨테이너를 시작할지 결정
REMOTE_BOOT_CONTAINER_RESTART_TIMEOUT_SECONDS	선택한 서버 전체의 컨테이너 처리 제한 시간
REMOTE_BOOT_CONTAINER_RESTART_POLL_SECONDS	실패한 서버를 다시 처리할 간격
REMOTE_BOOT_CONTAINER_POST_RESTART_CHECK_TIMEOUT_SECONDS	컨테이너별 SSH와 GPU 확인 제한 시간
REMOTE_BOOT_CONTAINER_POST_RESTART_CHECK_POLL_SECONDS	컨테이너 기동 후 점검을 다시 시도할 간격

처리 대상 이미지는 REMOTE_BOOT_CONTAINER_TARGET_IMAGE_REGEX 로 제한할 수 있다. 설정하지 않으면 decs 와 dguai Lab/decs repository를 대상으로 한다.

```
REMOTE_BOOT_CONTAINER_TARGET_IMAGE_REGEX='^(decs|dguai Lab/decs)(:|$)'
```

정규식을 바꿀 때는 컨테이너 모의 실행에서 선택·제외되는 이미지를 반드시 확인한다.

8. 로그와 알림

```
REMOTE_BOOT_ENABLE_HEALTH_LOGGING=true
REMOTE_BOOT_HEALTH_LOG_DIR="./logs/health"
REMOTE_BOOT_ALERT_STUB_LOG_FILE="./logs/alerts/remote_boot_alert_stub.log"
REMOTE_BOOT_ALERT_STATE_DIR="./state/alerts"
REMOTE_BOOT_LOG_FILE="/var/log/remote-boot.log"
REMOTE_BOOT_LOG_ROTATE_COUNT=14
```

변수	의미
REMOTE_BOOT_HEALTH_LOG_DIR	서버 상태 확인과 컨테이너 기동 후 점검의 실행별 로그 위치
REMOTE_BOOT_ALERT_STUB_LOG_FILE	Slack을 사용할 수 없을 때 실패를 남길 대체 로그
REMOTE_BOOT_ALERT_STATE_DIR	같은 실패의 반복 전송을 억제하는 상태 파일 위치

변수	의미
REMOTE_BOOT_LOG_FILE	systemd service의 전체 stdout-stderr 로그
REMOTE_BOOT_LOG_ROTATE_COUNT	daily logrotate 보존 개수

Slack 알림을 사용하려면 다음 값을 설정한다.

```
REMOTE_BOOT_SLACK_ENABLED=true
REMOTE_BOOT_SLACK_WEBHOOK_URL_FARM="<Slack webhook URL>"
REMOTE_BOOT_SLACK_MESSAGE_PREFIX="[remote_boot]"
```

LAB/FARM 알림은 현재 하나의 FARM webhook을 사용하며, 서버 ID는 메시지에 반영된다. 최종 Slack 전송은 source에 고정된 내부 알림 API를 통한다. webhook은 secret이므로 terminal 출력, 예제 파일이나 Git commit에 남기지 않는다.

설정 후 실제 message를 확인한다.

```
./test/test_slack_notification.sh --server-id FARM1
```

9. 독립 시험용 컨테이너 설정

REMOTE_BOOT_TEST_* 변수는 create_test_container.sh와 delete_test_container.sh를 수동으로 사용할 때만 적용된다. WOL → 서버 준비 상태 확인 → 기존 컨테이너 시작으로 이어지는 기본 부팅 흐름에서는 읽지 않는다.

독립 시험 도구는 Docker image, UID/GID, memory, runtime과 mount를 실제로 사용할 수 있으므로 운영 부팅 검증과 구분해 별도 시험 계정과 resource로 실행한다.

10. 첫 설정 검증 순서

1. 로컬 설정 파일의 권한이 0600 인지 확인한다.
2. --list-targets 로 FARM/LAB 목록을 확인한다.
3. 각 대상 서버의 Ansible ping을 확인한다.
4. WOL 모의 실행에서 MAC과 broadcast IP를 확인한다.
5. 서버 상태 모의 실행에서 mount source와 경로를 확인한다.
6. 컨테이너 모의 실행에서 대상 이미지와 기동 후 점검 계획을 확인한다.
7. Slack을 사용하는 경우 시험 메시지를 전송한다.
8. 단일 서버에서 실제 WOL, 서버 상태 확인과 컨테이너 기동 후 점검을 순서대로 실행한다.
9. 마지막으로 systemd 설치 스크립트를 실행한다.

명령과 실제 상태 변경 범위는 운영 문서를 따른다.

Kerberos/NFS 개요

원본: [md/system/kerberos-nfs/index.md](#), [md/system/kerberos-nfs/design.md](#), [md/system/kerberos-nfs/operations.md](#), [md/system/kerberos-nfs/debugging/index.md](#), [md/system/kerberos-nfs/debugging/lab9-why-sec-krb5.md](#), [md/system/kerberos-nfs/debugging/nfs-v41-session-slot-stuck.md](#), [md/system/kerberos-nfs/debugging/nfs-v41-session-slot-stuck-remediation.md](#)

kerberos-nfs는 FARM/LAB 서버에서 Kerberos로 사용자를 인증하고 NFS 공유 스토리지의 파일 권한을 동일한 UID/GID 기준으로 적용하는 구성을 설명한다.

이 영역을 이해하려면 [설계](#)에서 Kerberos의 주요 개념, NFS 인증 흐름과 FARM/LAB 설정을 확인한다. 실제 설정 변경과 장애 대응은 [운영](#), 과거 장애의 재현 조건과 분석 결과는 [디버깅 로그](#)에서 확인한다.

문서 구성

문서	내용
현재 페이지	Kerberos/NFS 영역의 목적과 문서 구성
설계	Kerberos 핵심 개념, NFS 인증 흐름, FARM/LAB 적용 설정과 선택 이유
운영	계정과 credential 관리, mount 점검, KVNO·NFS 장애 대응
디버깅 로그	재현된 장애의 증상, 실험 조건, 증거와 복구 결과

Kerberos/NFS 설계

[개요](#) · [운영](#) · [디버깅 로그](#)

1. 개요

Kerberos는 비밀번호를 매번 전달하는 대신 중앙 서버가 발급한 암호화된 티켓을 주고받아 신원을 증명하는 인증 시스템이다. **Kerberos/NFS** 구성의 목표는 사용자가 자신의 Kerberos identity로 NFS service에 인증하고, AD에 기록된 UID/GID와 스토리지의 파일 권한에 따라 공유 경로를 사용하는 것이다. Kerberos는 사용자와 service의 identity를 확인하고, NFS는 인증 결과와 numeric UID/GID, mode, ACL을 함께 사용해 파일 접근 권한을 결정한다.

이 문서는 다음 내용을 설명한다.

- Kerberos 인증에 사용되는 principal, ticket, keytab과 ccache
- Kerberos ticket이 NFS의 RPCSEC_GSS 인증으로 이어지는 과정
- FARM과 LAB에 적용한 realm, NFS service principal, protocol과 mount 설정

- 사용자 credential과 NFS service keytab을 관리하는 기준
- FQDN mount source, KVNO와 `sec=krb5` 를 선택한 이유

환경별 서버를 처음 구성하거나 현재 설정값을 확인할 때는 다음 가이드를 사용한다.

환경	설정 가이드	현재 기준
FARM	FARM Kerberos 설정 가이드	Samba AD, Synology NAS, NFSv4.0, <code>sec=krb5</code>
LAB	LAB Kerberos 설정 가이드	Samba AD, Linux storage, NFSv4.1, <code>sec=krb5</code>

2. 핵심 개념

2.1 Kerberos 개념

용어	의미	현재 구성에서의 사용
Realm	하나의 Kerberos 인증 관리 영역	<code>FARM.DECS.INTERNAL</code> , <code>LAB.DECS.INTERNAL</code>
Principal	Kerberos가 식별하는 사용자 또는 service 이름	<code><username>@REALM</code> , <code>nfs/<fqdn>@REALM</code>
KDC	principal의 비밀키를 관리하고 ticket을 발급하는 서버	FARM/LAB의 Samba AD DC가 KDC 역할을 함께 수행
TGT	사용자가 다른 service ticket을 요청할 때 사용하는 ticket	host의 사용자 ccache에 저장
Service ticket	특정 service에 접속하기 위해 KDC가 발급하는 ticket	NFS 접근 시 <code>nfs/<storage-fqdn>@REALM</code> 용 ticket 발급
SPN	service를 나타내는 principal 이름	NFS server의 FQDN을 포함한 <code>nfs/<fqdn></code> 등록
Keytab	principal의 장기 비밀키를 파일로 저장한 credential	사용자 keytab은 계산 host root가 보관하고 NFS service keytab은 storage가 보관
Ccache	발급된 TGT와 service ticket을 저장하는 credential cache	<code>/run/user/<uid>/krb5cc</code> 를 사용자 process가 사용
KVNO	principal 비밀키가 바뀔 때마다 올라가는 version 번호. 티켓을 만들 때 쓴 키 버전과 그걸 확인하는 keytab의 키 버전이 다르면 인증이 실패함.	AD의 NFS SPN과 storage keytab의 key version 일치 여부 확인
RPCSEC_GSS	Kerberos credential을 NFS RPC 인증에 사용하는 방식	NFS mount의 <code>sec=krb5</code> 가 GSS security context를 사용
RFC2307	AD에 Unix UID/GID를 저장하는 속성 체계	사용자·group을 스토리지와 container에서 같은 숫자로 표현

2.2 NFS 파일권한 개념

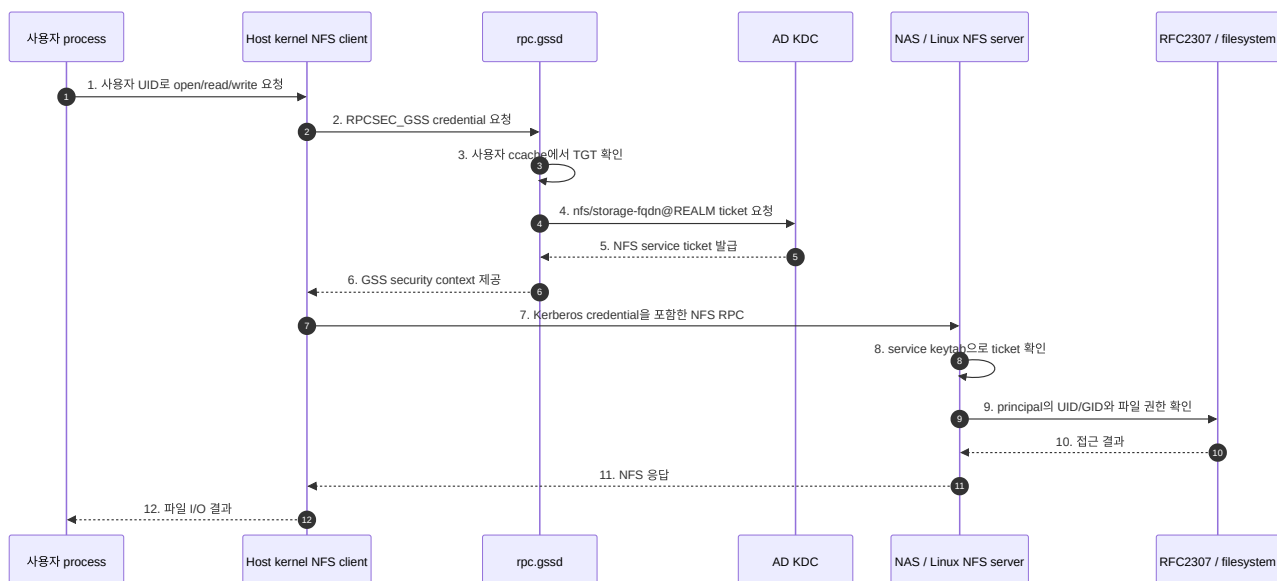
Kerberos가 다루지 않는, NFS 접근 결정에 함께 쓰이는 파일시스템 고유 개념이다.

용어	의미
UID/GID	사용자·그룹을 식별하는 숫자. Linux와 NFS는 이름이 아니라 이 숫자로 파일 권한을 판단한다
Identity	이 문서에서 "누가 누구인가"를 나타내는 정보 전체. Kerberos 쪽에서는 principal, 파일시스템 쪽에서는 UID/GID로 표현되며, 같은 사람을 가리키도록 맞추는 것이 이 설계의 핵심이다
Owner/Group	파일에 지정된 소유 사용자와 소유 그룹(둘 다 UID/GID로 저장됨)
Mode	소유자·그룹·기타 사용자별 읽기/쓰기/실행 권한을 나타내는 Unix 표준 권한 비트(예: <code>0600</code>)
ACL	mode의 소유자/그룹/기타 3단계보다 세밀하게, 특정 사용자·그룹 여러 개에 개별 권한을 지정할 수 있는 확장 권한 체계

Kerberos 인증과 filesystem 권한 판정은 이어지는 두 단계다. Kerberos는 요청자가 어떤 principal인지 확인하고, NFS server는 그 principal에 대응하는 UID/GID와 파일의 owner, group, mode, ACL을 비교해 접근 결과를 결정한다.

3. Kerberos가 NFS 접근에 적용되는 과정

사용자 credential은 계산 host에서 준비되고, 실제 파일 접근은 host kernel NFS client와 `rpc.gssd`가 처리한다.



각 단계는 다음과 같다.

1. 사용자 process가 UID로 파일 open/read/write를 요청한다.
2. Host kernel NFS client가 `rpc.gssd`에 RPCSEC_GSS credential을 요청한다.
3. `rpc.gssd`가 사용자 ccache에서 TGT를 확인한다.
4. `rpc.gssd`가 TGT로 KDC에 `nfs/storage-fqdn@REALM` service ticket을 요청한다.
5. KDC가 NFS service ticket을 발급한다.

6. `rpc.gssd`가 발급받은 ticket으로 GSS security context를 만들어 kernel에 전달한다.
7. Kernel이 이 credential을 포함한 NFS RPC를 NFS server에 보낸다.
8. NFS server가 자신의 service keytab으로 ticket을 검증한다.
9. NFS server가 ticket의 principal을 RFC2307로 UID/GID로 변환해 파일의 owner, group, mode, ACL과 비교한다.
10. 비교 결과(허용/거부)를 NFS server에 반환한다.
11. NFS server가 결과를 kernel에 응답한다.
12. Kernel이 결과를 사용자 process에 반환한다.

이 흐름에서 사용하는 credential은 세 종류다.

Credential	보관 위치	역할
사용자 keytab	계산 host <code>/etc/decs-krb/keytabs/<username>.keytab</code>	사용자 TGT를 새로 발급
사용자 ccache	계산 host <code>/run/user/<uid>/krb5cc</code>	NFS service ticket을 요청하고 사용자 process의 NFS 접근에 사용
NFS service keytab	Synology NAS 또는 Linux storage	NFS server가 받은 service ticket을 확인

사용자 process에는 `KRB5CCNAME=FILE:/run/user/<uid>/krb5cc`를 전달한다. Host의 refresh timer가 사용자 keytab으로 ccache를 갱신하며, container는 ccache와 읽기 전용 `/etc/krb5.conf`를 사용한다.

4. FARM과 LAB에 적용한 설정

FARM과 LAB은 같은 credential 모델과 UID/GID 기준을 사용한다. Storage 구현과 검증된 NFS protocol version에 맞춰 realm, endpoint, service principal과 mount option을 환경별로 관리한다.

4.1 공통 설정 기준

항목	설정 기준
사용자 identity	DB, AD RFC2307, NFS owner와 container에 같은 UID/GID 적용
사용자 keytab	계산 host root 소유, mode <code>0400</code>
사용자 ccache	<code>/run/user/<uid>/krb5cc</code> , 사용자 UID/GID 소유, mode <code>0600</code>
NFS mount source	NFS service principal과 같은 FQDN 사용
NFS 인증	기본 <code>sec=krb5</code>
Client GSS	<code>rpc.gssd</code> 가 호출 UID의 ccache로 GSS context 생성
Storage identity	AD RFC2307 값을 winbind 또는 SSSD/idmap으로 numeric UID/GID에 연결

4.2 환경별 설정값

항목	FARM	LAB
Kerberos realm	FARM.DECS.INTERNAL	LAB.DECS.INTERNAL
DNS domain	farm.decs.internal	lab.decs.internal
AD/KDC	dc1.farm.decs.internal	dc1.lab.decs.internal
Storage 구현	FARM AD에 가입한 Synology NAS	LAB AD에 가입한 Linux NFS server
NFS data 주소	100.100.100.120	100.100.100.100
Mount source	nas.farm.decs.internal:/volume1/share	lab-storage.lab.decs.internal:/294t/dcloud/share
NFS version	vers=4.0	vers=4.1
Security flavor	sec=krb5	sec=krb5
NFS service principal	nfs/nas.farm.decs.internal@FARM.DECS.INTERNAL	nfs/lab-storage.lab.decs.internal@LAB.DECS.INTERNAL
Service keytab	/etc/nfs/krb5.keytab	/etc/krb5.keytab
Server GSS process	Synology svcgssd	Linux rpc.svcgssd 또는 NFS server가 관리하는 service
Identity 연결	Samba winbind RFC2307	Storage는 winbind RFC2307, 일반 client는 SSSD RFC2307

4.3 NFS version 선택

FARM의 Synology NAS는 현재 vers=4.0,sec=krb5,hard,timeo=600,retrans=2 를 운영 기준으로 사용한다. NFSv4.1 과 RPCSEC_GSS 조합에서 확인된 timeout과 access denied 결과를 반영해 NFSv4.0을 선택했다. rsize/wsize 는 1 MiB를 요청하며 실제 연결값은 Synology와의 협상 결과에 따라 128 KiB로 표시될 수 있다.

LAB의 Linux storage는 vers=4.1,sec=krb5 를 사용한다. Linux NFS server와 client의 session 동작을 활용하며, session slot 관련 분석은 NFSv4.1 session slot 고착에서 관리한다.

5. 사용자 credential 관리

5.1 Keytab과 ccache { #keytab-ccache }

구분	Keytab	Ccache
저장 내용	principal의 장기 비밀키	발급된 TGT와 service ticket
용도	새 ticket 발급	ticket 제시와 service ticket 요청
수명	principal key rotation까지 유지	ticket lifetime과 renew lifetime 적용

구분	Keytab	Ccache
저장 위치	host root-only 경로	<code>/run/user/<uid>/krb5cc</code>
사용 주체	host credential refresh service	사용자 process와 <code>rpc.gssd</code>

Keytab은 새 ticket을 계속 발급할 수 있는 장기 credential이다. Host root가 keytab을 관리하고 사용자 runtime에는 수명이 제한된 ccache를 제공해 credential 노출 범위를 줄인다.

5.2 Ccache 갱신

`decs-krb-refresh@<username>.timer` 는 사용자별 oneshot service를 주기적으로 실행한다. 현재 일반적인 갱신 주기는 1시간이다.

유효 ccache 확인

```
-> renew 기간이 충분하면 임시 ccache에서 kinit -R
-> renew 기간이 짧거나 갱신이 실패하면 keytab으로 새 TGT 발급
-> klist로 결과 확인
-> uid:gid, mode 0600 적용
-> /run/user/<uid>/krb5cc로 원자 교체
```

AD group이 변경된 사용자는 keytab으로 새 TGT를 발급해 최신 group 정보를 ccache에 반영한다.

6. NFS service identity

NFS 서비스(NAS / Linux NFS server) 쪽도 Kerberos principal로 자신을 증명해야 한다. 이 절은 그 principal을 구성하는 이름(FQDN/SPN)과 비밀키 버전(KVNO)을 FARM/LAB이 어떻게 관리하는지 설명한다.

6.1 FQDN과 service principal

Kerberos NFS service는 host-based principal을 사용한다(FQDN은 호스트 이름과 도메인을 모두 포함한 완전한 이름을 뜻한다). Mount source의 hostname과 NFS service principal의 hostname을 같은 값으로 유지한다.

```
FARM mount source: nas.farm.decs.internal:/volume1/share
FARM service SPN:  nfs/nas.farm.decs.internal@FARM.DECs.INTERNAL
FARM transport:    addr=100.100.100.120

LAB mount source:  lab-storage.lab.decs.internal:/294t/dcloud/share
LAB service SPN:   nfs/lab-storage.lab.decs.internal@LAB.DECs.INTERNAL
LAB transport:     100.100.100.100
```

FQDN은 KDC가 발급할 NFS service ticket을 결정하고 data 주소는 실제 packet 전송 경로를 결정한다. 이 구분을 통해 service identity를 유지하면서 storage network를 사용한다.

6.2 KVNO와 service keytab

KDC가 발급한 service ticket의 KVNO와 storage keytab에 들어 있는 NFS principal의 KVNO가 일치해야 storage가 ticket을 확인할 수 있다.

FARM은 전용 AD service account `svc-nfs-farm`에 NFS SPN을 등록한다. Synology machine account password lifecycle과 NFS service key lifecycle을 분리해 관리자가 명시적으로 rotation할 때 KVNO를 변경한다. Synology keytab에는 FARM과 기존 AILAB NFS principal이 함께 있으므로 rotation 과정에서 두 acceptor를 모두 유지한다.

LAB은 Linux storage computer account `LAB-STORAGE$`의 NFS SPN과 `/etc/krb5.keytab`을 사용한다. 환경별 checker는 AD의 KVNO와 storage keytab의 KVNO를 비교해 drift를 확인한다.

7. NFS security flavor와 권한

Mount option	제공하는 보호
<code>sec=krb5</code>	사용자와 NFS service 인증
<code>sec=krb5i</code>	인증과 RPC payload 무결성
<code>sec=krb5p</code>	인증, 무결성과 RPC payload 암호화

현재 FARM/LAB 기본값은 `sec=krb5`다. 내부 storage network에서 Kerberos identity를 확인하고 payload wrapping 비용을 줄이는 설정이다. 무결성 또는 암호화가 필요한 공유 경로는 성능을 측정한 뒤 `krb5i` 또는 `krb5p`를 선택할 수 있다.

Kerberos 인증이 완료된 뒤에는 numeric UID/GID, file mode와 ACL이 최종 권한을 결정한다. 다음 값은 한 사용자에게 대해 같은 숫자를 유지한다.

```
AD RFC2307 uidNumber/gidNumber
= NFS storage의 파일 owner/group
= 계산 host가 조회한 UID/GID
= container process의 UID/GID
```

8. 설정 확인 위치

설정	기준 문서 또는 파일
FARM canonical runbook	kerberos-nfs/docs/farm.md
LAB canonical runbook	kerberos-nfs/docs/lab.md
계산 host mount와 readiness 설정	kerberos_nfs_client_recovery.yml
Monitoring 기대값	exporters.yml

Kerberos/NFS 운영

이 문서는 현재 Docker 기반 FARM/LAB 환경의 일상 점검과 장애 대응 절차를 설명한다. 환경별 endpoint와 적용 상태는 반드시 canonical runbook에서 다시 확인한다.

1. 운영 원칙

1. 관측과 변경을 분리한다. 5분 keytab checker는 자동 repair/rotation을 하지 않는다.
2. 사용자 keytab은 host root-only로 두고 container에는 ccache만 전달한다.
3. 컨테이너를 시작하기 전에 refresh timer와 최초 ccache 발급을 완료한다.
4. mount source는 FQDN을 사용하고 IP는 `addr=` transport 값으로만 사용한다.
5. `findmnt`, `klist`, `kvno`, readiness 순서로 원인을 좁힌 뒤 service restart나 remount를 마지막 수단으로 사용한다.
6. UID/GID 불일치는 AD·DB·NFS 숫자를 비교한 뒤 고친다. 먼저 `chown` 하지 않는다.

2. Kerberos container 생성 (추후 자동화 시스템으로 대체될 예정)

대표 CLI 형태는 다음과 같다. 실제 image/version, 사용자 정보와 대상 server는 요청에 맞게 지정한다.

```
cd /home/jy/server_manage/user-lifecycle/script

python3 -B -m uid_manager.cli create-container \
  --server-id FARM8 \
  --name "사용자 이름" \
  --username <username> \
  --group <groupname> \
  --image <image> \
  --version <version> \
  --created-by <operator> \
  --email <email> \
  --phone <phone> \
  --enable-kerberos
```

먼저 `--dry-run` 으로 UID/GID, target host, mount source, container command를 확인하는 것을 권장한다. 기존 사용자 password/key를 의도적으로 바꿀 때만 `--rotate-kerberos-keytab` 을 추가한다. 이 옵션은 AD user password와 KVNO를 바꾸므로 단순 재배포 옵션으로 사용하지 않는다.

생성 transaction은 다음 조건을 모두 통과한 후 Docker/DB를 확정해야 한다.

- AD `uidNumber/gidNumber` 가 선택한 DB UID/GID와 일치한다.

- target host keytab이 `root:root 0400` 이다.
- ccache timer가 enabled/active이고 최초 ccache가 유효하다.
- NAS/storage home의 numeric owner가 기대한 UID/GID다.
- 새로 만든 사용자 credential로 host NFS write/delete가 성공한다.
- Docker에는 ccache directory와 읽기 전용 `krb5.conf` 만 전달된다.

구현은 `create_container.py`와 `Kerberos command builder`에서 확인한다.

3. Host keytab과 ccache 확인

파일 권한

```
sudo stat -c '%U:%G %a %n' \
    /etc/decs-krb/keytabs/<username>.keytab \
    /etc/decs-krb/refresh.d/<username>.env

sudo stat -c '%U:%G %a %n' /run/user/<uid> /run/user/<uid>/krb5cc
```

기대값은 다음과 같다.

```
keytab:      root:root 400
refresh env: root:root 600
ccache dir:  <uid>:<gid> 700
ccache:      <uid>:<gid> 600
```

timer와 ticket

```
systemctl list-timers 'decs-krb-refresh@*.timer'
systemctl status 'decs-krb-refresh@<username>.timer' --no-pager
sudo systemctl start 'decs-krb-refresh@<username>.service'
sudo journalctl -u 'decs-krb-refresh@<username>.service' -n 100 --no-pager

sudo -u '#<uid>' env KRB5CCNAME=FILE:/run/user/<uid>/krb5cc \
    klist -c FILE:/run/user/<uid>/krb5cc
```

컨테이너 생성 때 timer unit을 설치·enable/start하고, 기존 ccache가 유효하지 않으면 oneshot service를 즉시 실행하도록 구현되어 있다. timer만 존재하고 최초 ccache가 없는 상태에서 컨테이너를 먼저 시작하면 Kerberized home 접근이 실패할 수 있다.

AD group을 변경했다면 renew만 기다리지 말고 fresh ticket을 발급한다.

```
sudo rm -f /run/user/<uid>/krb5cc
sudo systemctl start 'decs-krb-refresh@<username>.service'
sudo -u '#<uid>' klist -c FILE:/run/user/<uid>/krb5cc
```

4. 컨테이너 credential 확인

```
docker inspect <container> --format '{{range .Config.Env}}{{println .}}{{end}}' |
  grep -E '^(KRB5CCNAME|DECS_KRB5_PRINCIPAL|DECS_KERBEROS_ENABLED)= '

docker exec --user <username> <container> \
  sh -lc 'klist -c "$KRB5CCNAME" && touch ~/.__krb_test && rm ~/.__krb_test'
```

다음은 잘못된 상태다.

- container mount나 image 안에 *.keytab 이 있음
- KRB5CCNAME 이 host와 공유되지 않은 임시 경로를 가리킴
- container UID가 DB/AD UID와 다름
- Kerberos mode인데 unrestricted sudo와 광범위한 host mount를 함께 허용함

현재 저장소에는 Kubernetes Pod용 credential controller가 구현되어 있지 않다. Pod 운영을 추가한다면 Pod 생성 시 “어느 node에서 누가 keytab을 보관하고 ccache를 갱신할지”부터 구현해야 하며, Docker용 systemd timer 명령을 그대로 복사해 완료로 간주하면 안 된다.

5. NFS mount source 확인

환경마다 source, NFS version과 service principal이 다르다.

FARM의 올바른 형태는 다음과 같다.

```
nas.farm.decs.internal:/volume1/share /home/tako8/share nfs4
defaults,vers=4.0,rsz=1048576,wsz=1048576,sec=krb5,proto=tcp,hard,timeo=600,retrans=2,addr=100.100.100.120,
_netdev,exec,nouser 0 0
```

LAB의 올바른 형태는 다음과 같다. 실제 fstab에는 rpc-gssd 와 decs-kerberos-nfs-ready.service 에 대한 x-systemd.requires/after option도 playbook이 함께 추가한다.

```
lab-storage.lab.decs.internal:/294t/dcloud/share /home/tako9/share nfs4
defaults,vers=4.1,sec=krb5,proto=tcp,hard,_netdev,exec,nouser 0 0
```

핵심은 source가 각 realm의 NFS service principal과 일치하는 FQDN이라는 점이다.

FARM 올바른: `nas.farm.decs.internal:/volume1/share`
FARM 잘못된: `100.100.100.120:/volume1/share` 또는 `192.168.2.30:/volume1/share`
FARM 정상: runtime option의 `addr=100.100.100.120`

LAB 올바른: `lab-storage.lab.decs.internal:/294t/dcloud/share`
LAB 잘못된: `100.100.100.100:/294t/dcloud/share` 또는 `192.168.1.20:/294t/dcloud/share`
LAB 정상: FQDN을 해석한 실제 transport 주소 `100.100.100.100`

IP source를 쓰면 `nfs/nas.farm.decs.internal@FARM.DECS.INTERNAL` service principal 또는 `nfs/lab-storage.lab.decs.internal@LAB.DECS.INTERNAL` 과 이름이 맞지 않는다. `192.168.2.30` 과 `192.168.1.20` 은 관리 SSH 주소이며 운영 NFS data path가 아니다.

점검 명령:

```
MOUNT_TARGET=/home/tako9/share
STORAGE_FQDN=lab-storage.lab.decs.internal

findmnt -T "$MOUNT_TARGET" -o TARGET,SOURCE,FSTYPE,OPTIONS
nfsstat -m | sed -n "\\|${MOUNT_TARGET}|,+4p"
getent hosts "$STORAGE_FQDN"
```

FARM host에서는 값을 `/home/tako<번호>/share` 와 `nas.farm.decs.internal` 로 바꾼다. `findmnt` 의 `vers=4.0` 은 FARM, `vers=4.1` 은 LAB이어야 하고 두 환경 모두 `sec=krb5` 여야 한다.

`fstab` 설정은 [server-state playbook](#)에서 관리한다. 이미 mount된 legacy superblock은 유지보수 창에 `clean unmount/remount`한다. NFS caller가 D-state이면 강제 unmount를 반복하지 말고 포렌식 보존과 host reboot 판단으로 전환한다.

6. Machine credential과 NFS service ticket 확인

계산 호스트의 root mount에는 host machine keytab과 `rpc.gssd` 가 필요하다. realm과 NFS principal을 host 이름에 맞춰 선택한다.

```

short=$(hostname -s | tr '[:lower:]' '[:upper:]')

case "$short" in
    FARM*) KRB_REALM=FARM.DECS.INTERNAL
           KRB_NFS_PRINCIPAL=nfs/nas.farm.decs.internal@FARM.DECS.INTERNAL ;;
    LAB*)  KRB_REALM=LAB.DECS.INTERNAL
           KRB_NFS_PRINCIPAL=nfs/lab-storage.lab.decs.internal@LAB.DECS.INTERNAL ;;
    *)     printf '지원하지 않는 host: %s\n' "$short" >&2; exit 1 ;;
esac
KRB_MACHINE_PRINCIPAL="${short}\${KRB_REALM}"

sudo klist -kte /etc/krb5.keytab
sudo KRB5CCNAME=FILE:/run/decs-host-check.ccache \
    kinit -k -t /etc/krb5.keytab "$KRB_MACHINE_PRINCIPAL"
sudo KRB5CCNAME=FILE:/run/decs-host-check.ccache \
    kvno "$KRB_NFS_PRINCIPAL"
sudo rm -f /run/decs-host-check.ccache

systemctl status rpc-gssd --no-pager
systemctl cat rpc-gssd
pgrep -a rpc.gssd

```

`rpc.gssd`를 `-n`으로 시작하지 않는다. root mount는 host machine credential을 사용해야 하며, `-n`은 UID 0 사용자 credential을 찾게 하여 ticket이 없을 때 mount를 잃게 만들 수 있다.

7. Synology NAS와 Linux storage 운영 차이

FARM의 Synology NAS와 LAB의 Linux storage는 같은 NFS/Kerberos 개념을 사용하지만 설정을 관리하는 방식, service 이름과 keytab lifecycle이 다르다. client에서 확인하는 `findmnt`, `klist`, `kvno` 절차는 비슷해도 server 변경 명령은 서로 바뀌 실행하면 안 된다.

7.1 운영 경계 비교

항목	FARM Synology NAS	LAB Linux storage
관리 접속	<code>jy@192.168.2.30:6954</code>	<code>jy@192.168.1.20:6953</code>
AD join 도구	DSM UI 또는 <code>/usr/syno/sbin/synowin</code>	표준 Samba <code>net ads join</code>
Samba/winbind	Synology SMB package 경로와 <code>pkg-synosamba-*</code> unit	<code>/etc/samba/smb.conf</code> , 표준 <code>winbind.service</code>

항목	FARM Synology NAS	LAB Linux storage
RFC2307 NSS	<code>files winbind syno</code> 순서와 Synology idmap 설정을 함께 확인	<code>files systemd winbind</code> 와 Samba <code>idmap config</code> LAB : <code>backend = ad</code> 확인
NFS export root	<code>/volume1/share</code>	<code>/294t/dcloud/share</code> ; PoC는 별도 <code>test_krb</code> export
production export	storage IP별 <code>root_squash</code> , <code>sec=krb5:krb5i:krb5p</code> ; Synology <code>anonuid/anongid</code> , <code>insecure</code> option 포함	Linux <code>/etc/exports</code> 에서 <code>root_squash</code> , <code>sec=krb5</code> , <code>no_subtree_check</code> 를 명시
NFS service	Synology <code>/usr/sbin/svcsd</code> 와 <code>/usr/sbin/idmapd</code>	<code>nfs-server</code> 와 <code>rpc-svcgssd</code> 또는 배포판의 socket-activated GSS service
NFS keytab	<code>/etc/nfs/krb5.keytab</code> 이 ticket 검증 기준; <code>/etc/krb5.keytab</code> 도 동기화 검사	<code>/etc/krb5.keytab</code> 한 경로
NFS SPN 등록 계정	전용 user <code>svc-nfs-farm</code> ; <code>NAS\$</code> 와 분리	computer account <code>LAB-STORAGE\$</code>
추가 acceptor	같은 keytab의 AILAB <code>nfs/nas.ai lab.dgu@AILAB.DGU</code> 를 반드시 보존	현재 없음
운영 NFS version	v4.0	v4.1
keytab checker	<code>--profile farm</code> , 5분 user timer 운영	<code>--profile lab</code> ; profile은 준비됐지만 PoC 단계에서는 timer를 기본 활성화하지 않음

Synology에서 `getent` , `id` 또는 `wbinfo` 가 DECS UID/GID 대신 `96470xxx` 같은 내부 ID를 반환하면 RFC2307 경로가 정상인 상태가 아니다. 이때 symbolic `chown -R 'FARM\user':'FARM\group'` 을 실행하면 잘못 해석된 숫자가 filesystem에 저장된다. 먼저 DB-AD와 NAS의 numeric UID/GID가 일치하는지 확인하고, ownership 복구도 검증된 숫자로 수행한다.

FARM NAS에는 관리·호환성 목적으로 남은 `192.168.2.0/24 sec=sys,no_root_squash` export가 있을 수 있다. 이것은 production Kerberos data path가 아니며 FARM host나 container가 이 `sec=sys` export를 사용하도록 변경하면 안 된다.

7.2 읽기 전용 점검

FARM Synology NAS에서는 vendor 경로와 service 이름을 사용한다.

```
ssh -p 6954 jy@192.168.2.30

/usr/syno/sbin/synowin -getWorkgroup
SMB=/usr/local/packages/@appstore/SMBService/usr/bin
sudo "$SMB/net" ads testjoin
sudo "$SMB/wbinfo" --online-status

sudo klist -kte /etc/nfs/krb5.keytab
sudo klist -kte /etc/krb5.keytab
sudo exportfs -v | sed -n '/\/volume1\/share/,+8p'
ps -ef | grep -E 'svcgssd|idmapd' | grep -v grep
```

LAB Linux storage에서는 표준 Samba/NFS systemd unit과 keytab을 확인한다.

```
ssh -p 6953 jy@192.168.1.20

sudo net ads testjoin
wbinfo --online-status
getent passwd '<username>'

sudo klist -kte /etc/krb5.keytab \
  | grep -F 'nfs/lab-storage.lab.decs.internal@LAB.DECS.INTERNAL'
systemctl is-active winbind nfs-server
systemctl --no-pager --full status rpc-svcgssd.service 2>/dev/null || true
sudo exportfs -v | sed -n '/\/294t\/dcloud\/share/,+8p'
```

LAB 배포판에서는 `rpc-svcgssd` 가 독립 unit이 아니라 socket 또는 `nfs-server` 의 일부로 관리될 수 있다. unit 이름 하나만으로 실패를 판정하지 말고 process, service keytab과 실제 `kvno` /canary 결과를 함께 본다.

7.3 변경 작업에서 지킬 차이

- Synology 설정을 바꾸기 전에는 DSM/Samba/NFS 설정과 두 keytab을 먼저 backup한다. DSM 또는 SMB package가 관리하는 파일·unit을 일반 Ubuntu와 같다고 가정하지 않는다.
- FARM keytab repair는 AILAB principal과 이전 FARM KVNO를 보존하는 전용 `repair-farm-nas-nfs-keytab.sh` 을 사용한다. `svcgssd` 를 FARM principal 하나로 제한하는 `-p` 나 nameless acceptor `-n` 으로 시작하지 않는다.
- LAB storage는 표준 Samba join, `/etc/krb5.keytab` , `nfs-server` / `rpc-svcgssd` 와 Ansible desired state를 기준으로 한다. FARM repair/rotation script를 LAB에 실행하지 않는다.
- 두 환경 모두 identity/export 변경 후 `exportfs -ra` 와 필요한 NFS/RPC cache flush를 수행할 수 있지만, 먼저 활성 client와 D-state를 확인하고 유지보수 시간에만 실행한다.
- Synology 관리 IP와 LAB storage 관리 IP는 NFS source가 아니다. 변경 후 client `findmnt` 에서 각각 `nas.farm.decs.internal` 과 `lab-storage.lab.decs.internal` 이 유지되는지 확인한다.

환경별 keytab 상태는 같은 checker를 profile만 바꿔 읽기 전용으로 확인한다.

```
cd /home/jy/server_manage/monitoring
health-checks/kerberos-nfs-keytab/script/check-nfs-keytab.sh --profile farm
health-checks/kerberos-nfs-keytab/script/check-nfs-keytab.sh --profile lab
```

server-side 전체 설정 절차는 [FARM canonical runbook](#)과 [LAB canonical runbook](#)을 각각 따른다.

8. FARM NAS service account와 KVNO 운영

현재 정책

- NFS SPN 등록 계정: `svc-nfs-farm`
- `NAS$`: domain membership용 `HOST/*`, `RestrictedKrbHost/*` 만 유지
- 자동 password expiration/rotation: 없음
- keytab drift 관측: 5분마다 읽기 전용
- repair/rotation: 관리자 명시 실행만 허용

과거 `NAS$` machine password의 주기적 변경으로 KVNO와 NAS NFS keytab이 어긋났기 때문에 전용 service account를 만들었다. 운영 문서에는 “KVNO가 주기적으로 바뀌어서 매번 새 계정을 만든다”가 아니라, **한 번 만든 전용 계정으로 자동 machine-account rotation에서 NFS SPN을 분리했다고** 기록한다.

읽기 전용 점검

```
cd /home/jy/server_manage/monitoring
health-checks/kerberos-nfs-keytab/script/check-nfs-keytab.sh --profile farm

systemctl --user status check-nfs-keytab@farm.timer --no-pager
systemctl --user list-timers 'check-nfs-keytab@farm.timer'
```

checker가 확인하는 항목:

- AD `svc-nfs-farm`의 현재 `msDS-KeyVersionNumber`
- NAS `/etc/krb5.keytab`, `/etc/nfs/krb5.keytab`에 현재 KVNO와 NFS principal 존재
- 공유 keytab에 AILAB acceptor가 보존되어 있는지
- 실제 `kvno -k` service-ticket 복호화 성공 여부
- `svcgssd` 실행 여부와 잘못된 `-p` 제한 여부

코드는 공용 checker, profile 값은 [FARM config](#)에서 확인한다.

Drift 수동 복구

```
cd /home/jy/server_manage/kerberos-nfs

./script/keytab/repair-farm-nas-nfs-keytab.sh --check
./script/keytab/repair-farm-nas-nfs-keytab.sh --repair
```

`--repair` 는 새 key를 export·검증하고 기존 AILAB principal과 이전 FARM KVNO를 보존하면서 NAS keytab을 원자 교체한다. 필요하다면 `svcgssd` 를 `-p` 없이 재시작하므로 활성 client 영향과 되돌릴 backup을 먼저 확인한다. 구현은 [repair script](#)에 있다.

계획된 rotation

```
./script/keytab/rotate-farm-nfs-service-key.sh --check

# 유지보수 창, AD replication, NAS/client 상태를 확인한 뒤에만 실행
./script/keytab/rotate-farm-nfs-service-key.sh --rotate --apply
```

rotation은 random password 설정, AD KVNO 변경, 새 key export, NAS keytab merge, `svcgssd` 반영, replica sync를 한 작업으로 수행한다. 이전 KVNO는 설정된 24시간 ticket lifetime보다 긴 최소 48시간 보존한다. 구현은 [rotation script](#)에 있다.

9. 모니터링에서 확인할 항목

무엇을 확인하는가	대표 상태/지표	코드
AD KVNO와 NAS keytab 일치	profile status JSON, checker exit 0/1/2	keytab health check
host keytab→kinit→kvno→rpc-gssd	<code>cluster_monitor_nfs_gss_ready</code> 와 stage	readiness script
실제 service path	<code>cluster_monitor_nfs_gss_canary_success</code>	canary probe
운영 mount	<code>cluster_monitor_host_mount_up</code> , <code>..._responsive</code> , probe seconds	cluster collector
missing mount 복구	<code>cluster_monitor_nfs_gss_recovery_*</code>	guarded recovery
D-state/lease/hung task	<code>cluster_monitor_host_dstate_*</code> , <code>cluster_monitor_nfs_*</code>	NFS forensics collector
경보 조건	<code>ClusterMonitorNFSGSS*</code> , <code>ClusterMonitorNFS*</code> , mount alerts	FARM Prometheus values

상세한 Prometheus/Grafana 운영은 [monitoring 운영 문서](#)를 참고한다.

10. 장애 진단 순서

사용자 한 명만 실패

1. DB UID/GID와 AD `uidNumber/gidNumber` 를 비교한다.
2. user keytab permission과 principal을 확인한다.
3. refresh service journal과 ccache `klist` 를 확인한다.
4. 동일 UID로 host NFS write를 시험한다.
5. container UID, `KRB5CCNAME` , ccache bind mount를 확인한다.
6. AD group 변경 직후라면 fresh ticket을 발급한다.

여러 host/user가 동시에 실패

1. `getent hosts <storage-fqdn>` 과 storage RPC 연결을 확인한다.
2. keytab checker에서 AD KVNO/NAS keytab drift를 확인한다.
3. `kvno nfs/<storage-fqdn>@<REALM>` 을 확인한다.
4. NAS `svcgssd` 와 service keytab을 확인한다.
5. mount source가 IP나 관리망 주소로 바뀌지 않았는지 확인한다.
6. AD replication 실패 시 실제 쓰기 대상 DC와 NAS가 조회하는 DC를 확인한다.

Mount가 없거나 응답하지 않음

1. `findmnt` 로 존재/source/security를 확인한다.
2. readiness 상태에서 처음 실패한 stage를 확인한다.
3. D-state, lease expiry, hung-task와 forensics snapshot을 확인한다.
4. mount가 **없는 경우에만** guarded recovery timer 상태를 확인한다.
5. 이미 healthy한 mount를 자동/수동으로 재마운트하지 않는다.
6. D-state caller가 남아 있으면 forced unmount 반복 대신 증거 보존 후 reboot를 검토한다.

11. 변경 전후 체크리스트

변경 전

- 대상 realm, DC, storage FQDN과 NFS principal 확인
- 활성 client와 유지보수 창 확인
- AD replication health 확인
- NAS keytab backup과 AILAB principal 확인
- 현재 `findmnt` , `klist` , checker status 보존

변경 후

- AD current KVNO와 NAS keytab KVNO 일치
- `kvno -k` 복호화 성공
- `svcgssd` 가 `-p` 없이 실행
- host readiness와 실제 사용자 NFS write 성공
- refresh timer enabled/active, ccache 유효
- Prometheus target/rules와 Grafana dashboard 정상
- 이전 KVNO를 최소 48시간 보존

12. 문서에 추가로 유지할 내용

설계와 운영이 다시 섞이지 않도록 다음 내용을 계속 분리해 기록한다.

- 설계 문서: trust boundary, principal naming, UID/GID invariant, ticket lifetime, FQDN 선택 근거, failure domain, Docker와 향후 Pod의 credential 모델
- 운영 문서: 현재 endpoint/KVNO가 아니라 확인 명령, timer/SLO, rotation 승인 조건, rollback, incident 진단 순서, 마지막 검증 날짜
- canonical runbook: 실제 host별 mount, 현재 적용 상태, 예외와 잔여 위험
- 실험 결과: 당시 kernel/NFS/security flavor와 재현 조건; 현재 default와 분리

keytab, password, local environment, incident 개인정보와 packet capture는 위키와 Git에 넣지 않는다.

Kerberos/NFS 디버깅 로그

이 문서는 Kerberos/NFS에서 발생한 장애와 재현 실험을 축적하는 인덱스다. 정상 상태를 만드는 절차는 [운영 문서](#)에 두고, 여기에는 장애가 어떻게 관측됐고 어떤 가설을 어떤 조건에서 검증했는지 기록한다.

기록 목록

검증일	상태	문제	핵심 결론
2026-07-23	영구 조치 적용 · 운영 검증 완료	NFSv4.1 session slot 고착 해결 및 운영 적용	LAB storage를 6.12.0-250.el10으로 교체하고 LAB8 운영 mount의 15초 idmap 지연 시험에서도 영구 slot 고착이 없음을 확인함
2026-07-20	관측, 인과 미확정	LAB9에서 sec=krb5를 사용하는 이유	krb5p에서 krb5로 전환한 뒤 D-state 지속 시간, SUNRPC 적체와 mount probe 지연이 크게 감소함
2026-07-16	재현 완료, 과거 장애 원인 후보	NFSv4.1 session slot 고착	구형 storage kernel에서 지연된 idmap decode가 NFSv4.1 slot 하나를 영구 INUSE 상태로 남기는 경로를 재현함

상태는 다음 의미로 사용한다.

- **관측**: 증상과 증거는 있지만 원인 경로를 재현하지 못했다.
- **재현 완료**: 통제된 조건에서 같은 kernel 또는 protocol 경로를 다시 만들었다.
- **원인 후보**: 재현된 경로가 과거 장애를 설명하지만 장애 발생 시점의 직접 trace가 없어 동일 원인이었다고 확정할 수 없다.
- **원인 확정**: 장애가 시작된 시점의 증거와 재현 결과가 같은 경로를 가리킨다.

장애 발생 시 먼저 할 일

NFS **hard** mount 관련 process가 D-state에 들어간 경우에는 새로운 **find**, **du**, **stat**, canary I/O를 반복하지 않는다. 각 명령이 새로운 NFS request와 D-state process를 추가할 수 있다.

먼저 NFS 경로를 읽지 않는 local 상태만 수집한다.

```
date -Ins
uname -r
findmnt -rn -o TARGET,SOURCE,FSTYPE,OPTIONS

ps -e -o state=,pid=,ppid=,etimes=,comm=,wchan:48=,args= \
| awk '$1 ~ /^D/ {print}'

cat /proc/net/rpc/nfs 2>/dev/null || true
cat /run/decs-nfs-forensics/status.json 2>/dev/null || true
cat /var/lib/decs-nfs-gss/recovery.state 2>/dev/null || true
cat /var/lib/decs-nfs-gss/canary.state 2>/dev/null || true

systemctl --no-pager --full status rpc-gssd.service
journalctl -u rpc-gssd.service -b --no-pager -n 200
```

forensics가 배포된 host에서는 자동 snapshot이 생성됐는지 먼저 확인한다. 자동 trigger가 없었다면 local 상태만 읽는 수동 snapshot을 한 번 실행할 수 있다.

```
sudo systemctl start decs-nfs-forensics-snapshot.service
sudo cat /var/lib/decs-nfs-forensics/last_snapshot.state
```

이미 mount된 경로에 D-state caller가 있으면 **rpc-gssd** restart, forced unmount, 반복 remount를 먼저 실행하지 않는다. 증거를 보존한 뒤 storage·network 상태와 kernel wait path를 기준으로 복구 범위를 결정한다.

새 디버깅 문서 작성 형식

새 문제는 한 파일에 다음 내용을 남긴다.

1. **상태와 영향 범위**: 발생 시각, host, kernel, mount source/target/security

2. **증상**: 사용자 증상, process state, wait channel, alert
3. **확인한 사실과 가설**: 관측 사실과 아직 증명하지 못한 추론을 분리
4. **실험 목적**: 어떤 가설을 반증하거나 확인하려 했는지
5. **재현 조건**: server/client, version, cache, 동시성, fault와 안전장치
6. **실행 명령**: 사전 확인, fault 주입, workload, 관측, cleanup 순서
7. **결과와 판정 기준**: 성공/실패를 판단한 trace와 metric
8. **복구와 예방**: 임시 containment, 영구 수정, monitoring 변경
9. **원본 증거**: 결과 README, curated evidence, script와 checksum 링크

운영 환경에서 fault를 주입한 명령은 일반 runbook처럼 제시하지 않고 반드시 실행 일자와 승인·안전 조건을 붙인 **과거 실행 기록**으로 표시한다. 반복 실험은 가능하면 격리 VM harness로 옮긴다.

LAB9에서 **sec=krb5** 를 사용하는 이유

상태: LAB9에서 **sec=krb5p** 전환 전 24시간과 **sec=krb5** 전환 후 약 51시간을 비교한 운영 관측이다. 전환 뒤 NFS 지연과 적체가 크게 감소했지만, 관측 기간이 다르고 security flavor만 통제된 실험이 아니므로 **krb5p** 를 단일 원인으로 확정하지는 않는다.

1. 결론

LAB9의 Kerberos NFS 기본값은 **sec=krb5** 로 유지한다. 이 선택은 Kerberos principal 인증을 유지하면서 RPC payload 암호화 비용을 제외하고, LAB9에서 실제로 관측된 D-state 지속·SUNRPC 적체·mount probe 지연을 낮추기 위한 운영 기준이다.

krb5p 가 잘못된 방식이라는 뜻은 아니다. NFS payload 기밀성이 반드시 필요한 경로에서는 **krb5p** 를 사용해야 한다. 다만 현재 LAB9 사용자 공유의 기본 profile은 가용성과 지연 안정성을 우선해 **krb5** 를 사용한다. **krb5i** 또는 **krb5p** 는 요구사항이 있고 부하 검증을 통과한 경로에서만 명시적으로 선택한다.

2. security flavor의 차이

flavor	제공하는 보호	LAB9에서의 위치
sec=krb5	Kerberos principal 인증	기본값
sec=krb5i	인증 + RPC payload 무결성	변조 방지가 명시적으로 필요한 경로에서 검토
sec=krb5p	인증 + 무결성 + RPC payload 암호화	기밀성이 필요한 경로에서 성능 검증 후 사용

`sec=krb5`에서는 사용자와 NFS service가 Kerberos로 서로 인증되지만, NFS payload 자체는 암호화되지 않는다. 따라서 패킷 캡처와 forensics evidence는 root만 읽을 수 있게 보관하고, 보호되지 않은 네트워크에서 기밀 데이터를 전송해야 한다면 이번 성능 결과만으로 `krb5`를 선택해서는 안 된다.

3. 왜 전환했나

`sec=krb5p` 사용 중 LAB9에서는 NFS 관련 D-state가 관측 기간 내내 존재했고, SUNRPC task와 transport reply 대기가 누적됐다. mount responsiveness probe도 설정된 10초 timeout 상한에 도달했다. 이 상태에서는 home 경로를 읽는 SSH나 사용자 process가 NFS 대기에 연쇄적으로 묶일 수 있다.

`krb5p`는 각 RPC payload에 무결성 처리와 암호화를 추가한다. 이 비용이 장애의 단일 원인이라고 입증한 것은 아니지만, storage·kernel·network에서 이미 발생한 지연을 더 크게 보이게 하는 증폭 요인인지 확인할 운영상 이유가 있었다. 그래서 principal 인증은 유지하면서 privacy 처리를 제외하는 `sec=krb5`로 전환하고 동일한 NFS health 지표를 계속 관측했다.

4. 전환 전후 결과

측정 대상은 LAB9다.

지표	<code>krb5p</code> 전환 전 24시간	<code>krb5</code> 전환 후 약 51시간
NFS D-state 존재 비율	100%	약 4.1%
D-state 프로세스 최대	28개	8개
최장 연속 D-state	83,339초(약 23.1시간)	1,236초(약 20분 36초)
<code>xprt_pending</code> 평균	5.35	1.29
RPC task 최대	3,696	525
마운트 probe 최대	10초	0.429초
lease 만료	없음	없음

변화 폭은 다음과 같다.

비교 항목	변화
NFS D-state 존재 비율	95.9%p 감소
D-state 프로세스 최대	약 71.4% 감소
최장 연속 D-state	약 98.5% 감소
<code>xprt_pending</code> 평균	약 75.9% 감소
RPC task 최대	약 85.8% 감소

비교 항목	변화
마운트 probe 최대	약 95.7% 감소

가장 중요한 변화는 D-state가 “항상 존재하는 상태”에서 짧고 간헐적인 상태로 바뀐 점이다. `xprt_pending` 평균과 RPC task 최대도 함께 감소했으므로 단순히 D-state process 이름만 달라진 것이 아니라 NFS transport 적체 자체가 줄어든 방향과 일치한다.

전환 전 mount probe의 10초는 `cluster-monitor-exporter`의 command timeout 상한과 같다. 따라서 이 값은 정상 응답에 10초가 걸렸다가보다 probe가 timeout 또는 기존 in-flight probe 상태에 도달했을 가능성이 크다. 전환 후 최대 0.429초는 관측 구간에서 10초 상한에 도달하지 않았음을 보여준다.

두 구간 모두 lease 만료가 없었다. 따라서 이번 비교에서 확인한 개선은 lease 만료 복구 여부보다 D-state 지속, RPC queue와 mount 응답성에 관한 것이다.

5. 지표의 의미

지표	판정 방법	구현
NFS D-state 존재 비율	유효 sample 중 NFS/RPC 관련 D-state process가 하나 이상인 sample 비율	<code>cluster_monitor_nfs_forensics_d_state_processes</code>
D-state 프로세스 최대	관측 구간의 NFS/RPC 관련 D-state process 최대값	같은 metric의 <code>max_over_time</code>
최장 연속 D-state	가장 오래 연속 관측된 NFS/RPC 관련 D-state process의 시간	<code>cluster_monitor_nfs_forensics_d_state_oldest_seconds</code>
<code>xprt_pending</code> 평균	transport reply를 기다리는 SUNRPC task 수의 구간 평균	<code>cluster_monitor_nfs_xprt_pending</code>
RPC task 최대	NFS RPC client task 수의 구간 최대	<code>cluster_monitor_nfs_rpc_tasks</code>
마운트 probe 최대	required mount에 수행한 bounded <code>statfs</code> 시간의 구간 최대	<code>cluster_monitor_host_mount_probe_seconds</code>
lease 만료	local NFSv4 lease deadline을 넘긴 시간	<code>cluster_monitor_nfs_lease_expired_seconds</code>

수집 경로는 다음과 같다.

```
LAB9 kernel:/proc 상태
-> NFS forensics watcher status.json
-> cluster-monitor-exporter metrics
-> FARM Prometheus
-> 같은 server="LAB9" label의 전환 전후 구간 집계
```

구현은 `forensics watcher`, `forensics metric adapter`, `mount/D-state collector`에서 확인한다.

6. 이 결과로 말할 수 있는 것과 없는 것

말할 수 있는 범위는 다음과 같다.

- LAB9에서 **krb5** 전환 뒤 모든 주요 NFS 적체 지표가 같은 방향으로 개선됐다.
- 약 51시간의 전환 후 구간에는 24시간 전환 전 구간과 같은 장시간 고착이 관측되지 않았다.
- Kerberos 인증을 유지한 **krb5** 가 LAB9의 현재 운영 baseline으로 더 안정적이었다.

아직 확정할 수 없는 범위는 다음과 같다.

- **krb5p** 의 암호화 비용만으로 전환 전 D-state가 발생했다는 인과관계
- 같은 시기에 달라졌을 수 있는 workload, storage queue, network, kernel 상태의 영향
- FARM/LAB의 다른 host에서도 감소 폭이 동일할 것이라는 일반화
- 51시간보다 긴 기간이나 peak workload에서도 같은 결과가 유지된다는 보장

따라서 이 기록의 판정은 **LAB9 운영 선택을 지지하는 강한 전후 관측**이며, 통제된 benchmark나 원인 확정 실험은 아니다. 원인을 더 좁히려면 동일 workload와 동일 시간대에 **krb5 / krb5p** 를 반복 교차하는 격리 실험이 필요하다.

7. 운영 기준과 확인 방법

LAB9의 desired mount option은 **sec=krb5** 다. mount source는 IP가 아니라 NFS service principal과 일치하는 FQDN을 사용한다. D-state가 이미 존재할 때는 확인을 위해 mount를 반복해서 읽거나 즉시 remount하지 말고 먼저 local forensics 상태를 보존한다.

현재 mount의 security flavor는 다음처럼 정확히 확인한다.

```
LAB9_MOUNT_TARGET=/home/tako9/share

findmnt -T "$LAB9_MOUNT_TARGET" -n -o TARGET,SOURCE,FSTYPE,OPTIONS
findmnt -T "$LAB9_MOUNT_TARGET" -n -o OPTIONS \
| tr ' ,' '\n' \
| grep -Fx 'sec=krb5'
```

grep -F 'sec=krb5' 만 사용하면 **sec=krb5p** 도 일치하므로 option을 쉼표 단위로 분리해 exact match한다. 전환 이후에도 최소한 다음 조건을 함께 확인한다.

- NFS D-state 존재 비율과 최장 연속 시간이 다시 증가하지 않는가
- **xprt_pending** 평균과 RPC task 최대가 전환 전 수준으로 돌아가지 않는가
- mount probe가 10초 timeout 상한에 다시 도달하지 않는가
- **cluster_monitor_nfs_lease_expired_seconds** 가 계속 0인가
- payload 기밀성 요구가 새로 생겨 **krb5p** 가 필요한 경로가 되지 않았는가

기본 security policy는 Kerberos/NFS 운영 저장소, 배포 값은 **monitoring exporter** 변수를 기준으로 한다. 상세 장애 채증과 D-state 대응은 **디버깅 로그 개요**의 절차를 따른다.

8. 다음 측정에서 함께 남길 값

현재 비교표에는 구간 길이만 있고 정확한 시작·종료 timestamp와 원본 query/export artifact가 없다. 다음 재측정 부터는 아래 값을 함께 보관해야 같은 결과를 재계산할 수 있다.

- 전환 시각과 두 구간의 시작·종료 시각(타임존 포함)
- LAB9의 source FQDN, target, NFS version과 전체 mount option
- client/server kernel, storage 상태와 구간별 workload
- 사용한 PromQL 또는 `samples.tsv` 집계 script와 원본 결과
- exporter command timeout과 scrape interval
- 전환과 동시에 수행한 다른 설정 변경

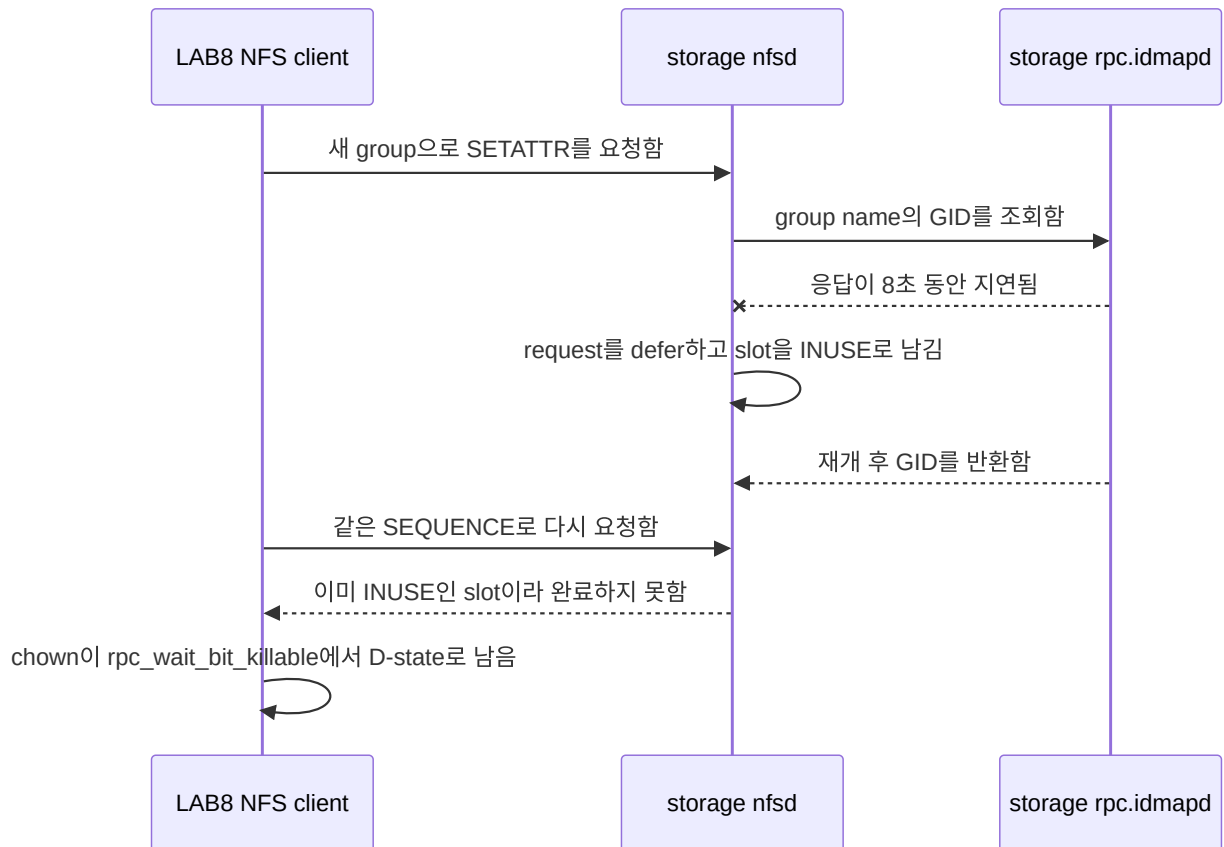
원본 artifact를 저장소에 추가하면 이 문서의 표 아래에 commit 고정 링크를 추가한다.

NFSv4.1 session slot 고착

상태: 2026-07-16에 slot leak 경로 재현 완료. 2026-06-29 LAB5 장애의 강한 원인 후보이지만, LAB5 장애 시작 시점의 storage trace가 없어 과거 장애의 최초 원인으로 확정하지는 않는다.

1. 요약

LAB storage의 구형 kernel `3.10.0-862.el7.x86_64` 에서 NFSv4 `SETATTR` 가 새로운 group 이름을 UID/GID로 해석하던 중 `rpc.idmapd` 응답이 지연되면 request가 defer될 수 있다. 이때 NFSv4.1 `SEQUENCE` 는 slot을 `INUSE` 로 표시하지만 response encode와 `nfsd4_sequence_done()` 이 실행되지 않아 slot이 반환되지 않는 경로를 실제로 재현했다.



한 slot만 leak한 최소 재현에서는 `ForeChannel` waiter가 0일 수도 있다. 따라서 waiter가 없다는 사실만으로 slot 문제를 배제하면 안 된다. session reset이나 recovery가 모든 slot의 drain을 기다리는 상황에서는 고착된 한 slot이 전체 복구를 막을 수 있다.

2. 어떤 증상이었나

2026-07-16 재현에서는 LAB8의 `chown` worker가 173초 이상 D-state에 남았다.

```

chown
-> nfs4_proc_setattr
-> nfs4_call_sync_sequence
-> rpc_wait_bit_killable
  
```

storage trace에서는 첫 처리와 재방문이 다음처럼 달랐다.

단계	관측
첫 idmap decode	<code>RQ_USEDEFERRAL</code> , <code>RQ_DROPME</code> 가 설정됨
첫 SEQUENCE 확인	<code>seqid=0x5f</code> , <code>slot_seqid=0x5e</code> , <code>INUSE=0</code>
첫 request 종료	dispatcher가 0을 반환했고 encode와 <code>sequence_done</code> 이 실행되지 않음
<code>rpc.idmapd</code> 재개 후	같은 request가 <code>seqid=0x5f</code> , <code>slot_seqid=0x5f</code> , <code>INUSE=1</code> 로 재방문

단계	관측
이후 retry	약 15초마다 같은 <code>INUSE=1</code> 을 확인하고 완료하지 못함

3. 왜 이 실험을 했나

조사는 다음 순서로 가설을 좁혔다.

1. 2026-06-29 LAB5에서는 NFS user home을 읽는 `sshd: ... [priv]` 가 D-state에 누적됐다.
2. 2026-07-10 LAB8에서 TCP/2049 transport를 일시 차단하자 같은 `rpc_wait_bit_killable` 계열 증상을 재현했지만, network가 복구된 뒤 process도 자연 복구됐다.
3. keytab, `rpc-gssd`, readiness와 mount 순서를 바로잡은 clean boot에서도 구형 storage kernel의 session state가 영구 고착될 수 있는지는 별도 검증이 필요했다.
4. upstream의 idmap deferral 수정 내용이 “NFSv4 decode 중 request drop으로 `NFSD4_SLOT_INUSE` 가 해제되지 않는다”는 경로를 설명하므로, 실제 LAB kernel과 mount에서 그 primitive를 최소 workload로 확인했다.

실험 목적은 많은 client를 동시에 멈추게 하는 것이 아니라, **idmap miss 하나가 slot 하나를 반환 불가능하게 만들 수 있는지**를 확인하는 것이었다.

4. 실험 결과

4.1 실제 LAB 환경의 최소 재현

첫 pass에서 `RQ_DROPME` 가 설정된 뒤 slot이 `INUSE` 로 바뀌었지만 encode와 `nfdsd4_sequence_done()` 이 호출되지 않았다. 8초 뒤 `rpc.idmapd` 가 재개된 후에도 같은 sequence는 이미 사용 중인 slot으로 판단됐다. client retry도 동일 상태를 반복했고 worker는 D-state에서 끝나지 않았다.

따라서 다음 좁은 결론은 확정할 수 있다.

- 당시 storage kernel에서는 idmap decode deferral이 NFSv4.1 session slot 하나를 leak할 수 있다.
- keytab 발급, `rpc-gssd` 시작 순서나 client mount 순서만 고쳐서는 이 server-side kernel 경로를 제거할 수 없다.
- 한 slot leak은 재현했지만 LAB5에서 나중에 관측한 전체 session recovery/drain stall까지 이 최소 실험 하나로 재현한 것은 아니다.
- LAB5 장애 시작 시점의 trace가 없으므로 어떤 실제 operation이 최초 idmap miss를 만들었는지는 알 수 없다.

4.2 A-G: 8초 idmap 지연 kernel matrix

최소 재현과 같은 `idmap-setattr`, NFSv4.1, `sec=krb5p` 조건에서 server와 client kernel 조합을 바꿔 각 5회 실행했다. client OS는 모두 Ubuntu 22.04.5 LTS이며, F/G만 client kernel을 `6.18.38` 로 올렸다.

Case	NFS server OS	Server kernel	Client kernel	결과
A	CentOS Linux 7.9.2009	<code>3.10.0-862.el7.x86_64</code>	<code>6.8.0-134-generic</code>	slot 고착 5/5

Case	NFS server OS	Server kernel	Client kernel	결과
B	CentOS Linux 7.9.2009	3.10.0-1160.el7.x86_64	6.8.0-134-generic	slot 고착 5/5
C	CentOS Stream 10	6.12.74	6.8.0-134-generic	slot 고착 5/5
D	CentOS Stream 10	6.12.75	6.8.0-134-generic	자동 복구 5/5
E	CentOS Stream 10	6.12.0-248.el10.x86_64	6.8.0-134-generic	자동 복구 5/5
F	CentOS Linux 7.9.2009	3.10.0-862.el7.x86_64	6.18.38	slot 고착 5/5
G	CentOS Linux 7.9.2009	3.10.0-1160.el7.x86_64	6.18.38	slot 고착 5/5

구형 server kernel에서는 client kernel을 6.18.38 로 올린 F/G도 모두 고착됐다. 반대로 idmap/slot fix가 포함된 D와 signed Stream kernel E는 모두 자동 복구했다. 이 결과는 문제와 복구 여부를 가르는 주된 변수가 client가 아니라 **NFS server kernel의 idmap deferral 처리**임을 지지한다.

4.3 D/E: 120·300·600초 장시간 idmap 지연 추가 실험

8초 지연만으로는 수정된 kernel이 짧은 지연에서만 우연히 복구한 것인지 판단하기 어려웠다. 그래서 자동 복구한 D/E만 대상으로 idmap cache miss 응답을 120초, 300초, 600초까지 지연하고 각 조건을 3회씩 다시 실행했다.

Case	Server kernel	idmap 지연	원래 chgrp	신규 RPC	최종 snapshot D / FC / lease
D-120s	6.12.75-nfs-slot	120초	성공 3/3	성공 3/3	모두 0 / 0 / 0*
D-300s	6.12.75-nfs-slot	300초	성공 3/3	성공 3/3	모두 0 / 0 / 0
D-600s	6.12.75-nfs-slot	600초	EINVAL 3/3	성공 3/3	모두 0 / 0 / 0
E-120s	6.12.0-248.el10.x86_64	120초	성공 3/3	성공 3/3	모두 0 / 0 / 0
E-300s	6.12.0-248.el10.x86_64	300초	성공 3/3	성공 3/3	모두 0 / 0 / 0
E-600s	6.12.0-248.el10.x86_64	600초	EINVAL 3/3	성공 3/3	모두 0 / 0 / 0

여기서 FC는 NFSv4.1 ForeChannel waiter 수다. 600초 조건의 EINVAL은 원래 chgrp 요청의 application 결과이며, slot 고착 판정과 분리해야 한다. 같은 run에서 idmap 지연을 해제한 뒤 실행한 신규 RPC는 모두 성공했고, 최종 snapshot의 D-state, ForeChannel waiter와 lease_expired도 모두 0이었다. 따라서 600초에서 원래 작업이 실패했더라도 **session slot은 반환됐고 후속 요청을 막지 않았다**.

* D-120s 2회차의 2초 sampler는 종료 직전 D-state 1을 한 번 기록했다. 그러나 직후 final snapshot은 D-state 0이었고, 신규 RPC 성공, ForeChannel 0, lease_expired=0, analyzer verdict=pass였다. 지속된 NFS slot 고착으로 판정하지 않았다.

5. 실제 재현 조건

항목	2026-07-16 조건
client	LAB8, 100.100.100.108
server	lab-storage.lab.decs.internal, 100.100.100.100
mount	/home/tako8/share, NFSv4.1, sec=krb5p, hard
storage kernel	3.10.0-862.el7.x86_64
client 시작 상태	D-state 0, NFS RPC task 0, readiness ready
test identity	LAB8 local group decs_idmap_repro_424242, GID 424242
test object	/home/tako8/share/.decs-idmap-slot-repro-20260716
fault	storage rpc.idmapd 에 8초간 SIGSTOP
workload	test file에 GID 424242 를 지정하는 chown(2) 한 번
안전장치	storage-local 8초·20초 SIGCONT timer를 먼저 예약
금지한 복구	재현 후 client reboot, unmount, signal, service restart를 하지 않음

boot 순서도 별도로 확인했다. LAB8은 rpc-gssd active → Kerberos/NFS readiness 완료 → mount 순서였고, 기존 /etc/krb5.keytab 의 KVNO는 2였다. 따라서 이번 결과를 keytab 생성 누락이나 잘못된 mount 시작 순서로 설명할 수 없다.

6. 당시 실행한 명령

경고: 아래 명령은 2026-07-16의 승인된 실제 실행 기록이다. storage의 rpc.idmapd 를 멈추므로 일반 점검이나 운영 재현 절차로 다시 실행하면 안 된다. 재실험은 8절의 격리 VM harness를 사용한다. 특히 kprobe 인자 register는 당시 x86_64 kernel symbol에만 맞춘 값이라 다른 kernel에서 그대로 사용하지 않는다.

6.1 client 사전 확인과 test 값 준비

LAB8에서 D-state와 RPC task가 없는지 확인한 뒤, storage에 없던 group 이름을 client가 encode하도록 local group과 test file을 준비했다.

```

ps -eo stat= | awk '$1 ~ /^D/ {n++} END {print "d_state=" (n+0)}'
grep -c '^RPC task' /proc/net/rpc/nfs 2>/dev/null || true
findmnt -T /home/tako8/share -n -o TARGET,SOURCE,FSTYPE,OPTIONS

sudo groupadd -g 424242 decs_idmap_repro_424242

sudo bash -c '
p=/home/tako8/share/.decs-idmap-slot-repro-20260716
rm -f "$p"
touch "$p"
chmod 0666 "$p"
stat -c "path=%n uid=%u gid=%g mode=%a ino=%i" "$p"
'

```

fault 전에 GID 424241 을 사용한 정상 `chown` 을 실행해 같은 경로가 idmap 지연 없이는 encode와 `sequence_done` 까지 완료되는 것을 먼저 확인했다.

```

sudo python3 -c 'import os; p="/home/tako8/share/.decs-idmap-slot-repro-20260716"; print("before",
os.stat(p).st_gid); os.chown(p, -1, 424241); print("after", os.stat(p).st_gid)'

```

6.2 storage trace 설정

당시 storage에서 idmap decode, slot 검사, dispatcher return, response encode와 slot 반환 여부를 한 trace instance에 기록했다.

```

T=/sys/kernel/debug/tracing
I="$T/instances/decs-idmap-slot-repro"
sudo mkdir -p "$I"

for event in uid gid check_slot dispatch dispatch_ret encode sequence sequence_done; do
    echo "-:decs_idmap/$event" | sudo tee -a "$T/kprobe_events" >/dev/null || true
done

echo 'p:decs_idmap/uid nfsd_map_name_to_uid rqstp=%di name=+0(%si):string namelen=%dx:u32' \
    | sudo tee -a "$T/kprobe_events" >/dev/null
echo 'p:decs_idmap/gid nfsd_map_name_to_gid rqstp=%di name=+0(%si):string namelen=%dx:u32' \
    | sudo tee -a "$T/kprobe_events" >/dev/null
echo 'p:decs_idmap/check_slot check_slot_seqid seqid=%di:u32 slot_seqid=%si:u32 slot_inuse=%dx:u32' \
    | sudo tee -a "$T/kprobe_events" >/dev/null
echo 'p:decs_idmap/dispatch nfsd_dispatch rqstp=%di' \
    | sudo tee -a "$T/kprobe_events" >/dev/null
echo 'r:decs_idmap/dispatch_ret nfsd_dispatch ret=$retval:s32' \
    | sudo tee -a "$T/kprobe_events" >/dev/null
echo 'p:decs_idmap/encode nfs4svc_encode_compoundres rqstp=%di' \
    | sudo tee -a "$T/kprobe_events" >/dev/null
echo 'p:decs_idmap/sequence nfsd4_sequence rqstp=%di cstate=%si seq=%dx' \
    | sudo tee -a "$T/kprobe_events" >/dev/null
echo 'p:decs_idmap/sequence_done nfsd4_sequence_done resp=%di' \
    | sudo tee -a "$T/kprobe_events" >/dev/null

echo 0 | sudo tee "$I/tracing_on" >/dev/null
echo 0 | sudo tee "$I/events/enable" >/dev/null
sudo sh -c ": > '$I/trace'"
echo 8192 | sudo tee "$I/buffer_size_kb" >/dev/null

for event in \
    decs_idmap/uid decs_idmap/gid decs_idmap/check_slot decs_idmap/dispatch \
    decs_idmap/dispatch_ret decs_idmap/encode decs_idmap/sequence \
    decs_idmap/sequence_done sunrpc/svc_process sunrpc/svc_defer \
    sunrpc/svc_revisit_deferred sunrpc/svc_drop sunrpc/svc_send
do
    echo 1 | sudo tee "$I/events/$event/enable" >/dev/null
done
echo 1 | sudo tee "$I/tracing_on" >/dev/null

```

6.3 fault와 단일 workload 실행

storage에서 반드시 자동 재개 timer 두 개가 active인지 확인한 뒤 `rpc.idmapd`를 멈췄다.


```
pid="$(pgrep -xo rpc.idmapd)"

sudo systemd-run --unit=decs-idmapd-auto-resume-8s \
  --on-active=8s --timer-property=AccuracySec=1s \
  /bin/kill -CONT "$pid"
sudo systemd-run --unit=decs-idmapd-auto-resume-20s \
  --on-active=20s --timer-property=AccuracySec=1s \
  /bin/kill -CONT "$pid"

sudo systemctl is-active --quiet decs-idmapd-auto-resume-8s.timer
sudo systemctl is-active --quiet decs-idmapd-auto-resume-20s.timer
sudo kill -STOP "$pid"
```

그 직후 LAB8의 별도 systemd worker에서 `chown(2)` 한 번만 실행했다.

```
sudo systemd-run --unit=decs-idmap-slot-repro-worker --no-block \
  /usr/bin/python3 -c \
  "import os; os.chown('/home/tako8/share/.decs-idmap-slot-repro-20260716', -1, 424242)"
```

6.4 결과 확인과 trace 보존

새로운 NFS I/O를 만들지 않고 worker, local kernel state와 이미 동작 중이던 forensics state를 확인했다.

```
systemctl --no-pager --full status decs-idmap-slot-repro-worker.service
ps -eo pid=,ppid=,stat=,comm=,wchan:32=,args= \
  | awk '$3 ~ /^D/ {print}'
grep -E 'lease_expired|RPC task|xprt_pending|ForeChannel' \
  /proc/net/rpc/nfs 2>/dev/null || true

cat /run/decs-nfs-forensics/status.json
cat /var/lib/decs-nfs-gss/recovery.state
cat /var/lib/decs-nfs-gss/canary.state
```

storage trace는 다음 pattern을 기준으로 추렸다.

```
I=/sys/kernel/debug/tracing/instances/decs-idmap-slot-repro
sudo sh -c "echo 0 > '$I/tracing_on'"
sudo cp "$I/trace" /var/tmp/decs-idmap-slot-repro-20260716.trace

sudo grep -E \
  'decs_idmap_gid|svc_defer|svc_revisit_deferred|svc_drop:|check_slot|sequence_done|dispatch_ret' \
  /var/tmp/decs-idmap-slot-repro-20260716.trace
```

7. Monitoring과 운영 대응

재현 당시 guardrail은 의도대로 동작했다.

- forensics watcher가 `nfs_d_state` 이유로 incident snapshot을 보존했다.
- recovery는 `status=blocked`, `diagnosis=nfs_d_state_detected`, `retry_inhibited=1`로 바뀌었다.
- canary도 같은 진단으로 차단돼 추가 NFS request를 만들지 않았다.
- packet ring과 kernel trace는 계속 증거를 수집했다.

운영에서 같은 증상이 보이면 다음 순서를 따른다.

1. NFS path를 읽는 `find`, `du`, `stat`, canary를 추가 실행하지 않는다.
2. D-state stack, `/proc/net/rpc/nfs`, forensics status와 snapshot을 보존한다.
3. mount가 이미 있으면 자동 remount와 `rpc-gssd` 반복 restart를 차단한다.
4. storage kernel, `rpc.idmapd`, network와 NFS session 증거를 함께 확인한다.
5. 상태가 자연 복구되지 않으면 유지보수 창외 client reboot 또는 storage 조치를 수동 결정한다. reboot는 상태를 지울 뿐 원인 수정은 아니다.
6. 영구 조치는 idmap deferral fix가 포함된 storage kernel로 올리고 격리 회귀 실험을 통과시키는 것이다.

운영 mount를 `soft`로 바꾸는 방식은 timeout을 application error로 노출하고 data integrity 위험을 만들 수 있어 해결책으로 사용하지 않는다.

8. 격리 VM에서 다시 검증하는 방법

반복 실험은 LAB8 local disk의 `decs-nfs-slot-isolated` network에서 수행한다. 이 network는 forwarding이 없고 `100.100.100.0/24`, `192.168.1.0/24`와 운영 storage hostname을 거부한다. VM disk, trace와 result도 NFS share가 아니라 LAB8 local filesystem에 둔다.

kernel 비교 기준은 다음과 같다.

Case	Server kernel	Client kernel	목적
A	CentOS 7 <code>3.10.0-862</code>	Ubuntu <code>6.8.0-134</code>	운영 취약 baseline
B	CentOS 7 <code>3.10.0-1160</code>	Ubuntu <code>6.8.0-134</code>	CentOS 7 최종 kernel 비교

Case	Server kernel	Client kernel	목적
C	upstream 6.12.74	Ubuntu 6.8.0-134	fix 이전 positive control
D	upstream 6.12.75	Ubuntu 6.8.0-134	idmap/slot fix 비교 control
E	signed Stream 6.12.0-248.el10	Ubuntu 6.8.0-134	운영 후보
F	CentOS 7 3.10.0-862	kernel.org 6.18.38	새 client kernel 비교
G	CentOS 7 3.10.0-1160	kernel.org 6.18.38	새 client kernel 비교

준비와 격리는 VM lab README의 절차를 따른다.

```
cd /home/jy/server_manage/kerberos-nfs/test/lab-kerberos-poc/vm-slot-regression

sudo ./bin/provision_lab8_vms.sh "$PWD"
./bin/configure_vm_stack.sh
sudo ./bin/prefetch_kernel_matrix.sh
sudo ./bin/verify_kernel_sources.sh
sudo ./bin/build_upstream_kernels_vm.sh
sudo ./bin/prepare_centos7_domains.sh "$PWD"
sudo ./bin/isolate_test_vms.sh
```

단일 비교는 같은 idmap-setattr, sec=krb5p, 8초 조건을 case별로 실행한다.

```
./bin/switch_kernel_case.sh A
NFS_SLOT_EXPECT_VULNERABLE=1 \
./bin/run_single_case.sh A krb5p idmap-setattr 8 1

./bin/switch_kernel_case.sh D
NFS_SLOT_EXPECT_VULNERABLE=0 \
./bin/run_single_case.sh D krb5p idmap-setattr 8 1
```

먼저 실행 수만 확인하려면 plan-only mode를 쓴다.

```
MATRIX_PLAN_ONLY=1 \
MATRIX_CASES='A C D E' \
MATRIX_SECURITY='krb5p' \
MATRIX_SCENARIOS='idmap-setattr' \
MATRIX_DURATIONS='8' \
MATRIX_REPETITIONS=3 \
./bin/run_matrix.sh
```

취약 case가 고착되면 harness는 evidence를 archive한 뒤 **격리 VM만** reset한다. 그 reset은 자동 복구 성공으로 계산하지 않는다. 운영 LAB8 host와 운영 NFS mount는 이 harness의 복구 대상이 아니다.

9. 원본 증거와 코드

- 2026-07-16 실제 재현 결과
- curated trace evidence
- 2026-07-10 NFS transport fault 실험
- 격리 VM slot regression lab
- single-case harness
- result analyzer
- upstream stable patch 설명
- NFS forensics 구현

보존된 storage raw trace의 SHA-256은 `5fe7fbab2b8aa712ff71afe5156438b4e880edcc39cf16a00592e22d578bac59` 다.

NFSv4.1 session slot 고착 해결 및 운영 적용

상태: 2026-07-21~22 LAB storage의 OS와 kernel을 교체하고 Kerberos/NFS 구성을 복원했다. 2026-07-23 LAB8 운영 mount에서 `rpc.idmapd` 를 15초 지연한 직접 시험에서도 session slot이 영구 고착되지 않았다.

이 문서는 NFSv4.1 session slot 고착에서 확인한 server-side kernel 문제를 실제 LAB storage에서 어떻게 제거하고 운영 구성을 복원했는지 기록한다. 원인 재현 과정과 fault-injection 명령은 기존 문서에 두고, 여기에는 해결 조치와 안전한 사후 검증만 정리한다.

1. 해결 결론

이 장애의 근본 조치는 NFS server인 **lab-storage** 의 kernel을 idmap deferral 수정이 포함된 kernel 계열로 교체하는 것이다. client 재부팅, NFS remount 또는 `rpc-gssd` 재시작은 이미 고착된 상태를 지울 수는 있지만 server-side slot leak 경로를 없애지 못한다.

항목	변경 전	변경 후
작업 서버	<code>lab-storage.lab.decs.internal</code>	동일
OS	CentOS Linux 7	CentOS Stream 10
kernel	<code>3.10.0-862.el7.x86_64</code>	<code>6.12.0-250.el10.x86_64</code>
데이터	<code>/294t</code> XFS	기존 <code>/dev/sda1</code> 을 포맷하지 않고 <code>/294t</code> 에 재연결

항목	변경 전	변경 후
AD/KDC	LAB2 Samba AD	동일, storage keytab은 LAB2에서 재생성
NFS 보안	Kerberos NFS	SSSD·gssproxy·NFS export를 복원

격리 VM 비교에서는 upstream 6.12.74 까지 slot 고착이 발생했고 6.12.75 에서 자동 복구했다. CentOS Stream 10 의 signed kernel 6.12.0-248.el10 도 같은 시험을 통과했다. 현재 운영 kernel 6.12.0-250.el10 은 그보다 뒤에 설치한 같은 배포판 kernel이다.

여기서 250.el10 은 upstream Linux 6.12.250 을 뜻하지 않고 CentOS/RHEL 계열의 package release 번호다. 초기 운영 적용은 격리 VM 회귀 결과와 교체 후 실제 서비스 상태를 근거로 판정했고, 2026-07-23에는 승인된 15초 지연 시험을 LAB8 운영 mount에서 한 번 추가해 현재 kernel에서도 영구 slot 고착이 없음을 확인했다.

2. 서버별 역할

서버	이번 조치에서의 역할
lab-storage.lab.decs.internal (100.100.100.100)	kernel 문제를 제거한 NFS server. OS, network, data mount, SSSD, keytab, gssproxy와 export를 복원한 대상
LAB2 / 100.100.100.102	운영 Samba AD DC, DNS/KDC. storage machine account와 NFS service principal의 원본
LAB1~LAB10	NFS client. 각 서버의 기존 /etc/fstab 으로 /home/takoN/share 를 다시 mount
LAB8 host	/home/tako8/share 운영 mount에서 15초 idmap 지연을 직접 검증한 client
LAB8의 격리 VM	kernel matrix와 장시간 지연 회귀 시험 전용

keytab은 예전 파일을 backup해서 되돌리지 않았다. LAB2 AD에 등록된 storage machine account와 SPN을 기준으로 새 keytab을 만든 뒤 lab-storage 의 /etc/krb5.keytab 에 설치했다. 따라서 keytab 생성 작업은 LAB2에서, 설치와 검증은 lab-storage 에서 수행한다.

3. 실제 적용 내용

3.1 lab-storage : OS와 kernel 교체

2026-07-21 22:38 KST에 kernel-core-6.12.0-250.el10.x86_64 를 설치했고 22:49 KST부터 이 kernel로 부팅해 운영 중이다. hostname과 두 network를 다음 상태로 복원했다.

용도	interface	주소
관리망	enp24s0f1	192.168.1.20/24
storage망	enp101s0f0	100.100.100.100/24

hostname은 `lab-storage.lab.decs.internal`, timezone은 `Asia/Seoul`이며 NTP 동기화 상태는 `yes`다.

3.2 lab-storage : 기존 데이터 디스크 재연결

RAID volume이 처음 보이지 않은 이유

OS 설치 직후에는 294TB 데이터 RAID가 정상적인 `/dev/sda`로 나타나지 않았다. 이 장비의 controller는 다음 Areca ARC-1284다.

```
17:00.0 RAID bus controller: Areca Technology Corp. ARC-12x4
Subsystem: Areca Technology Corp. ARC-1284 24 Port
PCI ID: 17d3:1214
```

CentOS Stream 10의 운영 kernel 설정을 확인하면 ARC-1284용 `arcmsr`가 빠져 있다.

```
grep CONFIG_SCSI_ARCMSR /boot/config-"$(uname -r)"
```

당시 결과는 다음과 같았다.

```
# CONFIG_SCSI_ARCMSR is not set
```

먼저 `kernel-modules-extra`를 설치했지만 이 kernel용 `arcmsr.ko`는 들어 있지 않았다.

```
sudo dnf install -y kernel-modules-extra
modinfo arcmsr
```

ELRepo의 `kmod-arcmsr`도 확인했지만 EL10용 package가 없었다. 최종 해결은 ELRepo package 설치가 아니라 **현재 실행 kernel에 맞춰 `arcmsr`를 외부 module로 직접 빌드하는 것이었다.**

`arcmsr` 외부 module 빌드와 설치

2026-07-21에 실제로 설치한 build dependency는 다음과 같다. `kernel-devel`의 version은 반드시 module을 사용할 kernel과 정확히 같아야 한다.

```
KVER=$(uname -r)

sudo dnf install -y \
    "kernel-devel-$KVER" \
    gcc make elfutils-libelf-devel xz
```

Linux 6.12의 `drivers/scsi/arcmsr`에서 `Makefile`, `arcmsr.h`, `arcmsr_attr.c`, `arcmsr_hba.c`를 임시 build directory로 가져왔다. Stream 10 kernel에 backport된 API에 맞게 다음 호환 조정을 적용한 뒤 외부 module로 빌드했다.

- `slave_configure` 를 `sdev_configure` 와 `struct queue_limits` signature로 변경
- `bios_param` 의 `struct block_device` 를 `struct gendisk` 로 변경
- `del_timer_sync()` 를 `timer_delete_sync()` 로 변경
- `from_timer()` 를 `timer_container_of()` 로 변경
- `sysfs bin_attribute` 인자를 `const` 로 변경
- PCI device ID table을 `const` 로 변경

실제 build 형태는 다음과 같다. upstream 6.12 source를 수정하지 않고 그대로 빌드하면 Stream 10 kernel API와 맞지 않으므로 위 호환 조정이 필요하다.

```
KVER=$(uname -r)
BUILD_DIR=$(mktemp -d /tmp/arcmsr-build.XXXXXX)

# BUILD_DIR에 호환 조정을 마친 다음 파일을 준비한다.
# Makefile arcmsr.h arcmsr_attr.c arcmsr_hba.c

make -C "/usr/src/kernels/$KVER" \
    M="$BUILD_DIR" \
    CONFIG_SCSI_ARCMSR=m \
    modules

modinfo "$BUILD_DIR/arcmsr.ko"
```

먼저 `insmod` 로 현재 boot에서 controller와 RAID volume이 정상적으로 인식되는지 확인했다.

```
sudo insmod "$BUILD_DIR/arcmsr.ko"

lspci -nnk -s 17:00.0
lsblk -e7 -o NAME,SIZE,TYPE,FSTYPE,UUID,MOUNTPOINTS,MODEL
```

`ARC-1284-VOL#000` 과 partition UUID가 확인된 후 module을 영구 경로에 설치하고 boot 때 자동으로 load되도록 등록했다.

```
KVER=$(uname -r)

sudo install -D -m 0644 "$BUILD_DIR/arcmsr.ko" \
    "/lib/modules/$KVER/extra/arcmsr.ko"
sudo depmod -a "$KVER"

printf 'arcmsr\n' | sudo tee /etc/modules-load.d/arcmsr.conf >/dev/null
```

현재 설치된 module은 다음 상태다.

```
path=/lib/modules/6.12.0-250.el10.x86_64/extra/arcmsr.ko
version=v1.51.00.14-20230915
sha256=8c78e3bb666abc49a17419904ab5911e0fed0c9cb885f0e2c18333b118d7865e
driver=arcmsr
volume=ARC-1284-VOL#000
```

이 module은 RPM 소유 파일이 아니며 서명되지 않은 외부 module이다. 현재 server는 Secure Boot가 disabled라 load됐고 kernel log에는 out-of-tree module taint가 기록된다.

kernel 업데이트 주의: 새 kernel을 설치하면 그 version에 맞는 `kernel-devel` 로 `arcmsr.ko` 를 다시 빌드해 새 kernel의 `/lib/modules/<새-version>/extra/` 에 설치하고 `depmod` 를 실행해야 한다. 이를 준비하지 않고 새 kernel로 reboot하면 ARC-1284 volume이 나타나지 않아 `/294t` mount가 실패할 수 있다.

XFS mount 복원

driver load 후 RAID volume은 `/dev/sda` , 기존 XFS partition은 `/dev/sda1` 로 나타났다. 데이터 디스크를 포맷하지 않고 다음 fstab 항목으로 다시 연결했다.

```
UUID=2a417935-7537-4301-bbcf-5ab2ca836af5 /294t xfs defaults 1 2
```

2026-07-22 확인 결과는 `/dev/sda1` → `/294t` , XFS, `rw` 다. 이 조치에서는 `/294t` 를 다시 포맷하지 않았다.

```
lsblk -e7 -o NAME,SIZE,FSTYPE,UUID,MOUNTPOINTS,MODEL
findmnt -rn -o TARGET,SOURCE,FSTYPE,OPTIONS /294t
lspci -nnk -s 17:00.0
lsmod | grep '^arcmsr'
```

2026-07-22 이후 kernel log에는 일부 allocation group의 AGFL을 reset했고 block이 leak됐다는 XFS warning도 기록됐다. 이는 `arcmsr` load와 `/294t` mount 성공 여부와 별개인 filesystem metadata 점검 항목이다. 운영 export가 연결된 상태에서 `xfs_repair` 를 실행하지 말고, 별도 유지보수 창에 NFS를 중단하고 `/294t` 를 unmount한 뒤 offline 점검·복구해야 한다.

3.3 LAB2와 lab-storage : AD 설정과 SSSD·keytab 복원

LAB2: AD server와 storage principal 준비

LAB2는 `LAB.DECS.INTERNAL` realm의 Samba AD DC와 KDC 역할을 한다. storage망 주소는 `100.100.100.102` 이며, `lab-storage` 의 Kerberos 설정에서 KDC와 admin server로 사용한다.

LAB2 AD에서 `lab-storage` machine account와 NFS service principal을 확인한 뒤 keytab을 새로 생성했다. OS 재설치 전 keytab을 backup해서 복원하지 않고 AD를 원본으로 재발급한 것이다.


```
LAB-STORAGE$@LAB.DECS.INTERNAL
nfs/lab-storage.lab.decs.internal@LAB.DECS.INTERNAL
```

생성된 keytab의 machine principal과 NFS service principal KVNO는 모두 2였다.

lab-storage : SSSD와 keytab 설치·검증

lab-storage 는 Kerberos realm을 LAB.DECS.INTERNAL , KDC와 admin server를 LAB2의 100.100.100.102 로 설정했다. SSSD는 다음 정책으로 AD의 RFC2307 UID/GID를 그대로 조회한다.

```
[domain/lab.decs.internal]
ad_domain = lab.decs.internal
krb5_realm = LAB.DECS.INTERNAL
id_provider = ad
access_provider = permit
ldap_id_mapping = False
use_fully_qualified_names = False
```

LAB2에서 재생성한 keytab은 lab-storage 의 /etc/krb5.keytab 에 root 전용 권한으로 설치했다.

lab-storage 에서 machine principal로 TGT를 받은 뒤 NFS principal을 keytab과 대조한 결과는 다음과 같다.

```
nfs/lab-storage.lab.decs.internal@LAB.DECS.INTERNAL: kvno = 2, keytab entry valid
```

NFS service principal 자체를 client principal처럼 kinit -k 하는 것은 이 구성의 검증 방법이 아니다. 아래 5.1절처럼 machine principal로 먼저 kinit 한 뒤 kvno -k 를 사용한다.

3.4 lab-storage : NFS service와 export 복원

nfs-server , sssd , gssproxy 를 활성화하고 NFSv4 port 2049/tcp 와 관련 firewall service를 복원했다. 2026-07-22 확인 시 nfsd thread는 16개였고 nfs-server , sssd , gssproxy 가 모두 active였다.

Kerberos 대상 export는 다음 상태다.

경로	client	security flavor
/294t/dcloud/share	100.100.100.101 ~ 110	krb5p:krb5i:krb5
/294t/share/test-krb	지정 시험 client	krb5p
/294t/health/nfs-gss-canary	100.100.100.101 ~ 110	krb5 , read-only

기존 sec=sys 전용 경로는 별도 호환 export로 유지했다. Kerberos export가 정상이라는 이유로 기존 sec=sys export를 임의로 삭제하지 않는다.

3.5 LAB1~LAB10: 기존 fstab mount 복원

client별 mount 설정을 새로 생성하지 않고 각 서버에 이미 있던 `/etc/fstab` 을 사용했다. mount가 없는 client에서 만 `sudo mount -a` 를 실행해 `/home/takoN/share` 를 다시 연결했다.

2026-07-22 재점검에서 LAB1~LAB10 모두 다음 공통 상태를 보였다.

```
source=lab-storage.lab.decs.internal:/294t/dcloud/share
fstype=nfs4
vers=4.1
hard
sec=krb5
addr=100.100.100.100
```

최초 병렬 점검 중 LAB7에서 짧은 D-state process 1개가 관측됐지만 즉시 수행한 local 상태 재확인에서는 사라졌다. 같은 시점에 NFS RPC task, transport pending, ForeChannel waiter와 NFS D-state가 모두 0이었고 recovery 상태도 `healthy` 였다. 이를 지속된 session slot 고착으로 판정하지 않았다.

3.6 LAB8 운영 mount: 15초 `rpc.idmapd` 지연 시험

과거 실행 기록: 아래 내용은 안전장치를 준비한 뒤 한 번 수행한 운영 검증 결과다. 일반 점검 절차가 아니며 이 문서만 보고 운영 storage에서 반복 실행하지 않는다.

격리 VM 결과와 별도로 현재 운영 중인 storage kernel에서 같은 idmap 지연 경로가 영구 slot 고착을 만들지 않는지 직접 확인했다.

항목	시험 조건
NFS client	LAB8 host의 운영 mount <code>/home/tako8/share</code>
mount	NFSv4.1, <code>sec=krb5</code> , <code>hard</code>
NFS server	<code>lab-storage.lab.decs.internal</code>
storage kernel	<code>6.12.0-250.el10.x86_64</code>
workload	시험 container 내부에서 새 GID <code>424242</code> 로 test file에 <code>chown</code>
idmap fault	storage의 <code>rpc.idmapd</code> 응답을 15초 지연
안전장치	15초 자동 재개 timer와 독립적인 30초 이중 자동 재개 timer

지연 중에는 `chown` 이 `rpc_wait_bit_killable` 경로에서 일시적으로 D-state에 들어갔다. 15초 자동 재개 후 worker는 더 이상 D-state에 남지 않고 종료됐다. 원래 `chown` 의 application 결과는 `EINVAL` 이었다.

```
/bin/chown: changing group of '.../chown-target': Invalid argument
```

이 `EINVAL` 은 “`chown` 성공”을 의미하지 않는다. 하지만 취약 kernel에서처럼 동일 요청이 `INUSE` slot에 계속 걸려 영구 D-state로 남지도 않았다. 이후 test file 삭제 RPC가 성공했고 client reboot, unmount, NFS service restart 없이 정상 상태로 돌아왔다. 후속 확인 결과는 다음과 같았다.

판정 항목	결과
지속 NFS D-state	0
NFS RPC task	0
transport pending	0
ForeChannel waiter	0
<code>lease_expired</code>	0
후속 NFS 작업	test file 삭제 성공

따라서 이 시험은 운영 kernel `6.12.0-250.el10` 에서 15초 idmap 지연이 일시적인 RPC 대기를 만들 수는 있지만 **NFSv4.1 session slot**을 영구 고착시키지는 않았음을 보여준다. 원래 operation의 application 결과와 session recovery 판정은 분리해야 한다.

4. 해결 판정 기준

이번 조치는 다음을 모두 만족해 운영 적용 완료로 판정했다.

- `lab-storage` 가 취약한 `3.10.0-862.el7` 이 아니라 `6.12.0-250.el10` 으로 부팅돼 있다.
- `/294t` 가 기존 XFS data disk에서 `rw` 로 mount돼 있다.
- LAB2 machine principal의 TGT 발급과 NFS service principal의 keytab 검증이 성공한다.
- SSSD에서 AD 사용자의 RFC2307 UID/GID가 조회된다.
- `nfs-server` , `sssd` , `gssproxy` 가 active이고 NFSv4 port가 listen한다.
- Kerberos NFS export가 LAB1~LAB10 storage망 주소에 열려 있다.
- LAB1~LAB10에서 기존 fstab의 NFSv4.1 `sec=krb5` mount가 확인된다.
- 지속되는 NFS D-state, RPC task, ForeChannel waiter 또는 lease expiry가 없다.
- LAB8 운영 mount의 15초 idmap 지연 후에도 workload가 영구 D-state에 남지 않고 후속 NFS RPC가 성공한다.

이 판정은 “2026-06-29 LAB5 장애의 최초 원인이 반드시 이 bug였다”는 뜻은 아니다. 당시 시작 시점의 storage trace가 없어 과거 장애 원인은 여전히 강한 후보로 남는다. 이번에 확정한 것은 구형 storage kernel에 실제 slot leak 경로가 있었고, 그 kernel을 운영에서 제거했다는 사실이다.

5. 안전한 사후 검증

5.1 작업 서버: `lab-storage`

```

hostname -f
uname -r
findmnt -rn -o TARGET,SOURCE,FSTYPE,OPTIONS /294t

systemctl is-active nfs-server sssd gssproxy
sudo exportfs -v
sudo ss -lnt | grep ':2049 '

DECS_KRB5CC_DIR=$(mktemp -d /tmp/decs-storage-kvno.XXXXXX)
DECS_KRB5CC="FILE:$DECS_KRB5CC_DIR/ccache"

sudo env KRB5CCNAME="$DECS_KRB5CC" \
  kinit -k -t /etc/krb5.keytab 'LAB-STORAGE$@LAB.DECS.INTERNAL'
sudo env KRB5CCNAME="$DECS_KRB5CC" \
  kvno -k /etc/krb5.keytab \
  nfs/lab-storage.lab.decs.internal@LAB.DECS.INTERNAL
sudo env KRB5CCNAME="$DECS_KRB5CC" kdestroy
rmdir "$DECS_KRB5CC_DIR"

```

기대 kernel은 6.12.0-250.el10.x86_64 이상이며, kvno 결과에는 keytab entry valid 가 나와야 한다.

5.2 작업 서버: 각 NFS client LAB1~LAB10

아래의 N 은 현재 접속한 LAB 서버 번호로 바꾼다. 이미 mount돼 있으면 반복 remount하지 않는다. mount가 없고 D-state도 없을 때만 기존 fstab으로 mount한다.

```

ps -e -o state=,pid=,etimes=,comm=,wchan:48=,args= \
  | awk '$1 ~ /^D/ {print}'

findmnt -T /home/takoN/share -o TARGET,SOURCE,FSTYPE,OPTIONS

# findmnt에 결과가 없고 D-state process도 없을 때만 실행
sudo mount -a
findmnt -T /home/takoN/share -o TARGET,SOURCE,FSTYPE,OPTIONS

```

container의 실제 사용자 권한까지 확인할 때는 container 안에서 해당 사용자로 실행한다. 다른 사용자의 홈이나 파일에 임의로 chown 하지 않는다.

```
id
ls -al "$HOME/share" | sed -n '1,40p'

probe="$HOME/share/.nfs-post-upgrade-check.${hostname -s}.${}"
printf 'nfs post-upgrade check\n' >"$probe"
grep -Fx 'nfs post-upgrade check' "$probe"
ls -ln "$probe"
rm -f "$probe"
```

`ls -al`의 소유자·그룹이 `nobody`가 아니라 AD의 사용자·그룹 이름으로 보이고, 생성·읽기·삭제가 모두 성공해야 한다.

6. 같은 증상이 다시 보일 때

NFS `hard` mount의 process가 D-state에 들어가면 NFS 경로를 읽는 `find`, `du`, `stat`, 반복 canary를 추가 실행하지 않는다. 새로운 NFS request와 D-state process를 더 만들 수 있다.

1. 먼저 client에서 `uname -r`, D-state stack, `/proc/net/rpc/nfs`, forensics와 recovery state를 local filesystem에서 수집한다.
2. `lab-storage`가 실수로 구형 kernel로 부팅되지 않았는지 확인한다.
3. 자동 remount, 강제 unmount, `rpc-gssd` 반복 restart를 중단한다.
4. storage의 network, nfsd, gssproxy와 session trace를 함께 확인한다.
5. 2026-07-23의 승인된 단일 시험을 일반 점검처럼 반복하지 않는다. 추가 재실험은 LAB8의 격리 VM harness에서 수행한다.
6. client reboot는 고착된 client state를 지우는 복구 수단일 뿐 근본 해결로 기록하지 않는다.

운영 mount를 `soft`로 바꾸는 것도 해결책으로 사용하지 않는다. timeout을 application error로 노출하고 data integrity 위험을 만들 수 있다.

7. 증거와 관련 문서

- 원인 재현과 kernel matrix
- 실제 LAB8 재현 결과
- 격리 VM 회귀 시험 절차
- LAB storage OS 재설치 후 복원 절차

Backend

원본: [md/backend/index.md](#), [md/backend/시작.md](#), [md/backend/개요.md](#), [md/backend/시스템-아키텍처.md](#), [md/backend/도메인-설명.md](#), [md/backend/코드-컨벤션.md](#), [md/backend/핵심-설계-패턴.md](#), [md/backend/인증-보안.md](#), [md/backend/외부-연동.md](#), [md/backend/Redis-키-카탈로그.md](#), [md/backend/에러-코드-카탈로그.md](#)

사용자가 서버 사용을 신청하고 관리자가 승인 버튼을 누르면, 그 뒤로는 사람 손을 거치지 않고 Ubuntu 계정 생성부터 컨테이너 배정, 만료 알림까지 전부 자동으로 처리됩니다. Admin BE(Spring Boot)는 그 자동화를 실제로 돌리는 서버 이고, 이 섹션은 그 코드를 넘겨받아 유지보수해야 하는 사람을 위한 문서입니다.

GitHub 레포: [CSID-DGU/admin_be](#)

읽는 순서

처음 합류했다면 아래 순서대로 읽습니다. 뒤에 나오는 문서일수록 앞 문서에서 나온 개념을 이미 안다고 가정하고 설명합니다.

순서	문서	이 문서가 하는 일
1	시작	로컬에서 서버를 처음 띄우기까지 순서대로 안내
2	개요	이 시스템이 왜 필요했고 무엇으로 만들어졌는지 소개
3	시스템 아키텍처	신청 버튼을 누른 순간부터 계정이 만들어지고 나중에 자동으로 정리되기까지, 실제로 일어나는 일을 순서대로 설명
4	도메인 설명	그 과정에서 다루는 데이터가 DB에 어떤 모양으로 저장되는지 설명 (ERD 포함)
5	코드 컨벤션	코드를 이런 방식으로 짜는 이유와 규칙 정리
6	핵심 설계 패턴	여러 도메인에서 반복해서 부딪히는 문제와 그 해결책 정리

참고자료

아래 문서들은 순서대로 읽기보다, 필요할 때 찾아보는 문서입니다.

문서	이 문서가 하는 일	이럴 때 읽으세요
인증·보안	로그인 이후 요청마다 사용자를 어떻게 증명하고 검증하는지 설명	로그인이 안 되거나 토큰 재발급 오류를 조사할 때
외부 연동	BE가 계정·컨테이너 작업을 넘기는 Infra Server, 지표를 가져오는 Prometheus와의 통신 방식 정리	Infra Server와 통신하는 코드를 수정하거나 디버깅할 때
Redis 키 카탈로그	BE가 쓰는 Redis 키를 패턴·TTL·저장과 삭제 시점까지 정리	Redis 관련 버그를 조사하거나 새 키를 추가할 때

문서	이 문서가 하는 일	이럴 때 읽으세요
에러 코드 카탈로그	API가 던지는 에러 59개를 상황별로 정리	API 에러 메시지의 원인을 조사하거나 새 에러 케이스를 추가할 때
운영 가이드	배포·장애 대응 등 운영 중 실행해야 할 절차 정리	배포하거나 운영 중 장애에 대응할 때

API 명세

컨트롤러에 붙은 Swagger 어노테이션에서 자동 생성되는 API 문서입니다. 요청·응답 형식을 코드 없이 바로 확인하고, 직접 호출도 해볼 수 있습니다.

항목	값
Swagger UI	http://210.94.179.18:30083/swagger-ui/index.html
Base URL	http://210.94.179.18:30083

시작

로컬 개발 환경을 처음 세팅할 때 따라야 할 순서를 안내합니다. IntelliJ 설치 확인부터 `application.yml` 준비, 로컬 DB 연결, 인코딩 설정, Swagger 확인까지 다룹니다. 신규 관리자는 이 문서부터 진행한 뒤 [개요](#)로 넘어갑니다.

1. 기술 스택 확인

- IDE: IntelliJ. 학생 계정이 있다면 Ultimate 버전 사용을 권장합니다.
- Java Version: Java 17. 로컬에 설치되어 있어야 합니다.
- Framework: Spring Boot. 별도 설정은 필요 없습니다.

2. 설정 파일(`application.yml`) 준비

보안상 설정 파일은 레포지토리에 포함되어 있지 않습니다.

- 관리자용 [Notion > BE 페이지 > application.yml 페이지](#)에서 `application.yml` 내용을 전체 복사합니다.
- 프로젝트 경로 `src/main/resources/`에 `application.yml` 파일을 새로 만들고 내용을 붙여넣습니다.
- `application.yml`을 수정할 때는 이 Notion 페이지도 함께 업데이트하고, PR 본문에도 수정 사실을 명시합니다.
- `application.yml`은 `.gitignore`에 등록되어 있어 커밋되지 않습니다.

3. 로컬 DB 및 프로파일 설정

로컬 MySQL에 `web-admin` 데이터베이스를 생성합니다. 사용자·비밀번호·포트는 자유롭게 설정합니다.

```
CREATE DATABASE `web-admin`;
```

`application.yml`의 `local` 프로파일 섹션에서 본인 로컬 DB의 `username`과 `password`가 맞는지 확인하고, 파일 맨 위의 `spring.profiles.active` 설정이 `local`로 되어 있는지 확인합니다.

```
server:
  port: 8080

spring:
  profiles:
    active: local #local, prod 중 선택
```

예시 설정입니다.

```
# 로컬 환경 설정(본인 로컬 환경에 맞게 수정)
---
api:
  base-url: http://localhost:8080

logging:
  level:
    org.hibernate.SQL: debug
    org.hibernate.type.descriptor.sql.BasicBinder: trace
spring:
  config:
    activate:
      on-profile: local
  datasource: # 본인 로컬 DB 설정
    url: jdbc:mysql://localhost:3306/web_admin?
serverTimezone=Asia/Seoul&useSSL=false&allowPublicKeyRetrieval=true
    username: root
    password: toni1234
```

4. 의존성 로드 (Gradle)

IntelliJ 우측의 Gradle 탭에서 Reload All Gradle Projects를 실행해 필요한 라이브러리를 모두 다운로드합니다.

5. 인코딩 설정

`message.properties`의 한글이 깨지지 않도록 아래 설정을 완료합니다.

- Settings(`Ctrl+Alt+S` 또는 `Cmd+,`) → Editor → File Encodings로 이동합니다.
- Properties Files (`*.properties`) 설정에서 Default encoding을 UTF-8로, Transparent native-to-ascii conversion을 체크합니다.
- Apply 후 확인합니다.

6. 애플리케이션 실행

`src/main/java/.../AdminBeApplication.java` (메인 클래스)를 찾아 Run으로 실행합니다. 콘솔에 에러 없이 로그가 올라오는지 확인합니다.

Mac OS에서 발생하는 netty 에러(`MacOSDnsServerAddressStreamProvider`)는 시스템 기본값으로 대체되어 로컬 실행에 문제가 없으므로 무시해도 됩니다.

7. Swagger 확인

Swagger는 FE 개발자와 API 명세를 공유하기 위한 문서입니다. `http://localhost:8080/swagger-ui/index.html#/`에 접속해 확인합니다.

접속되지 않으면 `application.yml`에 설정한 포트 번호와 8080이 일치하는지 확인합니다. 8080을 이미 사용 중이라면 해당 프로세스를 종료하거나 다른 포트를 사용합니다.

8. 문제가 발생했을 때

- 한글이 깨져서 보인다면: 5번 설정을 다시 확인하고, 이미 깨진 파일은 원본 텍스트로 다시 붙여넣습니다.
- DB 연결 에러가 난다면: `web-admin` 스키마 생성 여부와 비밀번호를 확인합니다. 해결되지 않으면 같은 포트에서 다른 DB가 실행 중인지 확인합니다.
- 그 외 문제는 팀원에게 문의합니다.

세팅을 마쳤다면 개요로 이동해 시스템 전체 구조를 확인합니다.

개요

코드를 보기 전에 이 시스템이 왜 만들어졌고 Admin BE가 그 안에서 무슨 몫을 맡는지부터 잡고 갑니다. 관리자의 반복 업무를 자동화한다는 목적, 그리고 그게 LAB-FARM이라는 서로 다른 서버 환경 때문에 왜 필요해졌는지를 먼저 다룹니다. 여기서 그림이 잡혀야 뒤에 나오는 개별 기능들이 왜 그렇게 설계됐는지 이해가 됩니다.

1. 시스템 소개 및 목적

동국대학교 AI LAB 자동화 시스템은 연구실 GPU 서버 자원 관리와 컨테이너 생성 자동화를 위한 시스템입니다. 기존에는 연구실 인원의 서버 사용 요청을 관리자가 수동으로 처리했으나, 신청 접수부터 계정 생성, 만료 처리까지 일련의 과정을 자동화하여 관리자 업무 부담을 줄이는 것을 목적으로 합니다.

연구실 인원(학생, 연구원)은 AI/ML 연구를 위해 GPU 자원이 필요합니다. 여러 인원이 한정된 수의 서버를 공유하므로, 각 사용자에게 독립된 컨테이너 환경을 제공합니다. 컨테이너는 K8s Pod로 생성되며, 사용자는 승인 후 발급받은 SSH 포트와 Jupyter 포트에 접속해 사용합니다.

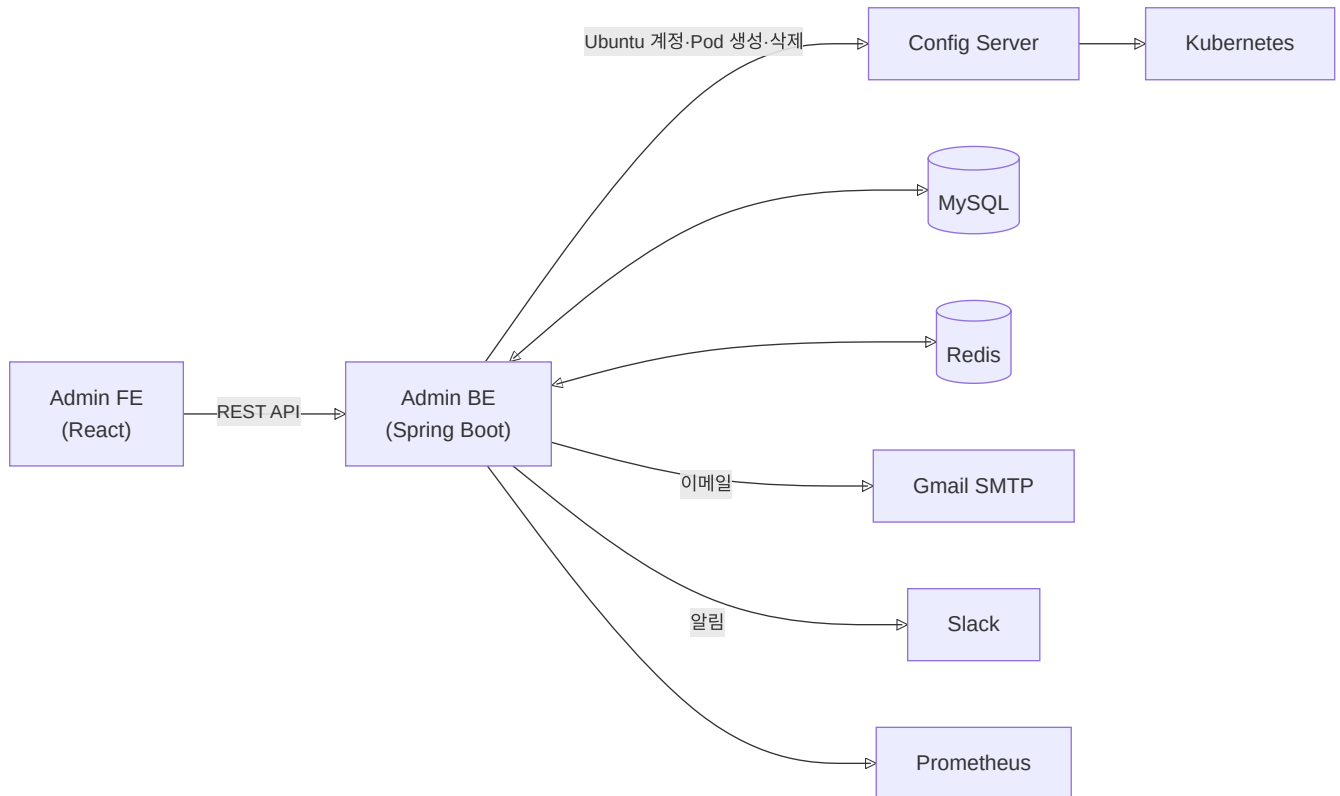
서버실에는 LAB과 FARM 두 개의 서버가 VLAN으로 분리되어 운영됩니다. 각 서버마다 탑재된 GPU 종류가 다르기 때문에, 사용자 신청 시 어느 서버에 컨테이너를 배치할지 적절히 오케스트레이션해야 합니다. 이처럼 이기종 서버 환경을 수동으로 관리하면 실수가 잦고 확장이 어렵기 때문에, 신청 접수부터 컨테이너 배치·회수까지 전 과정을 자동화하는 시스템이 필요합니다.

Admin BE는 이 자동화 시스템의 백엔드 서버로, 다음 역할을 담당합니다.

- 사용자/관리자 인증 및 권한 관리
- 서버 사용 신청 접수, 승인/거절 처리
- 승인 시 인프라 서버에 Ubuntu 계정 및 K8s 컨테이너 생성 요청
- 만료일 기반 컨테이너 자동 회수 및 사용자 알림 발송
- 장기 미접속 계정 자동 비활성화

본 문서는 Admin BE 인수인계 및 유지보수를 위한 기술 문서입니다.

2. 전체 구조



FE ↔ BE: 사용자·관리자의 모든 요청(신청, 승인, 조회 등)은 REST API로 들어옵니다.

BE → Infra: 승인 시 Infra Server에 Ubuntu 계정 생성과 K8s Pod 생성을 요청합니다. 만료·삭제 시에는 반대로 정리를 요청합니다.

BE → 알림: 승인, 만료 예고, 삭제 완료 등 주요 이벤트 발생 시 Gmail로 이메일을, Slack으로 알림을 보냅니다. Slack 알림은 Redis 큐를 통해 비동기로 발송합니다.

MySQL은 신청 내역, 사용자 정보, 포트 매핑 등 운영 데이터를 저장합니다. Redis는 JWT RefreshToken, Slack 알림 큐, 이메일 중복 방지에 사용합니다.

3. 기술 스택

- **Java 17:** 서버 사이드 애플리케이션 개발 언어
- **Spring Boot 3.5.3:** REST API 서버 구현, DI 컨테이너, 자동 설정
- **Spring Data JPA (Hibernate):** DB 접근 추상화. 엔티티 매핑 및 쿼리 처리에 사용
- **MySQL 8.0.32:** 신청, 사용자, 컨테이너 상태 등 모든 영속 데이터 저장
- **Redis:** JWT RefreshToken 저장, Slack 알림 비동기 큐, 이메일 중복 방지(SETNX)
- **JWT (jjwt 0.11.5):** 사용자 인증 토큰 생성 및 검증
- **Gradle 8.14.2:** 빌드 및 의존성 관리
- **Docker / Kubernetes / Helm (Fabric8 Kubernetes Client 6.10.0):** 컨테이너 패키징 및 K8s 클러스터 배포

- **JavaMailSender (Gmail SMTP)**: 사용자 이메일 알림 발송
- **Slack Webhook / DM API**: 관리자 채널 알림 및 사용자 DM 발송
- **Swagger (springdoc-openapi 2.8.0)**: REST API 명세 자동 문서화

4. 서버 환경

서버는 LAB(210.94.179.18)과 FARM(210.94.179.19) 두 곳을 운영합니다. 두 서버는 VLAN으로 네트워크가 분리되어 있으며, 각 서버마다 탑재된 GPU 종류가 상이합니다. LAB 서버는 김지희 교수님 연구실에서 사용 중입니다.

Admin BE는 K8s 네임스페이스 `default` 에 배포되며, NodePort 30083을 통해 외부에서 접근합니다(Pod 내부 포트 :8080). MySQL과 Redis는 `ailab-be` 네임스페이스에 있습니다.

- **API Base URL**: `http://210.94.179.18:30083`
- **Swagger UI**: `http://210.94.179.18:30083/swagger-ui/index.html`

클러스터 내부 서비스 주소는 아래와 같습니다.

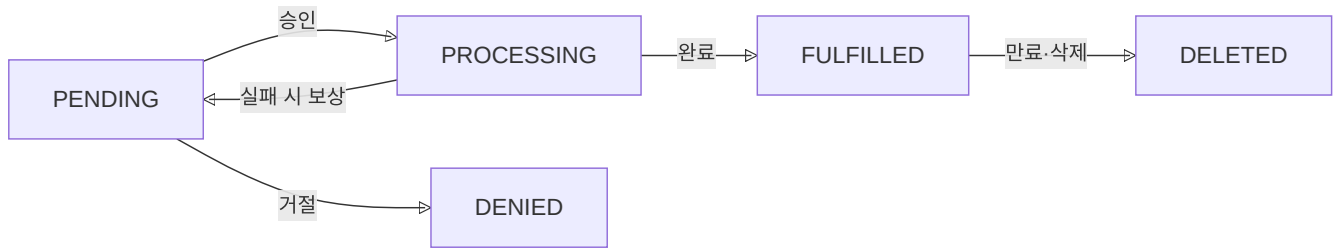
서비스	주소
MySQL	<code>my-mysql.ailab-be.svc.cluster.local:3306</code>
Redis	<code>my-redis-master.ailab-be.svc.cluster.local:6379</code>
Infra Server	<code>containerssh-config-service.ailab-infra.svc.cluster.local</code>

시스템 아키텍처

Request가 신청부터 승인·거절·만료까지 어떤 상태를 거치는지, 그리고 회원가입·로그인·신청·승인·삭제·만료 각 단계에서 FE·BE·MySQL·Redis·Infra Server·Gmail·Slack이 어떤 순서로 통신하는지를 시퀀스 다이어그램으로 정리합니다. 세 가지 스케줄러(만료 처리, 사용자 수명주기, Slack 큐 Worker)의 동작 방식과 BE·FE 내부 패키지 구조도 함께 다룹니다.

1. Request 생명주기

사용자가 서버 사용을 신청하면 Request가 생성되고, 관리자 승인 여부와 컨테이너 운영 상태에 따라 아래 상태를 따라 전이됩니다. PROCESSING은 승인 처리 중 외부 서버 호출이 실패했을 때 PENDING으로 되돌아오는 보상 처리를 위해 존재합니다.



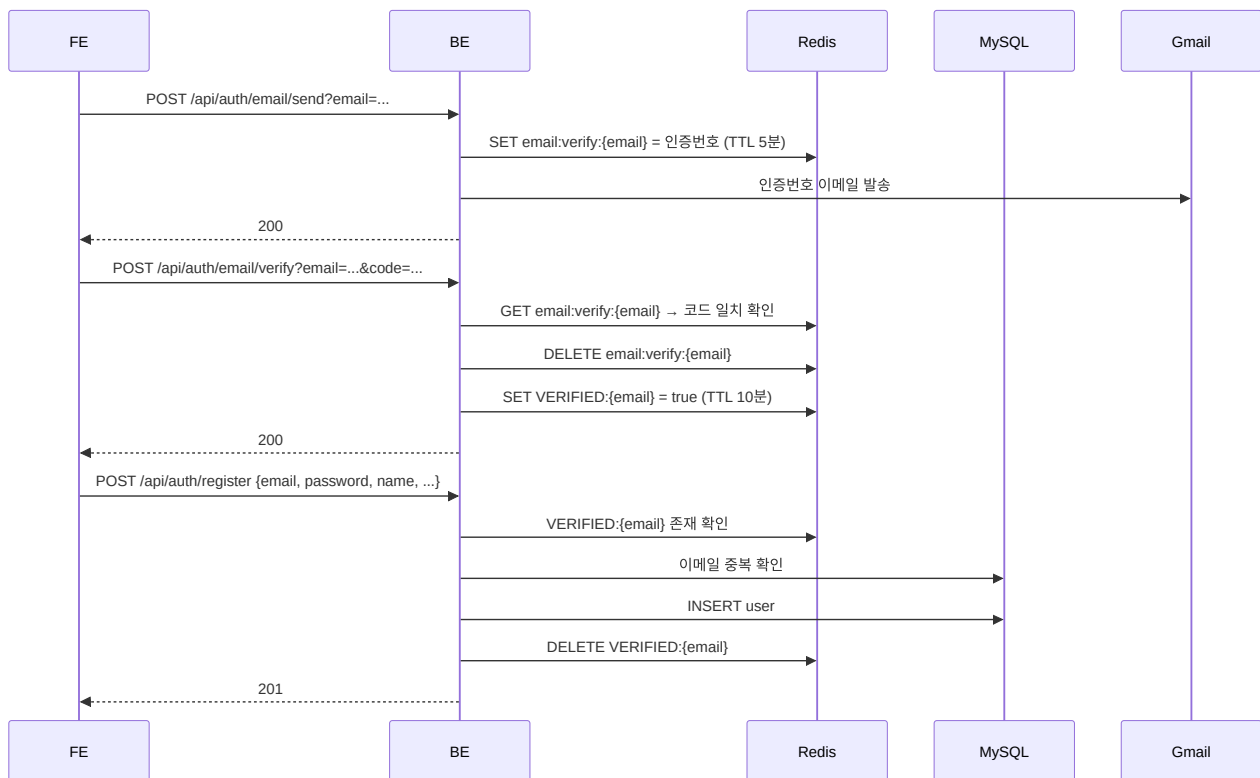
상태	설명
PENDING	신청 접수, 관리자 승인 대기
PROCESSING	승인 진행 중. 중복 승인 방지를 위해 사용
FULFILLED	승인 완료, 컨테이너 운영 중
DENIED	관리자 거절
DELETED	만료 또는 수동 삭제

2. 전체 흐름

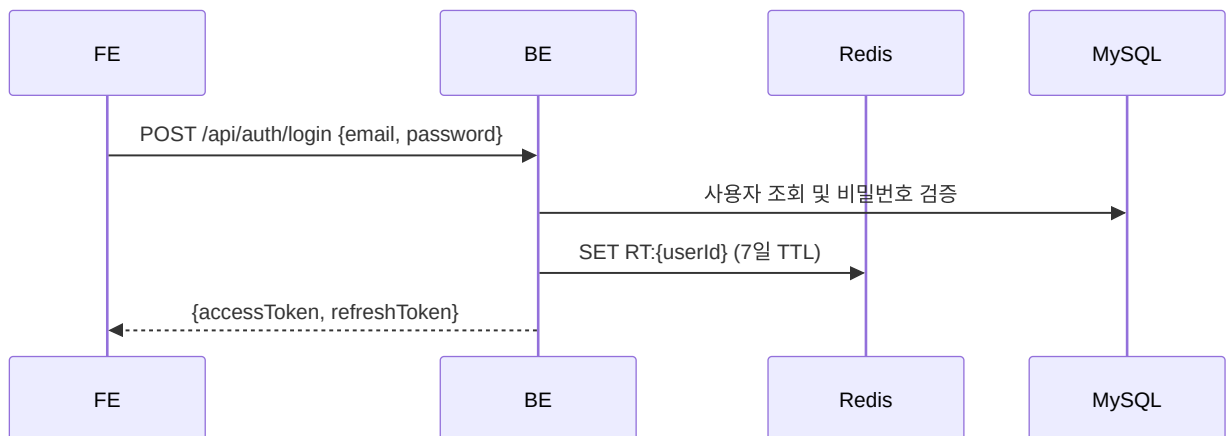
로그인부터 신청, 승인, 만료 처리까지 FE·BE·Infra Server 세 주체 간 통신 흐름입니다. 승인 과정에서는 Infra Server 에 Ubuntu 계정 생성과 Pod 생성을 순서대로 요청하며, 하나라도 실패하면 이전 단계를 되돌리는 보상 트랜잭션이 실행됩니다. 만료 처리는 스케줄러가 매일 08:00에 자동으로 수행합니다.

회원가입

이메일 인증 후 회원가입 순서입니다. 인증번호는 Redis에 5분 TTL로 저장하고, 인증 완료 상태(`VERIFIED:{email}`)는 10분간 유지됩니다. 회원가입 시 이 키가 있는지 확인하고, 저장 후 삭제합니다.

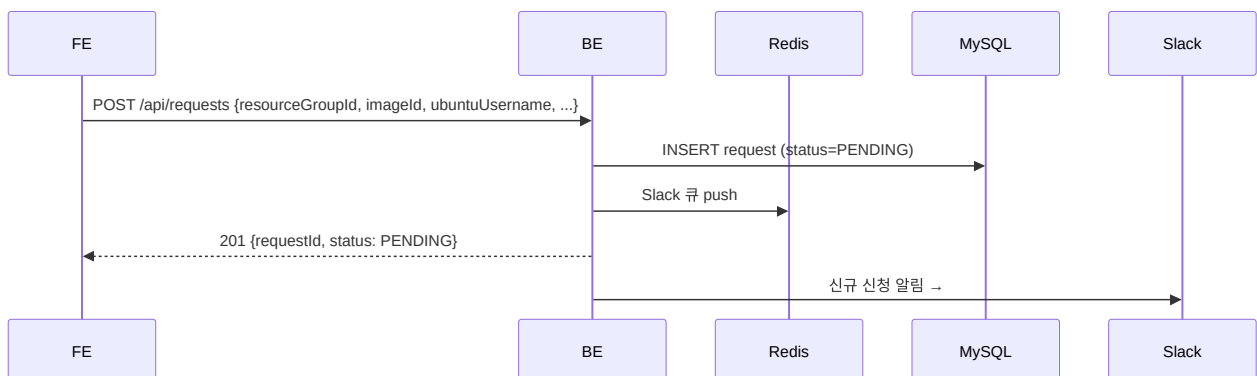


로그인

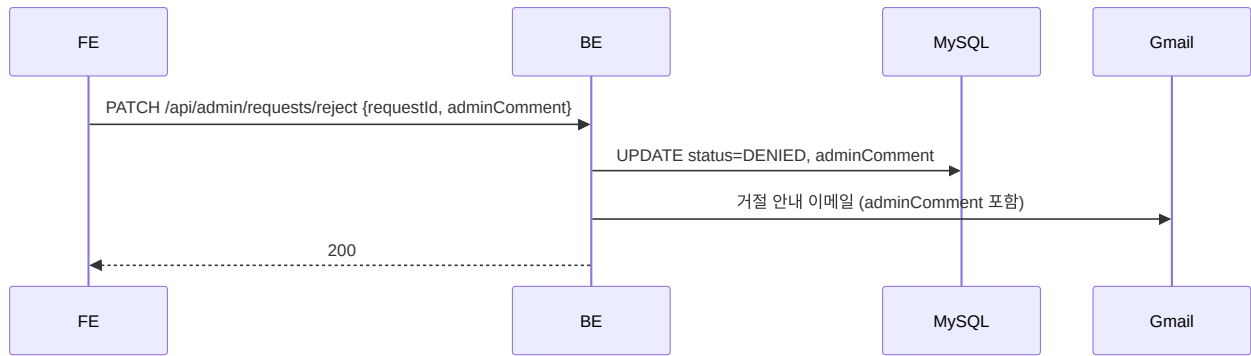


JWT 토큰 구조, 재발급, 로그아웃 흐름의 상세는 [인증·보안](#)을 참고합니다.

서버 사용 신청

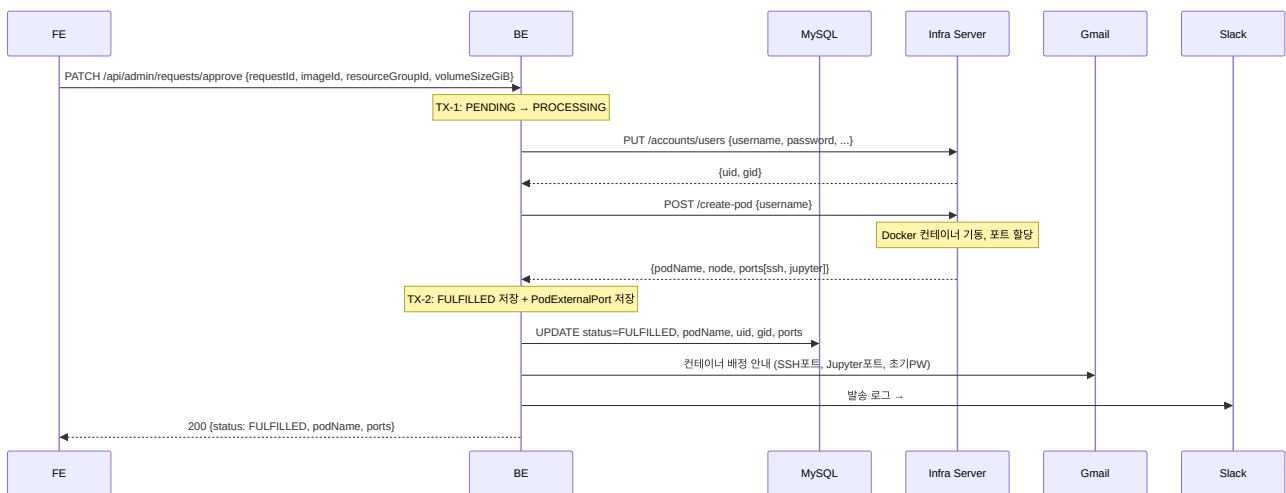


관리자 거절



이메일 발송은 try-catch로 감싸져 있어, 발송이 실패해도 거절 처리 자체는 그대로 완료됩니다. 실패 시 `AdminRequestCommandService` 에 WARN 로그만 남습니다.

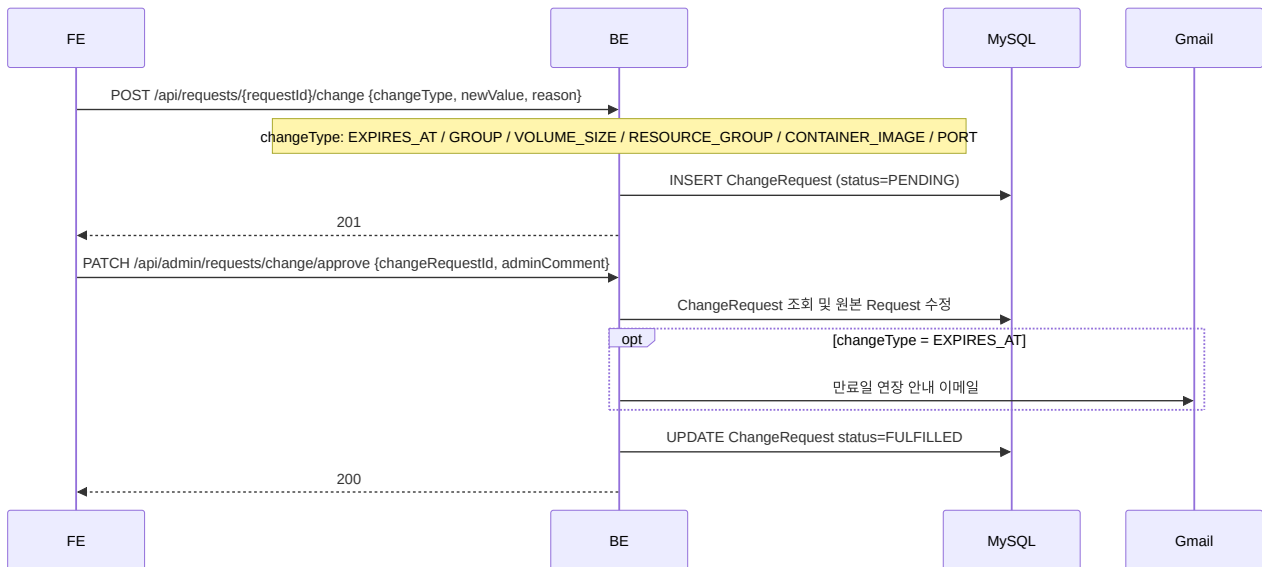
관리자 승인



Infra Server API 구조와 WebClient 타임아웃 설정 상세는 [외부 연동](#)을 참고합니다.

변경 요청 (만료일 연장·그룹 변경 등)

FULFILLED 상태인 컨테이너에 대해 사용자가 만료일 연장, 그룹 변경, 볼륨 크기 변경 등을 요청할 수 있습니다. 변경 요청은 별도 ChangeRequest 테이블에 PENDING으로 저장되고, 관리자가 승인하면 원본 Request에 반영됩니다. 만료일 연장이 승인된 경우에만 이메일이 발송됩니다.

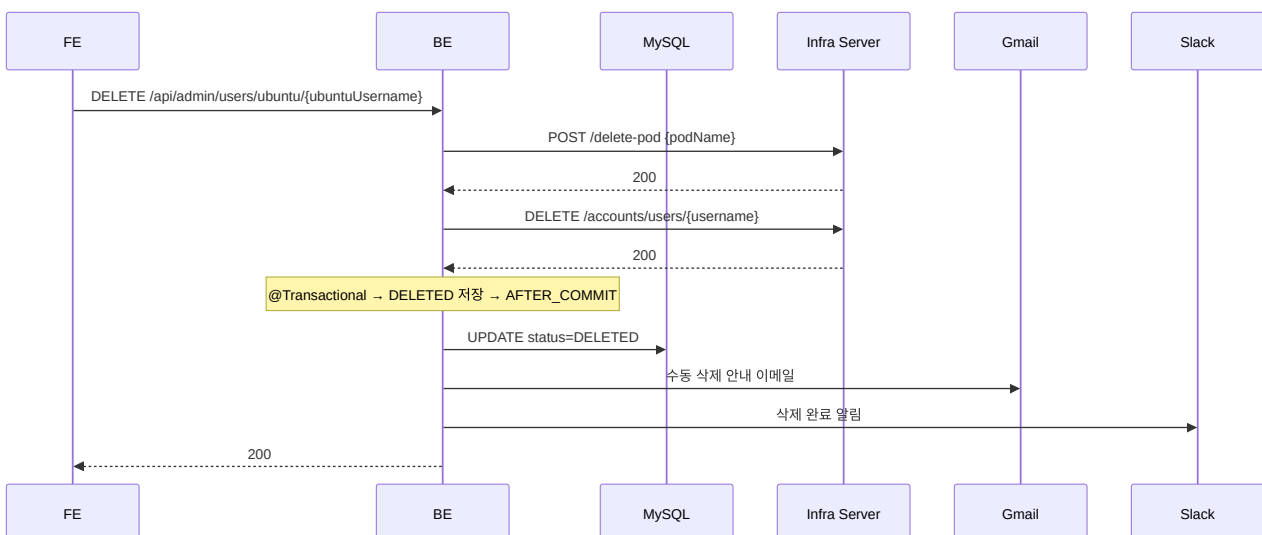


변경 요청이 거절되는 경우(`PATCH /api/admin/requests/change/reject`)에는 changeType과 무관하게 항상 거절 안내 이메일이 발송됩니다. 승인 흐름과 달리 EXPIRES_AT 조건이 없습니다.

관리자 수동 삭제

FULFILLED 상태인 컨테이너를 관리자가 직접 삭제합니다. 스케줄러 자동 삭제와 같은 Infra 호출·이메일 흐름을 따르지만, 즉시 처리된다는 점이 다릅니다.

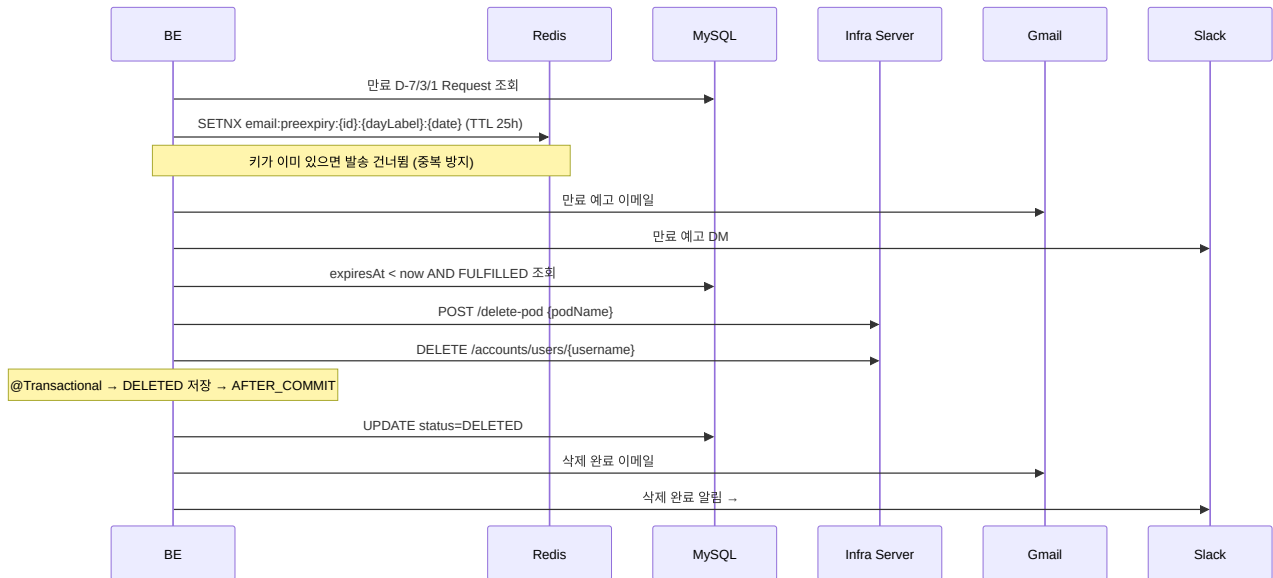
삭제 API는 두 가지입니다. 컨테이너 한 개만 지정해서 지우는 `DELETE /api/admin/users/ubuntu/{ubuntuUsername}` 과, 사용자 단위로 그 사람이 가진 FULFILLED 컨테이너를 모두 지우는 `DELETE /api/admin/users/{id}` (`AdminUserService.deleteUser()`)입니다. 아래 시퀀스는 전자(단일 컨테이너)를 기준으로 합니다. 후자는 같은 삭제 로직을 사용자의 Request 개수만큼 반복하며, 컨테이너별로 삭제 안내 이메일을 개별 발송합니다.



Pod 삭제나 Ubuntu 계정 삭제가 실패해도 DB는 `DELETED` 로 처리되며, Infra 리소스는 수동 정리가 필요합니다. 보상 트랜잭션은 승인 흐름에만 적용됩니다.

만료 처리 (매일 08:00 KST, 스케줄러)

만료 예고 이메일은 SETNX로 중복 발송을 막습니다. 스케줄러가 매일 실행되기 때문에, 같은 날 같은 사용자에게 D-7 이메일이 두 번 나가는 상황이 생길 수 있습니다. 발송 전에 `email:preexpiry:{id}:{dayLabel}:{date}` 키를 Redis에 SETNX로 쓰고, 이미 키가 존재하면 발송을 건너뛵니다. TTL은 25시간이라 하루가 지나면 자동으로 만료됩니다.



3. 스케줄러

BE 내부에 세 가지 스케줄러가 상시 동작합니다.

스케줄러	실행	역할
<code>RequestSchedulerService</code>	매일 08:00	D-7/3/1 만료 예고 알림 발송, 만료된 Request 삭제 처리
<code>UserSchedulerService</code>	매일 09:00	3개월 미접속 경고 → 비활성화(Soft Delete, 영구 보존)
<code>SlackNotificationWorker</code>	1초 간격	<code>slack:notification:queue</code> 에서 메시지를 꺼내 Slack API로 발송, 최대 3회 재시도
<code>InfraSlackNotificationWorker</code>	1초 간격	<code>slack:infra:notification:queue</code> 전용 Worker. 백엔드 알림 큐와 완전히 독립 운영, 최대 3회 재시도

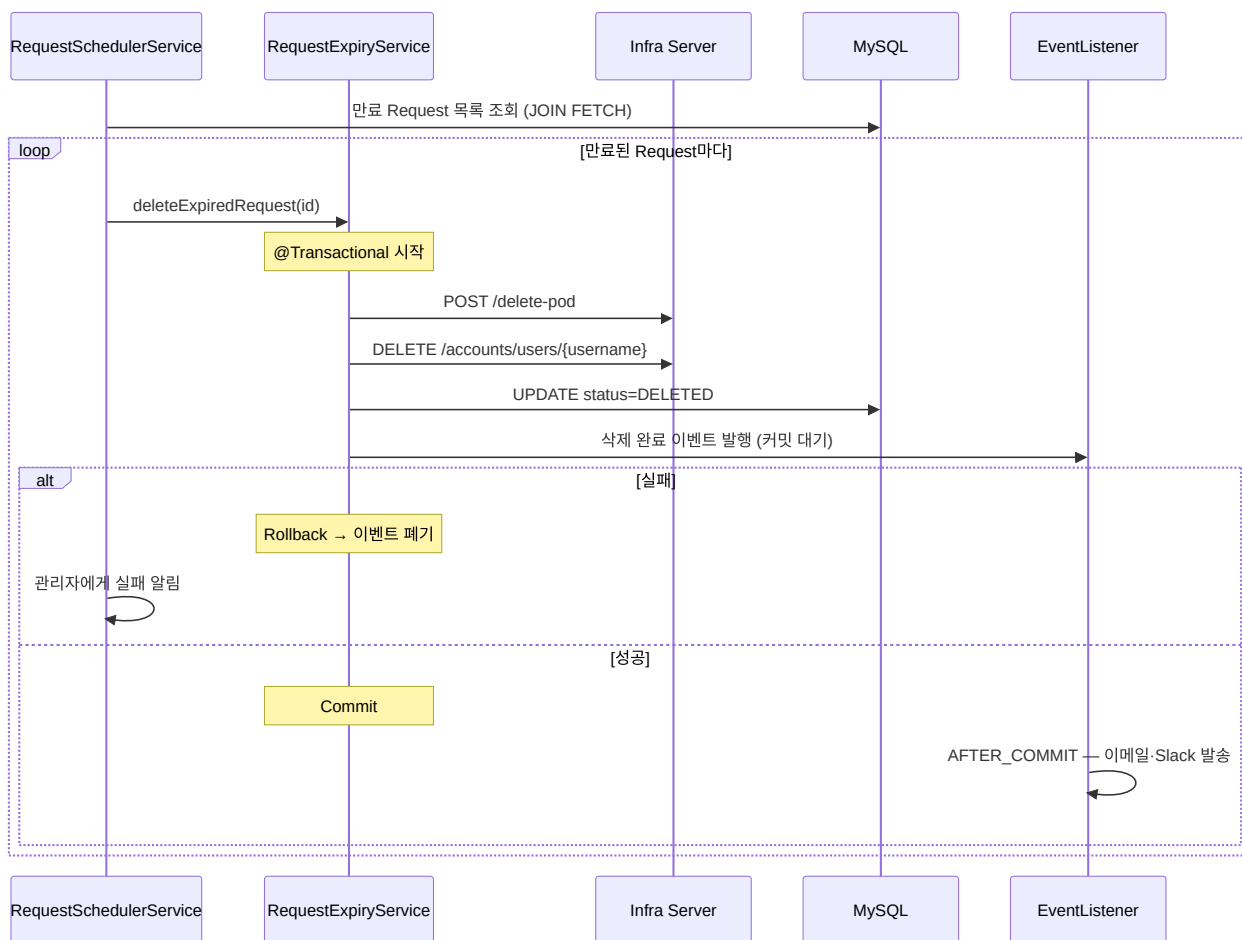
RequestSchedulerService

만료된 컨테이너를 삭제하는 흐름에서 주의할 점이 세 가지 있습니다.

첫째, 만료 대상 조회 시 `JOIN FETCH` 로 User·ResourceGroup을 한 번에 가져옵니다. 단순 조회 후 알림 발송 단계에서 연관 엔티티를 건당 추가 조회하는 N+1 문제를 막기 위해서입니다.

둘째, 삭제 메서드 호출 시 같은 클래스 내에서 `this.delete()` 로 호출하면 Spring AOP 프록시를 우회해 `@Transactional` 이 적용되지 않습니다. 이를 피하기 위해 `@Transactional` 메서드를 별도 클래스 (`RequestExpiryService`)로 분리하고, 해당 Bean을 주입받아 호출합니다.

셋째, 삭제 완료 알림은 `@TransactionalEventListener(phase = AFTER_COMMIT)` 를 통해 DB 커밋이 확정된 후에만 발송됩니다. 트랜잭션이 롤백되면 이벤트도 함께 폐기되므로, DB에 반영되지 않은 삭제에 대한 알림이 발송되는 상황을 방지합니다.



AFTER_COMMIT과 Self-Injection 패턴의 배경은 [핵심 설계 패턴 1·2번](#)을 참고합니다.

UserSchedulerService

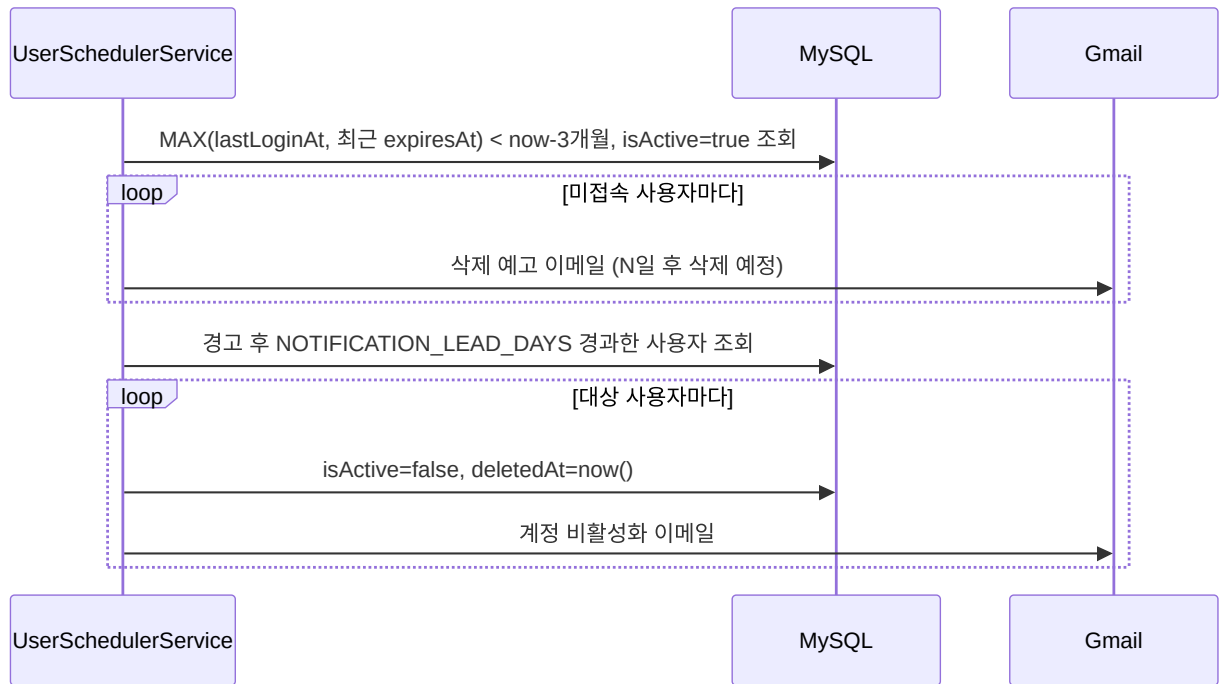
사용자 수명주기를 관리합니다. 매일 09:00에 실행되며 두 단계를 처리합니다. 이전에는 Soft Delete 1년 경과 후 DB row를 완전히 삭제하는 Hard Delete 단계가 있었으나, 이력 보존을 위해 제거되었습니다. 비활성화된 계정은 Soft Delete 상태로 영구 보존됩니다.

1단계 — 장기 미접속 경고

마지막 로그인 시각(`lastLoginAt`)과 해당 사용자의 가장 최근 컨테이너 만료일(`expiresAt`) 중 더 늦은 날짜를 기준으로, 3개월(`INACTIVE_MONTHS=3`) 이상 경과한 활성 사용자(`isActive=true`)를 조회해 삭제 예고 이메일을 발송합니다. 컨테이너가 활성 상태라면 로그인이 없어도 삭제 대상에서 제외됩니다. 이 시점에서는 계정을 삭제하지 않고 경고만 합니다.

2단계 — Soft Delete

경고 이메일을 받은 사용자가 일정 기간(`NOTIFICATION_LEAD_DAYS=8`) 내에 로그인하지 않으면 계정을 비활성화합니다. `isActive=false`, `deletedAt=now()` 로 설정하고 비활성화 이메일을 발송합니다. 이후 별도의 삭제 처리는 없습니다.



각 사용자 처리는 독립 트랜잭션으로 실행됩니다(`UserLifecycleTransactionalService`에 위임). 특정 사용자 처리 실패가 다른 사용자 처리에 영향을 주지 않습니다.

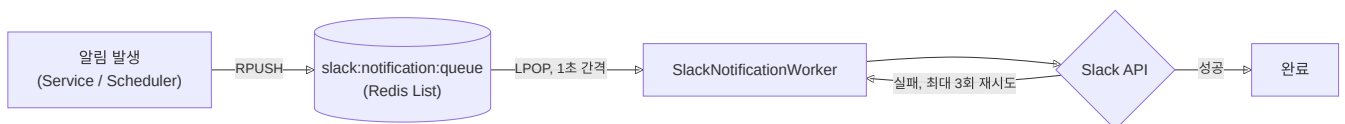
SlackNotificationWorker / InfraSlackNotificationWorker

승인, 삭제 등 비즈니스 로직이 Slack API 응답을 기다리며 블로킹되는 것을 막기 위해 Redis 큐를 사이에 둡니다. 알림이 발생하면 메시지를 큐에 즉시 적재하고, 전용 Worker가 1초마다 꺼내 Slack API를 호출합니다. 실패 시 최대 3회 재시도합니다.

일반 비즈니스 알림(`slack:notification:queue`)과 Infra 관련 알림(`slack:infra:notification:queue`)은 큐뿐 아니라 이를 소비하는 Worker 클래스(`SlackNotificationWorker`, `InfraSlackNotificationWorker`)도 완전히 분리되어 있습니다. 한쪽 큐가 밀려도 다른 쪽 발송에는 영향을 주지 않습니다.

1초 간격으로 설정한 이유는 Slack Webhook API의 rate limit이 채널당 초당 1건이기 때문입니다. 더 빠르게 폴링하면 제한에 걸려 메시지가 누락될 수 있습니다.

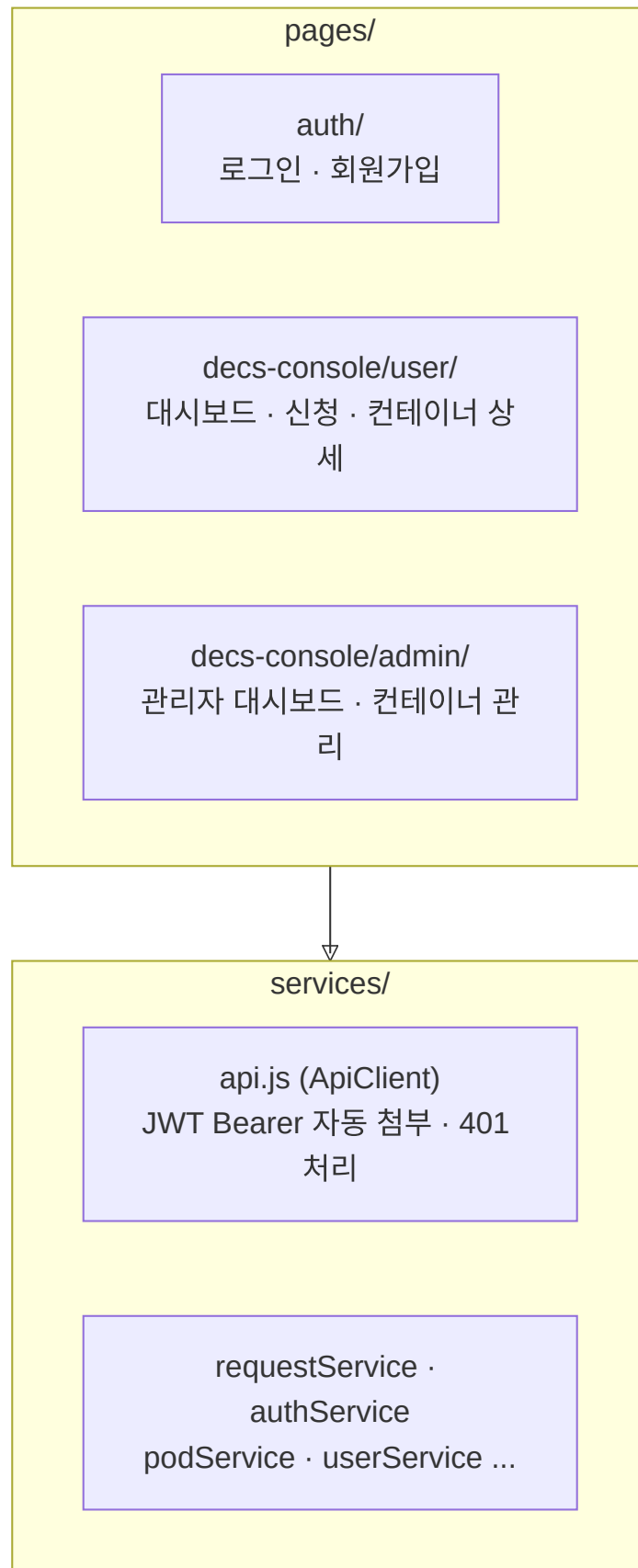
Redis List의 **RPUSH** (오른쪽 삽입)로 넣고 **LPOP** (왼쪽 꺼내기)으로 꺼내 FIFO 순서를 유지합니다. 자세한 내용은 [Redis 키 카탈로그](#) 4번을 참고합니다.



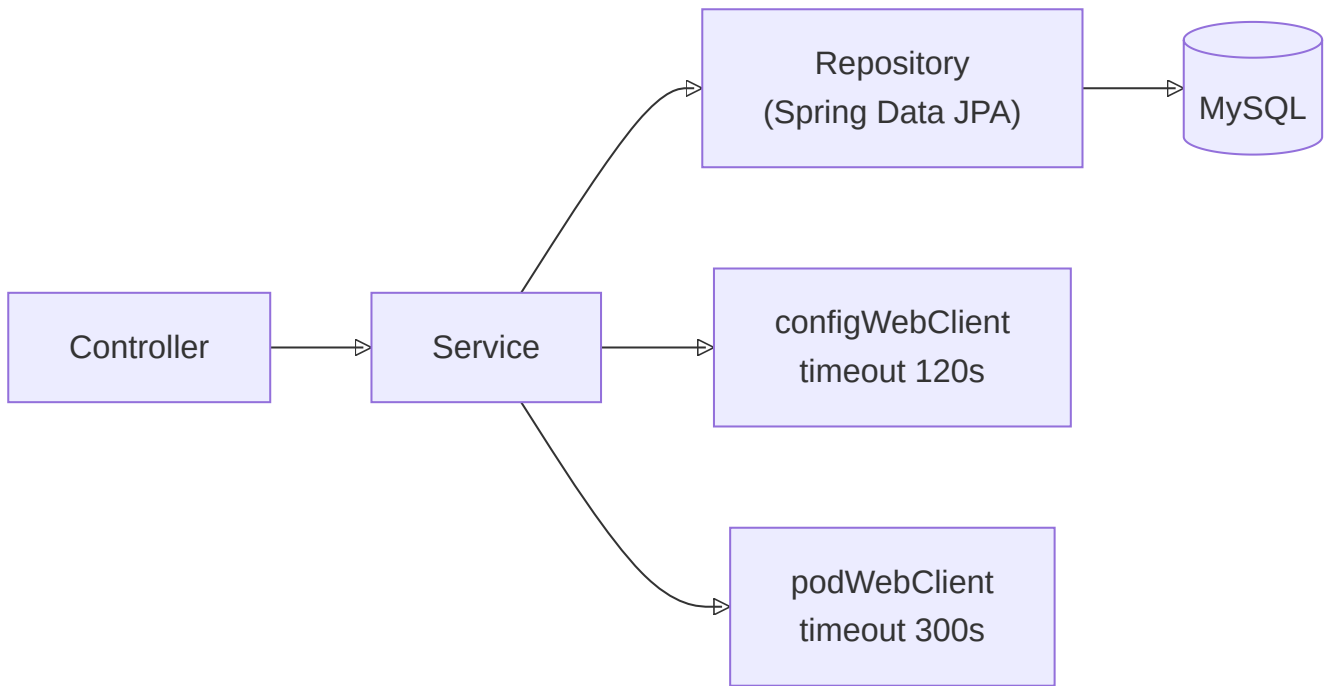
4. 내부 구조 및 통신

FE는 화면(pages)과 API 클라이언트(services)로 나뉩니다. BE는 도메인별 패키지로 구성되며 각각 Controller → Service → Repository 레이어를 따릅니다. Infra Server는 K8s 클러스터 내부 서비스로 Ubuntu 계정과 Docker 컨테이너 생성·삭제를 담당하며, BE에서만 접근합니다.

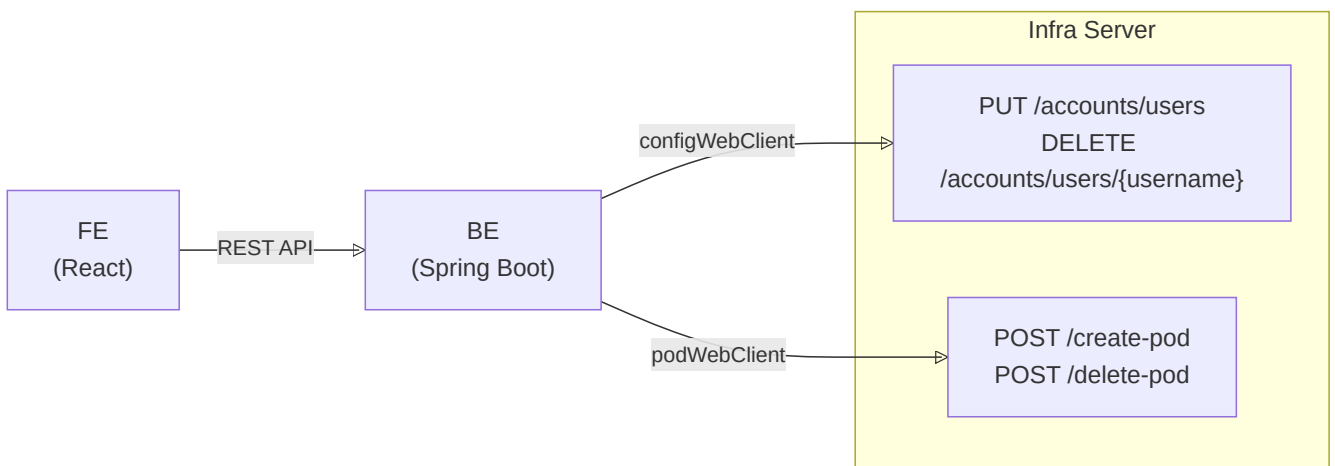
FE 내부 구조



BE 내부 구조



FE ↔ BE ↔ Infra 통신



BE 도메인 구조

domain/

requests	— 신청 접수, 승인/거절, 변경 요청 처리
users	— 사용자 관리, 인증
groups	— Ubuntu 그룹(GID) 관리
alarm	— 이메일·Slack 알림 발송
scheduler	— 만료·사용자 스케줄러, Slack 큐 Worker
pod	— Pod 정보 조회
containerImage	— 컨테이너 이미지 관리
resourceGroups	— 서버 그룹 관리
monitoring	— Prometheus 메트릭 조회
messageTemplate	— 알림 메시지 템플릿 관리

Infra Server API

K8s 클러스터 내부 서비스(`containersssh-config-service.ai lab-infra.svc.cluster.local`)로 BE에서만 접근합니다. 서비스 이름에 `containersssh`가 포함되어 있으나, ContainerSSH 제품을 사용하는 것이 아니라 자체 개발된 Infra Server입니다. BE는 두 개의 WebClient를 사용해 통신합니다. 계정 관련 요청은 `configWebClient` (timeout 120s), Pod 관련 요청은 `podWebClient` (timeout 300s)입니다. Pod 생성에 더 긴 타임아웃을 두는 이유는 Docker 이미지 pull 시간이 포함되기 때문입니다.

엔드포인트	역할
PUT /accounts/users	Ubuntu 계정 생성, uid·gid 반환
DELETE /accounts/users/{username}	Ubuntu 계정 삭제
POST /create-pod	Docker 컨테이너 기동, SSH·Jupyter 포트 할당
POST /delete-pod	Docker 컨테이너 종료

WebClient 설정·에러 처리·보상 트랜잭션 연계 상세는 [외부 연동](#)을 참고합니다.

5. 도메인 설명

BE는 아래 도메인으로 구성됩니다. 각 도메인의 엔티티 구조, 생명주기, 비즈니스 규칙 상세는 [도메인 설명](#)을 참고합니다.

도메인	역할
users	사용자 계정 관리, 인증. USER / ADMIN 역할 구분
requests	서버 사용 신청 접수·승인·거절. ChangeRequest(변경 요청) 포함
groups	Ubuntu 그룹(GID) 관리

도메인	역할
alarm	이메일·Slack 알림 단일 진입점. 이메일 중복 방지(SETNX), Redis 큐 연계
scheduler	만료 Request 처리(08:00), 사용자 수명주기(09:00), Slack 큐 소비(1초 간격)
pod	Pod 정보 및 외부 포트(SSH·Jupyter) 관리
containerImage	컨테이너 이미지 목록 관리
resourceGroups	FARM / LAB 서버 풀 관리
monitoring	Prometheus 메트릭 조회
messageTemplate	알림 메시지 템플릿 관리. messages.properties 기본값 + DB 오버라이드

6. Redis 키 사용 현황

BE는 7개의 Redis 키 패턴을 사용합니다.

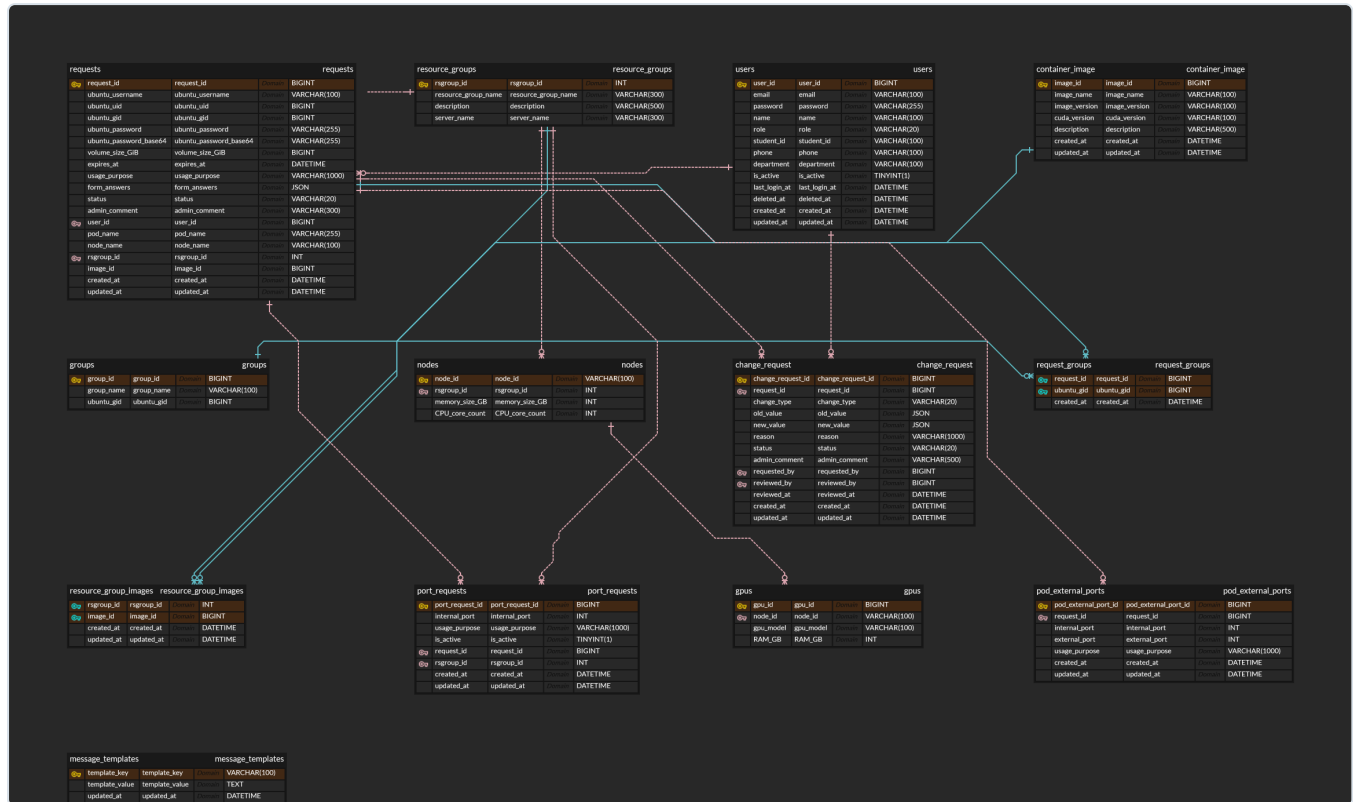
키 패턴	타입	TTL	목적
email:verify:{email}	String	5분	이메일 인증번호 임시 저장
VERIFIED:{email}	String	10분	인증 완료 상태. 이 키가 없으면 회원가입 불가
RT:{userId}	String	7일	Refresh Token. 이 키가 없으면 로그아웃 상태
email:preexpiry:{requestId}:{dayLabel}:{date}	String	25시간	만료 예고 이메일 중복 발송 방지 (SETNX)
slack:notification:queue	List	없음	Slack 알림 FIFO 큐 (1초 간격 소비)
slack:infra:notification:queue	List	없음	Infra 관련 Slack 알림 전용 큐
slack:cache:users:list	String	1시간	이메일 → Slack UserId 변환용 멤버 목록 캐시

키 패턴별 상세 설명(저장·삭제 시점, 값 형태, redis-cli 확인 명령)은 [Redis 키 카탈로그](#)를 참고합니다.

도메인 설명

코드에서 DB 조작하는 부분을 고치기 전에 "이 데이터가 왜 이런 모양으로 저장되는가"를 먼저 답해주는 문서입니다. 엔티티 하나하나가 필드를 그렇게 가진 데는 이유가 있고(예: Soft Delete 뒤에 즉시 지우지 않는 이유, 비밀번호를 두 가지 형태로 같이 저장하는 이유), 그 이유를 모르고 필드를 손대면 다른 기능이 조용히 깨집니다. 새 필드를 추가하거나 마이그레이션을 준비하기 전에는 반드시 여기서 해당 엔티티 절을 먼저 확인합니다.

ERD



ERD Cloud에서 직접 확인: [admin_be ERD](#)

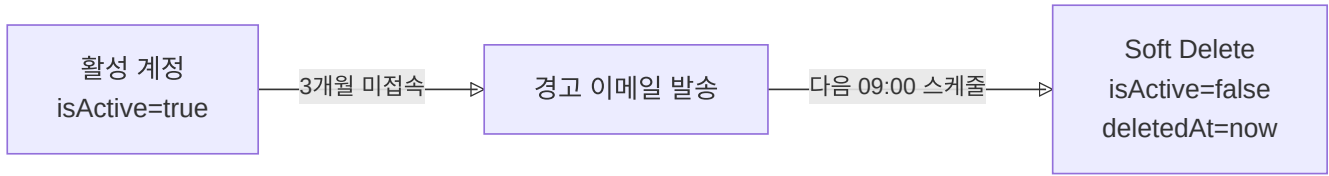
1. User

사용자 계정 정보와 생명주기를 관리합니다.

주요 필드

필드	설명
email	로그인 식별자, unique
password	BCrypt 해시
role	USER / ADMIN
isActive	계정 활성화 여부. false 면 로그인 불가
lastLoginAt	마지막 로그인 시각. 스케줄러가 미접속 여부 판단에 사용
deletedAt	Soft Delete 시각. null이면 활성화 계정

생명주기



Soft Delete 이후 DB row를 완전히 삭제하는 단계는 없습니다. 비활성화된 계정은 영구히 보존됩니다. 이력 보존을 위한 결정으로, `reactivate()` 로 계정을 복구할 수 있고 신청 이력·로그 추적도 계속 가능합니다.

2. Request

서버 사용 신청 한 건을 나타냅니다. 상태 전이와 보상 트랜잭션이 이 도메인의 핵심입니다.

주요 필드

필드	설명
<code>ubuntuUsername</code>	Ubuntu 계정명, unique
<code>ubuntuPassword</code> / <code>ubuntuPasswordBase64</code>	초기 비밀번호. <code>ubuntuPasswordBase64</code> 는 Infra Server API가 base64 인코딩된 평문을 요구하기 때문에 별도 보관. BCrypt 해시(<code>ubuntuPassword</code>)에서는 원문 복원이 불가능하므로 승인 시점의 base64 값을 함께 저장함
<code>ubuntuUid</code> / <code>ubuntuGid</code>	승인 후 Infra Server에서 받아 저장
<code>podName</code> / <code>nodeName</code>	승인 후 Pod 생성 결과 저장
<code>volumeSizeGiB</code>	할당 스토리지 크기
<code>expiresAt</code>	컨테이너 만료일
<code>status</code>	현재 상태 (<code>PENDING</code> / <code>PROCESSING</code> / <code>FULFILLED</code> / <code>DENIED</code> / <code>DELETED</code>)
<code>adminComment</code>	거절 사유 또는 관리자 메모
<code>formAnswers</code>	신청 양식 응답 (JSON). 양식 질문이 변경돼도 스키마 마이그레이션 없이 수용하기 위해 컬럼을 분리하지 않고 JSON으로 저장

상태 전이와 승인 흐름은 [시스템 아키텍처](#)를 참고합니다.

ChangeRequest

FULFILLED 상태의 컨테이너에 대해 사용자가 변경을 요청할 때 생성됩니다. 원본 Request와 분리된 테이블에 저장되므로, 변경 요청이 거절되거나 폐기돼도 원본 Request는 영향받지 않습니다.

changeType

타입	변경 내용
EXPIRES_AT	만료일 연장
VOLUME_SIZE	스토리지 크기 변경
RESOURCE_GROUP	서버 풀 변경
CONTAINER_IMAGE	컨테이너 이미지 변경
GROUP	Ubuntu 그룹 변경
PORT	추가 포트 요청

ChangeRequest는 PENDING → FULFILLED(승인) 또는 DENIED(거절) 상태로 전이됩니다. 만료일 연장 (EXPIRES_AT)이 승인된 경우에만 사용자에게 이메일을 발송합니다.

3. ResourceGroup / ContainerImage / Group

ResourceGroup

컨테이너를 배치할 서버 풀을 나타냅니다.

필드	설명
resourceGroupName	GPU 모델·풀 이름 (ex: 3090ti)
serverName	서버 구분 (FARM / LAB)
description	관리자용 설명

serverName 이 서버 군(FARM/LAB)을 나타내고, resourceGroupName 이 그 안의 특정 GPU 풀을 세분합니다. 관리자가 승인 시 ResourceGroup을 선택하면 Infra Server는 해당 풀 내의 노드에 Pod를 배치합니다.

ContainerImage

컨테이너 생성에 사용할 Docker 이미지 정보입니다.

필드	설명
imageName	이미지명
imageVersion	이미지 태그·버전
cudaVersion	해당 이미지가 지원하는 CUDA 버전
description	사용자에게 보여줄 설명

사용자는 서버 신청 시 목록에서 이미지를 선택합니다. CUDA 버전이 별도 필드로 관리되므로 사용자가 자신의 워크로드에 맞는 이미지를 고를 수 있습니다. `BaseTimeEntity` 를 상속하므로 `createdAt` / `updatedAt` 이 자동 관리됩니다.

Group

Ubuntu 시스템 그룹을 나타냅니다.

필드	설명
<code>groupName</code>	Ubuntu 그룹명
<code>ubuntuGid</code>	Ubuntu GID (unique)

Request와 Group은 `request_groups` 중간 테이블로 N:M 관계입니다. 하나의 신청에 여러 그룹을 지정할 수 있으며, 계정 생성 시 Infra Server의 `supplementary_groups` 필드에 이 목록이 전달됩니다. 이를 통해 컨테이너 안에서 특정 디렉터리나 리소스에 대한 접근 권한을 제어합니다.

API

엔드포인트	역할
<code>GET /api/groups</code>	전체 그룹 목록 조회. 그룹 ID·그룹명 반환
<code>POST /api/groups</code>	새 그룹 생성

그룹 삭제·수정 API는 없습니다. 잘못 만든 그룹을 정리해야 한다면 MySQL에 직접 접속합니다(운영 가이드 7절).

4. Alarm

모든 알림(이메일·Slack)은 `AlarmService` 를 단일 진입점으로 사용합니다. 직접 `JavaMailSender`나 `Slack API`를 호출하지 않고 `AlarmService`를 통해서만 발송합니다.

이메일 발송

Gmail SMTP를 통해 사용자에게 이메일을 발송합니다. 메시지 내용은 `messages.properties`의 기본값을 사용하거나, 관리자가 DB에 등록한 오버라이드 값을 우선 적용합니다(자세한 내용은 아래 `messageTemplate` 섹션 참고).

발송 대상 이벤트는 컨테이너 배정·삭제·만료일 연장·만료 예고·만료 완료뿐 아니라, 신청 거절과 변경 요청 거절도 포함합니다. 거절 이메일 발송은 try-catch로 감싸져 있어 발송 실패가 거절 처리 자체를 막지 않습니다.

만료 예고 이메일은 Redis SETNX로 중복 발송을 방지합니다. 키 형식은 `email:preexpiry:{requestId}:{dayLabel}:{date}` 이고 TTL은 25시간입니다. 스케줄러가 매일 실행되므로, 같은 날 같은 사용자에게 중복 발송되는 것을 막습니다.

Slack 발송

Slack 알림은 직접 보내지 않고 Redis List(`slack:notification:queue`)에 적재합니다. `SlackNotificationWorker` 가 1초마다 LPOP으로 꺼내 Slack API를 호출합니다. Slack Webhook rate limit이 채널당 초당 1건이기 때문입니다.

5. Pod / PodExternalPort

`PodExternalPort` 는 Infra Server가 Pod 생성 시 할당한 NodePort 매핑을 저장합니다. `pod_external_ports` 테이블에 저장되며, 하나의 Request에 여러 포트가 1:N으로 매핑됩니다.

필드	타입	설명
<code>usagePurpose</code>	<code>VARCHAR(1000)</code>	포트 용도 (<code>ssh</code> , <code>jupyter</code> 등)
<code>internalPort</code>	<code>INT</code> (1-65535)	컨테이너 내부 포트
<code>externalPort</code>	<code>INT</code> (1-65535)	외부에서 접속하는 Kubernetes NodePort 번호

`BaseTimeEntity` 를 상속하므로 `createdAt` / `updatedAt` 이 자동 관리됩니다.

흐름 요약

1. 관리자가 신청 승인 → `POST /create-pod` 호출
2. Infra Server가 NodePort를 자동 할당하고 `ports` 배열로 응답
3. BE가 배열을 순회하며 `PodExternalPort` row를 `request_id` 에 묶어 저장
4. 승인 완료 이메일에 포트 목록이 포함되어 사용자에게 발송

사용자는 이 포트를 통해 컨테이너에 SSH(`ssh -p {externalPort} ubuntu@서버IP`)나 Jupyter(`http://서버IP:{externalPort}`)로 접속합니다.

6. Node / Gpu

물리 서버 노드와 GPU 사양을 DB에서 관리합니다. Prometheus가 실시간 메트릭을 수집하는 것과 달리, Node/Gpu 테이블은 정적인 하드웨어 목록(어떤 서버에 GPU가 몇 개, 어떤 모델인지)을 저장합니다. 관리자 대시보드에서 서버 목록을 표시하거나, 신청 폼에서 ResourceGroup 설명을 구성할 때 이 정보를 사용합니다.

Node

필드	설명
<code>nodeId</code>	노드 식별자 (ex: <code>farm1</code> , <code>lab-dgx</code>) — 문자열 PK, Infra Server의 노드명과 일치
<code>rsgroup_id</code>	소속 ResourceGroup (FARM / LAB)
<code>memorySizeGB</code>	시스템 RAM (GB)
<code>cpuCoreCount</code>	CPU 코어 수

Gpu

필드	설명
gpuModel	GPU 모델명 (ex: RTX4050 , A3000)
ramGb	GPU VRAM (GB)
node_id	소속 노드 FK

하나의 노드에 여러 GPU가 장착될 수 있어 Node → Gpu는 1:N입니다.

7. PortRequests

사용자가 서버 신청 시 SSH·Jupyter 외에 추가 포트를 요청하는 경우 `port_requests` 테이블에 저장됩니다. 기본 제공 포트(`PodExternalPort`)와 달리, 사용자가 명시적으로 요청한 포트를 별도로 추적합니다.

필드	설명
internalPort	요청하는 컨테이너 내부 포트
usagePurpose	포트 용도 (ex: tensorboard , custom-api)
isActive	실제 Infra에 할당 완료 여부. 신청 시 <code>false</code> , 관리자 승인 후 포트가 열리면 <code>true</code>
request_id	연관 Request FK
rsgroup_id	어느 서버 풀에 요청하는지

`isActive=false` 는 "요청은 됐지만 아직 Infra에 포트가 열리지 않은 상태"를 의미합니다. `activate()` 가 호출되면 `isActive=true` 로 전환됩니다.

8. Monitoring / Dashboard

Monitoring

`GET /api/monitoring/metrics` — FE 대시보드가 주기적으로 폴링합니다. BE가 Prometheus HTTP API에 PromQL 쿼리를 날려 결과를 가공해 반환합니다. 인증 없이 접근 가능한 화이트리스트 경로(`/api/monitoring/**`)입니다.

지표	PromQL
서버별 평균 GPU 사용률 (%)	<code>avg by(Hostname)(DCGM_FI_DEV_GPU_UTIL)</code>
서버별 GPU 개수	<code>count by(Hostname)(DCGM_FI_DEV_GPU_UTIL)</code>
클러스터별 활성 컨테이너 수	<code>sum by(cluster)(cluster_monitor_container_running)</code>

Prometheus가 일시적으로 내려가도 API는 200을 반환합니다(빈 값 응답). 각 쿼리에 5초 타임아웃이 적용됩니다. Prometheus 연동 상세(WebClient 설정, 응답 구조 등)는 [외부 연동](#) 섹션 4를 참고합니다.

Dashboard

`GET /api/admin/dashboard` — 관리자 대시보드용 집계 데이터를 단일 API로 내려줍니다.

집계 항목	내용
신청 현황	상태별 Request 수 (<code>PENDING</code> / <code>FULFILLED</code> / <code>DENIED</code> 등)
사용자 현황	전체 사용자 수, 활성 사용자 수
서버별 사용량	ResourceGroup별 FULFILLED Request 수

9. MessageTemplate

알림 메시지를 재배포 없이 수정할 수 있도록 "DB 우선, properties 폴백" 구조로 관리합니다.

`messages.properties`에 모든 메시지 키의 기본값이 있습니다. JAR에 포함되므로 수정하려면 재배포가 필요합니다. 관리자 화면에서 특정 키의 값을 수정하면 `message_templates` 테이블에 row가 생성됩니다. `DbMessageSource`는 메시지를 읽을 때 DB를 먼저 확인하고, 없으면 `messages.properties`로 폴백합니다.

메시지 조회 순서:

1. `message_templates` 테이블에 해당 key row가 있으면 → DB 값 사용
2. 없으면 → `messages.properties` 기본값 사용

관리자 API를 통해 특정 키를 "초기화(reset)"하면 DB row가 삭제되어 기본값으로 돌아갑니다. 코드를 건드리지 않고 이메일/Slack 메시지 내용을 조정할 때 유용합니다.

코드 컨벤션

사람이 바뀌어도 코드는 한 사람이 짰 것처럼 보아야 유지보수가 됩니다. 그 일관성을 지키기 위한 규칙과, 그 규칙이 왜 그렇게 정해졌는지의 근거를 함께 정리했습니다. Entity에 `@Data`를 쓰지 않는 이유, Setter 대신 의도가 드러나는 메서드를 쓰는 이유가 그런 예입니다. 규칙만 외우면 애매한 새 상황에서 판단이 서지 않으므로, 이유까지 같이 알아둡니다. 코드에서 "왜 이렇게 짜여 있지?" 싶은 부분이 있다면 먼저 여기서 답을 찾습니다.

1. 패키지 구조

도메인 중심으로 패키지를 구성합니다. 각 도메인 안에는 아래 하위 패키지가 들어갑니다.

```

domain/
├── {도메인}/
│   ├── controller/
│   │   ├── docs/           # Swagger 인터페이스
│   │   ├── service/       # CommandService / QueryService 분리
│   │   ├── entity/
│   │   ├── repository/
│   │   ├── dto/
│   │   │   ├── request/
│   │   │   └── response/

```

전체 도메인 목록: `requests`, `users`, `groups`, `alarm`, `scheduler`, `pod`, `containerImage`, `resourceGroups`, `monitoring`, `messageTemplate`, `dashboard`

공통 기능은 `global` 패키지 아래 `auth`, `config`, `common`, `util` 로 분리합니다.

2. 클래스/파일 네이밍

사용자용과 관리자용 클래스를 분리합니다. 관리자 전용에는 반드시 `Admin` prefix를 붙여서, 파일명만 봐도 어느 쪽 기능인지 구분할 수 있게 합니다.

Controller는 역할이 명확하면 그 역할을 이름에 드러냅니다. 예를 들어 신청 변경 요청만 전담하는 컨트롤러는 `AdminRequestController` 가 아니라 `AdminRequestChangeController` 로 씁니다.

Service는 읽기(`QueryService`)와 쓰기(`CommandService`)를 분리합니다. 특정 역할에만 집중하는 서비스는 그 역할을 이름에 포함합니다(`UserLoginService`, `RequestExpiryService`, `UbuntuAccountService`).

DTO는 목적과 방향을 이름에 담습니다. Request DTO는 어떤 동작에 쓰이는지(`SaveRequestRequestDTO`, `ApproveRequestDTO`), Response DTO는 무엇을 반환하는지(`UserResponseDTO`, `MyInfoResponseDTO`) 이름에서 바로 알 수 있어야 합니다. suffix는 `DTO` 로 통일합니다.

유형	예시
Controller (사용자)	<code>RequestController</code> , <code>UserController</code> , <code>GroupController</code>
Controller (관리자)	<code>AdminRequestController</code> , <code>AdminUserController</code> , <code>AdminRequestChangeController</code>
Service (읽기)	<code>RequestQueryService</code> , <code>AdminRequestQueryService</code>
Service (쓰기)	<code>RequestCommandService</code> , <code>AdminRequestCommandService</code>
Service (단일 역할)	<code>UserLoginService</code> , <code>RequestExpiryService</code> , <code>UbuntuAccountService</code>
Repository	<code>RequestRepository</code> , <code>ChangeRequestRepository</code>
Entity	<code>Request</code> , <code>User</code> , <code>Group</code> , <code>ContainerImage</code> , <code>ChangeRequest</code>

유형	예시
Request DTO	SaveRequestRequestDTO , ApproveRequestDTO , CreateGroupRequestDTO
Response DTO	UserResponseDTO , SaveRequestResponseDTO , MyInfoResponseDTO

3. 메서드 네이밍

Service 메서드는 동사 접두사로 역할을 구분합니다. 조회는 `get*`, 생성은 `create*`, 수정은 `update*` 를 기본으로 쓰고, 승인/거절처럼 도메인 동사가 명확한 경우엔 그대로 씁니다.

동작	접두사	예시
조회	<code>get*</code>	<code>getMyInfo</code> , <code>getRequestsByUserId</code> , <code>getAllActiveContainers</code>
생성	<code>create*</code>	<code>createRequest</code> , <code>createGroup</code> , <code>createModificationRequest</code>
수정	<code>update*</code>	<code>updatePassword</code> , <code>updatePhone</code>
승인/거절	동사 그대로	<code>approveRequest</code> , <code>rejectRequest</code> , <code>approveModification</code>
상태 전환	동사 그대로	<code>markAsProcessing</code> , <code>revertToPending</code> , <code>deleteAfterCleanup</code>

엔티티 내부 메서드도 같은 방식입니다. `Request` 엔티티는 `approve()`, `reject()`, `markAsProcessing()`, `revertToPending()`, `assignPodInfo()` 처럼 동작을 직접 드러내는 이름을 씁니다. 외부에서 setter로 상태를 바꾸는 것을 막고, 엔티티 스스로 상태를 관리하게 하기 위해서입니다.

Repository 메서드는 Spring Data 네이밍 컨벤션을 따릅니다.

```

findByEmail(String email)
findAllByUser_UserId(Long userId)
existsByUbuntuUsernameAndStatusIn(String username, List<Status> statuses)

```

4. BaseTimeEntity

모든 주요 엔티티는 `BaseTimeEntity` 를 상속합니다. 엔티티마다 `createdAt`, `updatedAt` 컬럼을 중복 정의하지 않고 한 곳에서 관리하기 위해서입니다. `@EntityListeners(AuditingEntityListener.class)` 가 붙어 있어 생성/수정 시각이 자동으로 기록됩니다.

- 위치: `DGU_AI_LAB.admin_be.global.common.BaseTimeEntity`


```
public class Request extends BaseTimeEntity {  
    // createdAt, updatedAt 자동 관리  
}
```

5. Entity 작성 규칙

필수/권장 어노테이션

- `@Entity`, `@Table(name = "...")`
- `@Id`, `@GeneratedValue(strategy = GenerationType.IDENTITY)`
- `@Column(nullable = false, ...)` — 제약 조건은 명시
- `@Enumerated(EnumType.STRING)` — ORDINAL로 저장하면 enum 순서가 바뀔 때 데이터가 틀어지므로 STRING 사용
- 연관관계는 기본 `FetchType.LAZY` — EAGER는 불필요한 쿼리를 유발합니다

Lombok 조합

```
@Getter  
@NoArgsConstructor(access = AccessLevel.PROTECTED)  
@Entity  
public class User extends BaseTimeEntity { }
```

`@NoArgsConstructor(access = AccessLevel.PROTECTED)` 는 JPA가 요구하는 기본 생성자를 만들면서, 외부에서 무분별하게 빈 객체를 생성하는 것을 막기 위해 `PROTECTED` 로 제한합니다.

`@Data` 는 Entity에서 사용 금지입니다. 무분별한 Setter 생성, `toString()` 의 연관 엔티티 LAZY 로딩 트리거, JPA 프록시 충돌 문제가 생깁니다.

비즈니스 메서드

엔티티가 상태를 직접 변경하는 메서드를 가집니다. 외부에서 setter로 필드를 직접 바꾸는 대신, 엔티티에 의도를 드러내는 메서드를 두어 변경 경로를 하나로 통일합니다.

```

public void approve(ContainerImage image, ResourceGroup resourceGroup, ...) {
    this.containerImage = image;
    this.status = Status.FULFILLED;
    this.approvedAt = LocalDateTime.now();
}

public void markAsProcessing() {
    this.status = Status.PROCESSING;
}

```

6. DTO 작성 규칙

DTO는 불변(immutable)으로 작성합니다. 값이 바뀌면 버그 추적이 어렵기 때문입니다. Java `record` 를 기본으로 사용합니다.

Request DTO에는 Validation 어노테이션을 달아두고, Controller에서 `@Valid` 로 검증을 트리거합니다. Response DTO에는 `fromEntity()` static 메서드를 두어 엔티티 → DTO 변환을 한 곳에서 관리합니다. Swagger 문서화를 위해 `@Schema` 도 포함합니다.

```

@Builder
public record SaveRequestRequestDTO(
    @NotNull(message = "Resource group ID cannot be null")
    Integer resourceGroupId,

    @NotBlank(message = "Ubuntu username cannot be blank")
    @Size(min = 3, max = 50)
    String ubuntuUsername,

    @Future(message = "Expires date must be in the future")
    LocalDateTime expiresAt
) {
    public Request toEntity(User user, ...) { ... }
}

```

`@Data` 는 DTO에서도 사용하지 않습니다.

7. 각 클래스 어노테이션 정리

Controller

Controller는 HTTP 요청 수신, 인증 정보 추출, Service 호출, 응답 반환만 담당합니다. 비즈니스 로직은 Service에서 처리합니다. Swagger 문서는 컨트롤러 클래스에 직접 달지 않고 `controller/docs/` 아래 인터페이스를 만들어 분리합니다.

- `@RestController` — JSON API 응답 표준
- `@RequestMapping` — 클래스 레벨에서 공통 경로 prefix 지정 (예: `/api/requests`)
- `@Valid` — 요청 DTO 유효성 검증 트리거

Service

- `@Transactional` — 쓰기 작업에 적용. 중간에 실패하면 전체 롤백됩니다.
- `@Transactional(readOnly = true)` — 조회 작업에 적용. flush와 dirty checking을 건너뛰어 성능이 좋습니다.
- `@Slf4j` — `System.out.println` 대신 SLF4J 로깅 사용

8. CQS (Command Query Separation)

조회와 변경을 서비스 레벨에서 분리합니다. 한 서비스가 읽기와 쓰기를 모두 담당하면 트랜잭션 옵션을 일관되게 적용하기 어렵고, 책임이 불분명해집니다.

- **QueryService:** 조회만 담당. `@Transactional(readOnly = true)` 적용
- **CommandService:** 변경만 담당. `@Transactional` 적용

```
@Service
@Transactional(readOnly = true)
public class RequestQueryService {
    public List<SaveRequestResponseDTO> getRequestsByUserId(Long userId) { ... }
}
```

Query 코드에서 엔티티 값을 변경하지 않습니다.

9. 정적 팩토리 메서드

`new` 생성자 대신 `static` 팩토리 메서드를 사용합니다. 생성 의도를 이름으로 드러낼 수 있고, 생성 규칙을 한 곳에서 관리할 수 있습니다.

```
// Response DTO에서 엔티티 변환
public static UserResponseDTO fromEntity(User user) {
    return new UserResponseDTO(user.getUserId(), user.getName(), user.getIsActive());
}
```

10. 예외 처리 및 ErrorCode

예러는 아래 흐름으로 처리됩니다.

1. Service에서 조건 미충족 시 `BusinessException` 발생
2. `GlobalExceptionHandler` 가 예외를 가로챈
3. `ErrorCode` 에 정의된 HTTP 상태 코드와 메시지로 클라이언트에 응답

예외 계층

```
BusinessException
├── EntityNotFoundException
└── UnauthorizedException
```

사용 방법

예외는 Service 레이어에서만 던집니다. `ErrorCode` 를 직접 넘겨서 응답 메시지와 상태 코드를 한 곳에서 관리합니다.

```
// 조회 실패
User user = userRepository.findById(userId)
    .orElseThrow(() -> new BusinessException(ErrorCode.USER_NOT_FOUND));

// 비즈니스 규칙 위반
if (!passwordEncoder.matches(request.currentPassword(), user.getPassword())) {
    throw new BusinessException(ErrorCode.INVALID_PASSWORD);
}
```

ErrorCode 정의

위치: `DGU_AI_LAB.admin_be.error.ErrorCode`

새 에러 케이스가 생기면 enum에 추가합니다. 유사한 상태 코드끼리 묶어서 작성하고, 메시지는 FE에 바로 노출될 수 있으므로 명확한 한글로 작성합니다. 현재 정의된 59개 케이스 전체 목록은 [에러 코드 카탈로그](#)를 참고합니다.

```

@Getter
@RequiredArgsConstructor(access = AccessLevel.PRIVATE)
public enum ErrorCode {
    // 400 Bad Request
    INVALID_INPUT_VALUE(HttpStatus.BAD_REQUEST, "입력값이 올바르지 않습니다."),

    // 401 Unauthorized
    EXPIRED_ACCESS_TOKEN(HttpStatus.UNAUTHORIZED, "엑세스 토큰이 만료되었습니다."),

    // 404 Not Found
    RESOURCE_NOT_FOUND(HttpStatus.NOT_FOUND, "해당 리소스를 찾을 수 없습니다."),

    // 500 Internal Server Error
    INTERNAL_SERVER_ERROR(HttpStatus.INTERNAL_SERVER_ERROR, "서버 내부 오류입니다.");

    private final HttpStatus httpStatus;
    private final String message;
}

```

11. API 응답 포맷

성공 응답은 `SuccessResponse`, 실패 응답은 `ErrorResponse` 를 사용합니다. 응답 포맷을 통일해서 FE가 일관된 방식으로 처리할 수 있게 합니다.

```

// 성공
{ "status": 200, "message": "요청이 성공했습니다.", "data": { ... } }

// 실패
{ "status": 404, "message": "사용자를 찾을 수 없습니다." }

```

```

return SuccessResponse.ok(body);
return SuccessResponse.created(body);

```

12. 로깅 규칙

레벨을 구분해서 찍어야 운영 중 로그 필터링이 수월합니다. 레벨별 기준은 아래와 같습니다.

레벨	사용 상황
INFO	주요 비즈니스 이벤트 (요청 생성, 승인, 삭제 등)
WARN	비즈니스 규칙 위반, 중복 감지 등
DEBUG	상세 디버깅 정보
ERROR	시스템 오류, 외부 API 실패

메서드명을 **[대괄호]** 로 표시해 어디서 찍힌 로그인지 빠르게 파악할 수 있게 합니다.

```
log.info("[createRequest] 요청 생성: userId={}, resourceGroupId={}", userId, dto.resourceGroupId());
log.warn("[createRequest] 중복 username 감지: {}", dto.ubuntuUsername());
log.error("[approveRequest] 인프라 API 호출 실패", e);
```

13. 쿼리 최적화

연관 엔티티를 나중에 따로 조회하면 N+1 문제가 생깁니다. 한 번의 쿼리로 필요한 연관 엔티티를 모두 가져오려면 FETCH JOIN을 사용합니다.

```
@Query("SELECT DISTINCT r FROM Request r " +
        "JOIN FETCH r.user " +
        "LEFT JOIN FETCH r.resourceGroup " +
        "WHERE r.status = :status")
List<Request> findAllByStatusWithAssociations(@Param("status") Status status);
```

14. PR 및 코드 리뷰

기능 개발, 버그 수정 가릴 것 없이 반드시 PR을 남기고 팀원에게 코드 리뷰를 받은 뒤 머지합니다. 리뷰 없이 main/develop에 직접 푸시하지 않습니다.

PR 작성 시:

- 변경 사항과 이유를 설명합니다.
- `application.yml` 을 수정했다면 PR 본문에 명시하고 Notion 페이지도 함께 업데이트합니다.
- 리뷰어는 로직뿐 아니라 컨벤션 준수 여부도 확인합니다.
- **커밋 전에 반드시 테스트를 실행하고, 모든 테스트가 통과하는 상태에서만 커밋합니다.** 테스트가 깨진 코드를 커밋하지 않습니다.

15. 테스트 작성

로컬에서 테스트 실행: 별도 DB 설정 없이 바로 실행됩니다. `src/test/resources/application.yml` 이 `test` 프로파일과 H2 인메모리 DB(`jdbc:h2:mem:testdb`)를 자동으로 사용하기 때문입니다.

```
./gradlew test
```

CI(`deploy.yml`)는 배포 속도를 위해 `./gradlew build -x test`로 테스트를 건너뜁니다. 즉 테스트 통과 여부는 CI가 아니라 PR 리뷰와 로컬 실행에 전적으로 의존합니다. 커밋 전 테스트 실행이 규칙(14절)인 이유이기도 합니다.

테스트 케이스는 가능한 한 촘촘하게 작성합니다. 주요 흐름(happy path)뿐 아니라 엣지 케이스, 실패 케이스, 보상 트랜잭션 동작까지 커버합니다.

테스트는 세 가지 레이어로 구분합니다.

- **Service 단위 테스트:** 비즈니스 로직 검증. 외부 의존성은 Mockito로 대체합니다.
- **Controller 통합 테스트:** HTTP 요청/응답 검증. MockMvc로 실제 요청을 재현합니다.
- **Repository 테스트:** 커스텀 쿼리 검증. H2 인메모리 DB를 사용합니다.

테스트 메서드명

한국어로 작성하며 `@DisplayName`으로 의도를 명확히 설명합니다. 메서드명과 `DisplayName`을 조합해 어떤 상황에서 어떤 결과를 기대하는지 바로 알 수 있어야 합니다.

```
@Test
@DisplayName("이메일 인증이 완료되고 중복이 없으면 회원가입에 성공한다")
void register_success() { ... }

@Test
@DisplayName("이미 가입된 이메일로 회원가입하면 BusinessException을 던진다")
void register_throwsException_whenEmailAlreadyExists() { ... }
```

연관된 테스트 케이스는 `@Nested` + `@DisplayName`으로 묶어 가독성을 높입니다.

```

@Nested
@DisplayName("login (로그인)")
class Login {

    @Test
    @DisplayName("올바른 이메일과 비밀번호로 로그인하면 토큰을 반환한다")
    void login_success() { ... }

    @Test
    @DisplayName("비밀번호가 틀리면 UnauthorizedException을 던진다")
    void login_throwsException_whenPasswordWrong() { ... }
}

```

Service 단위 테스트

`@ExtendWith(MockitoExtension.class)` 를 사용합니다. 테스트 대상(`@InjectMocks`)과 의존성(`@Mock`)을 선언하고, `@BeforeEach` `setUp()` 에서 공통 픽스처를 구성합니다.

`@Value` 로 주입받는 필드는 Mockito가 처리하지 못하므로 `ReflectionTestUtils.setField()` 로 직접 설정합니다.

```

@ExtendWith(MockitoExtension.class)
class UserLoginServiceTest {

    @InjectMocks
    private UserLoginService userLoginService;

    @Mock private UserRepository userRepository;
    @Mock private PasswordEncoder passwordEncoder;
    @Mock private JwtProvider jwtProvider;
    @Mock private RedisTemplate<String, String> redisTemplate;

    @BeforeEach
    void setUp() {
        // @Value 필드는 Mockito가 주입하지 않으므로 직접 설정
        ReflectionTestUtils.setField(userLoginService, "REFRESH_TOKEN_EXPIRE_TIME", 604800000L);
    }
}

```

예외 발생 검증은 `assertThatThrownBy()` 로 씁니다.

```

assertThatThrownBy(() -> userLoginService.login(dto))
    .isInstanceOf(UnauthorizedException.class);

```


특정 ErrorCode까지 검증하려면 `extracting()` 을 사용합니다.

```
assertThatThrownBy(() -> service.approveRequest(dto))
    .isInstanceOf(BusinessException.class)
    .extracting(e -> ((BusinessException) e).getErrorCode())
    .isEqualTo(ErrorCode.INVALID_REQUEST_STATUS);
```

메서드 호출 횟수와 인자를 검증할 때는 `verify()` 와 `ArgumentCaptor` 를 활용합니다.

```
// 정확히 1회 호출 확인
verify(passwordEncoder, times(1)).matches("password123", "encodedPassword");

// 호출되지 않아야 함 확인
verify(podService, never()).deletePod(any());

// 실제 저장된 값 캡처 후 검증
ArgumentCaptor<PodExternalPort> captor = ArgumentCaptor.forClass(PodExternalPort.class);
verify(podExternalPortRepository, times(2)).save(captor.capture());
assertThat(captor.getAllValues().get(0).getUsagePurpose()).isEqualTo("ssh");
```

의존성이 많아 `@InjectMocks` 가 불안정한 경우, `@BeforeEach` 에서 생성자로 직접 주입합니다.

```
@BeforeEach
void setUp() {
    // @RequiredArgsConstructor 생성자 필드 선언 순서대로 주입
    service = new AdminRequestCommandService(
        alarmService, requestRepository, userRepository, ...
    );
}
```

Controller 통합 테스트

`@WebMvcTest` 로 특정 컨트롤러만 로드합니다. Spring Security는 컨트롤러 테스트 범위를 벗어나므로 `excludeAutoConfiguration` 으로 제외합니다. 모든 컨트롤러 테스트는 `WebMvcTestSupport` 를 상속해 공통 Mock 설정을 받습니다.

```

@WebMvcTest(
    value = AuthController.class,
    excludeAutoConfiguration = {SecurityAutoConfiguration.class}
)
class AuthControllerTest extends WebMvcTestSupport {

    @Autowired MockMvc mockMvc;
    @Autowired ObjectMapper objectMapper;

    @MockitoBean
    private UserLoginService userLoginService;
}

```

`WebMvcTestSupport`는 `SecurityConfig`가 필요로 하는 공통 빈(`JwtProvider`, `CustomUserDetailsService`, `RedisTemplate`, `JpaMetamodelMappingContext`)을 Mock으로 등록한 추상 클래스입니다. 새 컨트롤러 테스트를 작성할 때는 항상 이를 상속합니다.

HTTP 요청은 `mockMvc.perform()`으로, 응답은 `andExpect()`로 검증합니다.

```

// 성공 케이스: 201 Created
mockMvc.perform(post("/api/auth/register")
    .contentType(MediaType.APPLICATION_JSON)
    .content(objectMapper.writeValueAsString(dto)))
    .andExpect(status().isCreated());

// 실패 케이스: 예외 → HTTP 상태 코드 매핑 확인
doThrow(new BusinessException(ErrorCode.USER_ALREADY_EXISTS))
    .when(userLoginService).register(any());

mockMvc.perform(post("/api/auth/register")
    .contentType(MediaType.APPLICATION_JSON)
    .content(objectMapper.writeValueAsString(dto)))
    .andExpect(status().isConflict());

// 응답 본문 JSON 필드 검증
mockMvc.perform(post("/api/auth/login")...)
    .andExpect(status().isOk())
    .andExpect(jsonPath("$.accessToken").value("accessToken"));

```

Repository 테스트

`@DataJpaTest` + `@ActiveProfiles("test")` 를 사용합니다. H2 인메모리 DB로 실제 쿼리를 실행해 검증하므로, JPQL이나 `@Query` 의 적합성을 확인할 수 있습니다.

```
@DataJpaTest
@ActiveProfiles("test")
class UserRepositoryTest {

    @Autowired
    private UserRepository userRepository;

    @BeforeEach
    void setUp() {
        userRepository.save(User.builder()
            .email("active@dgu.ac.kr")
            .password("encoded1")
            .name("홍길동")
            ...
            .build());
    }
}
```

엔티티의 private 필드(예: `lastLoginAt`, `deletedAt`)를 테스트용으로 설정해야 할 때는 `ReflectionTestUtils.setField()` 를 사용합니다. 변경 후에는 `flush()` 를 호출해 DB에 즉시 반영합니다.

```
ReflectionTestUtils.setField(user1, "lastLoginAt", LocalDateTime.now().minusMonths(4));
userRepository.save(user1);
userRepository.flush();

List<User> result = userRepository.findInactiveUsers(LocalDateTime.now().minusMonths(3));
assertThat(result).extracting(User::getEmail).contains("active@dgu.ac.kr");
```

16. 사용자 권한 관리

별도로 유저 `Role` 을 변경하는 API를 제공하지 않습니다. 필요한 경우 직접 서버실을 방문하여 데이터베이스에서 변경합니다.

핵심 설계 패턴

같은 문제를 팀원마다 다른 방식으로 풀면 코드베이스가 제각각이 됩니다. BE에는 "DB 커밋 전에 알림이 나가면 안 된다", "배치가 하루에 두 번 돌아도 중복 발송되면 안 된다"처럼 여러 도메인에서 반복해서 마주치는 문제가 있고, 그때마다 이미 검증된 해결 방식이 정해져 있습니다. 새 기능을 만들다가 낯익은 문제를 만나면, 새로 고민하기 전에 여기에 이미 답이 있는지부터 확인합니다.

1. AFTER_COMMIT — DB 커밋 후 알림 발송

문제

서비스 메서드가 `@Transactional` 안에서 이메일·Slack 알림을 직접 발송하면, DB 커밋 전에 알림이 나갑니다. 이후 트랜잭션이 롤백되면 "DB엔 저장 안 됐지만 이메일은 발송된" 상태가 됩니다.

해결

Spring의 `@TransactionalEventListener(phase = AFTER_COMMIT)` 를 사용합니다. DB 커밋이 확정된 이후에만 이벤트 리스너가 실행되므로, 롤백 시 이벤트가 자동으로 폐기됩니다.

```
// 1. 이벤트 발행 (트랜잭션 안에서)
eventPublisher.publishEvent(new RequestExpiredEvent(userName, userEmail, ...));

// 2. 리스너 (커밋 후 실행)
@TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
public void onRequestExpired(RequestExpiredEvent event) {
    alarmService.sendEmail(event.userEmail(), ...);
}
```

적용 범위

만료 Request 삭제, 관리자 수동 삭제, 승인 완료 등 DB 변경과 알림이 짝을 이루는 모든 흐름에 적용합니다.

2. Self-Injection — 같은 클래스 내 @Transactional 호출

문제

Spring `@Transactional` 은 AOP 프록시로 동작합니다. 같은 클래스 내에서 `this.method()` 로 호출하면 프록시를 우회해 트랜잭션이 적용되지 않습니다.

```
// 잘못된 예 — @Transactional이 적용되지 않음
public void runScheduler() {
    this.deleteExpiredRequest(id); // 프록시 우회
}
```

해결

@Transactional 메서드를 별도 클래스로 분리하고, 해당 Bean을 주입받아 호출합니다. 분리된 Bean은 Spring이 프록시로 감싸기 때문에 트랜잭션이 정상 적용됩니다.

```
// RequestSchedulerService — 루프 제어만 담당
@Service
@RequiredArgsConstructor
public class RequestSchedulerService {

    private final RequestExpiryService requestExpiryService; // 별도 클래스 주입

    public void runScheduler() {
        requestExpiryService.deleteExpiredRequest(id); // 프록시를 통해 호출 → @Transactional 적용
    }
}

// RequestExpiryService — 트랜잭션 경계 담당
@Service
@RequiredArgsConstructor
public class RequestExpiryService {

    @Transactional
    public void deleteExpiredRequest(Long id) { ... }
}
```

Self-injection(`private final RequestSchedulerService self`)도 같은 문제를 해결하는 패턴이지만, 순환 의존 가능성을 피하기 위해 이 프로젝트에서는 별도 클래스 분리 방식을 사용합니다.

적용 범위

스케줄러처럼 루프 안에서 건별 독립 트랜잭션이 필요한 경우에 사용합니다. 한 건이 실패해도 나머지 건이 롤백되지 않습니다.

3. SETNX 중복 방지

문제

스케줄러는 매일 같은 시각에 실행됩니다. 같은 사용자에게 만료 예고 이메일이 하루에 두 번 발송되면 안 됩니다.

해결

Redis **SETNX** (SET if Not eXists)로 "오늘 이미 발송했는지"를 체크합니다. 키가 없으면 발송하고 키를 생성합니다. 키가 이미 있으면 건너뛵니다. TTL을 25시간으로 설정해 하루가 지나면 자동으로 만료됩니다.

```
String key = "email:preexpiry:" + requestId + ":" + dayLabel + ":" + date;
Boolean set = redisTemplate.opsForValue().setIfAbsent(key, "sent", Duration.ofHours(25));
if (Boolean.FALSE.equals(set)) {
    return; // 이미 발송됨 — 건너뛴
}
// 발송 로직
```

Fail-open

Redis 장애 시 **setIfAbsent()** 가 예외를 던집니다. 이때 **false** 를 반환해 이메일 발송을 허용합니다(fail-open). Redis 장애 때문에 사용자가 알림을 못 받는 것보다, 중복 발송이 낫다는 판단입니다.

적용 범위

동일 이벤트의 중복 처리를 막을 때 이 패턴을 사용합니다. 키 형식은 **{도메인}:{식별자}:{날짜}** 로 구성하고, TTL은 실행 주기보다 정확히 여유를 두고 설정합니다. 예를 들어 매일 08:00에 도는 스케줄러(24시간 주기)라면 TTL은 25시간으로, 실행 시각이 다소 밀려도 흡수할 수 있게 1시간을 여유로 둡니다.

4. 3단 보상 트랜잭션

문제

관리자 승인 흐름은 Ubuntu 계정 생성 → Pod 생성 → DB 저장 세 단계입니다. 중간에 실패하면 이미 완료된 단계를 되돌려야 합니다.

해결

각 단계 실패 시 역순으로 정리합니다.

```
Ubuntu 계정 생성 → 성공 → Pod 생성 → 실패 → Ubuntu 계정 삭제 → Request PENDING 복구
→ 실패 → BusinessException 전파 (Pod 생성 안 됐으니 정리 불필요)
```

```

try {
    ubuntuService.createAccount(username);    // 1단계
} catch (Exception e) {
    throw new BusinessException(UBUNTU_CREATION_FAILED); // 정리 불필요
}

try {
    podService.createPod(username);           // 2단계
} catch (Exception e) {
    ubuntuService.deleteAccount(username);    // 1단계 보상
    request.revertToPending();
    throw new BusinessException(POD_CREATION_FAILED);
}

// 3단계: DB 저장 (@Transactional 롤백이 자동 처리)

```

주의

보상 작업(`deleteUbuntuAccount`)이 실패해도 원래 예외(`POD_CREATION_FAILED`)를 그대로 전파합니다. 보상 실패는 로그로만 남기고, Slack으로 관리자에게 알립니다.

5. Redis 큐를 통한 비동기 처리

문제

Slack Webhook API는 채널당 초당 1건의 rate limit이 있습니다. 비즈니스 로직(승인, 삭제 등)이 Slack API 응답을 기다리면 전체 처리가 블로킹됩니다.

해결

알림이 발생하면 메시지를 Redis List에 즉시 적재하고(`R PUSH`), 별도 워커가 1초마다 꺼내(`L POP`) Slack API를 호출합니다.

```

비즈니스 로직 → R PUSH slack:notification:queue → 즉시 반환
SlackNotificationWorker → L POP (1초 간격) → Slack API 호출
                        → 실패 시 R PUSH 재삽입 (최대 3회)

```

비즈니스 로직은 Slack API 레이턴시 영향을 받지 않습니다. 워커가 순서대로 처리하므로 rate limit도 자연스럽게 지켜 집니다.

같은 패턴을 Infra 관련 알림에도 동일하게 적용합니다. 다만 큐(`slack:infra:notification:queue`)와 이를 소비하는 Worker(`InfraSlackNotificationWorker`)를 완전히 별도로 둡니다. 두 큐·두 Worker를 분리한 이유는 한쪽 알림이 밀리거나 실패해도 다른 쪽 발송에 영향을 주지 않게 하기 위해서입니다. 자세한 내용은 [Redis 키 카탈로그 4번](#)을 참고합니다.

적용 범위

rate limit이 있거나 응답이 느린 외부 API 호출을 비동기로 처리할 때 이 패턴을 사용합니다.

인증·보안

토큰을 왜 굳이 두 개로 나눴는지부터 설명하고, 거기서 로그인·재발급·로그아웃 흐름과 모든 요청을 가로채는 필터의 판단 기준까지 이어갑니다. 인증 코드는 한 군데만 잘못 고쳐도 전체 로그아웃되거나, 반대로 탈취된 세션이 계속 살아있는 사고로 이어질 수 있습니다. 그래서 "어떻게 동작하는가"보다 "왜 이렇게 만들었는가"를 먼저 알아야 안전하게 고칠 수 있습니다.

JWT와 Redis를 조합해 인증을 처리합니다. 여기서 AccessToken은 **AT**, RefreshToken은 **RT**로 줄여 씁니다. OAuth 스펙 용어는 아니고(스펙엔 `access_token` / `refresh_token` 으로 풀어 씀) 이 프로젝트에서만 쓰는 표현인데, RT는 실제로 코드에도 그대로 박혀 있습니다 — Redis 키를 `"RT:" + userId` 처럼 직접 만듭니다(`TokenService`, `UserLoginService`). AT는 Redis에 저장하지 않는 stateless 토큰이라 코드엔 등장하지 않고, RT와 짝을 맞추려고 문서에서만 쓰는 표현입니다.

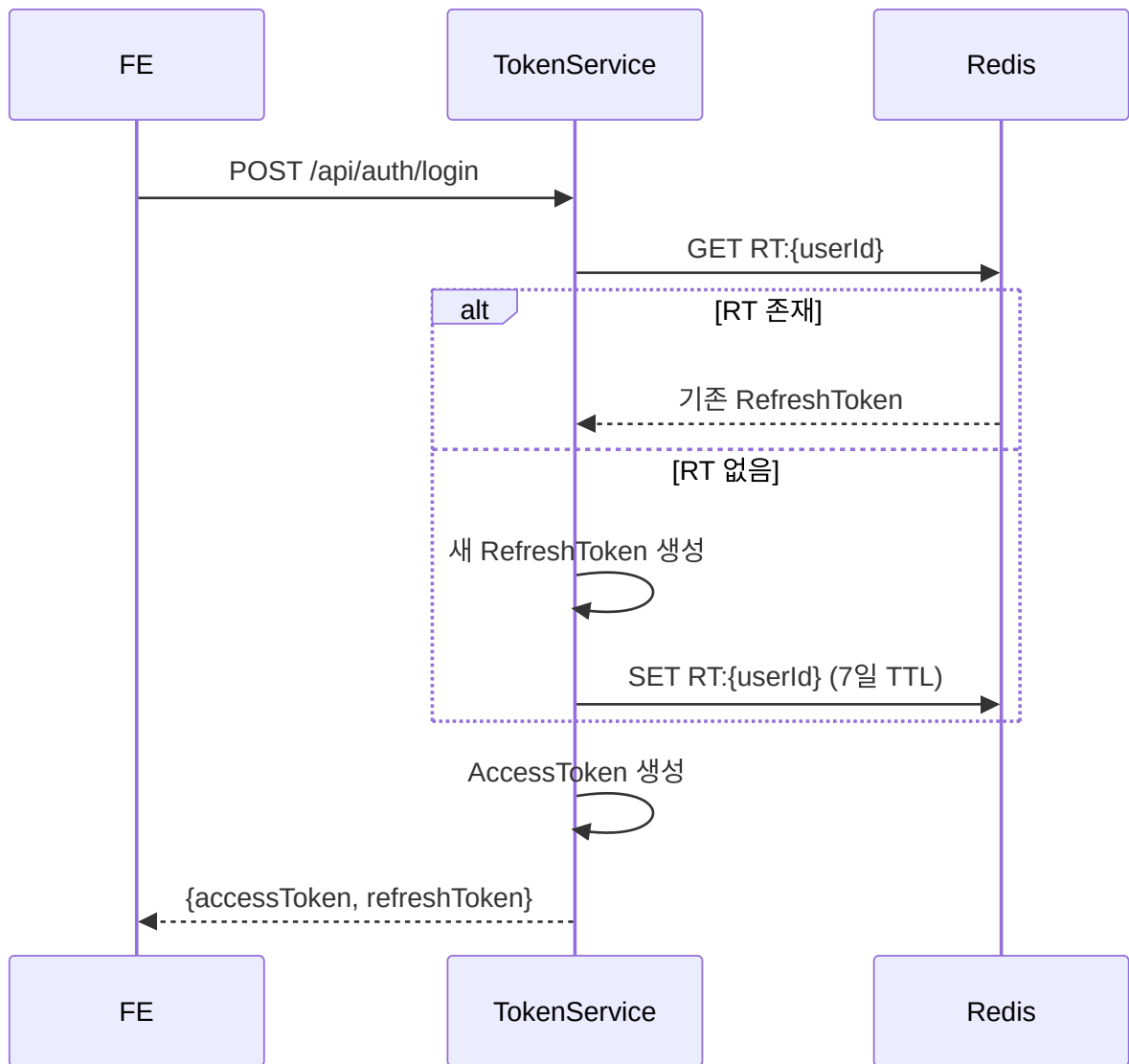
AT와 RT를 분리하는 이유는 **검증 속도**와 **세션 제어** 두 가지입니다. AT는 모든 API 요청에 붙어오기 때문에 Redis나 DB 조회 없이 서명만 검증해 빠르게 처리합니다(Stateless). 반면 RT는 Redis에 저장하기 때문에 언제든지 삭제해 즉시 세션을 종료할 수 있습니다. 로그아웃하면 Redis에서 RT를 지우고, 이후 AT가 만료돼 재발급을 시도할 때 Redis에 키가 없으므로 거부됩니다.

AccessToken	RefreshToken	TTL	application.yml 설정값 (기본 1시간)
application.yml 설정값 (기본 7일)	저장 위치	저장 안 함 (Stateless)	Redis RT:{userId}
검증 방식	서명만 확인, I/O 없음	Redis 조회 필요	사용 시점
모든 API 요청	AT 만료 시 재발급 한 번	세션 종료	만료 전 직접 취소 불가
Redis 키 삭제로 즉시 무효화			

토큰의 `subject` 에는 `userId` (Long)가 담깁니다. 서명 알고리즘은 HS256이며, secret key는 `application.yml` 의 `jwt.secret` 에서 읽어옵니다.

1. 토큰 발급 (로그인)

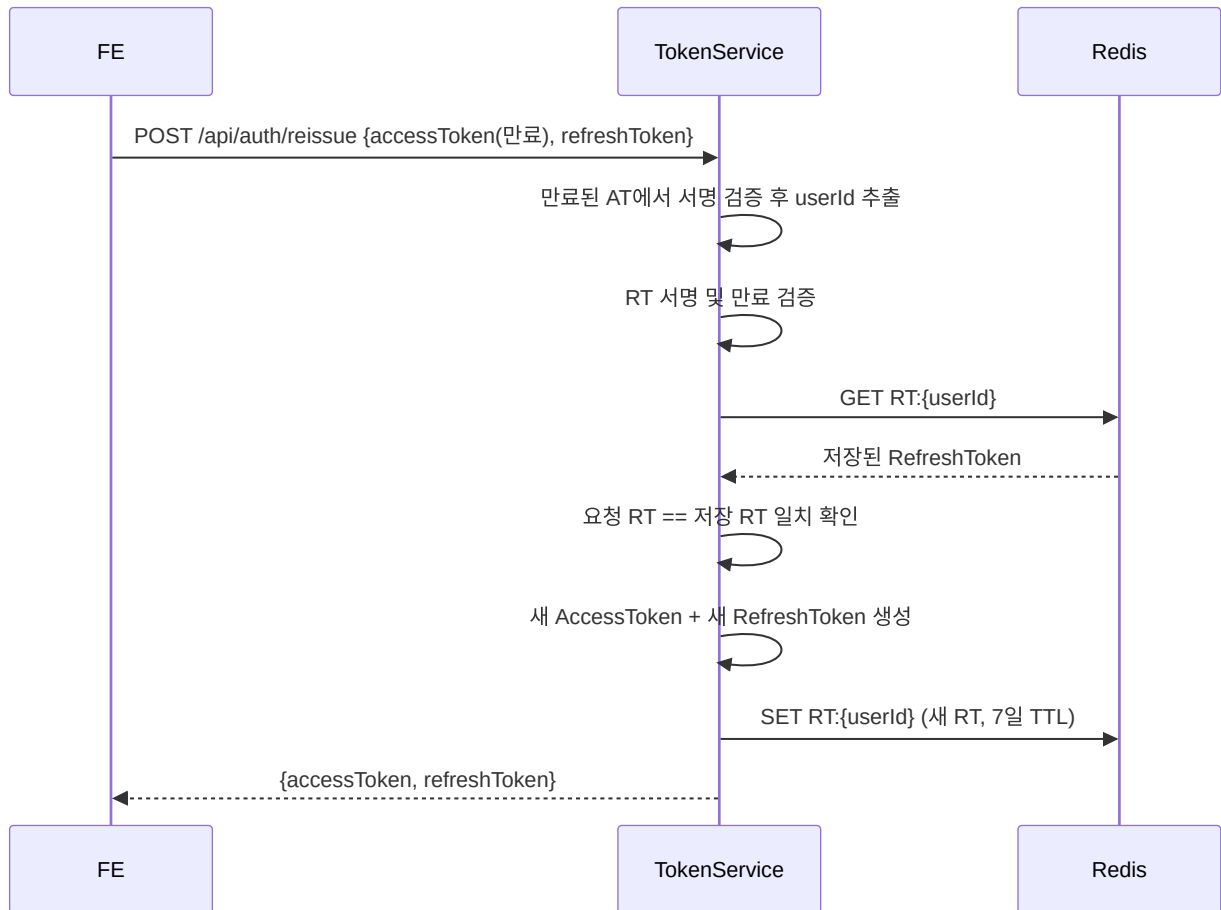
로그인 성공 시 AccessToken과 RefreshToken을 발급합니다. RefreshToken은 Redis에 이미 유효한 토큰이 있으면 재사용하고, 없으면 새로 발급합니다. 이 덕분에 여러 기기에서 로그인해도 RT가 중복 발급되지 않습니다.



2. 토큰 재발급

AT가 만료되면 클라이언트는 보관 중인 RT로 재발급을 요청합니다. 재발급 시 AT와 RT 모두 새로 발급됩니다. RT도 함께 교체하는 이유는 보안 때문입니다. RT가 탈취된 상황에서 정상 사용자가 먼저 재발급을 완료하면 Redis의 값이 새 RT로 교체되므로, 공격자가 이전 RT로 시도할 때 Redis 값과 일치하지 않아 거부됩니다.

재발급 흐름에서 주의할 점이 하나 있습니다. AT가 만료된 상태이므로 일반적인 검증(`parseClaimsJws`)을 쓰면 예외가 납니다. `JwtProvider`의 `getSubjectFromExpiredToken()`은 만료된 AT에서도 서명이 유효하면 `ExpiredJwtException`의 claims를 꺼내 `userId`를 반환합니다. 서명 자체가 잘못된 경우에는 예외가 그대로 전파돼 재발급이 거부됩니다.

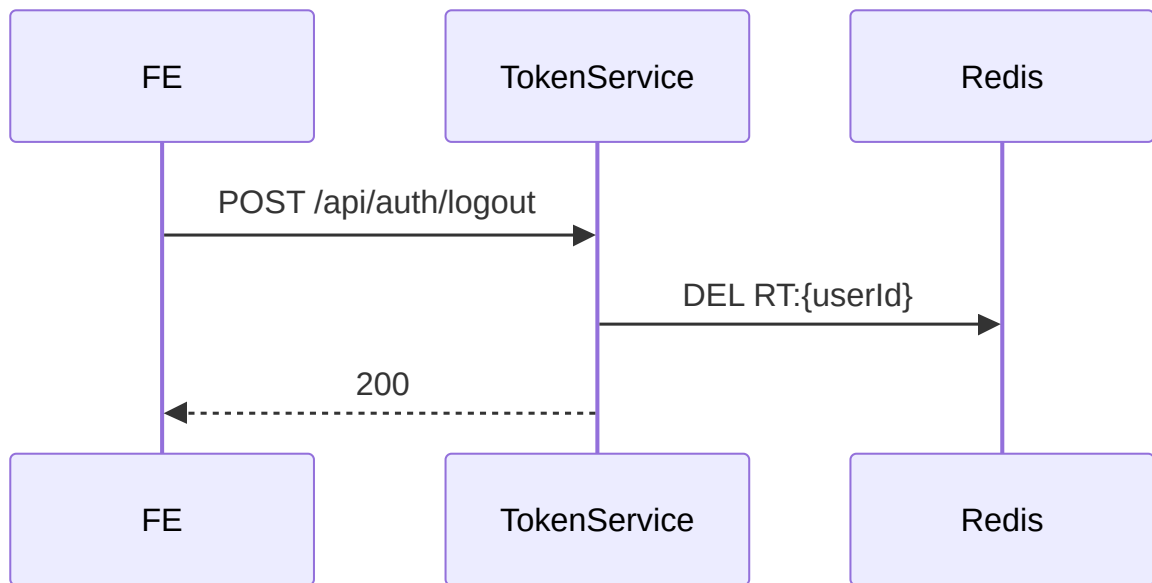


재발급이 거부되는 경우:

- 만료된 AT의 서명이 유효하지 않을 때
- RT가 만료됐을 때
- RT가 Redis에 없을 때 (로그아웃된 계정)
- 요청의 RT와 Redis에 저장된 RT가 다를 때

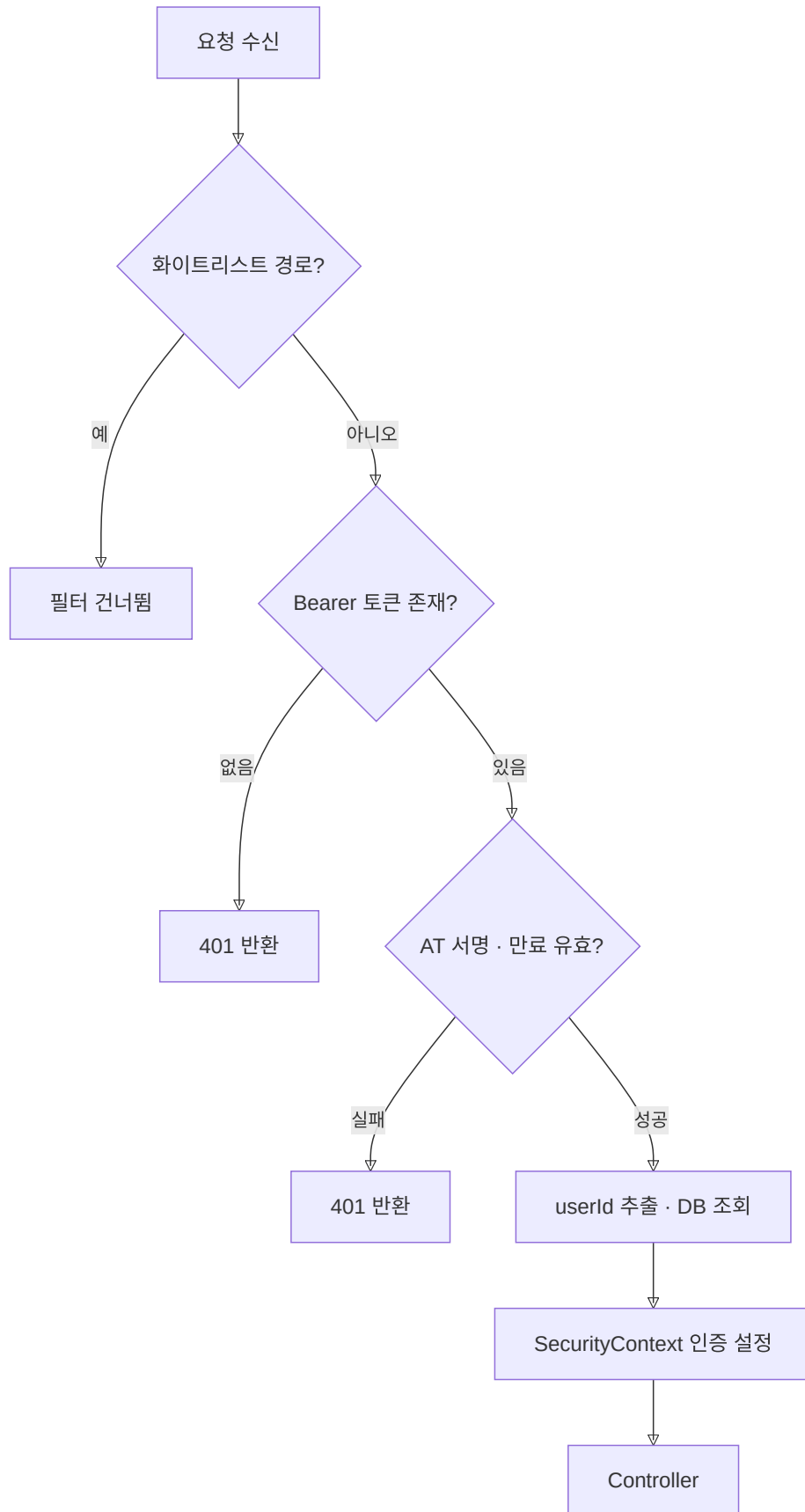
3. 로그아웃

Redis에서 `RT:{userId}` 키를 삭제합니다. 이후 해당 `userId`의 RT로 재발급 요청이 들어오면 Redis에 키가 없어 거부됩니다. AccessToken은 서버에 저장하지 않으므로 만료 전까지는 유효하지만, 짧은 TTL(1시간) 덕분에 실제 보안 영향은 제한됩니다.



4. 요청 인증 흐름 (JwtAuthenticationFilter)

모든 API 요청은 `JwtAuthenticationFilter` 를 통과합니다. `OncePerRequestFilter` 를 상속하므로 요청당 정확히 한 번만 실행됩니다.



필터가 건너뛰는 화이트리스트 경로(`UNPROTECTED_PATHS`)는 [운영 가이드 8절\(화이트리스트 설정\)](#)에서 관리합니다.

AccessToken은 `Authorization: Bearer {token}` 헤더에서 추출합니다. 헤더가 없거나 `Bearer` 로 시작하지 않으면 즉시 401을 반환합니다.

검증 실패 시 필터에서 바로 JSON 에러 응답을 씁니다. Spring의 `ExceptionHandlerFilter` 를 거치지 않기 때문에 `GlobalExceptionHandler` 가 아닌 필터 내부의 `setErrorResponse()` 에서 직접 응답을 만듭니다.

5. 역할 기반 접근 제어

사용자 `Role` 은 `USER` 와 `ADMIN` 두 가지입니다. 관리자 기능은 `/api/admin/**` 경로로 분리되어 있으며, `SecurityConfig` 에서 해당 경로에 `ADMIN` 권한을 요구합니다.

`Role` 을 변경하는 별도 API는 제공하지 않습니다. 필요한 경우 MySQL에 직접 접속해 수정합니다. 접속 방법은 [운영 가이드 7절\(MySQL 직접 접속\)](#)을 참고합니다.

외부 연동

BE는 계정을 만들거나 컨테이너를 띄우는 실제 작업을 스스로 하지 않고 Infra Server라는 바깥 시스템에 맡깁니다. 그 경계 너머에서 무슨 일이 벌어지는지, 그리고 경계 너머가 실패했을 때 BE가 자기 상태를 어떻게 지키는지를 다룹니다. "승인을 눌렀는데 왜 계정이 안 만들어지지", "Pod는 생겼는데 왜 신청이 PENDING으로 돌아갔지" — 이런 질문의 답은 대부분 여기 있습니다.

BE는 Ubuntu 계정·Pod 생성·삭제를 직접 구현된 Infra Server에 위임합니다. 두 종류의 작업은 타임아웃 특성이 달라 WebClient 인스턴스를 분리해 관리합니다.

WebClient	용도	타임아웃
<code>configWebClient</code>	Ubuntu 계정 생성·삭제	응답 120s(<code>config.timeout-seconds</code>), 연결 10s
<code>podWebClient</code>	Pod 생성·삭제	응답 300s(<code>config.pod-timeout-seconds</code>), 연결 30s

Bean 정의는 `WebClientConfig` 에 있습니다.

Pod 생성에 긴 타임아웃을 두는 이유는 Docker 이미지를 처음 pull할 때 시간이 크게 걸릴 수 있기 때문입니다. 두 WebClient 모두 같은 `config.base-url` (Infra Server)을 바라봅니다.

1. Ubuntu 계정 관리 (configWebClient)

계정 생성

관리자 승인 시 `AdminRequestCommandService` 에서 호출합니다. 생성 성공 시 Infra Server가 반환한 `uid` 와 `gid` 를 Request에 저장합니다.

요청

```
PUT /accounts/users
Content-Type: application/json
```

```
{
  "name": "ubuntu_username",
  "passwd_base64": "base64로 인코딩된 초기 비밀번호",
  "gecos": "사용자 실명",
  "primary_group_name": "ubuntu_username",
  "supplementary_groups": [
    { "name": "ailab", "gid": 2001 }
  ]
}
```

필수 필드는 `name` 과 `passwd_base64` 이며, 나머지는 선택입니다. `supplementary_groups` 는 사용자를 추가 그룹에 소속시킬 때 사용됩니다.

응답 (HTTP 201)

Swagger 스펙에 응답 스키마가 명시되어 있지 않으나, BE는 응답 본문에서 `uid` 와 `gid` 를 읽어 Request에 저장합니다.

계정 삭제

`UbuntuAccountService.deleteUbuntuAccount()` 에서 호출합니다. 만료 처리 스케줄러와 보상 트랜잭션 두 경로에서 호출됩니다.

```
DELETE /accounts/users/{username}
```

404 응답은 계정이 이미 없는 것이므로 정상 처리로 간주하고 넘어갑니다. 이미 지워진 계정을 다시 삭제 요청해도 결과는 똑같이 "계정 없음"이라, 같은 요청을 여러 번 보내도 안전합니다. 그래서 보상 트랜잭션 경로에서 계정이 애초에 생성되지 않았을 때도 걱정 없이 호출할 수 있습니다. 400이나 다른 에러 코드는

`BusinessException(UBUNTU_USER_DELETION_FAILED)` 으로 변환됩니다.

2. Pod 관리 (podWebClient)

Pod 생성

승인 처리 중 Ubuntu 계정 생성 성공 후 호출합니다. 응답에서 `podName`, `node`, `ports` 정보를 받아 `PodExternalPort` 테이블에 저장합니다.

요청

```
POST /create-pod
Content-Type: application/json
```

```
{
  "username": "ubuntu_username"
}
```

응답 (HTTP 201)

```
{
  "status": "created",
  "node": "farm1",
  "pod_name": "ai lab-user2100-1",
  "ports": [
    { "usage_purpose": "ssh", "internal_port": 22, "external_port": 30022 },
    { "usage_purpose": "jupyter", "internal_port": 8888, "external_port": 30888 }
  ]
}
```

4xx/5xx 응답이 오거나 응답 본문의 `pod_name` 이 null이면 `BusinessException(POD_CREATION_FAILED)` 를 던집니다. 이 예외는 보상 트랜잭션을 트리거해 앞서 생성한 Ubuntu 계정을 삭제하고 Request 상태를 PENDING으로 되돌립니다.

Pod 삭제

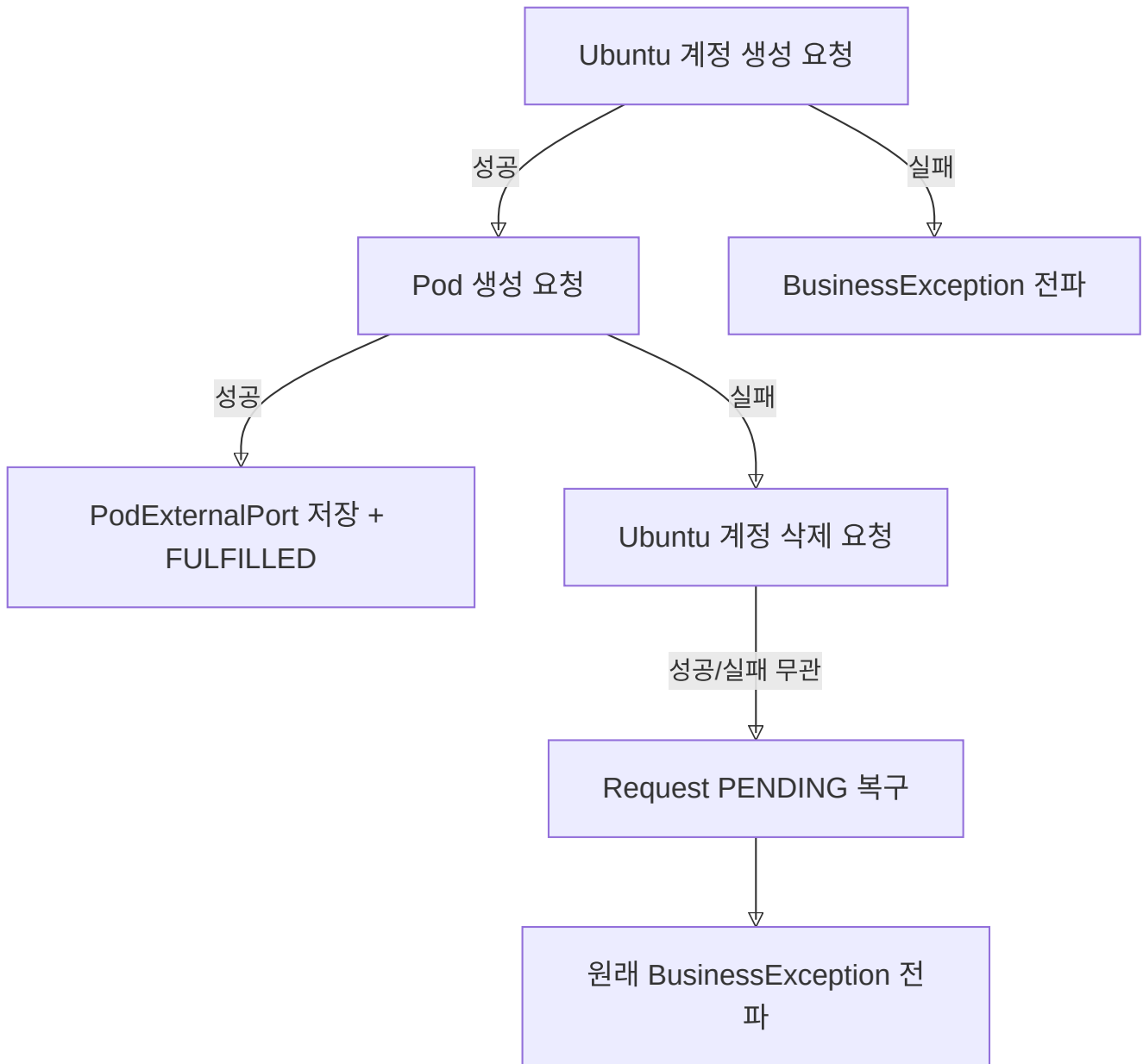
```
POST /delete-pod
Content-Type: application/json
```

```
{
  "pod_name": "ai lab-user2100-1"
}
```

`pod_name` 이 null이면 호출 자체를 건너뛰니다(이미 Pod가 없는 경우를 대비). 400이나 500 응답은 `BusinessException` 으로 변환됩니다.

3. 에러 처리와 보상 트랜잭션 연계

Ubuntu 계정 생성 → Pod 생성 순서로 진행하므로, 중간에 실패하면 이전 단계를 되돌려야 합니다.



Pod 생성 실패 시 Ubuntu 계정 삭제(`deleteUbuntuAccount`)가 추가로 실패하더라도 원래 예외(`POD_CREATION_FAILED`)가 그대로 전파됩니다. 로그에는 두 실패 모두 남습니다.

4. Prometheus 연동

FE 대시보드가 `GET /api/monitoring/metrics`를 폴링하면, BE가 Prometheus HTTP API에 PromQL 쿼리를 날려 결과를 가공해 반환합니다. 이 엔드포인트는 인증 없이 접근 가능한 화이트리스트 경로(`/api/monitoring/**`)입니다.

Prometheus 서버 주소는 `application.yml`의 `prometheus.base-url`에서 읽으며, 기본값은 클러스터 내부 주소(`http://10.98.198.241:9090`)입니다. 각 쿼리는 5초 타임아웃이 적용되고, 실패 시 예외를 전파하지 않고 빈 값을 반환합니다. Prometheus가 일시적으로 내려가도 대시보드 API 자체는 200으로 응답합니다.

실행하는 PromQL 쿼리

쿼리	설명
<code>avg by(Hostname)(DCGM_FI_DEV_GPU_UTIL)</code>	서버별 평균 GPU 사용률 (%)
<code>count by(Hostname)(DCGM_FI_DEV_GPU_UTIL)</code>	서버별 GPU 개수
<code>sum by(cluster)(cluster_monitor_container_running)</code>	클러스터별 활성 컨테이너 수

GPU 메트릭은 DCGM(NVIDIA Data Center GPU Manager) Exporter가 수집합니다. 활성 컨테이너 수는 별도로 구현된 `cluster_monitor_container_running` 메트릭에서 읽어옵니다.

응답 구조

```
{
  "gpuServers": [
    { "hostname": "farm1", "gpuUtil": 43.2, "gpuCount": 4 },
    { "hostname": "lab1", "gpuUtil": 12.5, "gpuCount": 2 }
  ],
  "activeContainers": {
    "farm": 8,
    "lab": 3
  }
}
```

`gpuUtil` 은 소수점 한 자리로 반올림해서 내려줍니다. 서버 목록은 `hostname` 기준 알파벳 순으로 정렬됩니다.

Redis 키 카탈로그

Redis는 코드를 안 보면 지금 이 키가 왜 이 값을 갖고 있는지 짐작하기 어렵습니다. 장애 상황에서 코드를 뒤질 필요 없이 "이 키가 뭘 의미하고 지워도 되는지, 왜 이 TTL인지"를 아래 표만 보고 판단할 수 있게 정리했습니다. 장애 대응·디버깅은 물론 새 캐시나 중복 방지 키를 설계할 때도 여기를 기준으로 삼습니다.

전체 요약

분류	키 패턴	타입	TTL
인증	<code>email:verify:{email}</code>	String	5분
인증	<code>VERIFIED:{email}</code>	String	10분
세션	<code>RT:{userId}</code>	String	7일

분류	키 패턴	타입	TTL
이메일 중복 방지	<code>email:preexpiry:{requestId}:{dayLabel}:{date}</code>	String	25시간
Slack 큐	<code>slack:notification:queue</code>	List	없음
Slack 큐	<code>slack:infra:notification:queue</code>	List	없음
Slack 캐시	<code>slack:cache:users:list</code>	String	1시간

1. 인증 키

이메일 인증 흐름에서 사용하는 두 개의 키입니다. 둘 다 TTL이 지나면 자동으로 사라지고, 정상 경로에서는 완료 즉시 명시 삭제됩니다.

`email:verify:{email}`

이메일 인증번호를 임시 저장합니다.

저장 POST /api/auth/email/send → SET email:verify:user@example.com = "183726" (TTL 5분)
 확인 POST /api/auth/email/verify → GET email:verify:user@example.com
 삭제 인증 성공 직후 → DEL email:verify:user@example.com
 (5분 안에 인증 안 하면 TTL 만료로 자동 삭제)

항목	값
값 형태	숫자 인증번호 문자열 (ex: <code>"183726"</code>)
TTL	5분 (<code>AUTH_CODE_EXPIRE_SECONDS = 300</code>)

`VERIFIED:{email}`

인증번호 확인이 완료됐음을 나타내는 상태 키입니다. 이 키가 없으면 회원가입 API가 400을 반환합니다.

저장 인증번호 일치 확인 후 → SET VERIFIED:user@example.com = "true" (TTL 10분)
 삭제 회원가입 완료 직후 → DEL VERIFIED:user@example.com
 (10분 안에 가입 안 하면 TTL 만료로 자동 삭제)

항목	값
값 형태	<code>"true"</code>

항목	값
TTL	10분

TTL 설계 이유: 인증번호 확인(5분)과 회원가입 폼 제출(10분)을 단계별로 분리한 이유는, 사용자가 인증 직후 바로 가입 폼을 제출하지 않을 수 있기 때문입니다. 더 짧은 TTL을 가진 `email:verify` 키를 먼저 삭제하고, `VERIFIED` 키에는 5분 더 긴 TTL(10분)을 줍니다.

흐름 요약



2. 세션 키

`RT:{userId}`

Refresh Token을 저장하는 핵심 세션 키입니다. 이 키가 없으면 해당 사용자는 로그아웃 상태입니다.

저장 로그인 성공	→ SET RT:42 = "<JWT>" (TTL 7일)
갱신 토큰 재발급	→ SET RT:42 = "<새 JWT>" (TTL 7일, 기존 키 재사용 가능)
삭제 로그아웃	→ DEL RT:42
(7일 미갱신 시 TTL 만료로 자동 삭제 — 자동 로그아웃)	

항목	값
값 형태	Refresh Token JWT 문자열
TTL	7일 (<code>jwt.refresh-token-expire-time</code>)
<code>{userId}</code>	User 테이블의 PK (<code>BIGINT</code>)

설계 포인트: Access Token은 Redis에 저장하지 않습니다(Stateless). Refresh Token만 Redis에 저장해서, 필요시 `DEL RT:{userId}` 로 즉시 세션을 무효화(강제 로그아웃)할 수 있습니다. 자세한 내용은 [인증·보안](#)을 참고합니다.

확인 방법

```
# 사용자 42번이 로그인 상태인지
EXISTS RT:42      # 1 = 로그인, 0 = 로그아웃

# 남은 세션 시간 (초)
TTL RT:42        # -2 = 만료됨, 양수 = 남은 초

# 현재 로그인된 사용자 전체 조회
KEYS RT:*
```

3. 이메일 중복 방지 키

email:preexpiry:{requestId}:{dayLabel}:{date}

스케줄러가 같은 날 같은 요청에 만료 예고 이메일을 두 번 보내지 않도록 막습니다.

키 예시: email:preexpiry:42:7일:2026-07-15

체크 이메일 발송 직전 → SETNX email:preexpiry:42:7일:2026-07-15 "sent" (TTL 25h)
키가 이미 있으면 → 발송 건너뛰
키가 없으면 → 발송 진행 후 키 생성
삭제 TTL 만료 (25시간 후 자동)

항목	값
값 형태	"sent"
TTL	25시간 (24시간 주기보다 1시간 길게 — 스케줄러 실행 시각 편차 흡수)
{dayLabel}	7일, 3일, 1일 (만료까지 남은 날 수)
{date}	만료 목표 날짜 yyyy-MM-dd (today+7/3/1 중 해당일, 발송 당일이 아님)

Fail-open 설계: Redis 장애 시 `setIfAbsent()` 가 예외를 던집니다. 이 경우 `false`를 반환해 이메일 발송을 허용합니다(fail-open). 중복 발송보다 누락이 더 나쁘다는 판단입니다. 자세한 패턴은 [핵심 설계 패턴 3번](#)을 참고합니다.

확인 방법

```
# 특정 요청의 중복 방지 키 전체 조회
KEYS email:preexpiry:42:*

# 오늘자 전체 만료 예고 키 조회
KEYS email:preexpiry:*:2026-07-15
```

4. Slack 알림 큐

Slack Webhook API는 채널당 초당 1건의 rate limit이 있습니다. 비즈니스 로직이 직접 Slack API를 호출하면 블로킹됩니다. 대신 Redis List를 큐로 사용해 비동기 처리합니다.

공통 구조

알림 발생 시 → RPush (오른쪽 삽입, FIFO)
전용 Worker → LPop (왼쪽 꺼내기, 1초 간격)
발송 실패 → RPush로 재삽입 (최대 3회, 3회 초과 시 폐기)

두 큐는 소비하는 Worker 클래스도 분리되어 있습니다. `slack:notification:queue`는 `SlackNotificationWorker`가, `slack:infra:notification:queue`는 `InfraSlackNotificationWorker`가 각각 전담합니다. 서로 완전히 독립 운영되므로 한쪽 큐가 밀려도 다른 쪽 발송에는 영향이 없습니다.

`slack:notification:queue`

일반 비즈니스 알림 큐입니다 (신청 접수, 승인, 거절, 삭제 등). `SlackNotificationWorker`가 소비합니다.

항목	값
값 형태	<code>SlackNotificationDTO</code> JSON 직렬화
TTL	없음 (영구 보존, 꺼낼 때까지 유지)

`slack:infra:notification:queue`

Infra 서버 관련 알림 전용 큐입니다. `slack:notification:queue`와 별도로 운영해 우선순위나 처리 경로를 분리합니다. `InfraSlackNotificationWorker`가 소비합니다.

항목	값
값 형태	<code>slack:notification:queue</code> 와 동일

확인 방법

```
# 현재 큐에 쌓인 메시지 수
LLEN slack:notification:queue
LLEN slack:infra:notification:queue

# 큐 내용 조회 (삭제하지 않고 확인)
LRANGE slack:notification:queue 0 -1
LRANGE slack:infra:notification:queue 0 -1
```

큐 길이가 계속 늘어난다면 `SlackNotificationWorker` 가 멈췄거나 Slack API 장애를 의심합니다. 자세한 대응은 [운영 가이드 11절](#)을 참고합니다.

동작을 직접 확인하고 싶을 때

메시지를 수동으로 넣어보고 Worker가 가져가는 과정을 실시간으로 볼 수 있습니다. DTO 필드는 실제 `SlackNotificationDTO` 와 맞춰야 역직렬화 예러가 나지 않습니다.

```
# 큐에 테스트 메시지 하나 삽입 (RPUSH는 오른쪽 끝에 추가)
RPUSH slack:notification:queue '{"type":"DM","username":"test_user","email":"test@test.com","message":"테스트"}'
# 결과가 (integer) 1이면 큐에 1건 쌓인 것

# 별도 터미널에서 실시간 명령 로그 확인
MONITOR

# 이 상태에서 스케줄러나 알림을 트리거하면 RPUSH → LPPOP이 순서대로 찍히는 것을 볼 수 있음
```

5. Slack 캐시

`slack:cache:users:list`

Slack 워크스페이스 멤버 목록을 캐싱합니다. DM 발송 시 이메일 → Slack UserId 변환에 사용됩니다. 매번 Slack `users.list` API를 호출하면 rate limit에 걸릴 수 있어 1시간 TTL로 캐싱합니다.

```
저장 Slack users.list API 호출 후 → SET slack:cache:users:list = <멤버 목록 JSON> (TTL 1h)
삭제 TTL 만료 (1시간 후 자동)
```

항목	값
값 형태	Slack 멤버 목록 JSON 직렬화
TTL	1시간

redis-cli 빠른 참조

—— Redis Pod 접속

```
kubectl exec -it <redis-pod-name> -n <namespace> -- redis-cli
```

—— 키 존재 여부 / 값 / TTL ——

EXISTS RT:42 # 1 = 있음, 0 = 없음

GET RT:42 # 값 조회

TTL RT:42 # 남은 TTL (초). -2 = 만료/없음

—— 인증 관련

EXISTS VERIFIED:user@example.com # 인증 완료 상태 확인

KEYS email:verify:* # 인증번호 대기 중인 이메일 목록

—— Slack 큐

LLEN slack:notification:queue # 큐 메시지 수

LRANGE slack:notification:queue 0 4 # 앞에서 5개 확인 (삭제 안 함)

—— 만료 예고 중복 방지 ——

KEYS email:preexpiry:42:* # requestId=42의 발송 이력

KEYS email:preexpiry:* # 전체 발송 이력 (주의: 많으면 느림)

—— 운영 환경 전체 키 스캔 (KEYS * 대신 사용) ——

```
SCAN 0 MATCH * COUNT 100
```

KEYS * 명령은 Redis를 블로킹합니다. 운영 환경에서는 반드시 **SCAN** 을 사용하세요.

에러 코드 카탈로그

API가 실패했을 때 클라이언트가 받는 에러 메시지와 HTTP 상태 코드는 전부 **ErrorCode** enum 한 곳에서 정의합니다. "이 에러가 무슨 뜻이고 어떤 상황에서 나는지"를 코드를 열어보지 않고 여기서 바로 찾을 수 있습니다. FE가 특정 에러 메시지를 받고 원인을 추적할 때, 또는 새 에러 케이스를 추가하기 전 비슷한 케이스가 이미 있는지 확인할 때 먼저 봅니다.

전체 59개 케이스가 있으며, **ErrorCode** 에 정의된 그룹 순서 그대로 정리했습니다. 새 에러를 추가할 때는 관련된 그룹 (예: Group Error, Pod Error) 안에 추가하고, 마땅한 그룹이 없다면 새 그룹 주석을 만듭니다.

위치: **DGU_AI_LAB.admin.be.error.ErrorCode**

HTTP 상태 코드 공통 에러

코드	상태	메시지
BAD_REQUEST	400	잘못된 요청입니다.
INVALID_INPUT_VALUE	400	입력값이 올바르지 않습니다.
MISSING_REQUEST_PARAMETER	400	필수 요청 파라미터가 누락되었습니다.
UNAUTHORIZED	401	리소스 접근 권한이 없습니다.
INVALID_ACCESS_TOKEN	401	액세스 토큰의 형식이 올바르지 않습니다. Bearer 타입을 확인해 주세요.
INVALID_ACCESS_TOKEN_VALUE	401	액세스 토큰의 값이 올바르지 않습니다.
EXPIRED_ACCESS_TOKEN	401	액세스 토큰이 만료되었습니다. 재발급 받아주세요.
INVALID_REFRESH_TOKEN	401	리프레시 토큰의 형식이 올바르지 않습니다.
INVALID_REFRESH_TOKEN_VALUE	401	리프레시 토큰의 값이 올바르지 않습니다.
EXPIRED_REFRESH_TOKEN	401	리프레시 토큰이 만료되었습니다. 다시 로그인해 주세요.
NOT_MATCH_REFRESH_TOKEN	401	일치하지 않는 리프레시 토큰입니다.
FORBIDDEN	403	리소스 접근 권한이 없습니다.
ACCESS_DENIED	403	접근이 거부되었습니다.
RESOURCE_NOT_FOUND	404	해당 리소스를 찾을 수 없습니다.
ENTITY_NOT_FOUND	404	엔티티를 찾을 수 없습니다.
METHOD_NOT_ALLOWED	405	잘못된 HTTP method 요청입니다.
CONFLICT	409	이미 존재하는 리소스입니다.
FILE_SIZE_EXCEEDED	413	이미지 최대 크기를 초과하였습니다.
INTERNAL_SERVER_ERROR	500	서버 내부 오류입니다.
FILE_UPLOAD_FAILED	500	파일 업로드에 실패하였습니다.
JSON_PARSING_FAILED	500	JSON 파싱에 실패하였습니다.
SLACK_DM_CHANNEL_FAILED	502	Slack DM 채널 열기를 실패하였습니다.
EXTERNAL_API_ERROR	502	외부 API 호출에 실패했습니다.
SLACK_SEND_FAILED	503	Slack 메시지 전송에 실패하였습니다.

이 그룹은 특정 도메인에 묶이지 않는 범용 에러입니다. 도메인 로직에서 던질 에러가 마땅치 않을 때가 아니라면, 아래 도메인별 에러를 우선 사용합니다.

사용자(User) 에러

코드	상태	메시지
EMAIL_NOT_VERIFIED	401	이메일 인증이 완료되지 않았습니다.
AUTHENTICATION_FAILED	401	사용자 인증에 실패하였습니다.
USER_NOT_FOUND	404	사용자를 찾을 수 없습니다.
USER_ALREADY_EXISTS	409	이미 가입된 사용자입니다.
USER_ALREADY_ACTIVE	409	이미 활성화된 사용자입니다.
DUPLICATE_NAME	409	중복된 닉네임입니다.
INVALID_LOGIN_INFO	400	잘못된 로그인 입력값입니다.
INVALID_AUTH_CODE	400	올바르지 않은 인증 코드입니다.
GROUP_NOT_FOUND	404	지정된 그룹을 찾을 수 없습니다.
UID_ALLOCATION_FAILED	502	외부 API 응답에서 UID/GID를 확인할 수 없습니다.
DUPLICATE_USERNAME	409	이미 사용하고 있는 username입니다. 같은 사용자이더라도 다른 username을 입력해주세요.
ACCOUNT_DISABLED	404	비활성화된 유저입니다. 관리자에게 문의하세요.
SLACK_USER_NOT_FOUND	404	Slack 사용자를 찾을 수 없습니다.
SLACK_USER_EMAIL_NOT_MATCH	404	이메일이 일치하는 Slack 사용자를 찾을 수 없습니다.
INVALID_PASSWORD	401	현재 비밀번호가 일치하지 않습니다.
PASSWORD_CHANGE_SAME_AS_OLD	400	새 비밀번호가 현재 비밀번호와 동일합니다.

그룹(Group) 에러

코드	상태	메시지
NO_AVAILABLE_GROUPS	404	존재하는 그룹 정보가 없습니다.
DUPLICATE_GROUP_ID	409	외부 API: 그룹명 또는 GID 충돌
DUPLICATE_GROUP_NAME	409	중복된 그룹 이름입니다.
GROUP_CREATION_FAILED	502	외부 API: 필수 필드 누락 또는 형식 오류

코드	상태	메시지
GID_ALLOCATION_FAILED	502	외부 API 응답에서 GID를 확인할 수 없습니다.
FORBIDDEN_REQUEST	400	요청된 우분투 사용자 이름은 로그인한 사용자의 계정이 아닙니다.
INVALID_GROUP_MEMBER	400	존재하지 않는 사용자입니다.

DUPLICATE_GROUP_ID , GROUP_CREATION_FAILED 처럼 "외부 API:"로 시작하는 메시지는 Infra Server 응답을 그대로 반영한 것입니다. Infra Server 쪽 원인까지 확인하려면 [외부 연동](#)을 참고합니다.

승인(Approval) 에러

코드	상태	메시지
USER_APPROVAL_NOT_FOUND	404	해당 유저의 승인 정보를 찾을 수 없습니다.

리소스 그룹(ResourceGroup) 에러

코드	상태	메시지
RESOURCE_GROUP_NOT_FOUND	404	리소스 그룹을 찾을 수 없습니다.
NO_AVAILABLE_RESOURCES	404	사용 가능한 리소스가 없습니다.

노드(Node) 에러

코드	상태	메시지
NODE_NOT_FOUND	404	노드를 찾을 수 없습니다.

신청(Request) 에러

코드	상태	메시지
INVALID_REQUEST_STATUS	409	이미 처리된 신청입니다.
UNSUPPORTED_CHANGE_TYPE	400	지원되지 않는 요청 타입(enum)입니다.

INVALID_REQUEST_STATUS 는 이미 승인/거절 처리된 Request를 다시 승인·거절하려 할 때, 또는 PROCESSING 상태인 Request에 중복 요청이 들어왔을 때 발생합니다. [시스템 아키텍처](#) 1번의 상태 전이표와 함께 보면 어느 전이에서 막힌 것인지 파악하기 쉽습니다.

계정(Account) 에러

코드	상태	메시지
UBUNTU_USER_DELETION_FAILED	502	우분투 계정 삭제에 실패했습니다.
USER_CREATION_FAILED	502	사용자 계정 생성에 실패했습니다.
INVALID_USERNAME_FORMAT	400	잘못된 사용자명 형식입니다.

Pod 에러

코드	상태	메시지
POD_CREATION_FAILED	502	Pod 생성 API 요청에 실패했습니다.
POD_DELETION_FAILED	502	Pod 삭제 API 요청에 실패했습니다.

`POD_CREATION_FAILED` 는 3단 보상 트랜잭션을 트리거하는 대표적인 에러입니다. 발생 시 Ubuntu 계정이 자동으로 정리 되고 Request가 PENDING으로 복구됩니다. 흐름은 [핵심 설계 패턴](#) 4번을 참고합니다.

Infra

원본: md/infra/index.md, md/infra/개요.md, md/infra/design/시스템-아키텍처.md, md/infra/design/기초-개념.md, md/infra/design/데이터베이스.md, md/infra/operations/시작.md, md/infra/operations/운영-매뉴얼.md, md/infra/operations/API-레퍼런스.md, md/infra/operations/Helm-차트-레퍼런스.md

이 묶음은 `admin_infra`의 config-server가 계정, 홈 디렉터리, Pod, NodePort, Kerberos를 어떻게 다루는지 설명한다. 처음 보는 사람도 어디서 시작해야 할지 바로 잡을 수 있게, 읽는 목적별 경로와 문서 지도를 함께 둔다.

먼저 읽을 길

지금 하려는 일	추천 순서
처음 합류해서 전체를 훑는다	개요 → 시스템 아키텍처 → 운영 매뉴얼
승인 뒤 무슨 작업이 일어나는지 본다	개요 → 시스템 아키텍처 → API 레퍼런스
배포나 장애 대응을 한다	운영 매뉴얼 → 데이터베이스 → Helm 차트 레퍼런스
용어가 낯설다	기초 개념 → 필요한 본문 문서
Kerberos, AD, keytab 흐름을 본다	kdc-setup → 설계 → 운영

문서 지도

묶음	문서	이 문서에서 보는 것	다음에 읽을 문서
입문	개요	config-server가 시스템에서 맡는 역할과 큰 흐름	시스템 아키텍처
설계	시스템 아키텍처	계정, 홈 디렉터리, Pod, NodePort가 만들어지는 순서	데이터베이스, API 레퍼런스
설계	기초 개념	UID/GID, Kubernetes, NFS, NodePort 같은 배경 용어	필요한 본문 문서
설계	데이터베이스	infra-mysql과 NodePort 배정 기록 구조	운영 매뉴얼
작업 시작	처음 작업할 때	브랜치, 로컬 실행, 배포 흐름, PR 규칙	운영 매뉴얼
운영	운영 매뉴얼	배포, 상태 점검, 장애 확인, 복구 순서	데이터베이스, Helm 차트 레퍼런스
운영	API 레퍼런스	config-server 엔드포인트, 입력, 응답, 호출 주체	시스템 아키텍처
운영	Helm 차트 레퍼런스	Helm values와 차트 템플릿이 어디에 쓰이는지	운영 매뉴얼
Kerberos	kdc-setup	AD, Secret, 노드, Pod 사이의 Kerberos 흐름 전체	설계, 운영

이 묶음의 범위

- `infra` 는 config-server가 직접 만드는 계정, 홈 디렉터리, Pod, NodePort, Kerberos 준비를 다룬다.
- NAS service keytab, NFS 서버 자체 설정, 사용자 이미지 내부 동작처럼 다른 묶음이 더 적합한 내용은 `system` 문서로 보낸다.
- 실제 값, 비밀번호, keytab, SSH 개인키는 여기서 설명하지 않고 운영 확인 절차와 위치만 적는다.

개요

이 문서에서 다루는 내용

- config-server가 전체 시스템에서 맡는 역할
- 승인 뒤 계정, 홈 디렉터리, Pod, NodePort가 만들어지는 큰 흐름
- 관련 저장소, 운영 주소, 다음에 읽을 문서

빠른 이동

궁금한 내용	바로 볼 문서
승인 뒤 세부 순서가 궁금하다	시스템 아키텍처
Kubernetes, UID/GID, NFS 용어가 낯설다	기초 개념
운영 점검과 장애 대응을 해야 한다	운영 매뉴얼
API 입력과 응답을 확인해야 한다	API 레퍼런스
Kerberos와 AD 설정이 궁금하다	kdc-setup

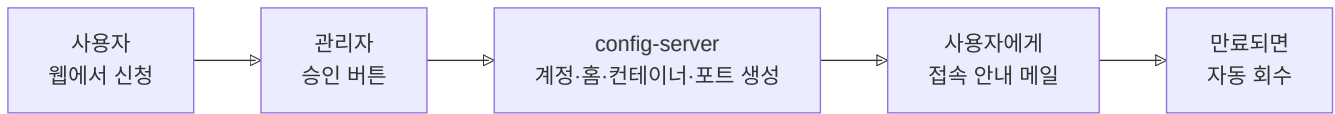
1. config-server가 하는 일

DECS는 연구실 GPU 서버를 웹에서 신청하고 승인받아 사용하는 시스템이다. 세 서버가 다음 일을 나눠 한다.

서버	하는 일
admin_fe	사용자와 관리자가 신청과 승인 결과를 확인하는 화면을 제공한다.
admin_be	신청 내용과 승인 상태를 저장하고 config-server에 작업을 요청한다.
config-server (admin_infra)	계정, 홈 디렉터리, Pod, 접속 포트를 만들거나 지운다.

admin_be가 승인 여부를 정하고, config-server가 Kubernetes와 NAS에 실제 작업을 수행한다. config-server는 스스로 사용자를 승인하지 않는다.

2. 승인 뒤에 일어나는 일



관리자가 승인하면 config-server는 다음 순서로 작업을 한다.

1. **계정 만들기** — 리눅스 계정(UID)을 공용 계정 파일에 등록
2. **홈 폴더 만들기** — NAS(공용 저장 장비)에 그 사용자의 홈 디렉터리를 생성
3. **GPU Pod 만들기** — GPU 사용량이 낮은 노드를 골라 Pod를 만든다.
4. **접속 포트 열기** — SSH·Jupyter로 접속할 수 있는 포트를 배정한다.
5. **결과 전달** — Pod 이름과 포트를 admin_be에 보내고, admin_be가 사용자에게 안내 메일을 보낸다.

만료되면 Pod와 포트를 지운 뒤 계정과 홈 디렉터리를 정리한다.

3. config-server가 없을 때 하던 일

이 시스템이 없던 때에는 관리자가 서버에 접속해 계정·컨테이너·포트를 모두 명령어로 처리했다.

- 신청이 몰리면 **관리자 한 사람이 병목**이 된다.
- UID 중복·포트 충돌 같은 **수작업 실수**가 발생한다.
- 만료된 자원을 사람이 기억해 삭제해야 하므로 **GPU가 방치**될 수 있다.

config-server는 이 작업을 API로 처리한다. 계정, GPU 노드, 포트가 정해지는 방법은 **시스템 아키텍처**에 있다.

4. 원본 인프라 저장소의 구성

아래 디렉터리는 이 위키가 아닌 `CSID-DGU/admin_infra` 저장소에 있다.

디렉터리	내용
<code>config-server/</code>	Flask API 서버와 Helm 차트
<code>infra-sql/</code>	NodePort 기록을 저장하는 MySQL 배포 파일
<code>pvc-image-chart/</code>	사용자 이미지 파일을 저장하는 PVC 차트

5. 서버 환경 요약

항목	값
배포 위치	FARM Kubernetes, namespace <code>ailab-infra</code>
외부 접근	NodePort 30082 (Service <code>containerssh-config-service</code>)
Swagger UI	<code>http://210.94.179.18:30082/apidocs/</code>
전용 DB (config-server가 관리)	infra-mysql <code>pod_port_db</code> — 데이터베이스

6. 필요한 문서 찾기

상황	추천 페이지
처음 작업한다	이 페이지 → 처음 작업할 때
계정, Pod, GPU 노드, 포트가 어떻게 정해지는지 본다	시스템 아키텍처
API 요청과 응답이 필요하다	API 레퍼런스
점검하거나 장애를 처리한다	운영 매뉴얼
Helm 값이나 차트를 바꾼다	Helm 차트 레퍼런스

시스템 아키텍처

이 문서에서 다루는 내용

- `admin_infra`의 config-server가 계정, 홈 디렉터리, Pod, NodePort를 어떤 순서로 만드는지
- 어떤 정보가 `admin_be`, infra-mysql, `/kube_share`, NAS에 나뉘어 저장되는지
- 계정 생성, Pod 생성, 삭제 흐름에서 어디서 실패할 수 있는지

빠른 이동

필요할 때	문서
실제 운영 명령과 점검 순서를 본다	운영 매뉴얼
요청 형식과 응답을 확인한다	API 레퍼런스

필요할 때	문서
Kerberos 준비 단계를 자세히 본다	kdc-setup

1. 서버마다 하는 일

신청을 받고 승인하는 일, Pod와 계정을 만드는 일, 파일과 인증을 제공하는 일은 서로 다른 서버가 맡는다.

하는 일	서버 또는 서비스	설명
신청과 승인	<code>admin_fe</code> , <code>admin_be</code>	신청을 받고 승인 결과와 사용자 설정을 저장한다.
Pod와 계정 만들기	<code>config-server</code>	계정, 홈 디렉터리, Pod, NodePort Service를 만들거나 지운다.
Pod를 만드는 데 필요한 서비스	Kubernetes, NAS, KDC, Prometheus	컨테이너를 실행하고, 파일을 저장하고, Kerberos 인증과 GPU 노드 정보를 제공한다.

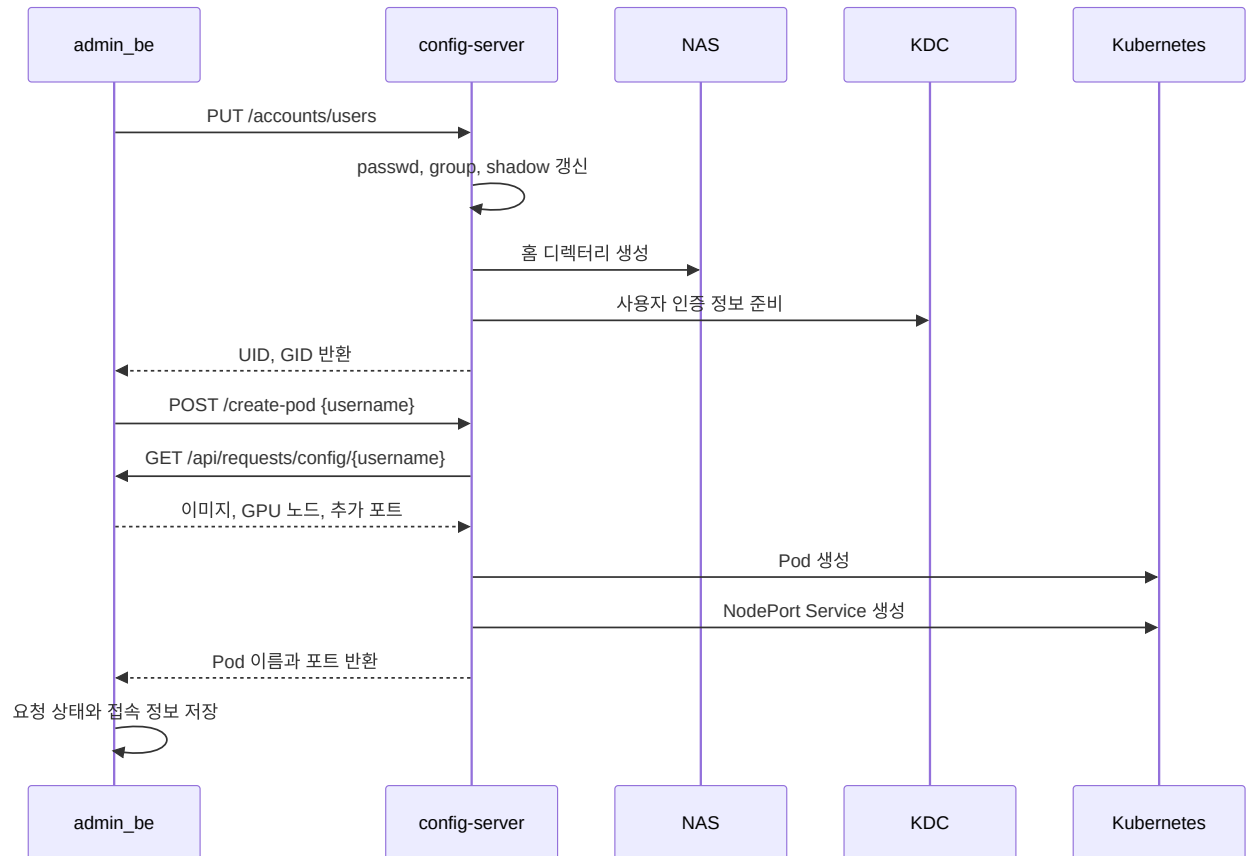
중요한 정보는 다음 위치에 저장한다.

정보	저장 위치	사용하는 곳
신청 상태와 사용자 설정	<code>admin_be</code> DB	<code>admin_be</code> , <code>config-server</code>
NodePort 배정	<code>infra-mysql</code> 의 <code>nodeport_allocations</code>	<code>config-server</code>
UID, GID, 그룹	<code>/kube_share</code> 의 계정 파일	<code>config-server</code> , 사용자 Pod
사용자 홈과 이미지 저장소	NAS	Kubernetes 노드, <code>config-server</code>

`admin_be`는 어떤 사용자에게 Pod를 만들지 정한다. `config-server`는 그 요청을 받아 Pod, 홈 디렉터리, 접속 포트를 만든다. Pod를 만들 때 필요한 이미지, GPU 노드, 추가 포트는 `admin_be`에서 다시 받는다.

2. 승인 후 사용자 환경을 만드는 순서

승인이 끝나면 `admin_be`는 먼저 계정을 만들고, 그다음 Pod를 만든다.



2.1 계정과 홈 디렉터리

`PUT /accounts/users` 는 `/kube_share` 의 `passwd` , `group` , `shadow` 파일에 새 사용자 정보를 쓴다. 새 UID와 GID를 정하는 방법은 [계정 정보](#)에서 확인한다.

홈 디렉터리는 Pod보다 먼저 NAS에 만든다. NFS의 `root_squash` 설정 때문에 Pod 안의 root 사용자는 NAS에 홈 디렉터리를 만들 수 없다. config-server가 NAS에 SSH로 접속해 디렉터리를 만들고, 새 UID와 GID로 소유자를 지정한 뒤 권한을 `700` 으로 설정한다. 홈 생성이 실패하면 계정 파일에 쓴 내용도 되돌리고 계정 생성 요청을 실패로 끝낸다.

2.2 사용자 설정과 Pod 생성

`POST /create-pod` 의 입력은 사용자 이름이다. config-server는 admin_be 의 `GET /api/requests/config/{username}` 을 호출해 다음 값을 가져온다.

- 사용할 이미지
- 사용할 GPU 수와 후보 노드
- 기본 포트 외에 열 포트

config-server는 받은 값으로 Pod를 만든다. 추가 포트에 내부 포트 `6080` 이 있거나 용도가 `novnc` , `vnc` 이면 Pod 환경 변수에 `ENABLE_VNC=true` 도 넣는다. Pod가 접속을 받을 준비가 된 뒤에 NodePort Service를 만들고, Pod 이름과 NodePort 목록을 admin_be 에 돌려준다.

2.3 Pod와 계정 삭제

사용자 환경을 지울 때는 Pod 관련 작업과 계정 관련 작업을 나누어 처리한다.

구분	API	정리 대상
Pod 관련 작업	<code>POST /delete-pod</code>	Pod, NodePort Service, NodePort 배정 기록
계정 관련 작업	<code>DELETE /accounts/users/{username}</code>	계정 파일, NAS 홈, Kerberos 관련 파일

두 요청을 모두 처리해야 계정과 Pod가 함께 삭제된다. 요청 형식과 응답은 [API 레퍼런스](#)에서 확인한다.

3. GPU 노드와 포트 고르는 방식

3.1 노드 선택

config-server는 `admin_be`가 전달한 `gpu_nodes` 목록에서 Pod를 올릴 GPU 노드를 고른다. 노드 이름은 소문자로 바꾸고, 목록에 들어 있던 순서대로 비교한다. `gpu_nodes`가 비어 있으면 Kubernetes에서 준비가 끝난 워커 노드를 가져온다. control-plane 노드에 붙은 `NoSchedule` 표시는 관리용 노드라는 뜻이므로 후보에서 뺀다.

지표	사용 방식
<code>DCGM_FI_DEV_GPU_UTIL</code>	노드 GPU 사용률 평균을 그대로 사용한다.
<code>DCGM_FI_DEV_FB_USED</code>	GPU 메모리 사용량 평균을 GiB 단위로 환산한다.
<code>DCGM_FI_DEV_GPU_TEMP</code>	GPU 온도 평균을 보조 값으로 사용한다.

$$\text{score} = \text{avg}(\text{GPU_UTIL}) + \text{avg}(\text{FB_USED}) / 1024 + \text{avg}(\text{GPU_TEMP}) / 100$$

점수가 가장 낮은 노드를 고른다. 점수가 같으면 후보 목록에서 앞에 있던 노드를 고른다.

각 지표 조회에는 `or vector(0)`이 들어 있다. 어떤 지표가 없으면 그 값은 0으로 계산한다. Prometheus에 연결하지 못했거나 결과가 비어 있으면 그 노드는 고르지 않는다. 모든 후보를 조회하지 못하면 Pod를 만들지 않는다.

고른 노드가 `gpu_nodes` 목록에 있으면 그 항목의 CPU, 메모리, GPU 개수를 Pod 설정에 넣는다. `gpu_nodes` 없이 워커 노드를 찾은 경우에는 config-server에 설정된 기본값을 사용한다.

3.2 NodePort 배정

사용자 Pod에는 SSH(22), Jupyter(8888), `admin_be`가 전달한 추가 포트마다 NodePort Service를 하나씩 만든다. 추가 포트는 받은 순서대로 SSH와 Jupyter 뒤에 붙인다. 어떤 Pod에 어떤 NodePort를 줬는지는 `infra-mysql`의 `nodeport_allocations` 테이블에 적는다.

포트를 고르기 전에는 MySQL 기록과 Kubernetes Service를 비교한다. 이 비교는 마지막 시도 뒤 5분이 지나야 다시 실행되며, 성공하지 못한 시도도 5분 간격에 포함한다. `app=ai lab-nodeport` 라벨과 같은 `pod_name`을 가진 Service가 Kubernetes에 하나도 없고 MySQL에만 남아 있으면 그 Pod의 포트 기록을 지운다. 이 비교는 포트별이 아니라 Pod별로 한다.

포트를 고르고 MySQL에 기록하는 일은 한 번의 DB 작업으로 처리한다. 중간에 실패하면 새 기록은 저장하지 않는다.

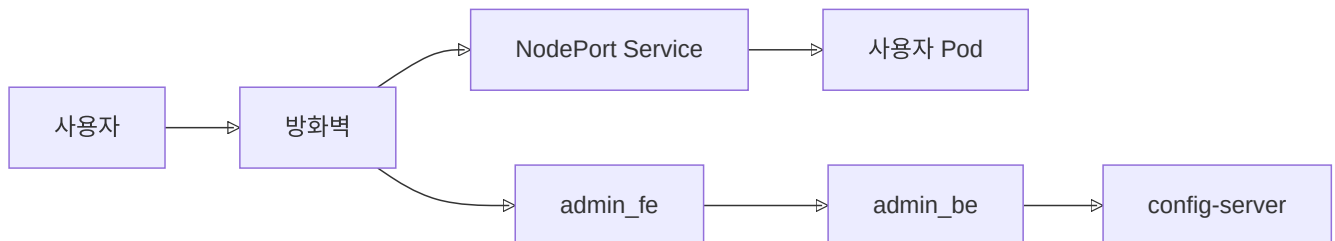
1. `nodeport_allocations`의 포트 기록을 잠가, 동시에 들어온 요청이 같은 번호를 고르지 못하게 한다.
2. MySQL에 적힌 포트와 클러스터 전체 Service가 이미 쓰는 NodePort를 모은다. 클러스터 전체 Service 조회가 실패하면 오류를 로그에 남기고 MySQL 기록만으로 계속 진행한다. 이 경우 다른 Service가 이미 쓰는 포트를 고를 수 있고, 뒤의 Service 생성 단계가 실패할 수 있다.
3. 30000부터 32767까지 오름차순으로 훑어 빈 포트를 필요한 개수만큼 고른다.
4. 고른 포트를 `username`, `pod_name`, `node_name`, 내부 포트, 용도와 함께 MySQL에 저장한다.
5. Pod가 Ready 상태가 되면 기록된 포트마다 NodePort Service를 만든다.

Pod 또는 Service 생성에 실패하면 해당 Pod의 Service, NodePort 배정 행, 생성된 Pod를 정리한다.

구분	저장 위치	의미
요청 포트	<code>admin_be</code> DB의 <code>port_requests</code>	사용자가 요청한 컨테이너 내부 포트
확정 포트	<code>admin_be</code> DB의 <code>pod_external_ports</code>	실제 NodePort와 내부 포트의 연결 정보
배정 기록	<code>infra-mysql</code> 의 <code>nodeport_allocations</code>	Pod와 NodePort Service를 정리할 때 사용하는 기록

3.3 외부 접근 경로

외부 사용자는 방화벽을 거쳐 `admin_fe` 또는 사용자 Pod의 NodePort Service에 접속한다. 방화벽은 외부 포트를 NodePort로 매핑하므로 외부 포트와 NodePort 번호가 같을 필요는 없다. 외부 접속이 되지 않으면 해당 Service의 NodePort와 pfSense 매핑 대상이 같은지 확인한다.



4. 계정, 홈 디렉터리, 인증

4.1 계정 정보 {#account-information}

UID와 GID는 `/kube_share`의 계정 파일에 적힌 값을 사용한다. NAS 파일의 소유자도 이 숫자로 표시되므로, UID와 GID를 다른 데이터베이스에 따로 저장하지 않는다.

계정을 추가할 때는 `passwd` 파일을 잠가, 동시에 들어온 요청이 같은 UID를 고르지 못하게 한다. `/home/`을 쓰는 사용자 중 UID가 20000 이상인 값 가운데 가장 큰 값에 1을 더해 새 UID 후보를 만든다. 이 번호가 시스템 계정을 포함해 이미 쓰이고 있으면, 비어 있는 번호가 나올 때까지 1씩 올린다. 요청에 UID가 들어 있어도 config-server는 사용하지 않고 이 방식으로 정한 값을 반환한다.

새 사용자의 기본 GID는 새 UID와 같은 번호로 정한다. 기본 그룹 이름은 요청의 `primary_group_name`을 쓰고, 이 값이 없으면 사용자 이름을 쓴다. 추가 그룹은 요청에 들어 있는 `{name, gid}` 목록으로 처리한다. 같은 GID의 그룹이 이미 있으면 사용자를 그 그룹에 넣고, 없으면 요청에 적힌 GID로 그룹을 새로 만든다. 추가 그룹의 GID는 config-server가 새

로 정하지 않는다.

`passwd`에 사용자 항목을 쓴 뒤 기본 그룹과 추가 그룹을 `group`에 기록하고, SHA-512 암호 해시를 `shadow`에 기록한다. `sudo` 허용 명령이 설정된 경우에만 사용자별 `sudoers` 파일도 만든다. 이 단계에서 실패하면 앞에서 쓴 계정·그룹 정보를 지운다. NAS 홈 디렉터리의 소유자도 같은 UID와 GID로 바꾼다. Pod를 만들 때 `config-server`는 UID, 기본 GID, 그룹 목록을 환경 변수로 넣는다. 컨테이너가 시작할 때 실행하는 스크립트는 이 값으로 컨테이너 안의 사용자를 만든다. 사용자 Pod는 `/kube_share`를 직접 마운트하지 않는다.

4.2 홈 디렉터리와 이미지 저장소

사용자 홈은 NAS의 NFS 공유를 노드에 마운트한 뒤 Pod의 `/home`에 연결한다. 사용자마다 PVC를 만들지는 않는다. 홈 디렉터리의 소유자와 권한으로 사용자별 접근을 나눈다.

사용자 이미지 tar 파일은 별도의 `pvc-image-store`에 저장한다. `config-server`가 이미지 저장과 로드를 담당하고, Redis에는 이미지 작업의 진행 상태를 기록한다.

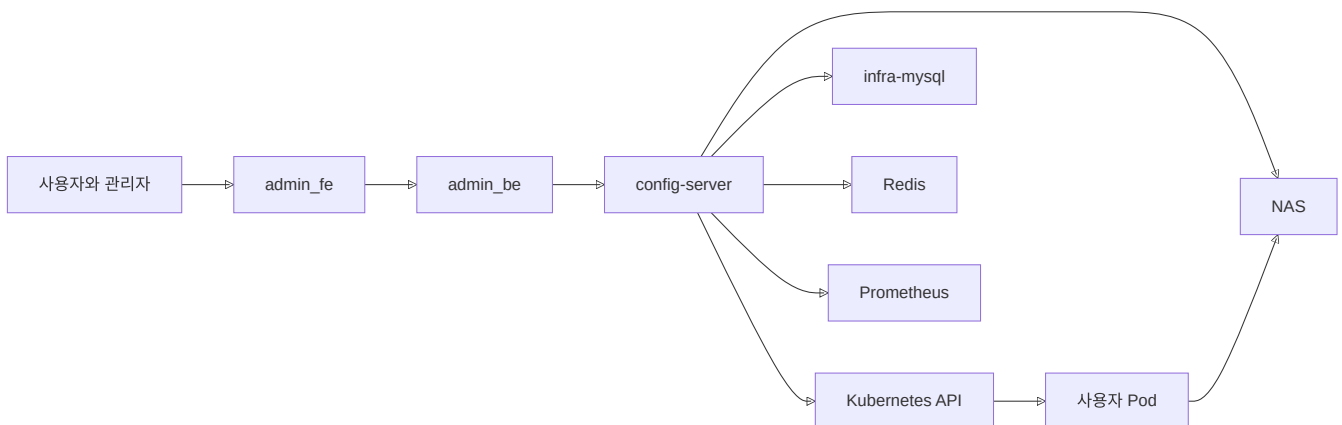
4.3 Kerberos를 사용하는 방식

FARM 환경에서는 NFS 홈을 쓰기 전에 Kerberos 인증을 준비한다. 계정을 만들 때 `config-server`는 사용자 principal을 만들고 keytab을 Kubernetes Secret에 저장한다. Pod를 만들 때는 선택된 노드에 keytab과 갱신용 설정을 두고 ccache를 만든다. Pod에는 keytab을 넣지 않고 ccache만 연결한다.

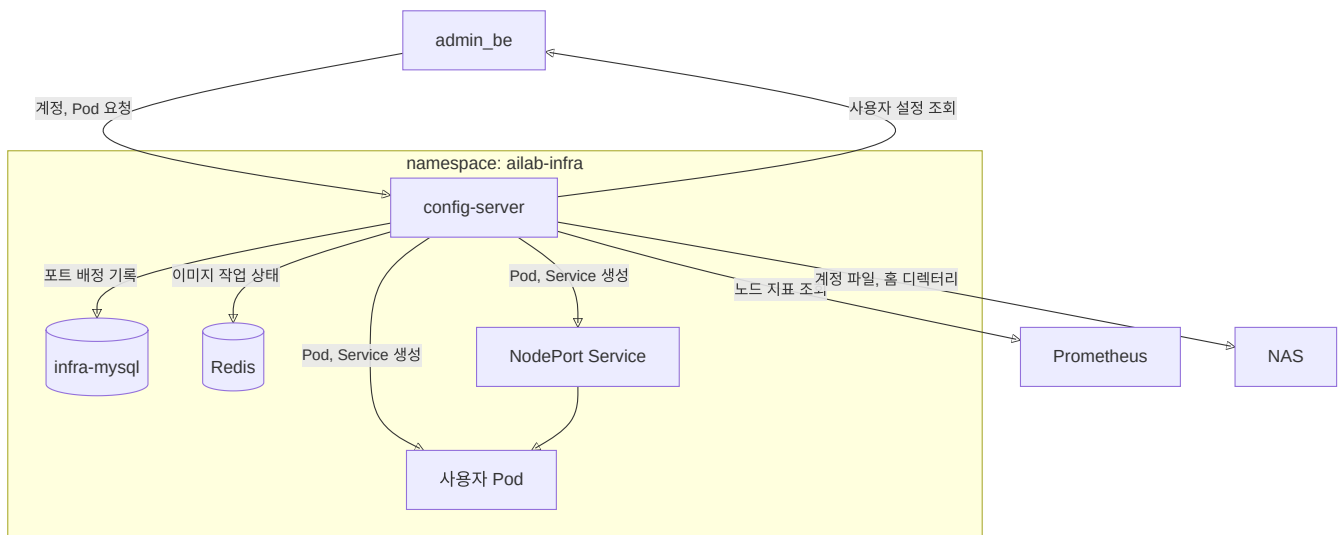
Kerberos 서버와 설정 값은 `kdc-setup`에서 확인한다.

5. 서버와 서비스가 연결되는 모습

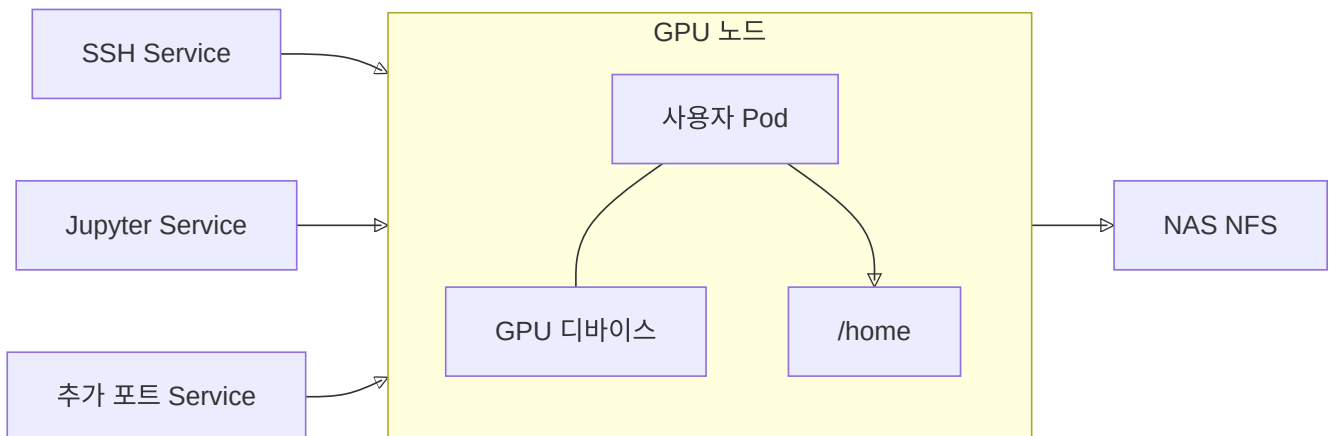
5.1 전체 연결



5.2 config-server가 연결하는 서비스



5.3 사용자 Pod



사용자 Pod는 접속용 Service, GPU 디바이스, 홈 디렉터를 사용한다. UID와 GID, 그룹 정보는 Pod를 만들 때 넣고, 홈 디렉터리는 노드의 NFS 마운트를 통해 연결한다.

기초 개념

이 문서에서 다루는 내용

- 시스템 아키텍처에 나오는 UID/GID, 계정 파일, 권한, Kubernetes, NFS, NodePort 기본 개념
- config-server 문서를 읽을 때 자주 헷갈리는 배경 용어
- 운영 절차가 아니라 개념 설명만 먼저 보고 싶을 때 필요한 내용

빠른 이동

필요할 때	문서
실제 생성 순서를 본다	시스템 아키텍처
배포와 장애 대응을 본다	운영 매뉴얼
포트 배정 기록과 DB 구조를 본다	데이터베이스

1. 리눅스 계정과 파일 권한

1.1 UID와 GID

리눅스는 사용자와 그룹을 이름뿐 아니라 숫자로도 구분한다. 사용자 번호는 UID, 그룹 번호는 GID이다.

파일에도 사용자 이름이 아닌 UID와 GID가 소유권 정보로 기록된다. 예를 들어 파일의 소유자가 `dongmin`으로 보이더라도, 실제 디스크에는 `UID 2041`과 같은 숫자가 저장된다.

이름과 숫자의 대응 관계는 다음 파일에서 관리한다.

파일	역할
<code>passwd</code>	사용자 이름, UID, 기본 GID를 기록한다.
<code>group</code>	그룹 이름, GID, 그룹 구성원을 기록한다.

UID를 정하는 곳

NFS에 저장된 사용자 홈 디렉터리 역시 UID 숫자를 기준으로 소유권을 판단한다. 따라서 동일한 NAS를 사용하는 FARM 서버와 Kubernetes Pod는 반드시 동일한 UID 대응 관계를 사용해야 한다.

서버마다 같은 UID를 다른 사용자에게 배정하면, 특정 사용자가 다른 사용자의 홈 디렉터리를 자신의 파일처럼 읽거나 수정할 수 있다.

이를 막기 위해 config-server가 `/kube_share/passwd`를 계정 정보의 기준 파일로 사용하고 UID도 여기서 정한다.

1.2 passwd, group, shadow, sudoers

파일	저장 내용	주의사항
<code>passwd</code>	사용자 이름, UID/GID, 홈 경로, 로그인 셸	암호 정보는 없으며 누구나 읽을 수 있다.
<code>group</code>	그룹 이름, GID, 그룹 멤버	그룹 권한 관리에 사용한다.

파일	저장 내용	주의사항
shadow	사용자 암호 해시	root만 읽을 수 있다.
sudoers	sudo 사용 권한	문법 오류가 발생하면 sudo 전체가 동작하지 않을 수 있다.

일반적인 리눅스 환경에서는 `/etc/passwd`, `/etc/group` 등을 `useradd` 와 같은 명령으로 관리한다.

계정 파일을 쓰는 방법

여기서는 config-server가 NFS의 `/kube_share` 에 저장된 계정 파일을 직접 수정해 UID/GID 배정 기록으로 사용한다.

사용자 Pod에는 이 파일들을 직접 마운트하지 않는다. 대신 다음 순서로 사용자를 생성한다.

1. config-server가 `/kube_share/passwd` 에서 UID/GID를 조회한다.
2. 조회한 값을 Pod 환경변수로 주입한다.
3. 컨테이너 entrypoint가 전달받은 UID/GID로 내부 사용자를 생성한다.

계정 파일을 컨테이너의 `/etc` 에 직접 마운트하던 방식은 더 이상 쓰지 않는다.

1.3 chmod 700

`chmod` 의 세 숫자는 차례대로 다음 권한을 의미한다.

1. 파일 소유자
2. 소유 그룹
3. 기타 사용자

각 권한 값은 다음 숫자의 합으로 표현한다.

| 권한 | 값 | | ----- | -: | | 읽기 | 4 | | 쓰기 | 2 | | 실행 또는 디렉터리 진입 | 1 |

따라서 `chmod 700` 은 다음과 같은 권한 설정이다.

대상	권한
소유자	읽기, 쓰기, 진입
그룹	권한 없음
기타 사용자	권한 없음

홈 디렉터리 권한

모든 사용자 홈 디렉터리는 `chmod 700` 으로 생성한다.

현재는 사용자별 PVC를 만들지 않는다. 사용자 홈 사이의 접근은 UID 소유권과 `chmod 700` 권한으로 막는다.

2. NFS와 NAS

2.1 NFS와 NAS

NFS(Network File System)는 다른 서버의 디렉터리를 내 서버의 디렉터리처럼 쓰게 하는 방식이다.

서버가 특정 디렉터리를 외부에 공개하면(`export`), 클라이언트는 해당 경로를 자신의 파일시스템에 마운트한다. 이후 `ls`, `cat`, 파일 저장 등의 작업은 로컬 파일처럼 보이지만 실제로는 네트워크 통신을 통해 처리된다.

NAS(Network Attached Storage)는 이 NFS 저장소를 제공하는 장비다.

NAS에 저장하는 데이터

이 시스템의 NAS에는 다음 데이터가 저장된다.

- 사용자 홈 디렉터리
- `/kube_share` 계정 파일
- `infra-mysql` 데이터

따라서 NAS에 문제가 생기면 사용자 파일, 계정 관리, 데이터베이스가 함께 영향을 받는다. 이 저장소를 대신할 장비가 현재 구성되어 있지 않으므로 NAS 장애는 전체 서비스에 영향을 준다.

2.2 root_squash

`root_squash`는 NFS의 기본 보안 기능이다.

NFS 클라이언트의 `root` 사용자가 공유 디렉터리에 접근하면, NFS 서버는 그 요청을 `nobody` 같은 익명 사용자 권한으로 처리한다.

그래서 한 클라이언트 서버에서 `root` 권한을 얻더라도 NAS 파일 전체를 마음대로 바꾸기 어렵다.

홈 디렉터리를 만드는 방법

Pod 안의 프로세스가 `root`로 실행되더라도 NFS 위에서는 `root` 권한을 사용할 수 없다. 따라서 Pod가 NFS에 사용자 홈 디렉터리를 직접 생성하거나 소유권을 변경할 수 없다.

`config-server`는 Pod를 생성하기 전에 NAS에 SSH로 접속하여 다음 작업을 수행한다.

```
mkdir <사용자 홈>
chown <UID>:<GID> <사용자 홈>
chmod 700 <사용자 홈>
```

이 과정이 실패하면 Pod 생성도 실패하도록 처리한다.

2.3 NFS 파일 락

파일 잠금은 여러 프로세스가 같은 파일을 동시에 바꾸지 못하게 하는 장치다.

NFS 위의 파일 잠금은 네트워크와 별도 잠금 서비스에 의존하므로, 이 시스템에서는 계정 파일에 직접 적용하지 않는다.

현재 잠금 방식

config-server의 `LockedFile`은 NFS 파일을 수정하기 전에 config-server Pod 안의 `/tmp`에 잠금 파일을 만든다.

```
/tmp/cssh_lock*
```

현재 config-server는 한 개만 실행한다. 이 전제에서 같은 서버 안의 요청은 이 잠금으로 순서대로 처리된다.

config-server를 여러 개로 늘리면

config-server를 여러 개로 늘리면 이 방식만으로는 부족하다. 각 인스턴스의 `/tmp` 잠금 파일은 서로 보이지 않기 때문이다.

3. Kubernetes

3.1 클러스터와 노드

Kubernetes는 여러 서버를 하나의 컨테이너 실행 환경으로 묶어 관리하는 오케스트레이션 시스템이다.

- 클러스터는 여러 노드로 구성된 전체 실행 환경이다.
- 노드는 Pod가 실제로 실행되는 개별 서버이다.

Kubernetes에는 필요한 Pod와 Service의 내용을 API로 등록한다. Kubernetes는 등록된 내용과 실제 상태가 다르면 다시 맞추려고 한다.

Pod와 Service 생성

현재는 config-server가 Pod와 Service 정보를 Kubernetes API에 보내고, Kubernetes가 이를 실행한다.

3.2 Pod

Pod는 Kubernetes가 컨테이너를 실행하는 최소 단위이다.

Pod 스펙에는 다음과 같은 정보가 포함된다.

- 컨테이너 이미지
- 환경변수
- 볼륨 마운트

- GPU 개수
- 실행할 노드
- 포트 설정

사용자 Pod 규칙

여기서는 사용자 한 명마다 Pod 하나를 만든다.

config-server는 Kubernetes 클라이언트 라이브러리를 이용해 사용자 Pod 스펙을 생성한다.

보통은 Kubernetes가 실행 노드를 고르지만, 이 시스템은 config-server가 Prometheus GPU 지표로 먼저 노드를 고른 뒤 Pod에 그 노드를 지정한다.

3.3 Service와 NodePort

Pod의 IP는 Pod가 재생성될 때 변경될 수 있다. Service는 이러한 Pod에 고정된 네트워크 접근 지점을 제공한다.

NodePort 타입 Service는 모든 Kubernetes 노드의 특정 포트를 열고, 해당 포트로 들어온 요청을 대상 Pod로 전달한다.

기본 NodePort 범위는 다음과 같다.

```
30000~32767
```

사용자 Pod에 여는 포트

사용자 Pod의 각 포트마다 별도의 NodePort Service를 생성한다.

- SSH는 컨테이너 포트 22를 사용한다.
- Jupyter는 컨테이너 포트 8888을 사용한다.
- 사용자가 추가로 신청한 포트도 별도 Service로 생성한다.

config-server가 배정한 포트는 infra-mysql의 `nodeport_allocations` 테이블에 기록한다. 포트를 고를 때는 이 테이블뿐 아니라 클러스터 전체 Service의 NodePort도 함께 확인한다.

NodePort와 방화벽 매핑

Kubernetes의 NodePort는 클러스터 안에서 쓰는 포트이고, pfSense는 외부 포트를 이 NodePort로 전달한다. 외부 포트와 NodePort 번호는 같을 필요가 없다. 외부에서 접속하지 못하면 Service의 NodePort와 pfSense의 전달 대상이 같은지 확인한다.

3.4 Namespace

Namespace는 클러스터 리소스를 용도나 팀별로 구분하는 논리적 공간이다.

이 시스템의 주요 Namespace는 다음과 같다.

Namespace	용도
ailab-frontend	사용자 화면
default	admin_be
ailab-infra	config-server와 사용자 Pod
monitoring	Prometheus와 Grafana

리소스를 조회할 때는 Namespace를 정확히 지정해야 한다.

```
kubectl get pods -n ailab-infra
```

Namespace를 생략하면 다른 공간의 리소스를 조회하여 장애 원인을 놓칠 수 있다.

3.5 볼륨, hostPath, subPath

hostPath

hostPath는 노드 호스트의 특정 파일이나 디렉터리를 컨테이너에 직접 마운트하는 방식이다.

이 시스템에서는 다음 항목에 사용한다.

- 노드에 마운트된 NFS 홈 경로
- GPU 디바이스 `/dev/nvidia*`

subPath

subPath는 볼륨 전체가 아니라 볼륨 안의 특정 파일이나 디렉터리만 골라 마운트하는 옵션이다.

과거에는 `/kube_share/passwd`와 `shadow`를 컨테이너의 `/etc/passwd`, `/etc/shadow`에 직접 마운트했다. 현재 Pod 스펙에는 이 마운트가 없고, config-server가 UID/GID를 환경변수로 전달하면 컨테이너 시작 스크립트가 내부 사용자를 만든다.

코드 주석과 예전 배포 기록에는 **subPath** 방식이 남아 있을 수 있다. 현재 Pod 스펙에는 이 마운트가 없다.

3.6 imagePullPolicy와 Ready

imagePullPolicy

Pod를 생성할 때 컨테이너 이미지를 어떻게 가져올지 결정하는 정책이다.

값	동작
Always	Pod 생성 시마다 이미지를 내려받는다.

값	동작
<code>IfExists</code>	노드에 이미지가 없을 때만 내려받는다.

`Always` 는 최신 이미지를 보장하지만 시작 시간이 길어질 수 있다.

Ready

컨테이너 프로세스가 시작된 상태와 실제 요청을 받을 준비가 된 상태는 다르다.

`config-server`는 다음 순서로 Pod를 생성한다.

1. Pod 생성 요청을 보낸다.
2. Pod가 Ready가 될 때까지 상태를 확인한다.
3. Ready가 되면 NodePort Service를 만든다.
4. 생성 결과를 응답한다.

NFS 마운트가 실패하여 `FailedMount` 상태가 되면 Pod는 Ready가 되지 않으며, `config-server`의 대기는 타임아웃으로 종료된다.

3.7 RBAC와 ServiceAccount

RBAC(Role-Based Access Control)는 Kubernetes 리소스를 누가 다룰 수 있는지 정하는 권한 설정이다.

ServiceAccount는 사람이 아닌 애플리케이션이나 Pod가 사용하는 Kubernetes 내부 신원이다.

config-server의 Kubernetes 권한

`config-server`는 자신의 ServiceAccount 권한으로 Kubernetes API를 호출하여 다음 작업을 수행한다.

- Pod 생성과 삭제
- Service 생성과 삭제
- Pod 상태 조회

관련 ServiceAccount, Role, RoleBinding은 Helm 차트 배포 과정에서 함께 생성한다.

3.8 Helm

Helm은 여러 Kubernetes YAML 파일과 설정값을 한 묶음으로 관리하는 도구다.

Deployment, Service, RBAC 등의 정의를 템플릿으로 구성하고, 환경별 값만 변경해 배포할 수 있다.

`config-server`는 `Chart/` 디렉터리의 Helm 차트로 배포하며, 다음 명령이 배포 절차를 감싼다.

```
make deploy
```

4. 모니터링

4.1 Prometheus와 Pull 모델

Prometheus는 시스템 지표를 수집하고 저장하는 모니터링 시스템이다.

Prometheus는 각 서버가 데이터를 보내는 push 방식이 아니라, Prometheus가 각 대상의 HTTP 엔드포인트를 주기적으로 조회하는 **pull 방식**을 사용한다.

Prometheus 형식으로 지표를 제공하는 프로그램을 exporter라고 한다.

4.2 DCGM Exporter

DCGM은 NVIDIA GPU의 상태를 수집하는 도구이며, DCGM exporter는 GPU 지표를 Prometheus 형식으로 제공한다.

config-server는 다음 지표를 노드 선택 점수에 사용한다.

```
DCGM_FI_DEV_GPU_UTIL
DCGM_FI_DEV_FB_USED
DCGM_FI_DEV_GPU_TEMP
```

GPU 지표가 없을 때

DCGM exporter가 중단되면 해당 노드의 GPU 지표가 Prometheus에서 사라진다.

현재 쿼리는 값이 없을 때 다음 표현으로 0을 반환한다.

```
... or vector(0)
```

이 때문에 지표가 없는 노드를 사용률이 0인 노드, 즉 가장 한가한 노드로 잘못 판단할 수 있다.

지표 없음과 실제 사용률 0은 구분해야 한다.

4.3 PromQL 기본 문법

PromQL은 Prometheus에 저장된 지표를 조회하는 질의 언어이다.

예를 들어 farm5의 GPU 평균 사용률은 다음과 같이 조회한다.

```
avg(DCGM_FI_DEV_GPU_UTIL{Hostname="farm5"})
```

주요 문법은 다음과 같다.

표현	의미
<code>metric{label="value"}</code>	특정 레이블을 가진 지표를 조회한다.
<code>avg(...)</code>	여러 GPU 값의 평균을 계산한다.
<code>or vector(0)</code>	조회 결과가 없을 경우 0을 반환한다.

`or vector(0)` 은 계산을 단순화하지만, 장애로 인한 지표 부재까지 정상적인 0으로 처리한다는 문제가 있다.

4.4 Grafana

Grafana는 Prometheus에 저장된 지표를 대시보드 형태로 보여주는 시각화 도구이다.

관리자는 다음 주소에서 노드와 GPU 상태를 확인할 수 있다.

```
http://210.94.179.18:30080
```

PromQL을 직접 실행하려면 Prometheus에 port-forward로 접속한다.

5. 네트워크와 방화벽

5.1 공인 IP, 사설 IP, NAT

- **사설 IP**는 내부 네트워크에서만 사용하는 주소이다.
- **공인 IP**는 인터넷에서 직접 접근할 수 있는 주소이다.
- **NAT**는 사설 IP와 공인 IP 사이에서 주소나 포트를 변환하는 기술이다.
- **포트포워딩**은 특정 공인 IP와 포트로 들어온 요청을 내부 서버의 특정 포트로 전달하는 규칙이다.

예시는 다음과 같다.

```
공인 IP:9300 → farm6:30000
```

5.2 pfSense

pfSense는 외부 네트워크와 내부 서버 사이에서 방화벽과 라우터 역할을 한다.

공인 IP	정책
210.94.179.18	사전에 허용한 포트만 통과시킨다.
210.94.179.19	외부 포트를 사용자 Pod의 NodePort로 전달한다.

외부 접속이 안 될 때

NodePort 번호만 보고 외부 접속 주소를 추정하지 않는다. Service의 NodePort, pfSense가 전달하는 대상, 사용자에게 안내한 외부 주소를 함께 확인한다.

6. 저장소와 동시성

6.1 트랜잭션과 SELECT ... FOR UPDATE

트랜잭션은 여러 SQL 작업을 하나의 작업 단위로 묶는 기능이다.

트랜잭션 내부의 작업은 모두 성공하거나 모두 취소된다.

`SELECT ... FOR UPDATE` 는 조회한 행에 잠금을 걸어, 현재 트랜잭션이 끝날 때까지 다른 트랜잭션이 해당 행을 수정하지 못하게 한다.

NodePort를 배정하는 순서

NodePort 배정은 다음 순서로 처리한다.

1. 트랜잭션을 시작한다.
2. 사용 중인 포트 목록을 `FOR UPDATE` 로 조회한다.
3. 빈 포트를 선택한다.
4. 배정 정보를 insert한다.
5. 트랜잭션을 commit한다.

두 개의 Pod 생성 요청이 동시에 들어오더라도 같은 포트가 중복 배정되는 것을 방지한다.

6.2 Redis

Redis는 메모리 기반 키-값 저장소이다.

이 시스템에서는 사용자 이미지 저장과 로드 작업의 진행 상태만 기록한다.

```
img:<username>
```

이미지 commit 또는 save 작업이 오래 걸리므로, Redis는 현재 작업 단계를 API에서 조회할 수 있도록 상태판 역할을 한다.

6.3 정보별 기준 저장 위치

같은 정보가 여러 곳에 있으면 값이 달라질 수 있다. 그래서 정보마다 어느 곳의 값을 기준으로 사용할지 정해 둔다.

이 시스템에서는 다음 위치의 값을 기준으로 사용한다.

정보	기준으로 보는 곳
사용자 UID/GID	/kube_share/passwd
요청 상태	admin_be DB의 requests 테이블
NodePort 배정	infra-mysql의 nodeport_allocations 테이블

admin_be가 config-server에 UID를 전달하더라도 config-server는 전달값을 신뢰하지 않고 /kube_share/passwd 를 기준으로 동작한다.

반대로 요청의 PENDING, FULFILLED, DELETED 상태는 admin_be가 관리하며 config-server는 해당 상태를 저장하지 않는다.

6.4 삭제 표시(soft delete)

삭제 표시는 데이터베이스 행을 지우지 않고 삭제되었다는 상태만 기록하는 방식이다.

admin_be의 요청 상태 DELETED 가 이에 해당한다.

주요 장점은 다음과 같다.

- 삭제 이력을 보존한다.
- 운영 감사와 장애 추적이 가능하다.
- 필요할 경우 데이터를 복구할 수 있다.

7. 실행 환경

7.1 Flask와 Gunicorn

Flask는 config-server에 사용한 Python 웹 프레임워크이다.

Flask의 내장 서버는 개발용이므로 운영 환경에서는 Gunicorn을 사용한다. Gunicorn은 여러 워커 프로세스로 Flask 애플리케이션을 실행하여 동시 요청을 처리한다.

config-server의 로컬 파일 락은 Gunicorn의 여러 워커가 동시에 계정 파일을 수정하는 상황을 제어하는 역할도 한다.

7.2 WAS와 Spring Boot

WAS(Web Application Server)는 웹 요청을 받아 비즈니스 로직을 실행하는 서버를 의미한다.

이 시스템에서는 Spring Boot로 작성한 `admin_be`가 WAS 역할을 수행한다.

운영 문서나 대화에서 “WAS”라고 표현하면 일반적으로 `admin_be`를 의미한다.

7.3 Swagger

Swagger는 API 목록과 요청 형식을 웹 페이지로 제공하고, 브라우저에서 직접 API를 호출할 수 있게 하는 도구이다.

config-server Swagger 주소는 다음과 같다.

```
http://210.94.179.18:30082/apidocs/
```

Swagger를 사용할 때

config-server API에는 별도의 인증 기능이 없다.

따라서 Swagger의 `Try it out` 기능은 단순한 테스트가 아니라 실제 Kubernetes와 NAS 리소스를 생성·수정·삭제할 수 있는 운영 명령이다.

Swagger 호출은 실제 인프라를 변경하는 작업이므로 운영 권한과 동일하게 취급해야 한다.

핵심 구조 요약

여기서는 다음 방식으로 동작한다.

1. 사용자 식별과 파일 권한의 기준은 UID/GID이다.
2. UID/GID는 `/kube_share/passwd`에 적힌 값을 사용한다.
3. 사용자 홈은 NFS에 저장하며 `chmod 700`으로 다른 사용자의 접근을 막는다.
4. `root_squash` 때문에 사용자 홈은 Pod 생성 전에 NAS에서 미리 생성해야 한다.
5. 사용자 한 명당 하나의 Pod와 여러 NodePort Service를 생성한다.
6. 실행 노드는 config-server가 Prometheus GPU 지표를 바탕으로 선택한다.
7. NodePort는 infra-mysql에서 트랜잭션과 행 잠금으로 배정한다.
8. 외부 접속은 pfSense가 외부 포트를 해당 NodePort로 전달해야 가능하다.

9. NAS, 계정 파일, NodePort DB 등 주요 기준 정보가 손상되면 전체 시스템에 영향을 줄 수 있다.
10. Swagger와 config-server API는 인증이 없으므로 실제 인프라 관리 인터페이스로 취급해야 한다.

데이터베이스

이 문서에서 다루는 내용

- infra-mysql이 어떤 테이블을 가지고 있고 config-server가 어디에 쓰는지
- NodePort를 동시에 안전하게 배정하기 위해 어떤 기록과 잠금을 쓰는지
- DB를 확인하거나 복구할 때 Kubernetes 상태와 함께 무엇을 봐야 하는지

빠른 이동

필요할 때	문서
승인부터 Pod 생성까지 전체 흐름을 본다	시스템 아키텍처
운영 중 포트 충돌이나 잔여 기록을 확인한다	운영 매뉴얼
API 응답과 오류 형식을 확인한다	API 레퍼런스

1. 사용하는 데이터베이스

DB	관리 주체	용도	관련 저장소
infra-mysql <code>pod_port_db</code>	config-server 전용	NodePort 배정 기록과 FARM 노드 Kerberos 정리 재 시도 목록을 저장한다.	이 저장소(admin_infra)
admin_be MySQL	admin_be(W AS)	신청·사용자 정보를 관리한다.	별도 저장소이며, 이 문서에서는 이름만 언급한다.

NodePort는 동시에 두 요청이 들어와도 같은 번호를 두 번 배정하면 안 된다. config-server는 포트를 배정할 때 infra-mysql의 행 잠금을 사용해 이 문제를 막는다.

신청과 사용자 정보는 admin_be MySQL에서 관리한다. 두 시스템은 서로의 DB를 직접 조회하지 않고 API로만 통신한다.

포트 배정 기록은 infra-mysql에 남는다. 다만 실제 Service가 남아 있는지도 Kubernetes에서 함께 확인해야 한다.

2. infra-mysql 배포 형태

infra-mysql은 `infra-sql/` 디렉터리의 매니페스트로 배포한다.

리소스	내용
StatefulSet <code>infra-mysql</code>	<code>mysql:8.0.36</code> 이미지, replicas는 1이고 Namespace는 <code>ailab-infra</code> 이다.
Service <code>infra-mysql</code>	ClusterIP 타입이며 3306 포트를 제공. config-server는 <code>db.host: "infra-mysql"</code> 로 접속한다.
StorageClass <code>sc-mysql</code>	NFS CSI인 <code>nfs.csi.k8s.io</code> 를 사용. NAS export를 백엔드로 사용하며 <code>reclaimPolicy: Retain</code> , <code>nfsvers=3</code> 으로 설정.
PVC <code>mysql-data</code>	StatefulSet의 <code>volumeClaimTemplate</code> 으로 생성되며 용량은 10Gi. <code>/var/lib/mysql</code> 에 마운트.

StatefulSet은 Pod 이름과 스토리지를 고정하는 Kubernetes 워크로드이다. 따라서 infra-mysql Pod의 이름은 항상 `infra-mysql-0` 이다.

계정과 DB 생성 주체

계정과 데이터베이스는 MySQL 공식 이미지의 초기화 동작이 생성한다.

공식 MySQL 이미지는 데이터 디렉터리가 비어 있는 최초 기동 시 다음 환경변수를 읽어 초기화를 수행한다.

환경변수	역할
<code>MYSQL_DATABASE</code>	지정한 데이터베이스를 생성. 이 시스템에서는 <code>pod_port_db</code> .
<code>MYSQL_USER</code>	일반 사용자 계정을 만든다. 이 시스템에서는 <code>pod_port_user</code> 다.
<code>MYSQL_PASSWORD</code>	생성한 사용자 계정의 비밀번호를 설정.

이미지는 생성한 사용자에게 `MYSQL_DATABASE` 로 지정한 데이터베이스의 전체 권한을 부여한다.

즉 `CREATE USER`, `GRANT` 같은 계정 생성과 권한 부여는 MySQL 이미지가 처음 시작할 때 처리한다.

현재 상태는 다음과 같다.

- 배포 매니페스트와 코드에 수동 `GRANT` 구문이 없다.
- 계정과 권한 생성은 MySQL 이미지의 초기화 동작에 위임한다.
- 별도의 init SQL 파일이나 초기 스키마 스크립트가 없다.

config-server는 Kubernetes Secret인 `config-server-db-secret` 에서 비밀번호를 `DB_PASSWORD` 환경변수로 주입받는다. infra-mysql 측 비밀번호는 배포 manifest나 Secret을 `kubectrl` 로 조회한다.

3. 스키마와 DDL 관리

infra-mysql에서 현재 코드가 사용하는 테이블은 두 개다.

테이블	쓰는 곳	용도
<code>nodeport_allocations</code>	<code>main.py</code>	사용자 Pod의 NodePort 배정 기록
<code>krb5_cleanup_pending</code>	<code>main.py</code> , <code>reconcile_krb5.py</code>	FARM 노드에서 실패한 Kerberos 파일 정리를 다음 CronJob 실행 때 다시 시도할 목록

`server_nodes` 와 같은 다른 테이블은 현재 코드에서 사용하지 않는다.

3.1 `nodeport_allocations`

컬럼	역할
<code>username</code>	포트를 소유한 사용자를 기록한다.
<code>pod_name</code>	포트를 사용하는 사용자 Pod 이름이다. 포트 해제와 동기화 작업의 키로 사용한다.
<code>node_name</code>	Pod가 배치된 Kubernetes 노드를 기록한다.
<code>internal_port</code>	컨테이너 내부 포트이다. 예시는 22, 8888 등이다.
<code>node_port</code>	외부 접근을 위해 배정한 NodePort이다. 기본 Kubernetes 범위는 30000~32767이다.
<code>purpose</code>	포트의 용도를 기록한다. 예시는 <code>ssh</code> , <code>jupyter</code> , <code>custom</code> 등이다.

3.2 `krb5_cleanup_pending`

Kerberos를 쓰는 배포에서는 Pod 또는 계정을 지울 때 FARM 노드의 keytab, timer, ccache도 지운다. 이 작업이 실패하면 config-server는 아래 정보를 `krb5_cleanup_pending` 에 기록한다. 30분마다 실행되는 `reconcile_krb5.py` 는 이 기록을 다시 처리하고, 성공하면 행을 지운다.

컬럼	역할
<code>username</code>	정리할 사용자 이름
<code>node_name</code>	정리 작업이 실패한 FARM 노드
<code>failed_at</code>	마지막으로 정리에 실패한 시각

코드의 `ON DUPLICATE KEY UPDATE` 구문상 `username`, `node_name` 조합에는 중복을 막는 키가 있어야 한다. 정확한 기본 키·인덱스·컬럼 타입은 코드만으로 확인할 수 없으므로, 재구축할 때는 기존 DB의 DDL을 사용한다.

3.3 DDL 생성 방식

DDL(Data Definition Language)은 테이블 구조를 생성하거나 변경하는 SQL이다. 대표적인 구문은 `CREATE TABLE`, `ALTER TABLE` 이다.

현재 코드에는 `CREATE TABLE` 구문이 없다.

따라서 DB를 새로 구축하면 `nodeport_allocations` 와 `krb5_cleanup_pending` 을 직접 만들어야 한다. 자동으로 테이블을 만드는 기능은 없다.

위 컬럼 목록은 코드의 `INSERT` 와 `SELECT` 에서 실제로 쓰는 컬럼을 기준으로 작성했다.

자동 증가 ID, 기본 키, 인덱스, 컬럼 타입은 코드만으로 확인할 수 없다. DB를 다시 만들 때는 기존 DB에서 아래 결과를 먼저 가져와 사용한다.

```
SHOW CREATE TABLE nodeport_allocations;
SHOW CREATE TABLE krb5_cleanup_pending;
```

기존 스키마 백업이 없을 때만 위 컬럼 목록을 바탕으로 새 DDL을 작성한다.

재구축용 DDL 실행 절차

```
kubectl exec -it infra-mysql-0 -n ailab-infra -- \
mysql -u pod_port_user -p pod_port_db
```

MySQL 접속 후 두 테이블의 생성 DDL을 실행한다.

```
CREATE TABLE nodeport_allocations (...);
CREATE TABLE krb5_cleanup_pending (...);
```

실제 운영 환경에서는 두 테이블의 `SHOW CREATE TABLE` 결과를 사용해야 한다.

4. Flask에서의 사용 패턴

4.1 접속 방식

config-server의 MySQL 라이브러리는 PyMySQL이다.

`utils.py` 의 `get_db_connection()` 함수가 다음 환경변수를 읽어 DB에 접속한다.

환경변수	역할
<code>DB_HOST</code>	MySQL 호스트이다.
<code>DB_USER</code>	MySQL 사용자이다.

환경변수	역할
DB_PASSWORD	MySQL 비밀번호이다.
DB_NAME	사용할 데이터베이스 이름이다.

포트 배정, 해제, 동기화와 같은 작업 단위마다 새로운 DB 연결을 생성하고, 작업이 끝나면 `finally` 구문에서 연결을 닫는다.

`autocommit=False` 로 동작하므로 SQL 실행 후 명시적으로 `commit()` 을 호출해야 변경 사항이 반영된다.

4.2 포트 배정 트랜잭션

NodePort 배정 트랜잭션은 `main.py` 의 `allocate_nodeports()` 함수가 담당한다.

처리 순서는 다음과 같다.

1. 사용 중인 포트 목록을 `FOR UPDATE` 로 조회한다.

```
SELECT node_port
FROM nodeport_allocations
FOR UPDATE;
```

1. 클러스터 모든 namespace의 Service가 쓰는 NodePort도 조회해 사용 중인 목록에 합친다. 이 조회가 실패하면 오류를 로그에 남기고 MySQL 목록만으로 계속 진행한다. 이때 다른 Service가 이미 쓰는 포트를 고르면 Service 생성 단계에서 실패할 수 있다.
2. NodePort 범위인 30000~32767에서 사용하지 않는 포트를 찾는다.
3. 요청한 포트 개수만큼 `nodeport_allocations` 에 행을 삽입한다.
4. 모든 삽입이 성공하면 `conn.commit()` 으로 트랜잭션을 확정한다.
5. 예외가 발생하면 `conn.rollback()` 으로 전체 작업을 취소한다.

트랜잭션은 여러 SQL 작업을 묶어서 모두 반영하거나, 하나라도 실패하면 모두 취소하는 기능이다.

4.3 SELECT ... FOR UPDATE를 사용하는 이유

두 개의 승인 요청이 거의 동시에 들어오면 두 트랜잭션이 같은 시점에 동일한 빈 포트 목록을 읽을 수 있다.

잠금이 없다면 다음과 같은 경쟁 상태가 발생할 수 있다.

1. 트랜잭션 A가 30042번 포트가 비어 있다고 판단한다.
2. 트랜잭션 B도 30042번 포트가 비어 있다고 판단한다.
3. A와 B가 모두 30042번 포트를 배정하려고 한다.

4. 같은 NodePort가 두 사용자에게 중복 배정된다.

`SELECT ... FOR UPDATE` 는 현재 작업이 끝날 때까지 조회한 행을 다른 작업이 바꾸지 못하게 한다.

두 번째 요청은 첫 번째 요청이 `commit` 하거나 `rollback` 할 때까지 기다린다. 그래서 동시에 들어온 포트 배정 요청도 차례대로 처리된다.

이 구조를 통해 NodePort 중복 배정을 방지한다.

4.4 포트 해제와 동기화

함수	동작
<code>release_nodeports(pod_name)</code>	해당 <code>pod_name</code> 과 연결된 행을 삭제한 후 <code>commit</code> 한다.
<code>reconcile_nodeport_allocations(namespace)</code>	MySQL 기록과 Kubernetes Service를 비교해 남은 기록을 정리한다.

`release_nodeports`

`release_nodeports(pod_name)` 은 특정 Pod가 사용하던 NodePort 배정 정보를 삭제한다.

처리 방식은 다음과 같다.

```
DELETE FROM nodeport_allocations
WHERE pod_name = <pod_name>;
```

삭제 후 `commit()` 을 호출한다.

`reconcile_nodeport_allocations`

`reconcile_nodeport_allocations(namespace)` 는 infra-mysql 기록과 Kubernetes의 실제 NodePort Service 목록을 비교한다.

이 동기화 작업은 Kubernetes에 실제로 남아 있는 Service를 기준으로 한다.

`app=ai lab-nodeport` 라벨과 같은 `pod_name` 을 가진 Service가 Kubernetes에 하나도 없는데 infra-mysql에 그 Pod의 배정 기록이 남아 있으면 해당 Pod의 행을 삭제한다. 포트별 Service를 하나씩 비교하는 방식은 아니다. 따라서 같은 Pod의 Service가 하나라도 남아 있으면 그 Pod의 DB 행은 모두 남는다.

포트 배정 직전에 이 함수를 호출하지만, Kubernetes API와 DB를 매 요청마다 전체 조회하지 않도록 마지막 시도부터 5분(300초)이 지나야 다시 실행한다. 실패한 시도도 이 5분 간격에 포함한다.

동기화에 실패하면 트랜잭션을 `rollback`한다. 다만 동기화 실패는 치명적 오류로 처리하지 않으며, 포트 배정 작업은 계속 진행한다.

5. 운영 절차

5.1 상태 확인

접속 방법은 [운영 매뉴얼](#)의 관리 주소와 명령 표를 따른다.

`kubectl exec` 로 `infra-mysql-0` Pod 내부에서 MySQL 클라이언트를 실행한다. 비밀번호는 Kubernetes Secret에서 조회한다.

```
kubectl exec -it infra-mysql-0 -n ailab-infra -- \
mysql -u pod_port_user -p pod_port_db
```

현재 점유 중인 포트 전체는 다음 쿼리로 조회한다.

```
SELECT
    username,
    pod_name,
    node_port,
    purpose
FROM nodeport_allocations
ORDER BY node_port;
```

특정 사용자의 포트 배정 현황은 다음 쿼리로 조회한다.

```
SELECT *
FROM nodeport_allocations
WHERE username = '<username>';
```

FARM 노드에서 다시 정리해야 할 Kerberos 파일 목록은 다음 쿼리로 확인한다.

```
SELECT username, node_name, failed_at
FROM krb5_cleanup_pending
ORDER BY failed_at;
```

5.2 백업

현재 자동 백업은 구성되어 있지 않다.

배포 매니페스트와 코드 어디에도 다음 구성은 존재하지 않는다.

- `mysqldump` 자동 실행

- Kubernetes CronJob
- 외부 백업 스크립트
- 정기 스냅샷 설정

현재 데이터 보호 장치는 StorageClass의 다음 설정뿐이다.

```
reclaimPolicy: Retain
```

Retain 정책은 PVC나 StatefulSet을 삭제하더라도 NFS 위의 PV 데이터를 자동으로 제거하지 않도록 한다.

다만 이는 백업이 아니라 삭제 방지 장치이다. 파일 손상, 잘못된 SQL 실행, NAS 장애에는 대응하지 못한다.

infra-mysql에는 NodePort 배정 기록과 Kerberos 정리 재시도 기록이 있다. 데이터가 사라지면 Kubernetes의 실제 Service를 보고 포트 기록을 다시 만들 수 있다. Kerberos 정리 재시도 기록은 삭제 실패 로그와 FARM 노드의 파일을 대조해 다시 만들어야 한다.

스키마 정의는 코드에서 자동으로 생성되지 않으므로 두 테이블의 DDL을 함께 백업해야 한다.

```
SHOW CREATE TABLE nodeport_allocations;
SHOW CREATE TABLE krb5_cleanup_pending;
```

5.3 재구축 순서

infra-mysql 재구축은 다음 순서로 진행한다.

1. **nfs-mysql.yaml** 을 적용하여 StorageClass **sc-mysql** 을 생성한다.
2. **infra-mysql.yaml** 을 적용하여 StatefulSet과 Service를 생성한다.
3. MySQL 최초 기동 과정에서 **pod_port_db** 데이터베이스와 **pod_port_user** 계정이 자동 생성되는지 확인한다.
4. **nodeport_allocations** , **krb5_cleanup_pending** 테이블의 DDL을 수동으로 실행한다.
5. config-server를 재시작한다.
6. 사용자 Pod 생성 요청을 한 건 실행한다.
7. infra-mysql에 NodePort 배정 행이 정상적으로 삽입되는지 확인한다.
8. Kubernetes에 NodePort Service가 정상적으로 생성되는지 확인한다.

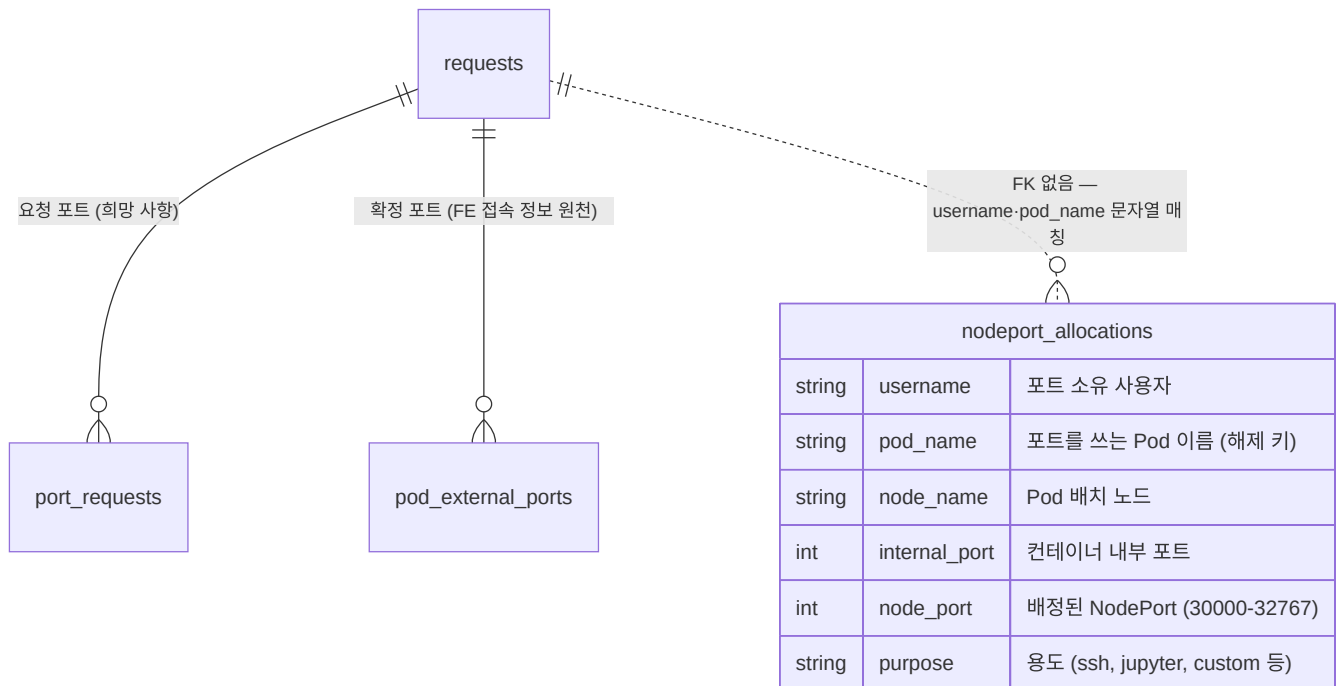
자동 스키마 bootstrap이 없으므로 4단계의 수동 DDL 실행은 필수이다.

6. admin_be DB와 포트 정보를 비교하는 방법

다음 ERD는 포트 요청과 배정 정보를 두 데이터베이스에서 어떻게 비교하는지 보여 준다.

`nodeport_allocations`의 컬럼은 코드에서 실제 사용하는 컬럼을 기준으로 작성한다. 타입과 기본 키 구성은 미확인 상태이므로 개념 수준으로 표기한다.

admin_be의 테이블은 별도 저장소에서 관리하므로 테이블 이름과 관계만 표현하고 상세 컬럼은 생략한다.



infra-mysql과 admin_be MySQL은 물리적으로 분리된 데이터베이스이다.

두 데이터베이스 사이에는 외래 키가 존재하지 않는다.

`nodeport_allocations`와 admin_be 테이블은 `username`, `pod_name` 문자열로만 비교할 수 있다. 외래 키가 없으므로 두 DB를 SQL 조인으로 조회할 수 없다.

대조 작업은 다음 방식으로 수행한다.

1. admin_be MySQL에서 요청과 확정 포트 정보를 조회한다.
2. infra-mysql에서 `nodeport_allocations`를 조회한다.
3. `username`과 `pod_name`을 기준으로 결과를 비교한다.
4. 불일치가 있으면 Kubernetes의 실제 Service 상태까지 함께 확인한다.

ERD의 점선은 외래 키 없이 `username`, `pod_name`을 비교한다는 표시다.

핵심 구조 요약

이 시스템의 데이터베이스 구조는 다음과 같다.

1. MySQL은 infra-mysql과 admin_be MySQL 두 개로 분리된다.
2. infra-mysql은 NodePort 배정 기록과 Kerberos 정리 재시도 기록을 관리한다.
3. admin_be MySQL은 사용자와 신청 정보를 관리한다.

4. 두 DB는 직접 연결하지 않고 API를 통해 통신한다.
5. infra-mysql의 데이터베이스와 계정은 MySQL 공식 이미지가 최초 기동 시 생성한다.
6. `nodeport_allocations`, `krb5_cleanup_pending` 테이블은 자동으로 생성되지 않으므로 수동 DDL이 필요하다.
7. NodePort 배정은 `SELECT ... FOR UPDATE` 와 트랜잭션으로 처리한다.
8. 포트 해제는 `pod_name` 을 기준으로 DB 행을 삭제한다.
9. Kubernetes 실제 Service와 DB 기록의 차이는 reconcile 작업으로 정리한다.
10. 자동 백업은 없으며 `reclaimPolicy: Retain` 은 삭제 방지 장치일 뿐 백업은 아니다.
11. 두 데이터베이스 사이에는 외래 키가 없으며 `username` 과 `pod_name` 으로 연결해 비교한다.
12. 장애 조사 시 infra-mysql, admin_be MySQL, Kubernetes Service 상태를 각각 조회해 비교한다.

처음 작업할 때

이 문서에서 다루는 내용

- `develop`, `main`, `feature/*`, `hotfix/*` 브랜치의 역할
- 로컬에서 config-server 이미지를 띄워 Swagger와 `/health` 를 확인하는 방법
- 클러스터에서 배포 결과를 확인할 위치와 PR 규칙

빠른 이동

필요할 때	문서
운영 배포와 장애 대응을 바로 본다	운영 매뉴얼
Helm 값과 차트 구조를 본다	Helm 차트 레퍼런스
config-server가 실제로 무슨 일을 하는지 먼저 본다	개요

1. 브랜치와 배포

코드는 `develop` 에서 모으고, `main` 에 병합되면 운영 서버에 배포된다.

브랜치	역할	배포 여부	비고
<code>main</code>	운영 서버에 배포되는 브랜치	O	PR을 병합하면 배포한다.
<code>develop</code>	기능을 합쳐 테스트하는 브랜치	X	기능 개발이 끝난 뒤 이 브랜치로 PR을 만든다.
<code>feature/*</code>	기능별 작업 브랜치	X	<code>develop</code> 에서 만든다.
<code>hotfix/*</code>	긴급 수정 브랜치	X	<code>main</code> 에서 만들고, 배포하려면 다시 <code>main</code> 에 병합한다.

2. 로컬에서 API 확인하기

config-server는 Python 3.10 slim과 gunicorn으로 만든 컨테이너로 실행한다. 로컬에서는 Docker로 띄워 Swagger와 `/health` 응답을 확인할 수 있다.

```
cd admin_infra/config-server

# 이미지 빌드
docker build -t config-server:local .

# 실행 (컨테이너 포트 8000)
docker run --rm -p 8000:8000 config-server:local
```

- Swagger: `http://localhost:8000/apidocs/`
- 상태 확인: `http://localhost:8000/health`

계정 생성과 Pod 생성은 Kubernetes API, infra-mysql, `/kube_share` NFS, NAS SSH가 있어야 동작한다. 로컬 컨테이너에서는 Swagger와 코드 변경을 확인하고, 실제 생성은 클러스터에서 시험 사용자 한 명으로 확인한다.

3. 클러스터에서 확인할 것

- 운영 config-server는 FARM 클러스터의 `ailab-infra` namespace에서 실행한다.
- `kubectl` 은 control-plane에서 실행한다. CI/CD는 운영 서버 `farm8` 에 SSH로 접속해 `helm upgrade` 를 실행한다.
- 배포 뒤 확인할 명령은 운영 매뉴얼에 있다.

```
# 배포된 config-server 확인
kubectl get pods -n ailab-infra | grep config-server
```

4. 커밋과 PR

1. `develop` 에서 `feature/#이슈번호-작업명` 브랜치를 만든다. 예: `feature/#155-scheduler`
2. 커밋 메시지는 `[분류] #이슈번호 설명` 으로 쓴다. 예: `[feat] #4 메인 기능 만들기`
3. 작업이 끝나면 `feature` 에서 `develop` 으로 PR을 만든다.
4. 정기 배포는 `develop` 에서 `main` 으로 PR을 만든다. 제목은 `[deploy] develop -> main (설명)` 으로 쓴다.
5. 한 명 이상 승인한 뒤 병합한다. `main` 에 병합하면 운영 서버에 배포된다.

배포 전에 다른 사람이 같은 클러스터에서 테스트하거나 Helm 값을 바꾸고 있는지 확인한다. 현재 실행 중인 이미지가 어느 커밋에서 만들어졌는지도 확인한 뒤 배포한다.

운영 매뉴얼

이 문서에서 다루는 내용

- config-server 배포 상태 확인, 운영 주소, 주요 리소스 이름
- 계정과 Pod 생성·삭제 뒤에 무엇을 점검해야 하는지
- 오류 응답, 잔여 리소스, NodePort 기록, Secret을 어떤 순서로 확인해야 하는지

빠른 이동

필요할 때	문서
생성 순서와 실패 지점을 먼저 본다	시스템 아키텍처
API 입력과 오류 형식을 본다	API 레퍼런스
포트 DB와 복구 기준을 본다	데이터베이스
Helm 값과 Secret 이름을 본다	Helm 차트 레퍼런스

config-server 배포, 계정과 Pod 작업, 문제 확인·복구 방법을 정리한다. 계정·Pod·포트가 만들어지는 순서는 [시스템 아키텍처](#), 용어는 [기초 개념](#)에서 확인한다. Helm 값과 Secret 이름은 [Helm 차트 레퍼런스](#)에 있다. **keytab·개인키·비밀번호의 실제 값은 문서나 로그에 적지 않는다.**

1. 작업 전에 확인할 것

작업을 시작하기 전에 다음을 지킨다.

1. **계정과 Pod는 따로 지운다.** 계정을 지운 뒤 Pod가 남거나, Pod를 지운 뒤 계정과 홈이 남을 수 있다. 삭제할 때는 두 API를 모두 호출했는지 확인한다.
2. **작업 전후 값을 기록한다.** 사용자 이름, Pod 이름, UID/GID, 선택 노드, NodePort를 적어 둔다. 문제가 생기면 이 기록으로 어느 단계에서 멈췄는지 확인한다.
3. **API 오류 뒤에는 같은 요청을 바로 반복하지 않는다.** 계정·Pod 생성은 여러 외부 시스템을 차례로 바꾼다. 오류 응답의 `stage` 와 `rollback` 을 먼저 보고 일부만 만들어진 것이 없는지 확인한다.
4. **비밀 값은 출력하지 않는다.** keytab, SSH 개인키, DB 비밀번호, shadow 내용은 화면·로그·티켓 어디에도 남기지 않는다.

- 5. **Kubernetes 리소스를 직접 지우기 전에 기록을 확인한다.** config-server와 MySQL에 남은 포트 배정 기록을 먼저 본다. Pod나 Service 한쪽만 지우면 기록과 실제 상태가 더 달라질 수 있다.
- 6. **예전 chart·스크립트는 복구에 쓰지 않는다.** 레포에 남아 있는 레거시(ContainerSSH, 사용자 PVC chart, restart_*.sh)는 현행 구조와 무관하다(10절 금지 사항).
- 7. **config-server 제어 API는 승인된 경로에서만 접근한다.** 이 API에는 인증이 없으므로(API 레퍼런스의 공통 사항), WAS와 승인된 운영망 밖으로 노출되지 않게 유지한다.
- 8. **배포 뒤에도 실제 동작을 확인한다.** 배포가 끝나면 승인 한 건으로 계정·Pod·접속 포트가 만들어지는지 확인한다.

2. 이름과 접속 주소

2.1 주요 이름과 포트

항목	값
운영 네임스페이스	ailab-infra
Helm release	containersssh-config-server
config-server Deployment	containersssh-config-server
config-server Service	containersssh-config-service
config-server HTTP 포트	Service 80 → 컨테이너 8000
config-server 외부 NodePort	30082
사용자 Pod 접두사	ailab-
사용자 NodePort 범위	30000-32767
Pod Ready 최대 대기	300초 (배포 브랜치에 따라 다를 수 있음)
Pod 삭제 최대 대기	60초
진행 상태 보존	최종 갱신 후 1시간

config-server 관련 리소스에는 과거 이름인 containersssh 접두사가 남아 있다. 현재 ContainerSSH 프록시는 배포하지 않는다.

2.2 서비스별 namespace

namespace는 Kubernetes 리소스를 나누어 두는 단위다.

namespace	배포 대상	분리한 이유
ailab-infra	config-server, infra-mysql, redis, 사용자 Pod 전체	사용자 Pod를 만드는 서비스와 Pod를 함께 둔다.

namespace	배포 대상	분리한 이유
ailab-frontend	admin_fe (정적 서빙)	화면 파일을 제공하는 서비스만 둔다.
default (admin-prod)	admin_be (Spring Boot WAS)	신청과 승인 정보를 관리한다. config-server가 사용자 설정을 조회할 때 admin-prod.default 를 사용한다.
monitoring	kube-prometheus-stack (Prometheus-Grafana)	애플리케이션을 다시 배포해도 모니터링 데이터를 계속 볼 수 있게 따로 둔다.

레거시 namespace `containerssh`, `cssh` 가 클러스터에 남아 있을 수 있다. 현재 서비스는 이 namespace를 쓰지 않는다. 리소스를 지우려면 실제 사용 중인지 먼저 확인한다.

2.3 관리할 때 쓰는 주소와 명령

대상	방법	비고
Grafana 대시보드	<code>http://210.94.179.18:30080</code>	admin 로그인. 비밀번호는 K8s Secret에서 kubectl로 조회한다
Swagger (config-server API)	<code>http://210.94.179.18:30082/apidocs/</code>	현재 API 목록을 볼 수 있다. 인증이 없으므로 <code>Try it out</code> 은 실제 계정·Pod·파일을 바꾼다. README의 9732 포트는 잘못된 값이며 NodePort는 30082다.
infra-mysql (pod_port_db)	<code>kubectl exec -it infra-mysql-0 -n ailab-infra -- mysql -u pod_port_user -p pod_port_db</code>	비밀번호는 K8s Secret에서 kubectl로 조회한다
config-server 로그	<code>kubectl logs -f deploy/containerssh-config-server -n ailab-infra</code>	승인·삭제 실패의 1차 확인 지점

Secret 값 조회는 이름을 찾은 뒤 jsonpath로 디코딩한다.

```
kubectl get secrets -n ailab-infra # Secret 이름 확인
kubectl get secret <이름> -n ailab-infra -o jsonpath='{.data.<키>}' | base64 -d
```

3. 작업 전 상태 확인

매일 한 번, 또는 배포·계정 작업을 하기 전에 아래 항목을 확인한다.

3.1 config-server가 떠 있는지

```
kubectl -n ailab-infra get deploy,sts,pod
kubectl -n ailab-infra get svc
kubectl -n ailab-infra get events --sort-by=.lastTimestamp
kubectl -n ailab-infra rollout status deploy/containersssh-config-server
```

다음 네 가지를 확인한다.

- config-server Deployment가 1/1 Ready이다.
- infra-mysql과 Redis 구성요소가 Ready이다.
- config-server Service가 30082 NodePort를 유지한다.
- 새 Warning 이벤트와 반복 재시작이 없다.

3.2 현재 배포 설정

비밀 값을 출력하지 않고 배포 구조를 확인한다.

```
kubectl -n ailab-infra get deploy containersssh-config-server -o custom-
columns=IMAGE:.spec.template.spec.containers[0].image,SA:.spec.template.spec.serviceAccountName,NODE:.spec.tem
plate.spec.nodeSelector
kubectl -n ailab-infra describe deploy containersssh-config-server
kubectl -n ailab-infra get svc containersssh-config-service
helm -n ailab-infra get manifest containersssh-config-server
```

현재 배포에는 아래 값이 들어가야 한다.

항목	확인할 값
replica	1
프로세스	gunicorn worker 4개, timeout 700초
ServiceAccount	config-server
config-server 배치	지정 control-plane 노드의 nodeSelector와 NoSchedule toleration
컨테이너 포트	8000
readiness	GET /health, 초기 15초, 10초 주기
liveness	별도 probe 없음
리소스	request 500m/512Mi, limit 2000m/1024Mi
API Service	NodePort 30082, Service 80 → container 8000
계정 저장소	NFS kube_share → /kube_share

항목	확인할 값
비밀 mount	NAS, FARM, FARM AD SSH Secret이 read-only
Kerberos 설정	krb5-conf ConfigMap → /etc/krb5.conf
image-store	pvc-image-store → /image-store

config-server 계정의 Kubernetes 권한도 확인한다.

```
kubectl auth can-i list pods -n ailab-infra --as=system:serviceaccount:ailab-infra:config-server
kubectl auth can-i create services -n ailab-infra --as=system:serviceaccount:ailab-infra:config-server
kubectl auth can-i list nodes --as=system:serviceaccount:ailab-infra:config-server
kubectl auth can-i list services --all-namespaces --as=system:serviceaccount:ailab-infra:config-server
```

마지막 명령은 다른 namespace에서 사용 중인 NodePort도 확인하는 권한이다. **no**가 나오면 권한을 바로 바꾸지 말고, 현재 배포의 Role과 ClusterRole을 먼저 확인한다.

3.3 API 응답과 로그

```
config_server_url='운영_config-server_URL'

curl -fsS "$config_server_url/health"
kubectl -n ailab-infra logs deploy/containersssh-config-server --since=30m
```

/health는 config-server 프로세스만 확인한다. admin_be, MySQL, Redis, Prometheus, NAS, FARM SSH, AD 상태는 확인하지 않는다. /health가 OK여도 승인이 실패하면 이 서비스들을 차례로 확인한다.

Pod 생성 로그에는 다음 태그가 차례로 나온다.

- **FETCH_USER_CONFIG** — admin_be 사용자 설정 조회
- **SELECT_NODE** — Prometheus 점수 기반 노드 선택
- **NODEPORT** — 포트 배정
- **WAIT_POD_READY** — Pod Ready 폴링
- **FARM SSH**, **FARM AD SSH** — 노드·AD 원격 작업

3.4 사용자 Pod와 접속 Service

```
kubectl -n ailab-infra get pod -l username
kubectl -n ailab-infra get svc -l app=ailab-nodeport
kubectl -n ailab-infra get pod -o wide
```

각 Service의 `pod_name` 라벨이 실제 Pod 이름과 같은지 확인한다. Error·Failed Pod나 연결 대상이 없는 Service를 발견해도 바로 지우지 않는다. 사용자 이름, 포트 배정 기록, 생성·삭제 로그를 먼저 확인한다(1절 5번).

특정 사용자 Pod의 실행 설정과 마운트를 확인할 때는 jsonpath로 필요한 필드만 뽑는다.

```
pod_name='점검할_ailab_Pod_이름'

kubectl -n ailab-infra get pod "$pod_name" -o jsonpath='{.spec.nodeName}{"\n"}{.spec.restartPolicy}{"\n"}{.spec.containers[0].image}{"\n"}{.spec.containers[0].resources}{"\n"}'
kubectl -n ailab-infra get pod "$pod_name" -o jsonpath='{range .spec.volumes[*]}{.name}{"\t"}{.hostPath.path}{"\t"}{.hostPath.type}{"\n"}{end}'
kubectl -n ailab-infra get pod "$pod_name" -o jsonpath='{range .spec.containers[0].volumeMounts[*]}{.name}{"\t"}{.mountPath}{"\t"}{.readOnly}{"\n"}{end}'
kubectl -n ailab-infra get pod "$pod_name" -o jsonpath='{range .spec.containers[0].env[*]}{.name}{"\n"}{end}'
```

사용자 Pod에서 다음 항목을 확인한다.

- FARM 홈이 `/home` 에 연결되어 있다. 노드에 마운트된 NFS 경로를 hostPath로 연결한다.
- Kerberos 활성 시 `/run/user/<uid>` 가 같은 경로로 연결되어 있다.
- GPU를 배정했다면 `/dev/nvidia<N>` 과 추가 NVIDIA 디바이스가 있다.
- **사용자 Pod에는 keytab, kube_share, image-store가 없어야 한다.** 이 셋이 보이면 잘못된 스펙이다.

현재 Pod 스펙에는 securityContext와 사용자 probe가 없다. 컨테이너 안의 UID/GID, SSH, Jupyter, noVNC 동작은 guest image의 entrypoint까지 확인해야 알 수 있다.

아래 명령은 중복된 NodePort만 출력한다. 아무것도 나오지 않아야 한다.

```
kubectl -n ailab-infra get svc -l app=ailab-nodeport -o jsonpath='{range .items[*].spec.ports[*]}{.nodePort}{"\n"}{end}' | sort -n | uniq -d
```

3.5 GPU와 NFS 상태

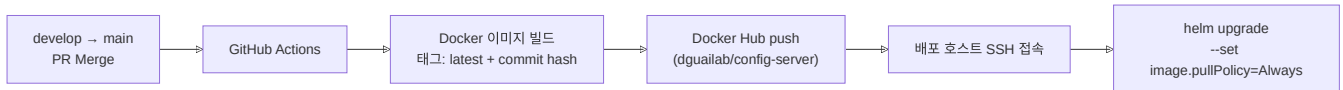
대상	방법
통합 대시보드 (Grafana)	<code>http://210.94.179.18:30080</code> admin 로그인
스토리지 응답성	Grafana 대시보드 <code>storage-latency-health</code> — NFS 장애 의심 시 가장 먼저 본다
Prometheus 직접 질의	<code>kubectl -n monitoring port-forward svc/monitoring-kube-prometheus-prometheus 9090:9090</code>

Prometheus 값은 대시보드뿐 아니라 Pod를 올릴 노드를 고를 때도 쓴다. Prometheus 조회가 모두 실패하면 `create-pod` 도 실패한다. GPU 지표가 없을 때 노드를 고르는 방식은 **시스템 아키텍처**의 GPU 노드 선택 방식에 있다.

4. 배포와 롤백

4.1 배포가 진행되는 순서

배포 자동화는 GitHub Actions를 사용하며, **main** 브랜치에 **push** 이벤트가 발생할 때만 실행된다. 즉 **develop** → **main** PR이 Merge 되는 순간 운영 배포가 시작된다.



CI는 다음을 실행한다.

1. **config-server/Dockerfile** 로 이미지를 빌드한다.
2. **latest** 와 소스 버전을 나타내는 commit hash 태그 두 가지로 레지스트리에 푸시한다.
3. **config-server/Chart** 를 배포 호스트로 전달한다.
4. CI Secret의 NFS, NAS, FARM, AD, Realm 값을 Helm에 주입한다.
5. **ailab-infra** release의 이미지 태그를 commit hash로 갱신한다.

CI/CD용 GitHub Secrets는 **DOCKER_USERNAME**, **DOCKER_PASSWORD**, **K8S_HOST**, **K8S_USERNAME**, **K8S_PRIVATE_KEY**, **K8S_PORT** 이다. 현재 username이 toni 기준으로 등록되어 있어 관리자 변경 시 인수인계가 필요하다. 비밀 값은 repository 파일, 일반 운영 절차, 로그에 복사하지 않는다.

4.2 배포 전 확인

코드를 병합하기 전에 다음을 확인한다.

1. **다른 작업 진행 여부** — 다른 사람이 같은 클러스터에서 인프라 테스트를 하거나 Helm 값을 바꾸고 있으면, 변경 시점과 범위를 먼저 맞춘다.
2. **배포 브랜치** — main과 hotfix 계열 브랜치가 분기된 이력이 있어 Ready 대기·krb5 방식·마운트 구성이 브랜치마다 다르다. "지금 클러스터에 떠 있는 이미지가 어느 커밋인가"부터 확인한다.

명령으로는 다음을 확인한다.

```
helm lint config-server/Chart
git diff -- config-server .github/workflows/deploy-config-server.yaml
kubectl -n ailab-infra get deploy/containersssh-config-server -o
jsonpath='{.spec.template.spec.containers[0].image}{"\n"}'
helm -n ailab-infra history containersssh-config-server
```

다음에 맞아야 한다.

- 변경 범위가 config-server와 chart 의도에 맞다.
- 이미지 태그가 commit hash인지 확인한다.
- chart가 참조하는 Secret과 ConfigMap 이름이 운영 네임스페이스에 존재한다(Helm 차트 레퍼런스 1.4절).
- FARM 노드 목록과 AD DC 목록의 이름이 실제 Kubernetes 노드 이름과 일치한다.

- `FARM_HOME_MOUNT_ROOT` 가 모든 후보 노드에서 동일한 FARM 홈을 가리킨다.

4.3 배포 뒤 확인

```
kubectl -n ailab-infra rollout status deploy/containersssh-config-server --timeout=5m
kubectl -n ailab-infra get pod -l app=containersssh-config-server -o wide
kubectl -n ailab-infra logs deploy/containersssh-config-server --since=10m
kubectl -n ailab-infra get svc containersssh-config-service
```

로그에서 먼저 볼 오류는 다음과 같다.

- `ModuleNotFoundError` — requirements.txt 누락 또는 파일명 불일치이다.
- `WORKER TIMEOUT` — 초기 로딩 시간이 긴 경우이다 (Dockerfile 타임아웃 설정 확인).

이후 시험 사용자 또는 승인된 기존 사용자 한 명으로 실제 동작을 확인한다.

1. `/health` 를 호출한다.
2. 사용자 조회 API를 호출한다.
3. 시험 사용자 또는 승인된 기존 사용자로 Pod 생성 단계를 확인한다.
4. NodePort Service와 실제 접속을 확인한다.
5. 선택 노드의 Kerberos ccache와 FARM 홈 읽기·쓰기를 확인한다.
6. Pod 삭제 후 Service, 포트 예약, 노드 타이머가 정리되는지 확인한다.

4.4 롤백

배포 뒤 기존에 되던 기능이 동작하지 않으면 Helm release 이력을 보고 이전 버전으로 되돌린다.

```
helm -n ailab-infra history containersssh-config-server
helm -n ailab-infra rollback containersssh-config-server '정상_리비전'
kubectl -n ailab-infra rollout status deploy/containersssh-config-server --timeout=5m
```

롤백은 **config-server Deployment**만 이전 버전으로 되돌린다. 계정 파일, MySQL 포트 배정 기록, Kubernetes Service, FARM 노드의 keytab은 자동으로 이전 상태가 되지 않는다. 배포 중 실패한 요청이 있으면 해당 요청의 `stage`, `rollback`, 실제 리소스를 함께 확인한다.

5. 계정 관리

계정 정보는 `/kube_share` 의 passwd 파일에 기록한다(시스템 아키텍처의 계정 정보). 아래 API만 이 파일을 수정한다. passwd 파일을 직접 편집하지 않는다.

5.1 조회

```
config_server_url='운영_config-server_URL'
target_user='사용자명'

curl -fsS "$config_server_url/accounts/users"
curl -fsS "$config_server_url/accounts/users/$target_user"
```

계정 상세에서 사용자 이름, UID, GID, 홈, 셸을 확인한다. shadow와 keytab 내용은 조회하지 않는다(1절 4번).

5.2 생성

계정은 관리자 승인 뒤 admin_be가 만든다. 운영자가 직접 호출하는 것은 예외이며, 호출 전에는 같은 사용자가 없는지 확인한다.

```
curl -fsS -X PUT "$config_server_url/accounts/users" -H 'Content-Type: application/json' --data '{
  "name": "사용자명",
  "passwd_base64": "BASE64로_인코딩한_비밀번호",
  "gecos": "사용자 설명",
  "supplementary_groups": [
    {"name": "그룹명", "gid": 20050}
  ]
}'
```

성공 응답의 UID/GID를 기록하고(1절 2번) 다음 순서로 확인한다.

1. 사용자 조회 API
2. NAS 홈 디렉터리의 숫자 소유권과 0700
3. FARM AD 사용자와 <user>_gid
4. Kubernetes keytab Secret의 **존재 여부만** 확인 (내용은 보지 않는다)

```
target_user='사용자명'
kubectl -n ailab-infra get secret "krb5-keytab-$target_user" -o custom-
columns=NAME:.metadata.name,CREATED:.metadata.creationTimestamp
```

5.3 그룹

그룹 생성은 이름을 필수로 받고, GID를 생략하면 다음 사용 가능한 값을 할당한다.

```
curl -fsS -X PUT "$config_server_url/accounts/groups" -H 'Content-Type: application/json' --data '{"name":"그룹
명","members":["사용자명"]}'
```

기존 그룹에 사용자를 추가한다.

```
curl -fsS -X PUT "$config_server_url/accounts/users/사용자명/groups" -H 'Content-Type: application/json' --data '{"groups":["그룹명"]}'
```

사용자가 기본 그룹으로 쓰는 그룹은 삭제할 수 없다.

5.4 삭제

계정 삭제 전에 사용자의 Pod를 먼저 삭제한다(6.2절). Pod 이름과 선택 노드를 기록하고, Service와 NodePort 정리가 끝난 뒤 계정을 삭제한다. 순서를 지키지 않으면 주인 없는 Pod가 남는다.

```
target_user='사용자명'
curl -fsS -X DELETE "$config_server_url/accounts/users/$target_user"
```

삭제 뒤에는 다음을 확인한다.

- 계정 조회가 404를 반환한다.
- 사용자 Pod와 Service가 없다.
- keytab Secret이 없다.
- 모든 FARM 노드에서 사용자 타이머와 keytab이 없다.
- NAS 홈 디렉터리가 현재 보존·삭제 규칙에 맞게 처리되었다.

NAS 홈 삭제가 실패해도 계정 삭제는 계속 진행된다. **deleted** 응답 뒤에도 홈 디렉터리가 남을 수 있으므로 로그와 NAS 경로를 확인한다.

6. 사용자 Pod 관리

6.1 만들기

계정과 WAS 사용자 설정이 있는지 확인한 뒤 호출한다.

```
target_user='사용자명'

curl -fsS "$config_server_url/accounts/users/$target_user"
curl -fsS -X POST "$config_server_url/create-pod" -H 'Content-Type: application/json' --data '{"username":"'${target_user}'"}'
```

노드 선택, 포트 배정, krb5 배포, Pod 생성, Ready 대기를 차례로 처리하므로 응답까지 시간이 걸린다. 기다리는 동안 다른 터미널에서 진행 상태를 조회할 수 있다.

```
curl -fsS "$config_server_url/pods/$target_user/status"
```

성공 응답에서 `pod_name`, `node`, `ports` 를 기록하고(1절 2번), 실제 리소스를 확인한다.

```
pod_name='응답의_Pod_이름'

kubectl -n ailab-infra get pod "$pod_name" -o wide
kubectl -n ailab-infra get svc -l "pod_name=$pod_name"
kubectl -n ailab-infra describe pod "$pod_name"
```

Pod가 Ready이고, 응답의 각 포트에 해당하는 NodePort Service가 있으며, SSH·Jupyter·선택 기능에 실제로 접속 되면 생성이 끝난 것이다.

6.2 지우기

삭제 전 사용자에게 중단 사실을 확인하고 Pod 이름을 정확히 기록한다.

```
pod_name='삭제할_ailab_Pod_이름'

curl -fsS -X POST "$config_server_url/delete-pod" -H 'Content-Type: application/json' --data
'{"pod_name":"$pod_name"}'
```

응답의 `progress` 에서 각 단계의 완료 여부를 확인한다.

필드	의미
<code>servicesDeleted</code>	연결된 NodePort Service 삭제 완료
<code>nodeportsReleased</code>	MySQL 예약 해제 완료
<code>podDeleteRequested</code>	Kubernetes 삭제 요청 접수
<code>podDeleted</code>	실제 Pod 삭제 확인

삭제 후 실제로 사라졌는지 조회한다.

```
kubectl -n ailab-infra get pod "$pod_name"
kubectl -n ailab-infra get svc -l "pod_name=$pod_name"
```

Pod가 이미 없다는 성공 응답은 노드 이름을 읽지 못해 Kerberos 노드 정리를 건너뛰었을 수 있다. 이 경우 사용자와 마지막 노드를 기준으로 `kdc-setup` 운영 문서의 keytab, ccache, timer를 확인한다.

수동으로 지울 때 **kubectl** 로 Pod만 지우지 않는다. Service와 MySQL 배정 행이 남는다. 배정 정리(reconcile)는 같은 `pod_name` 의 Service가 하나라도 남아 있으면 그 Pod의 배정 행을 지우지 않는다. Service가 남은 상태에서는 이 배정이 계속 남는다(시스템 아키텍처의 NodePort 배정 방식).

6.3 사용자 Pod에 들어가 확인하기

어떤 경로든 Pod 이름 찾기부터 시작한다. 사용자 Pod 이름은 `ailab-<username>--<랜덤>` 형식이다.

```
kubectl get pods -n ailab-infra | grep <username>
```

(a) **kubectl exec** 로 확인하기. 사용자의 SSH 설정·비밀번호와 관계없이 root로 바로 들어간다. 컨테이너 프로세스, 마운트, 홈 권한을 확인할 때 쓴다.

```
kubectl exec -it ailab-user2100-1a2b3c4d -n ailab-infra -- bash
```

(b) **사용자가 쓰는 SSH 경로로 확인하기.** 사용자가 접속하지 못한다는 문의는 이 경로로 확인한다. NodePort는 Service 목록이나 `infra-mysql` `nodeport_allocations` 에서 확인한다. 이 번호는 pfSense 전달 대상 확인에 쓰고, SSH 접속에는 사용자에게 안내한 외부 주소와 포트를 쓴다.

```
kubectl get svc -n ailab-infra | grep <username> # ssh 용도 Service의 NodePort 확인
ssh -p <사용자에게_안내한_외부_포트> <username>@<사용자에게_안내한_주소>
```

교내망에서는 되는데 외부에서만 접속하지 못하면 **시스템 아키텍처**의 외부 접근 경로를 보고 Service의 NodePort와 pfSense 전달 대상이 같은지 확인한다.

(c) **컨테이너에 들어갈 수 없을 때.** Pod가 CrashLoop 등으로 `exec` 할 수 없으면 로그와 노드에서 확인한다.

```
kubectl logs ailab-user2100-1a2b3c4d -n ailab-infra # 컨테이너 stdout 로그
kubectl describe pod ailab-user2100-1a2b3c4d -n ailab-infra # 이벤트(FailedMount 등)
```

노드에 SSH로 들어가 컨테이너 런타임을 직접 보는 방법도 있다.

```
crictl ps | grep <username> # 해당 노드의 컨테이너 목록
crictl logs <container-id>
```

6.4 Pod가 죽었거나 노드에 문제가 있을 때

현재 config-server는 Pod가 죽은 것을 감지해 다른 노드에 자동으로 다시 만들지 않는다. 운영자가 다음 순서로 확인하고 복구한다.

1. 사용자 이름, 기존 Pod 이름, resource group, 노드, 이미지, NodePort를 기록한다.
2. Pod가 Failed인지, 노드가 NotReady인지, 애플리케이션만 응답하지 않는지 구분한다.
3. 같은 사용자의 다른 Running Pod가 없는지 확인한다.
4. FARM 홈과 AD·keytab Secret은 삭제하지 않는다. Pod를 다시 만들어도 계정 관련 파일은 그대로 써야 한다.
5. 장애 노드가 후보에서 다시 선택되지 않도록 WAS resource group 또는 노드 상태를 먼저 정리한다.
6. `/create-pod` 로 대체 Pod를 만들고 새 노드의 ccache, 홈, SSH·Jupyter를 확인한다.
7. 현재 구현은 새 NodePort를 할당하므로 **새 접속 정보를 사용자에게 전달한다.**

8. 대체 Pod가 정상인지 확인한 뒤에야 기존 Service, Pod, NodePort 배정 기록, 노드의 Kerberos 파일을 정리한다.

```
target_user='사용자명'

kubectl -n ailab-infra get pod -l "username=$target_user" -o wide
kubectl -n ailab-infra get svc -l "username=$target_user"
curl -fsS "$config_server_url/pods/$target_user/status"
```

자동으로 다시 만드는 기능이 없으므로 Failed Pod를 발견했다고 기존 Service부터 지우지 않는다.

6.5 Pod를 다른 노드로 옮길 때

`/migrate` 는 같은 resource group의 다른 노드가 충분히 여유 있을 때 새 Pod를 먼저 만드는 API다. 그러나 사용자 image-store 연결과 기존 NodePort 유지가 검증되지 않았으므로 **운영 사용자에게 일반적으로 실행하지 않는다.**

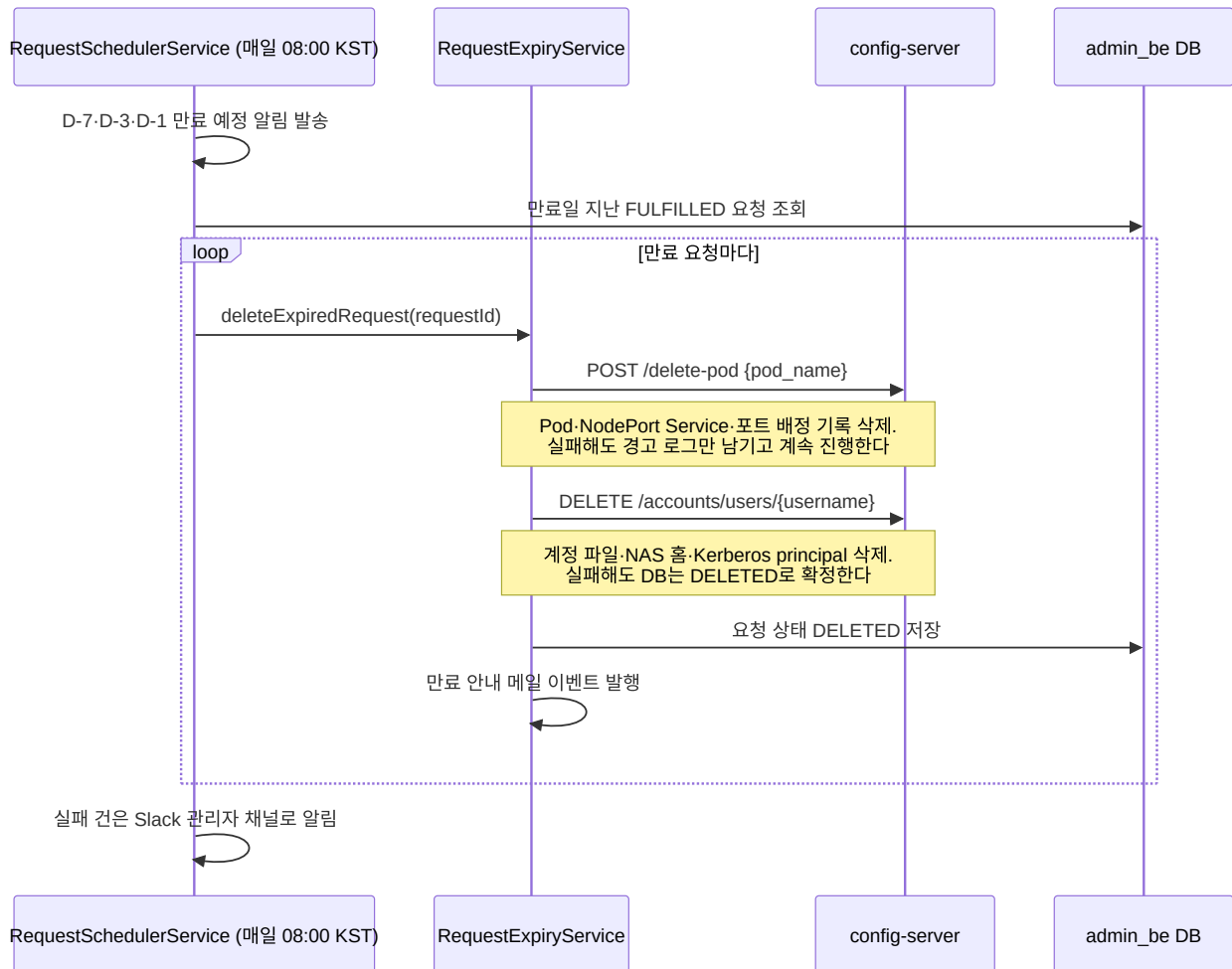
시험 환경에서 확인할 때는 다음을 모두 준비한다.

- 삭제해도 되는 시험 사용자와 재현 가능한 이미지
- 두 개 이상의 정상 GPU 후보 노드
- FARM 홈 백업 또는 시험 데이터
- 현재 Pod·Service·NodePort·선택 노드 기록
- 새 Pod 실패 시 기존 Pod를 유지하는 확인 절차
- 새·이전 노드의 Kerberos 자산 점검

새 Pod가 Ready여도 홈 데이터, 설치 패키지, SSH·Jupyter·noVNC, 포트 연속성, 이전 노드 정리까지 확인해야 한다.

6.6 만료된 사용자 정리

자원 회수는 admin_be의 일일 스케줄러가 자동 수행한다. `RequestSchedulerService` 가 매일 **08:00(KST)** 에 돌면서 만료 예정 알림을 보내고, 만료일이 지난 요청을 `RequestExpiryService` 에 넘겨 config-server API 두 개를 순서대로 호출한다. (사용자 비활성 스케줄러는 별도로 09:00에 돈다.)



만료 처리에서는 다음 두 가지를 확인한다.

1. **삭제할 때는 두 API를 모두 호출한다.** `POST /delete-pod` 는 Pod, Service, 포트만 지우고 `DELETE /accounts/users` 는 계정, 홈, principal만 지운다. 상세 명세는 [API 레퍼런스](#)를 참고한다.
2. **외부 정리가 실패해도 DB는 DELETED로 확정된다.** 인프라에 남은 자원이 있어도 admin_be 화면에는 삭제 완료로 보인다. 만료 처리가 있었던 날에는 8절에서 Pod, Service, 포트 기록을 확인한다. 과거에는 만료 처리에서 계정만 지우고 Pod·NodePort를 남기는 결함 T-KI-01이 있었고 26-07-06에 수정 배포했다. 만료 처리 경로는 아직 실제로 검증하지 않았다.

7. 문제가 생겼을 때

7.1 먼저 확인할 순서

문제가 생기면 API 응답부터 실제 리소스까지 다음 순서로 확인한다.

API HTTP 상태

- └ error stage / error code / rollback
 - └ config-server 로그
 - └ 연결된 서비스 (WAS·MySQL·Redis·Prometheus·NAS·FARM)
 - └ Kubernetes 실제 리소스
 - └ MySQL·FARM 노드 잔존 상태

같은 요청을 재시도하기 전에 부분 생성 여부를 확인한다. 특히 계정 생성과 Pod 생성은 여러 외부 저장소를 순차 갱신하므로, 중간 실패는 "일부만 만들어진" 상태를 남긴다.

7.2 증상별로 먼저 볼 곳

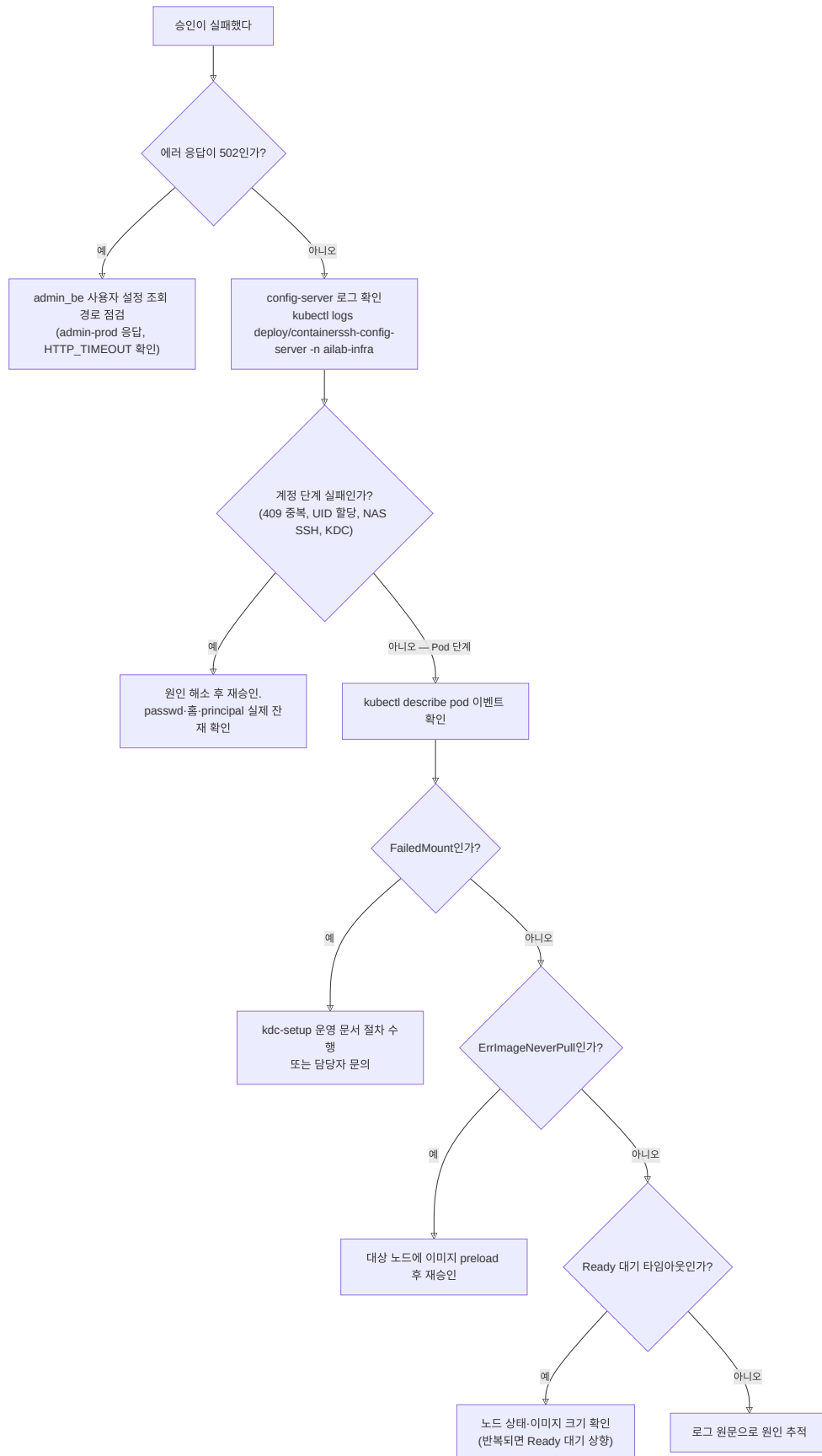
자주 생기는 문제와 먼저 볼 곳을 정리했다. 각 항목의 자세한 절차는 이어지는 절에 있다.

장애	증상	확인	해결
Pod Pending / ErrImageNeverPull	승인 후 Pod가 Pending, Ready 대기 초과로 승인 롤백	<code>kubectl -n ailab-infra describe pod <pod></code> 이벤트, 대상 노드 `cricctl images \	grep decs`
FailedMount (krb5)	<code>mount.nfs: Key has expired</code> 로 신규 Pod 홈 마운트 실패	<code>kubectl -n ailab-infra describe pod <pod></code> 의 FailedMount 이벤트	FARM NAS 홈 마운트에는 Kerberos 인증이 전제된다. kdc-setup 운영 문서 절차를 따르거나 담당자에게 문의한다
계정 생성 실패	승인이 계정 단계에서 실패(409, UID 할당 실패), PENDING 복귀	<code>kubectl -n ailab-infra logs deploy/containersssh-config-server, /kube_share/passwd</code> , NAS SSH 수동 확인	원인(중복 username, NAS SSH, KDC) 해소 후 재승인. 롤백 로그를 맹신하지 말고 passwd·홈·principal의 실제 잔재를 확인한다
NodePort 외부 접속 불가	내부(교내망)는 정상인데 외부 접속만 안 됨	<code>nodeport_allocations</code> 와 Service의 NodePort, pfSense 전달 대상, 사용자에게 안내한 외부 주소를 대조	pfSense 전달 대상이나 안내한 주소를 바로잡고 다시 접속 확인

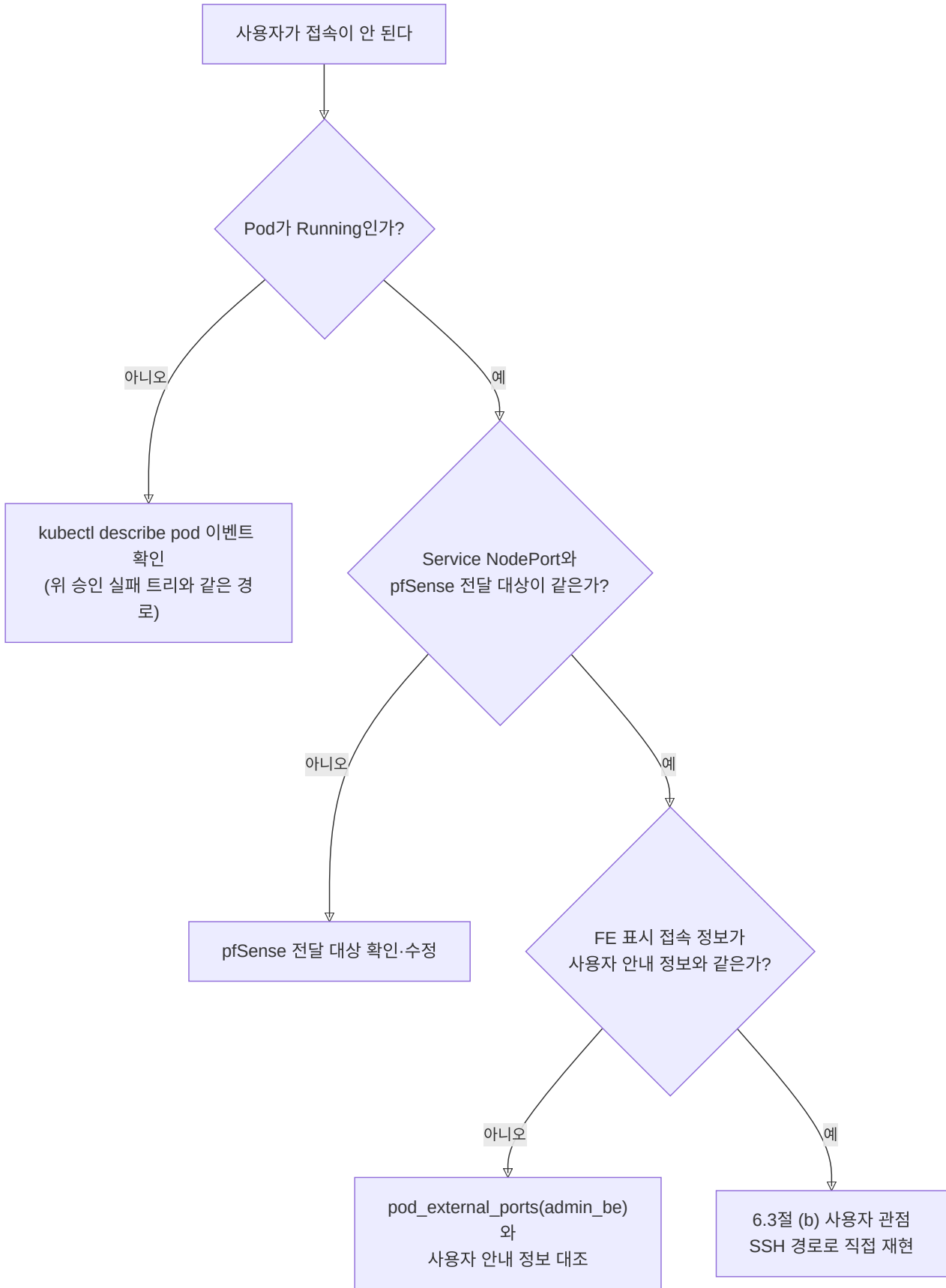
7.3 승인과 접속 문제 확인 순서

승인이 실패했을 때와 사용자가 접속하지 못할 때는 아래 순서로 확인한다.

트리 1 — 승인이 실패했다



트리 2 — 사용자가 접속이 안 된다



7.4 사용자 설정 조회 실패 (**FETCH_USER_CONFIG**)

config-server가 admin_be에서 사용자 설정을 가져오지 못한 경우다. 이때 승인은 되었지만 Pod 생성만 실패할 수 있다.

1. `admin-prod` Deployment와 Service 상태
2. 대상 사용자 승인·설정 존재 여부
3. config-server에서 admin_be Service DNS와 HTTP 응답
4. 응답 JSON의 image, gpu_nodes, additional_ports 형식
5. config-server의 3초 외부 HTTP 제한(`HTTP_TIMEOUT_SEC`) 안에 응답하는지

WAS가 HTTP 200 본문에 404 상태를 넣는 경우도 사용자 없음으로 처리된다.

7.5 노드 선택 실패 (`SELECT_NODE` , `NODE_LIST_FAILED`)

```
kubectl get node
kubectl get node -o custom-columns=NAME:.metadata.name,TAINTS:.spec.taints
kubectl -n ailab-infra logs deploy/containersssh-config-server --since=20m
```

- WAS가 내려준 후보 이름이 실제 Kubernetes 노드와 일치하는지 확인한다.
- `gpu_nodes` 가 비어 있을 때 사용할 Ready 상태의 일반 워커 노드가 있는지 확인한다.
- Prometheus가 각 후보의 GPU 지표를 반환하는지 확인한다. exporter가 죽은 노드는 0점으로 오인 선택될 수 있다(시스템 아키텍처의 GPU 노드 선택 방식).
- 선택된 노드에 FARM 홈과 Kerberos 관리 채널이 준비되어 있는지 확인한다.

7.6 NodePort 할당 또는 Service 생성 실패 (`NODEPORT` , `CREATE_NODEPORT_SERVICE`)

1. 요청 포트가 숫자이며 내부 포트가 중복되지 않는지 확인한다.
2. 30000-32767 범위의 실제 Service 사용 포트를 조회한다.
3. MySQL `nodeport_allocations` 의 같은 Pod·포트 예약을 조회한다(데이터베이스).
4. 오류 응답의 `nodeportsReleased` 와 `servicesDeleted` 를 확인한다.
5. Pod가 남아 있으면 연결된 Service 라벨을 대조한다.

포트 할당 기록과 Service가 다르다는 이유만으로 DB 행이나 Service를 바로 삭제하지 않는다. 사용자 접속 여부와 생성·삭제 로그를 먼저 확인한다.

7.7 Pod Ready 실패

```
kubectl -n ailab-infra describe pod "$pod_name"
kubectl -n ailab-infra logs "$pod_name" --all-containers
kubectl -n ailab-infra get events --field-selector "involvedObject.name=$pod_name"
```

증상별로 볼 곳이 다르다.

증상	우선 확인
<code>ImagePullBackOff</code> , <code>ErrImagePull</code> , <code>ErrImageNeverPull</code>	이미지 이름, 태그, 레지스트리 접근, 노드 캐시 (<code>crictl images</code>)

증상	우선 확인
CreateContainerConfigError	환경 변수, hostPath, Secret, ConfigMap
CrashLoopBackOff	guest image entrypoint, UID/GID, ccache, 셀 설정
Pending	직접 지정된 nodeName, 노드 Ready, hostPath, 디바이스
Ready 시간 초과	이미지 크기, 노드 I/O, 컨테이너 readiness 상태

생성 실패 응답이 Pod와 NodePort를 실제로 정리했는지 반드시 확인한다.

7.8 Kerberos 배포 또는 홈 접근 실패

deploying_krb5 단계에서 실패하면 다음을 확인한다.

1. 사용자 keytab Secret 메타데이터
2. 선택 노드와 FARM 노드 설정의 일치
3. ai lab-krb5 forced-command SSH 채널
4. 선택 노드의 decs-krb-refresh@<user> 로그
5. ccache 소유권과 FARM TGT
6. FARM 홈 마운트와 NAS 소유권

상세 명령과 머신 ccache 안전 규칙은 kdc-setup 운영 문서를 따른다.

7.9 MySQL 또는 Redis 장애

MySQL 장애는 NodePort 예약 실패로 Pod 생성을 중단시킨다. infra-mysql Pod, Service, PVC, 연결 제한과 config-server DB 오류를 확인한다.

Redis 장애는 진행 상태 기록만 누락시킨다 — 생성 자체는 성공할 수 있다. 생성 API와 Kubernetes 리소스가 성공했는데 상태 API가 unknown 이거나 오래된 단계라면 Redis 연결과 TTL을 확인한다. Redis 복구를 위해 정상 사용자 Pod를 삭제하지 않는다.

7.10 삭제가 중간에 멈췄을 때

오류 응답의 progress 와 실제 상태를 비교해 어느 단계까지 처리됐는지 확인한다.

실제 상태	다음 조치
Service만 삭제됨	사용자 접속 중단을 알리고 포트 할당 기록과 Pod 상태를 확인한다
Service·예약 삭제, Pod 남음	같은 Pod 이름으로 삭제 API 재시도 여부를 판단한다
Pod 삭제, 노드 Kerberos 파일 남음	Kerberos 재조정 작업 상태를 확인하고 대상 노드에서 정리한다
계정 삭제, NAS 홈 남음	데이터 보존 여부를 확인한 뒤 NAS 운영 절차로 처리한다

Kerberos 파일 정리가 실패한 경우에는 infra-mysql의 `krb5_cleanup_pending`도 확인한다. 이 테이블에 행이 있으면 30분마다 실행되는 CronJob이 다시 정리한다. CronJob도 실패하면 해당 사용자와 노드의 keytab, timer, ccache를 [kdc-setup 운영 문서](#) 순서대로 확인한다.

수동으로 정리하기 전에는 만료 스케줄러나 krb5 reconcile이 같은 파일을 다시 처리하지 않는지 확인한다.

8. 기록과 실제 상태 비교

MySQL·계정 파일의 기록이 Kubernetes·NAS·AD의 실제 상태와 같은지 정기적으로 확인한다. 만료 처리가 있었던 날에는 반드시 확인한다.

8.1 Pod·Service

```
kubectl -n ailab-infra get pod -l username -o custom-  
columns=NAME:.metadata.name,USER:.metadata.labels.username,NODE:.spec.nodeName,PHASE:.status.phase  
kubectl -n ailab-infra get svc -l app=ailab-nodeport -o custom-  
columns=NAME:.metadata.name,USER:.metadata.labels.username,POD:.metadata.labels.pod_name,NODEPORT:.spec.ports[0].nodePort
```

Pod가 없는 Service, Service가 없는 Running Pod, 같은 Pod에 중복으로 열린 목적 포트를 목록으로 남긴다. **이 단계에서는 지우지 않는다.** 기록과 실제 상태를 비교한 뒤에 처리 방법을 결정한다.

만료가 있었던 날의 간이 확인 명령이다.

```
kubectl get pods -n ailab-infra # 만료 사용자 Pod 잔류 여부  
kubectl get svc -n ailab-infra | grep ailab- # NodePort Service 잔류 여부
```

잔류가 보이면 수동 정리는 `POST /delete-pod` → `DELETE /accounts/users` 순서를 지킨다(6.2절, 5.4절).

8.2 계정·AD·NAS

시험 사용자 또는 문제가 있는 사용자에 대해 다음 값을 한 줄씩 비교한다. 값이 다른 행이 문제를 찾아볼 곳이다.

확인 위치	비교 값
config-server	사용자 이름, UID, GID
FARM AD	sAMAccountName, uidNumber, gidNumber, 전용 그룹
NAS	홈 경로, 숫자 소유권, 모드
선택 노드	ccache 소유 UID, principal
Pod	<code>id</code> , 홈 소유권, ccache 경로

9. 코드나 설정을 바꾼 뒤 확인할 것

코드나 설정을 바꾼 뒤 기존 기능이 깨지지 않았는지 확인한다.

9.1 API 또는 계정 로직

- 기존 응답 코드와 `stage`, `error_code`, `rollback` 형식을 유지한다.
- `passwd`·`group`·`shadow` 쓰기 순서와 실패했을 때 되돌리는 처리를 함께 시험한다.
- NAS, AD, Secret 중간 실패를 각각 시험한다.
- 사용자 이름과 그룹 입력 검증을 우회할 경로가 없는지 확인한다.

9.2 Pod 명세 또는 guest image

- UID/GID와 추가 그룹 환경 변수를 양쪽에서 같은 이름으로 사용한다.
- `/home` 과 `/run/user/<uid>` hostPath를 함께 검증한다.
- SSH 22, Jupyter 8888, noVNC 6080, 추가 포트 회귀 시험을 수행한다.
- GPU 디바이스와 nodeName 변경 뒤 다른 Pod가 같은 GPU 디바이스에 접근하지 않는지 확인한다.

9.3 Helm과 권한

- ServiceAccount의 namespaced 권한과 Node 조회 권한을 최소 범위로 유지한다.
- Secret 값은 values 파일과 로그에 남기지 않는다.
- 배포 후 실제 이미지 태그와 source 버전을 기록한다.

9.4 문제 확인에 쓰는 설정값

문제를 확인할 때 바꿔 보며 원인을 좁힐 수 있는 값들이다. 위치는 `config-server/main.py` 상단 `app.config` (코드 상수와 env), `config-server/Chart/values.yaml` (Helm 배포값) 두 곳이다.

키	위치	역할	디버깅 때 언제 바꾸나
<code>NAMESPACE</code> (<code>ai lab-infra</code>)	main.py 상수	모든 Pod·Service를 만드는 namespace	다른 리소스와 분리한 시험 클러스터에서만
<code>POD_READY_MAX_WAIT_SEC</code> (300초)	main.py	Ready 대기 초과 시 생성 롤백. 배포 브랜치에 따라 값이 다를 수 있음	큰 이미지 첫 기동으로 타임아웃 롤백이 반복될 때 상향
<code>imagePullPolicy</code>	main.py 사용자 Pod 스펙	<code>IfNotPresent</code> — 노드에 이미지가 없을 때만 내려받는다	<code>ErrImagePull</code> 원인을 확인할 때 이미지 이름·태그와 노드의 이미지 목록을 먼저 확인한다
<code>PROM_URL</code>	main.py 상수	노드 선택용 Prometheus 주소	모니터링 스택 주소·포트가 바뀌었을 때
<code>WAS_URL_TEMPLATE</code>	main.py 상수	admin_be 사용자 설정 조회 주소	admin_be 배포 위치(namespace·Service명) 변경 시

키	위치	역할	디버깅 때 언제 바꾸나
<code>HTTP_TIMEOUT_SEC</code> (3.0)	main.py 상수	admin_be·Prometheus 호출 제한 시간	느린 응답으로 502가 날 때 원인을 확인할 때
<code>BASE_ETC_DIR</code> (<code>/kube_share</code>)	main.py 상수	계정 파일(passwd 등) 루트 경로	운영에서는 바꾸지 않는다. 시험 환경에서만 변경한다.
<code>NFS_SERVER</code> / <code>NFS_USER_SHARE_PATH</code>	env ← values.yaml <code>nfs.*</code>	사용자 홈 NFS 마운트 원천	NAS 이전·export 경로 변경 시
<code>KRB5_REALM</code>	env ← values.yaml <code>krb5.*</code>	비우면 Kerberos 설정을 만들지 않는다	KDC 설정을 관리하는 사람과 확인한 뒤 변경
<code>db.*</code> 블록	values.yaml	NodePort 배정 DB(infra-mysql) 접속점	포트 배정 실패 시 접속 대상이 맞는지 확인
<code>redis.*</code> 블록	values.yaml	사용자 이미지 저장/로드 상태 Redis	이미지 상태 조회 실패 시 접속 대상 확인
<code>namespace</code> / <code>nodeSelector</code>	values.yaml	배포 namespace·config-server 배치 노드 고정	고정 노드 점검으로 임시 이전 시 (tolerations 짝 확인)

변경 방법은 재배포 1회이다. values.yaml 값은 `helm upgrade` 로 반영하고, main.py 상수는 이미지 재빌드가 필요하므로 CI/CD(main 머지) 경로를 쓴다.

```
helm upgrade --install containerssh-config-server ./Chart -n ailab-infra
```

10. 하지 않는 작업

- shadow, keytab, SSH 개인키, DB 비밀번호를 화면·로그·티켓·일반 운영 문서에 붙이지 않는다. 관리자 전용 문서 외에는 원문을 남기지 않는다.
- 사용자 확인 없이 Running Pod와 홈을 삭제하지 않는다.
- MySQL NodePort 행만 지우거나 Service만 지우지 않는다. 기록과 실제 리소스가 더 어긋날 수 있다.
- 대체된 ContainerSSH·사용자 PVC chart를 현재 복구 경로로 재배포하지 않는다.
- 레거시 **restart 스크립트**(`restart_auth.sh`·`restart_containerssh.sh`·`restart_pvc.sh`)를 운영 배포에 사용하지 않는다. 전부 레거시 namespace(`containerssh`·`cssh`)를 향하던 것으로 레거시 차트 정리와 함께 레포에서 제거됐다. 과거 체크아웃에 남아 있어도 쓰지 않는다. 운영 서비스는 전부 `ailab-infra`에 있으며, config-server 재배포는 4.1절 CI/CD 또는 `config-server/Makefile` (`make deploy`) 경로를 쓴다.
- 이미지 저장·마이그레이션 경로를 검증 없이 운영 사용자에게 실행하지 않는다(6.5절).
- D-state(디스크 I/O에 걸려 죽지도 살지도 못하는 프로세스 상태)가 있는 FARM 노드에서 NFS 복구를 반복하지 않는다.

11. 참고 문서

- 계정·Pod·포트가 만들어지는 방식 — 시스템 아키텍처
- 배경 용어 — 기초 개념
- API 전체 명세 — API 레퍼런스
- 배포값·Secret 이름·차트 구조 — Helm 차트 레퍼런스
- NodePort 배정 테이블 — 데이터베이스
- FARM AD·Kerberos·NFS 장애 — kdc-setup 운영 문서

API 레퍼런스

이 문서에서 다루는 내용

- config-server HTTP API의 엔드포인트, 호출 주체, 입력, 응답
- 어떤 API가 실제 admin_be 에서 호출되고 어떤 API는 수동 확인용인지
- Pod·계정 생성과 삭제 요청에서 주의할 오류 형식

빠른 이동

필요할 때	문서
승인 뒤 내부 처리 순서를 먼저 본다	시스템 아키텍처
운영 중 오류 응답을 해석한다	운영 매뉴얼
Kerberos 준비 실패를 본다	kdc-setup 운영

config-server의 HTTP API 명세다. main.py 의 Flask route와 /accounts blueprint를 합쳐 총 12개 엔드포인트를 제공한다.

현재 배포된 API 목록은 Swagger UI http://210.94.179.18:30082/apidocs/ 에서도 확인할 수 있다.

1. 공통 사항

항목	값
Base URL (운영)	http://210.94.179.18:30082 (namespace ailab-infra , Service containersssh-config-service)

항목	값
요청/응답 형식	JSON (<code>Content-Type: application/json</code>)
인증	없음 — 승인된 운영망 밖으로 노출하지 않는다. 현재 Helm 차트에는 NetworkPolicy가 없으므로 클러스터의 별도 접근 제어가 있는지 확인한다.
에러 응답 형태	Pod 계열은 <code>infra_error()</code> 포맷(<code>step</code> , <code>error</code> , <code>detail</code> , <code>progress</code> 등), 계정 계열은 <code>{"error": "..."} </code> 단순 포맷

호출 주체는 `admin_be`(Spring Boot WAS)의 서비스 클래스이다. `admin_be` 실코드 기준으로 확인한 매핑이다.

admin_be 서비스	호출 엔드포인트	시점
<code>AdminRequestCommandService</code>	<code>PUT /accounts/users</code>	관리자 승인 시
<code>PodService</code>	<code>POST /create-pod</code> , <code>POST /delete-pod</code>	승인 시 / 만료 또는 실패를 되돌리는 삭제 시
<code>UbuntuAccountService</code>	<code>DELETE /accounts/users/{username}</code>	만료 또는 실패를 되돌리는 삭제 시
<code>GroupService</code>	<code>PUT /accounts/groups</code>	그룹 생성 시
(호출처 없음 — 미확인)	<code>GET /pods/{username}/status</code> , <code>POST /migrate</code> , <code>GET /accounts/*</code> , <code>DELETE /accounts/groups</code> , <code>PUT /accounts/users/{u}/groups</code>	<code>admin_be</code> 코드에서 호출처가 발견되지 않았다. 운영자 수동 호출·실험용으로 보인다

2. GET /health

항목	값
메서드·경로	<code>GET /health</code>
호출 주체	헬스체크(서버 생존 확인). <code>admin_be</code> 서비스 호출은 없다

입력. 없다.

처리 순서. 즉시 응답한다. 외부 의존성(DB, Kubernetes)을 검사하지 않으므로, 200이 와도 DB 연결 실패 상태일 수 있다.

성공 응답. 200 본문 `"OK"` (문자열).

대표 실패. 없다(무조건 200).

3. POST /create-pod

항목	값
메서드·경로	POST /create-pod
호출 주체	admin_be PodService (승인 트랜잭션의 마지막 단계)

입력.

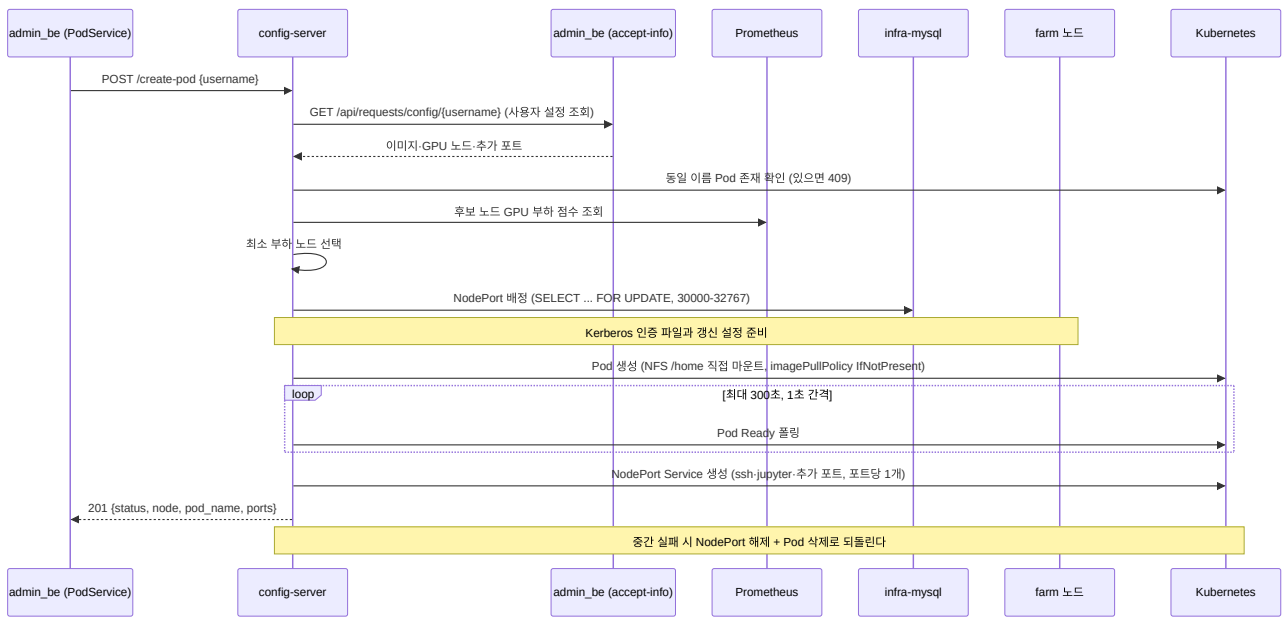
```
{"username": "user2100"}
```

`username` 하나만 받는다. 이미지, GPU 노드, 추가 포트는 config-server가 admin_be에 다시 요청해 가져온다.

처리 순서.

1. `username` 없으면 400을 반환한다.
2. admin_be에 GET `http://admin-prod.default/api/requests/config/{username}` 을 요청해 사용자 설정(이미지, GPU 노드 목록, 추가 포트)을 받아온다. 통신 실패나 JSON 파싱 실패는 502, 사용자가 없으면 404이다.
3. Pod 이름을 `ai lab-<username>-<랜덤>` 형식으로 생성한다.
4. 같은 이름의 Pod가 이미 있으면 409를 반환한다. 같은 username이라도 이름이 다른 Pod는 막지 않으므로, 사용자 단위 중복 호출은 admin_be가 막아야 한다.
5. 후보 노드 목록을 만든다. admin_be가 준 `gpu_nodes` 를 사용하고, 비어 있으면 Ready 상태의 워커 노드 전체를 사용한다.
6. Prometheus(서버 지표를 수집하는 모니터링 시스템)에 각 후보 노드의 GPU 부하를 물어 **가장 한가한 노드**를 고른다.
7. Pod 스펙을 조립한다(`build_pod_spec`). 이 단계 안에서 아래 일이 함께 일어난다.
 - `/kube_share` 계정 파일 확인 — **passwd에 사용자가 없으면 실패**한다(계정 생성이 선행 조건).
 - 저장된 사용자 이미지(`/image-store/images/user-<username>.tar`)가 있으면 로드하고, 없으면 WAS가 준 base 이미지를 쓴다.
 - 기본 포트 22(ssh), 8888(jupyter), 추가 포트에 NodePort를 배정한다. DB에서 30000~32767 중 빈 번호를 고르고 `SELECT ... FOR UPDATE` 로 동시에 같은 번호를 고르지 못하게 한다. 클러스터 전체 Service 조회에 실패하면 DB 기록만으로 배정하므로, 이미 다른 Service가 쓰는 포트를 고르면 뒤의 Service 생성이 실패할 수 있다.
 - 추가 포트에 내부 포트 6080 이 있거나 용도가 `novnc` , `vnc` 이면 Pod 환경 변수에 `ENABLE_VNC=true` 를 넣는다.
 - Kerberos 설정이 켜져 있으면 선택된 FARM 노드에 인증 파일과 갱신 설정을 준비한다. 실패하면 여기서 중단된다. 점검과 복구는 [kdc-setup 운영 문서](#)를 따른다.
 - NFS user-share를 `/home` 에 직접 마운트하도록 볼륨을 구성한다. 사용자 Pod의 `imagePullPolicy` 는 `IfNotPresent` 이므로 노드에 이미지가 없을 때만 레지스트리에서 가져온다.
1. Kubernetes에 Pod를 생성한다.
2. 최대 300초 동안 1초 간격으로 Pod가 Ready 상태가 되기를 기다린다.
3. Ready가 되면 포트마다 NodePort Service(`ai lab-<username>-<용도>-<외부포트>`)를 생성한다.

4. 성공 응답을 반환한다. 8~10단계 어디서든 실패하면 배정한 NodePort 해제 + 만든 Pod 삭제(+Service 삭제)로, 그때까지 만든 것을 되돌린다.



성공 응답. 201

```

{
  "status": "created",
  "node": "farm5",
  "pod_name": "ailab-user2100-1a2b3c4d",
  "ports": [
    {"internal_port": 22, "external_port": 30000, "usage_purpose": "ssh"},
    {"internal_port": 8888, "external_port": 30001, "usage_purpose": "jupyter"}
  ]
}
  
```

대표 실패.

상 태	의미
400	<code>username</code> 누락, 또는 스펙 조립 단계의 검증 실패(알 수 없는 노드, passwd에 사용자 없음 등)
404	WAS에 해당 사용자가 없음 (<code>USER_CONFIG_NOT_FOUND</code>)
409	동일 이름 Pod가 이미 존재 (<code>POD_ALREADY_EXISTS</code>)
502	admin_be 사용자 설정 조회 실패 — 통신 오류·비정상 응답 (<code>USER_CONFIG_FETCH_FAILED</code>)
500	노드 선택·NodePort 배정·Kerberos 준비·Pod 생성·Ready 대기·Service 생성 실패. 응답의 <code>rollback</code> 필드로 어디까지 되돌렸는지 알 수 있다

4. GET /pods/<username>/status

항목	값
메서드·경로	GET /pods/<username>/status
호출 주체	생성 진행 상황을 조회하는 운영자·클라이언트용 API. admin_be 호출처는 미확인

POST /create-pod 는 Pod 준비가 끝날 때까지 최대 300초 동안 기다리는 동기 API이다. 이 API는 생성 요청이 끝나기 전에도 사용자별 진행 상황을 가볍게 조회한다.

입력. 경로의 `username`.

성공 응답. 200

```
{
  "username": "user2100",
  "stage": "waiting_ready",
  "message": "이미지 pull / 컨테이너 기동 대기 중",
  "updated_at": "2026-07-22T01:23:45+00:00"
}
```

stage 는 `unknown`, `started`, `selecting_node`, `building_pod_spec`, `allocating_nodeport`, `deploying_krb5`, `creating_pod`, `waiting_ready`, `creating_services`, `ready`, `failed` 중 하나이다. 생성 이력이 없으면 stage 는 `unknown` 이고 `updated_at` 은 없다. `failed` 의 `message` 에는 보안을 위해 상세 예외나 Kubernetes 오류 원문을 넣지 않으므로, 원인은 config-server 로그에서 확인한다.

대표 실패. 상태 저장소 조회에 실패하면 500 (`POD_STATUS_LOOKUP_FAILED`)을 반환한다.

5. POST /delete-pod

항목	값
메서드·경로	POST /delete-pod
호출 주체	admin_be PodService (만료 스케줄러, 실패를 되돌리는 삭제)

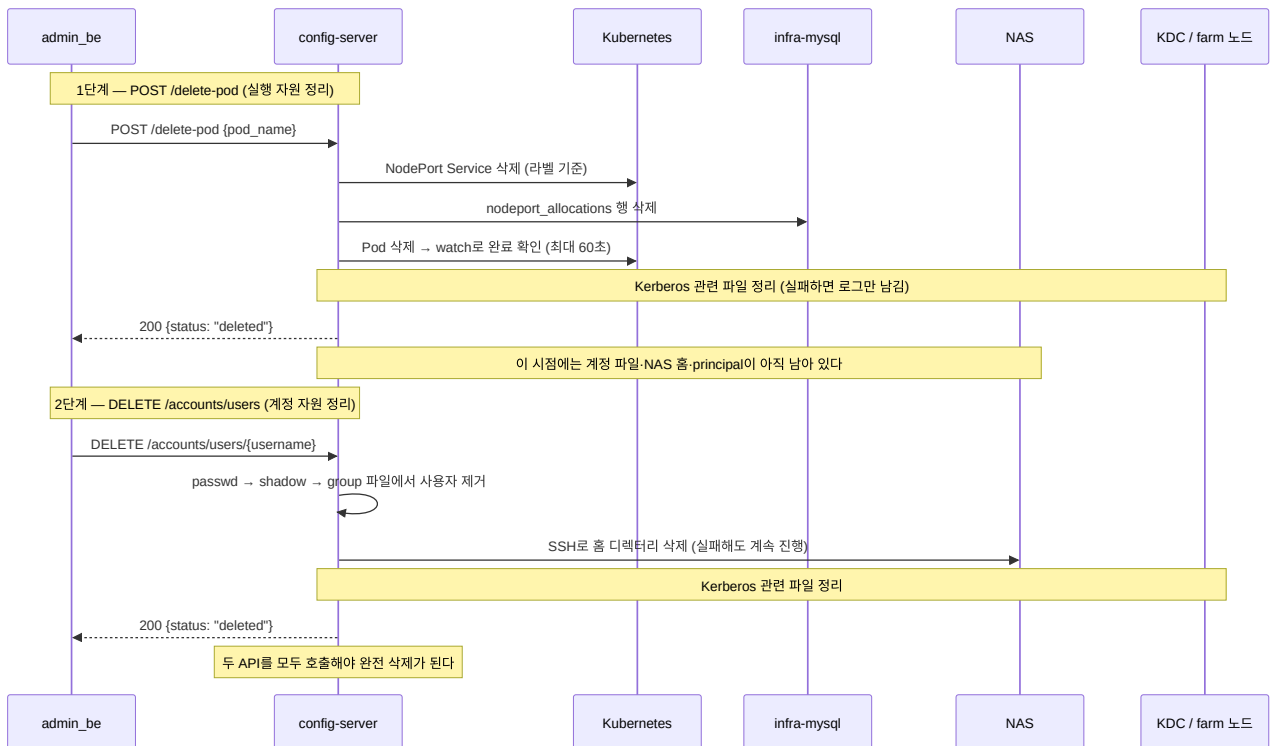
입력.

```
{"pod_name": "ai lab-user2100-1a2b3c4d"}
```

처리 순서.

1. `pod_name` 없으면 400, `ai lab-` prefix가 아니면 400(`INVALID_POD_NAME`)을 반환한다. 이름에서 username을 파싱한다.
2. 해당 Pod의 NodePort Service들을 라벨(`app=ai lab-nodeport, pod_name=<pod>`) 기준으로 삭제한다.
3. `nodeport_allocations` 테이블에서 해당 Pod의 포트 배정 기록 행을 삭제한다.
4. Kubernetes Pod를 삭제하고, watch(쿠버네티스 이벤트 스트림 구독)로 **실제 삭제 완료**를 최대 60초 기다린다. Pod가 원래 없었으면(404) `already_absent: true` 로 200을 반환한다.
5. Kerberos 관련 파일을 정리한다. 이 정리가 실패해도 Pod 삭제 결과는 실패로 바꾸지 않고 로그만 남긴다.

이 API는 Pod, Service, NodePort DB 행만 정리한다. 계정 파일(passwd 등), NAS 홈, Kerberos principal은 그대로 남고, 아래 10절의 `DELETE /accounts/users` 가 정리한다. 두 API 중 하나만 호출하면 Pod 관련 자원이나 계정 관련 파일이 남는다.



성공 응답. 200

```

{
  "status": "deleted",
  "pod_name": "ai lab-user2100-1a2b3c4d",
  "progress": {
    "servicesDeleted": true,
    "nodeportsReleased": true,
    "podDeleteRequested": true,
    "podDeleted": true
  }
}
  
```

`progress` 는 롤백 결과가 아니라 **각 단계의 완료 여부**이다. Pod가 원래 없던 경우 `already_absent: true` 가 추가된다.

대표 실패.

상태	의미
400	<code>pod_name</code> 누락, 또는 <code>ai lab-</code> prefix가 아닌 이름
500	Service 삭제·NodePort 해제·Pod 삭제 실패, 또는 60초 내 삭제 미완료(<code>POD_DELETE_TIMEOUT</code>)

6. POST /migrate

항목	값
메서드·경로	<code>POST /migrate</code>
호출 주체	admin_be 호출처 없음 — 운영자 수동·실험용 (미확인)

입력.

```
{
  "username": "user2100",
  "nodes": ["farm5", "farm6"],
  "min_improvement_ratio": 0.2
}
```

`username` 과 `nodes` (후보 노드 목록)가 필수이고, `min_improvement_ratio` (이만큼은 좋아져야 옮긴다는 개선 문턱, 기본 0.2)는 선택이다.

처리 순서.

- 사용자별 락 파일(`/tmp/migrate-<username>.lock`)을 잡아 같은 사용자의 동시 마이그레이션을 막는다.
- 후보 노드 이름을 실제 클러스터 노드 이름으로 정규화한다. 모르는 노드가 있으면 400이다.
- 실행 중인 사용자 Pod를 찾는다. 없으면 404이다. 현재 노드가 후보 목록에 없으면 400이다.
- 현재 노드를 뺀 후보가 없으면 `skipped` (`no_candidate_node`)로 200을 반환한다.
- Prometheus GPU 점수를 현재 노드와 후보들에 대해 계산한다. 최고 후보가 `min_improvement_ratio` 만큼 충분히 좋지 않으면 `skipped` (`no_significant_improvement`)로 200을 반환한다.
- WAS에서 사용자 설정을 다시 조회한다.
- 기존 Pod 안에서 이미지 commit/save를 실행해 사용자 상태를 image-store에 저장한다. 실패하면 500이다.
- 새 Pod 이름을 만들고, 새 노드에 Pod를 생성한 뒤 Ready를 최대 60초 기다리고 NodePort Service를 만든다. 실패하면 새 Pod와 새로 배정한 포트를 정리하고 500이다.
- 새 Pod가 완전히 성공한 뒤에야 기존 Pod의 Service·포트 배정·Pod를 삭제한다.

성공 응답. `200`

```
{
  "status": "migrated",
  "from": "farm5",
  "to": "farm6",
  "new_pod": "ailab-user2100-9z8y7x6w",
  "ports": [{"internal_port": 22, "external_port": 30005, "usage_purpose": "ssh"}]
}
```

건너뛴 경우에도 200이며 `{"status": "skipped", "reason": ...}` 형태이다.

대표 실패.

상태	의미
400	<code>username</code> / <code>nodes</code> 누락, 알 수 없는 노드, 현재 노드가 후보 목록에 없음
404	실행 중인 Pod 없음
500	이미지 commit 실패(<code>image_commit_failed</code>), 새 Pod 기동 실패, Service 생성 실패

7. GET /accounts/users

항목	값
메서드 · 경로	<code>GET /accounts/users</code>
호출 주체	admin_be 호출처 없음 — 운영자 점검용 (미확인)

입력. 없다.

처리 순서. NFS 계정 파일 `/kube_share/passwd` (리눅스 계정 목록 파일)를 읽어 전체 사용자를 반환한다.

성공 응답. 200

```
{
  "users": [
    {
      "name": "user2100",
      "uid": 20001,
      "gid": 20001,
      "gecos": "GPU User",
      "home": "/home/user2100",
      "shell": "/bin/bash"
    }
  ]
}
```

대표 실패. 파일 읽기 예외 시 500(`{"error": ...}`).

8. GET /accounts/users/<username>

항목	값
메서드·경로	GET /accounts/users/<username>
호출 주체	admin_be 호출처 없음 — 운영자 점검용 (미확인)

입력. 경로의 `username`.

처리 순서.

- passwd에서 사용자를 찾는다. 없으면 404이다.
- group 파일을 읽어 기본 그룹과 추가 그룹을 함께 반환한다.

성공 응답. 200

```
{
  "user": {"name": "user2100", "uid": 20001, "gid": 20001, "gecos": "", "home": "/home/user2100", "shell":
"/bin/bash"},
  "groups": [
    {"name": "user2100", "gid": 20001, "type": "primary"},
    {"name": "developers", "gid": 20005, "type": "supplementary"}
  ]
}
```

대표 실패.

상태	의미
404	사용자 없음
500	파일 읽기 예외

9. PUT /accounts/users

항목	값
메서드·경로	PUT /accounts/users
호출 주체	admin_be AdminRequestCommandService (승인 시)

입력.

```
{
  "name": "user2100",
  "passwd_base64": "cGFzc3dvcmQ=",
  "gecos": "GPU User",
  "primary_group_name": "user2100",
  "supplementary_groups": [{"name": "ailab", "gid": 20005}]
}
```

필수는 `name`, `passwd_base64` (Base64로 감싼 평문 비밀번호)이다. 나머지는 선택이다.

이전 문서에는 `uid`, `gid`, `passwd_sha512` 가 필수라고 적혀 있지만 현재 코드와 다르다. UID/GID는 서버가 빈 번호를 골라 정하고, 비밀번호 해시(SHA-512 crypt)도 서버가 만든다.

처리 순서.

1. 필수 필드와 `supplementary_groups` 형식(`{name, gid}` 목록)을 검증한다. `passwd_base64` 디코딩에 실패하면 400이다.
2. `/kube_share` 계정 파일 구조가 비어 있으면 `base_etc/` 템플릿으로 초기화한다.
3. `passwd` 파일을 잠가 다른 요청이 동시에 같은 UID를 고르지 못하게 한다. NFS 파일에는 잠금을 걸지 않고 로컬 `/tmp` 잠금 파일을 쓴다. 사용자가 이미 있으면 409를 반환한다. 없으면 20000 이상이고 `/home/` 을 쓰는 사용자 중 가장 큰 UID 다음 번호를 찾아 `passwd`에 추가한다. GID는 UID와 같은 값이다.
4. `group` 파일에 기본 그룹을 만들고, 요청에 든 추가 그룹에는 사용자를 구성원으로 넣는다. 실패하면 `passwd`에 쓴 내용도 되돌리고 500을 반환한다.
5. `shadow`(암호 해시 파일)에 SHA-512 crypt 해시를 기록한다. 실패하면 롤백 후 500이다.
6. `SUDO_ALLOWED_COMMANDS` 설정이 있으면 사용자별 `sudoers`(sudo 사용 권한을 정의하는 파일)를 0440 권한으로 만든다.
7. NAS(`192.168.2.30:6954`)에 SSH로 접속해 홈 디렉터리를 만든다(`mkdir`, `chown`, `chmod 700`). NFS의 `root_squash` 설정 때문에 Pod는 홈 디렉터리를 직접 만들 수 없다. 실패하면 계정 파일을 되돌리고 500을 반환한다.
8. Kerberos 설정이 켜져 있으면 사용자 `principal`과 `keytab Secret`을 만든다. 실패하면 홈 디렉터리를 지우고 계정 파일을 되돌린 뒤 500을 반환한다. 점검과 복구는 [kdc-setup 운영 문서](#)를 따른다.

성공 응답. 201

```
{
  "status": "created",
  "user": {"name": "user2100", "uid": 20001, "gid": 20001, "home": "/home/user2100", "shell": "/bin/bash"},
  "group": {"name": "user2100", "gid": 20001},
  "supplementary_groups": [{"name": "ailab", "gid": 20005}],
  "sudoers": "/kube_share/sudoers.d/user2100"
}
```

대표 실패.

상 태	의미
400	필수 필드 누락, <code>passwd_base64</code> 디코딩 실패, <code>supplementary_groups</code> 형식 오류
409	사용자 이미 존재
500	<code>group/shadow/sudoers</code> 기록 실패, NAS SSH 실패(<code>NAS_SSH_FAILED</code>), KDC 실패(<code>KDC_FAILED</code>) — 모두 계정 파일을 되돌린 뒤 반환한다

10. DELETE /accounts/users/<username>

항목	값
메서드 · 경로	<code>DELETE /accounts/users/<username></code>
호출 주체	<code>admin_be UbuntuAccountService</code> (만료, 실패를 되돌리는 삭제)

입력. 경로의 `username`.

처리 순서.

1. `passwd`에서 사용자를 제거한다. 없으면 404이다.
2. `shadow`에서 해당 행을 제거한다.
3. `group` 파일을 정리한다 — 모든 그룹의 멤버 목록에서 사용자를 빼고, 이 사용자가 쓰던 그룹(명시적 멤버였거나 primary GID 그룹)이 비게 되면 그룹 자체를 삭제한다.
4. NAS 홈 디렉터리를 SSH로 삭제한다. **실패해도 경고 로그만 남기고 계속 진행한다**(계정 파일은 이미 지워진 상태).
5. Kerberos 관련 파일을 정리한다. 이 단계가 실패하면 오류를 기록하고 다음 작업을 계속한다.

`POST /delete-pod`도 함께 호출해야 Pod와 계정이 모두 정리된다(5절의 시퀀스 다이어그램 참조). 이 API만 호출하면 Pod와 NodePort가 남고, `delete-pod`만 호출하면 계정, 홈, principal이 남는다.

성공 응답. `200` — `{"status": "deleted", "user": "user2100"}`

대표 실패.

상태	의미
404	사용자 없음
500	Kerberos 정리 등 처리 중 예외 (NAS 홈 삭제 실패는 500이 아니라 경고 후 진행)

11. PUT /accounts/groups

항목	값
메서드·경로	PUT /accounts/groups
호출 주체	admin_be GroupService

입력.

```
{"name": "developers", "gid": 20005, "members": ["user2100", "user2101"]}
```

필수는 **name** 이다. **gid** 는 생략하면 group 파일 기준으로 빈 번호를 골라 자동 배정(20000 이상)하고, **members** 는 생략 가능하다.

처리 순서.

- gid** 타입을 검증한다(정수 또는 정수 문자열만 허용, 아니면 400).
- members** 에 적힌 사용자가 passwd에 전부 존재하는지 확인한다. 없는 사용자가 있으면 400이다.
- group 파일에 락을 걸고 — 같은 이름이 있으면 409, 지정한 gid가 이미 쓰이면 409, 아니면 새 그룹 행을 추가한다.

성공 응답. 201 — {"group": {"name": "developers", "gid": 20005}}

대표 실패.

상태	의미
400	name 누락, gid 타입 오류, 존재하지 않는 멤버
409	그룹 이름 또는 gid 중복

12. DELETE /accounts/groups/<groupname>

항목	값
메서드·경로	DELETE /accounts/groups/<groupname>
호출 주체	admin_be 호출처 없음 (미확인)

입력. 경로의 **groupname** .

처리 순서.

- group 파일에서 그룹을 찾는다. 없으면 404이다.

- 이 그룹을 기본 그룹으로 쓰는 사용자가 passwd에 있으면 삭제를 거부하고 400을 반환한다. 오류 응답에는 해당 사용자 이름이 들어간다.
- 그룹 행을 삭제한다.

성공 응답. 200 — {"status": "deleted", "group": "developers", "gid": 20005}

대표 실패.

상태	의미
400	기본 그룹으로 사용 중
404	그룹 없음

13. PUT /accounts/users/<username>/groups

항목	값
메서드·경로	PUT /accounts/users/<username>/groups
호출 주체	admin_be 호출처 없음 (미확인)

입력.

```
{"groups": ["developers", "ai-lab"]}
```

처리 순서.

- groups 목록이 비어 있으면 400이다.
- 사용자가 passwd에 없으면 404이다.
- 지정한 그룹들의 멤버 목록에 사용자를 추가한다(이미 있으면 그대로 둔다).
- 요청한 그룹 중 존재하지 않는 것이 있으면 404를 반환한다.

성공 응답. 200 — {"status": "updated", "user": "user2100", "groups": ["ai-lab", "developers"]}

대표 실패.

상태	의미
400	groups 누락
404	사용자 없음, 또는 존재하지 않는 그룹 포함

14. 호출할 때 확인할 점

- 위 12개 API는 요청자를 확인하지 않는다. 승인된 운영망 밖에서 호출할 수 없게 해야 한다. 현재 Helm 차트는 NetworkPolicy를 만들지 않으므로, 네트워크 접근 제어는 클러스터 설정에서 따로 확인한다.
- `create-pod`를 같은 사용자에게 여러 번 호출하면 Pod와 Service가 여러 개 만들어질 수 있다. 사용자 단위 중복 호출은 admin_be가 막아야 한다.
- `POST /delete-pod` (Pod, Service, 포트)와 `DELETE /accounts/users` (계정, 홈, principal)를 모두 호출해야 관련 자원이 남지 않는다.
- 8888 포트도 다른 포트처럼 NodePort Service로 열린다. 게스트 이미지의 Jupyter에 인증이 없으면 외부에서 접속할 수 있으므로 내부망에서만 접근하게 해야 한다.
- 만료 스케줄러가 이 API들을 어떤 순서로 부르는지는 [운영 가이드](#)의 만료 흐름 절을 참고한다.

Helm 차트 레퍼런스

이 문서에서 다루는 내용

- config-server Helm 차트가 어떤 리소스를 만드는지
- `values.yaml`과 템플릿이 Deployment, Service, RBAC, CronJob에 어떻게 반영되는지
- 배포 값을 바꾸기 전에 어디를 확인해야 하는지

빠른 이동

필요할 때	문서
Kubernetes 기본 용어를 먼저 본다	기초 개념
배포와 장애 대응 순서를 본다	운영 매뉴얼
Kerberos 설정 값의 위치를 본다	kdc-setup 설정

쿠버네티스 기본 용어(Pod, Service, PVC, namespace 등)는 [기초 개념](#)을 참고한다.

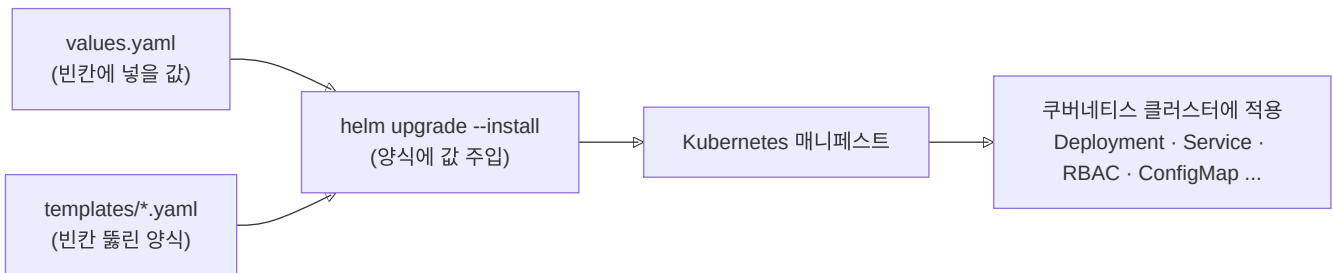
0. Helm 기본 개념

0.1 왜 Helm을 쓰는가

쿠버네티스에 무언가를 배포하려면 YAML 파일(매니페스트)을 여러 장 써야 한다. config-server 하나만 해도 Deployment, Service, 권한(RBAC), 설정(ConfigMap) 등 파일이 많다. 문제는 이 파일들 곳곳에 **배포 환경마다 달라지는 값**(이미지 태그, NFS 서버 주소, 포트 번호)이 박혀 있다는 점이다. 값이 바뀔 때마다 여러 파일을 뒤져 고치는 것은

실수하기 좋다.

- **템플릿(templates/)** — 매니페스트의 구조를 적어 둔 양식이다. 달라지는 자리는 `{{ .Values.image.tag }}` 처럼 빈칸으로 뚫어 둔다.
- **values.yaml** — 그 빈칸에 채워 넣을 값들의 모음이다. "이번 배포의 설정 전체"가 이 파일 한 장에 모인다.
- **차트(Chart)** — 템플릿 묶음 + values.yaml + 메타데이터(Chart.yaml)를 합친 패키지이다.
- **릴리스(release)** — 차트를 클러스터에 설치한 결과물의 이름이다. 같은 차트를 이름만 다르게 여러 번 설치할 수도 있다. 이 시스템의 릴리스 이름은 `containerssh-config-server` 이다.



배포 명령(`helm upgrade`)을 실행하면 Helm이 값을 양식에 넣어 Kubernetes 매니페스트를 만들고 클러스터에 적용한다. 예를 들어 NFS 서버 주소가 바뀌면 템플릿을 고치지 않고 values.yaml의 값을 바꾸면 된다.

0.2 자주 쓰는 명령

```
# 배포(설치+업데이트 겸용) — ./Chart 디렉터리의 차트를 릴리스에 반영한다
helm upgrade --install containerssh-config-server ./Chart -n ailab-infra

# 새 Pod가 정상적으로 떴는지 확인한다
kubectl rollout status deploy/containerssh-config-server -n ailab-infra

# 잘못됐으면 직전 배포 상태로 되돌린다 (Helm이 릴리스 이력을 기억한다)
helm rollback containerssh-config-server -n ailab-infra
```

`upgrade --install` 은 "릴리스가 없으면 설치하고, 있으면 갱신하라"는 겸용 명령이라 설치와 업데이트를 구분해 외울 필요가 없다. `rollback` 이 가능한 이유는 Helm이 릴리스마다 배포 이력을 버전으로 쌓아 두기 때문이다.

0.3 --set 으로 바꾼 값

`helm upgrade ... --set image.tag=abc123` 처럼 명령어 뒤에 값을 직접 붙이면 values.yaml을 고치지 않고 그 배포 한 번에만 값을 바꿀 수 있다. 다음 배포 때 values.yaml의 값으로 돌아가므로, 계속 유지할 변경은 values.yaml이나 별도 운영 values 파일에 기록한다.

1. Chart/ — config-server 차트 (현행)

config-server 배포용 Helm 차트이다. 위치는 저장소의 `config-server/Chart/` 이다.

1.1 이 차트가 클러스터에 만드는 것

리소스	이름	왜 필요한가
Deployment	containerssh-config-server	config-server 컨테이너 1개(replicas: 1)를 실행한다. 15초 뒤부터 10초마다 /health 를 확인해 Ready 상태를 판단한다.
Service (NodePort)	containerssh-config-service	config-server의 고정 접속 주소이다. 내부 80 → 컨테이너 8000(gunicorn), 외부 NodePort 30082
ServiceAccount	config-server	config-server가 Kubernetes API를 호출할 때 사용하는 계정이다.
Role + RoleBinding	config-server-role / config-server-rolebinding	ailab-infra namespace 안에서 Pod(exec-log 하위 리소스 포함)·Service·PVC·Secret을 만들고 지을 권한을 위 ServiceAccount에 붙인다. 사용자 Pod 생성·삭제뿐 아니라 Pod 안에서 명령 실행(exec)·로그 조회까지 이 권한으로 이뤄진다
ClusterRole + Binding	...-node-reader	노드 목록 조회 권한이다. 노드는 namespace에 속하지 않는(cluster-scoped) 리소스라서 Role로는 못 주고 ClusterRole이 따로 필요하다
ConfigMap	krb5-conf	Kerberos 클라이언트 설정(krb5.conf)을 담아 config-server Pod에 주입한다
CronJob	...-krb5-reconcile	30분마다 reconcile_krb5.py 를 실행한다. krb5_cleanup_pending 에 남은 실패 건을 다시 정리하고, FARM 노드의 keytab 목록과 nodeport_allocations 를 비교해 Pod가 없는 사용자의 keytab도 지운다.

1.2 파일 구성

파일	역할
Chart.yaml	차트 이름(containerssh-config-server)·버전 메타데이터이다
values.yaml	배포마다 달라지는 설정값 모음이다(1.3절)
templates/deployment.yaml	Deployment 양식이다 — 이미지·환경변수·마운트가 여기서 조립된다. values의 값 대부분이 컨테이너 환경변수로 주입된다
templates/service.yaml	NodePort Service 양식이다
templates/rbac.yaml	Role/RoleBinding + ClusterRole/ClusterRoleBinding 양식이다
templates/serviceaccount.yaml	ServiceAccount 양식이다
templates/configmap.yaml	krb5.conf ConfigMap 양식이다
templates/cronjob-krb5-reconcile.yaml	30분마다 Kerberos 정리 재시도와 남은 keytab 확인을 실행하는 CronJob 설정이다.
templates/_helpers.tpl	리소스 이름 조립 헬퍼이다 — 릴리스 이름을 그대로 리소스 이름으로 쓴다

값은 `values.yaml` 에서 `deployment.yaml` 템플릿의 `env` 를 거쳐 컨테이너 환경 변수로 들어간다. `main.py`, `utils.py` 는 이 환경 변수를 읽는다. 예를 들어 `nas.ssh.host` 는 `NAS_SSH_HOST` 가 되어 NAS SSH 접속에 사용된다.

1.3 values.yaml에 넣는 값

아래 기본값은 저장소 `values.yaml` 을 기준으로 적었다. `""` (빈 값)인 키는 실제 배포 때 별도 values 파일이나 `--set` 으로 넣는다. 저장소에는 주소·계정 같은 민감값을 두지 않는다.

image — 무엇을 배포하나

CI/CD가 새 이미지를 push한 뒤 이 값 기준으로 배포된다.

키	의미	기본값
<code>image.repository</code>	운영 이미지 저장소	<code>dguai lab/config-server</code>
<code>image.tag</code>	이미지 태그 (CI가 latest + commit hash 이중 태깅)	<code>latest</code>
<code>image.pullPolicy</code>	이미지를 매번 새로 받을지 정하는 정책	<code>Always</code>

service — 어떻게 접속하나

`admin_be`와 관리자가 이 포트로 API를 호출한다.

키	의미	기본값
<code>service.type</code>	Service 타입	<code>NodePort</code>
<code>service.port</code>	클러스터 내부 포트	<code>80</code>
<code>service.targetPort</code>	컨테이너 포트 (gunicorn)	<code>8000</code>
<code>service.nodePort</code>	외부 노출 NodePort	<code>30082</code> (README의 9732는 오기, kubectl 실측)

resources — config-server 자신의 몫

사용자 Pod의 리소스가 아니라 **관리 서버 본인**이 쓸 CPU/메모리이다. `requests`는 스케줄링 시 보장받는 최소치, `limits`는 상한이다.

키	의미	기본값
<code>resources.requests</code>	CPU/메모리 요청	<code>500m</code> / <code>512Mi</code>
<code>resources.limits</code>	CPU/메모리 상한	<code>2000m</code> / <code>1024Mi</code>

nfs / nas / farm — 연결할 서버 설정

사용자 홈 NFS 주소, NAS SSH 접속 정보, keytab을 배포할 FARM 노드 목록을 정한다.

키	의미	기본값
<code>nfs.server</code>	NFS 서버 주소	<code>""</code> (배포 시 주입)

키	의미	기본값
<code>nfs.userSharePath</code>	사용자 홈 루트 export 경로	""
<code>nfs.kubeSharePath</code>	계정 파일(<code>/kube_share</code>) export 경로 — config-server Pod 자신이 이 NFS를 마운트한다	""
<code>nas.ssh.host</code> / <code>port</code> / <code>user</code> / <code>keyPath</code>	NAS 홈 생성용 SSH 접속 정보 (키 실체는 Secret, 1.4절)	""
<code>farm.ssh.user</code> / <code>keyPath</code> / <code>nodes</code>	farm 노드 keytab 배포용 SSH 정보와 대상 노드 목록	"" / []
<code>farm.adSsh.user</code> / <code>keyPath</code> / <code>nodes</code>	AD/DC(krb5 principal 발급) 노드용 SSH 정보	"" / []
<code>farm.homeMountRoot</code>	노드 호스트에 마운트된 홈 루트 — 사용자 Pod <code>/home</code> 의 hostPath 원천이다 (시스템 아키텍처의 사용자 Pod 그림)	""

security / krb5 — 권한과 인증

키	의미	기본값
<code>security.sudoAllowedCommands</code>	사용자별 sudoers whitelist 명령 목록. 비어 있으면 sudoers 파일 자체를 만들지 않는다	""
<code>krb5.realm</code>	Kerberos realm(인증 도메인 이름) — 운영은 <code>AILAB.DGU</code> 이다	""
<code>krb5.kdcHost</code>	KDC(티켓 발급 서버) 호스트. 기본 <code>values.yaml</code>과 현재 CI 배포 명령에는 이 값이 없다. ConfigMap 템플릿은 이 값을 읽으므로, 현재 배포 결과의 <code>krb5.conf</code> 에는 <code>kdc</code> 와 <code>admin_server</code> 값이 비어 있다. 이 ConfigMap을 실제 Kerberos 클라이언트 설정으로 쓸 계획이면 <code>values</code> 와 CI에 주입 경로를 추가해야 한다.	(키 없음)

`krb5.realm`이 비어 있으면 config-server는 사용자 keytab과 ccache를 준비하지 않는다. NAS의 Kerberos NFS 설정에 이 값이 필요하면 새 Pod가 홈에 접근하지 못할 수 있다. 실제 NFS mount 옵션은 FARM 노드에서 확인한다.

db / redis — 상태 저장소

NodePort 배정 기록은 MySQL에, 이미지 저장·로드 상태는 Redis에 둔다(시스템 아키텍처의 NodePort 배정 방식).

키	의미	기본값
<code>db.host</code>	infra-mysql 호스트	<code>infra-mysql</code>
<code>db.name</code>	데이터베이스명	<code>pod_port_db</code>
<code>db.user</code>	DB 사용자	<code>pod_port_user</code>
<code>redis.host</code>	이미지 상태용 Redis	<code>redis-bg-master.ailab-infra.svc.cluster.local</code>
<code>redis.port</code>	Redis 포트	<code>6379</code>

DB 비밀번호는 values에 없다 — Secret `config-server-db-secret` 에서 주입된다(1.4절).

배치 관련 — config-server가 어디에 뜨나

키	의미	기본값
<code>namespace / config.namespace</code>	배포 대상 namespace. ConfigMap 템플릿은 <code>.Values.namespace</code> 를 읽고 나머지 템플릿은 <code>.Values.config.namespace</code> 를 읽는다. 두 값은 항상 같게 둔다.	<code>ailab-infra</code>
<code>nodeSelector</code>	config-server 배포 노드 고정	<code>kubernetes.io/hostname: csid-dgu-desktop</code>
<code>tolerations</code>	control-plane의 NoSchedule taint 허용. taint는 노드에 걸린 "여기 배치 금지" 표식이고, toleration은 그 금지를 통과하는 허가증이다. 고정 대상 노드가 control-plane이라 nodeSelector와 반드시 짝으로 있어야 Pod가 뜬다	<code>node-role.kubernetes.io/control-plane</code>
<code>cleanup.image</code>	현재 템플릿에서는 읽지 않는 값이다. 지워도 배포 결과는 달라지지 않는다.	<code>config-server:latest</code>

1.4 미리 만들어 둘 Secret

민감값(비밀번호, SSH 개인키)은 values.yaml에 넣지 않고 쿠버네티스 Secret으로 관리한다. Secret은 차트가 만들어 주지 않으므로 **클러스터에 미리 존재해야** 하며, 없으면 Pod가 뜨지 못한다.

Secret 이름	내용	소비처
<code>config-server-db-secret</code>	DB 비밀번호 (key: <code>password</code>)	Deployment-CronJob의 <code>DB_PASSWORD</code> 환경변수
<code>nas-ssh-key</code>	NAS 접속용 SSH 개인키	<code>/etc/nas-ssh</code> 에 read-only 마운트
<code>farm-ssh-key</code>	farm 노드 접속용 SSH 개인키	<code>/etc/farm-ssh</code> 에 read-only 마운트
<code>farm-ad-ssh-key</code>	AD/DC 노드 접속용 SSH 개인키	<code>/etc/farm-ad-ssh</code> 에 read-only 마운트

이 외에 config-server는 사용자별 `krb5-keytab-<user>` Secret을 만든다. `rbac.yaml` 의 Secret 생성 권한은 이 작업에 필요하다.

1.5 값을 바꿀 때

먼저 위치부터 잡는다. `config-server/` 는 GitHub 저장소(CSID-DGU/admin_infra) 안의 폴더이다. 저장소를 내려받아(`git clone https://github.com/CSID-DGU/admin_infra.git`) 그 안의 `config-server/` 로 이동한 상태에서 아래를 실행한다. 단, ②~④는 클러스터에 연결된 관리 서버(kubectl과 helm이 설정된 컴퓨터)에서만 동작한다. 개인 노트북에서 clone만 받아서는 ①(파일 수정)까지만 가능하다.

기본 `values.yaml` 에는 NFS·NAS·FARM·Kerberos의 실제 값이 비어 있다. 운영에서 수동으로 `helm upgrade` 를 실행할 때는 CI가 넣는 것과 같은 별도 values 파일 또는 `--set` 값을 함께 사용한다. 빈 기본값만으로 아래 명령을 실행하지 않는다.

```
# ① values.yaml에서 값을 수정한다
vi Chart/values.yaml

# ② 릴리스에 반영한다 (재배포)
helm upgrade --install containerssh-config-server ./Chart -n ailab-infra

# ③ 새 Pod가 정상적으로 떴는지 확인한다
kubectl rollout status deploy/containerssh-config-server -n ailab-infra

# ④ 잘못됐으면 직전 배포 상태로 되돌린다
helm rollback containerssh-config-server -n ailab-infra
```

반영 뒤에는 [운영 매뉴얼](#)의 배포 확인 절차를 따른다. 일회성 실험에는 `--set` 을 쓸 수 있지만, 계속 유지할 값은 0.3절처럼 파일에 남긴다.

2. pvc-image-chart/ — 이미지 저장소 (현행)

사용자 컨테이너 이미지를 tar로 보관하는 공용 PVC `pvc-image-store` 를 배포하는 작은 차트이다. **현행 구조에서 이 PVC를 마운트하는 것은 config-server뿐이다**(`/image-store` — 사용자별 이미지 저장/로드 경로는 `/image-store/images/user-<username>.tar`). 과거에는 게스트 Pod에도 붙였지만 마운트 실패 문제로 사용자 Pod 스펙에서는 제거됐다. 사용자별 홈 PVC가 폐기된 뒤에도 **image-store만 PVC로 남았다**([시스템 아키텍처](#)의 홈 디렉터리와 이미지 저장소).

키/장치	값	의미
<code>imageStore.size</code>	<code>500Gi</code>	PVC 요청 용량
<code>storageClass</code>	<code>nfs-nas-v3-expandable</code>	NFS CSI 기반 StorageClass — 어떤 방식으로 저장소를 만들지 정한 템플릿
<code>accessModes</code>	<code>ReadWriteMany</code>	여러 Pod가 동시에 마운트할 수 있는 모드. 현행 소비자는 config-server이지만, 이미지 저장소를 공유 스토리지로 유지하는 계약을 나타낸다
<code>helm.sh/resource-policy: keep</code>	<code>annotation</code>	릴리스를 지워도 PVC는 남긴다. 차트를 실수로 삭제해도 이미지 저장소 PVC가 함께 지워지지 않게 한다.

3. pvc-chart/ (폐기)

사용자/그룹별 홈 PVC를 동적 생성하던 Helm 차트(`containerssh-pvc-chart`)의 잔재이다. 개별 사용자 PVC 방식은 26-07-02에 폐기되어 현행 흐름에서는 사용하지 않는다(현행은 NFS 직접 마운트 — [시스템 아키텍처](#)의 홈 디렉터리와 이미지 저장소 참조).

이 디렉터리는 저장소에 남아 있지만 현재 배포에는 쓰지 않는다. `Chart.yaml`, `values.yaml`, `README.md`, StorageClass 매니페스트 두 장(`sc-nfs-nas-v3-resizable.yaml`, `old-sc.yaml`)은 이전 클러스터 상태를 확인할 때만 본다. 새로 배포하거나 복구할 때는 이 차트를 쓰지 않고 NFS 직접 마운트와 `pvc-image-chart/`를 사용한다.

4. infra-sql/ (현행)

config-server의 NodePort 배정 기록 등 인프라 상태를 저장하는 MySQL을 배포하는 **일반 매니페스트**다. Helm 차트가 아니라 `kubectl apply`로 YAML을 적용한다.

파일	역할
<code>nfs-mysql.yaml</code>	MySQL용 NFS CSI StorageClass <code>sc-mysql</code> 을 만든다 (NFS server <code>192.168.2.30</code> , share <code>/volume1/share</code> , reclaimPolicy <code>Retain</code> — PVC를 지워도 데이터 볼륨을 삭제하지 않고 남긴다)
<code>infra-mysql.yaml</code>	<code>ailab-infra</code> namespace에 <code>infra-mysql</code> StatefulSet(고정 이름과 저장소를 유지하는 Pod 관리 객체, 저장소 10Gi) + ClusterIP Service(클러스터 내부 전용 접속 주소)를 생성한다

DB에는 StatefulSet을 쓴다. Deployment와 달리 Pod가 다시 만들어져도 같은 이름과 저장소(PVC)를 다시 사용하므로 데이터가 이어진다. Service는 ClusterIP(클러스터 내부 전용)다. NodePort 배정 기록 DB를 클러스터 밖에 열지 않는다.

infra-mysql은 `nodeport_allocations`와 `krb5_cleanup_pending` 테이블을 자동으로 만들지 않는다. DB를 다시 만들면 두 테이블의 생성 SQL을 직접 실행해야 한다. 스키마는 [데이터베이스](#)를 참고한다.

kdc-setup

원본: md/infra/kdc-setup/index.md, md/infra/kdc-setup/design.md, md/infra/kdc-setup/operations.md, md/infra/kdc-setup/config.md

이 문서에서 다루는 내용

- config-server가 FARM AD 계정, 사용자 keytab, 노드 ccache, Kerberos NFS 홈에 쓰는 흐름 전체
- 이 묶음 안에서 설계, 운영, 설정 문서를 어떤 순서로 읽으면 되는지
- `system/kerberos-nfs` 문서와 여기 문서의 경계

빠른 이동

필요할 때	문서
파일과 단계의 설계를 본다	설계
점검과 장애 대응 순서를 본다	운영
Helm, CI, Secret 값의 위치를 본다	설정
NAS와 NFS 쪽 Kerberos 설정을 본다	System Kerberos/NFS

`kdc-setup` 은 config-server가 사용자 Pod를 만들 때 쓰는 Kerberos 설정 문서다. NAS service keytab, NAS KVNO, 다른 컨테이너 환경의 Kerberos 설정은 [System Kerberos/NFS](#)에서 다룬다.

문서 안내

문서	읽어야 하는 경우	핵심 내용
현재 페이지	문서의 역할을 먼저 볼 때	AD, Secret, 노드, Pod가 이어지는 순서
설계	계정과 Pod를 만들 때 어떤 파일이 필요한지 볼 때	AD, Secret, 선택 노드, ccache, NFS
운영	점검, 장애 대응, 노드 추가를 할 때	확인 순서와 명령
설정	Helm, CI, Secret 설정을 바꿀 때	값이 들어가는 곳과 확인 방법

계정부터 Pod까지의 순서

승인 시스템

- > config-server
 - > FARM AD DC: 사용자·전용 그룹·RFC2307·keytab
 - > Kubernetes Secret: 사용자 keytab 보관
 - > 선택된 FARM 노드: root-only keytab·refresh timer·ccache
 - > 사용자 Pod: keytab 없이 ccache와 FARM 홈을 사용

누가 무엇을 하나

- config-server는 계정과 Pod 생성·삭제를 요청하고 AD와 노드 관리 명령을 실행한다.
- AD는 사용자, 전용 그룹, RFC2307 UID/GID, 사용자 keytab을 만든다.
- 선택된 노드는 keytab을 root만 읽을 수 있는 경로에 두고 사용자 ccache를 갱신한다.
- Pod는 keytab을 받지 않고 ccache만 사용한다.
- NAS 홈 디렉터리 생성·삭제는 NAS 관리 경로에서 처리한다.
- 비밀 값과 변경 이력은 관리자 전용 시크릿과 배포 설정 문서에만 둔다.

이전 Pod 처리

새 계정과 새 Pod는 FARM AD 설정을 사용한다. 전환 전에 만든 Pod는 다시 만들기 전까지 이전 Realm이나 홈 마운트를 쓸 수 있다. 이전 Pod에 새 keytab이나 timer 파일을 덮어쓰지 말고, 현재 Pod의 환경 변수와 hostPath를 확인한 뒤 다시 만든다.

관련 문서

- [System Kerberos/NFS](#): NAS와 NFS의 Kerberos 설정
- [System container-images](#): 이미지가 ccache를 쓰는 방식
- [설정](#): config-server 배포 설정과 확인 방법

kdc-setup 설계

이 문서에서 다루는 내용

- AD 사용자, Secret, 노드 keytab, ccache가 어디에 놓이고 누가 쓰는지
- 계정 생성, Pod 시작, Pod·계정 삭제 때 Kerberos 관련 파일이 어떻게 바뀌는지
- 바꾸면 안 되는 보안·운영 규칙

빠른 이동

필요할 때	문서
전체 흐름을 먼저 본다	개요
점검과 장애 대응을 본다	운영
실제 설정값의 위치를 본다	설정

1. 이 설정에서 지킬 것

- FARM AD의 RFC2307 UID/GID와 NAS의 숫자 소유권을 같은 값으로 유지한다.
- 사용자 keytab은 Pod에 전달하지 않고, 선택 노드의 root 전용 경로에만 둔다.
- Pod를 실행하기 전에 해당 노드에서 사용자 TGT와 ccache 소유권을 검증한다.
- AD를 관리하는 접속과 노드를 관리하는 접속에 같은 키를 쓰지 않는다.
- NFS에서 D-state가 보이면 mount나 서비스를 반복해서 다시 시작하지 않는다.

2. 파일과 서비스가 있는 곳

config-server
├─ AD 관리 채널 -> AD DC
│ 사용자 + <user>_gid + RFC2307 UID/GID + 사용자 keytab
├─ Kubernetes API -> Secret krb5-keytab-<user>
└─ 노드 관리 채널 -> Pod가 배치될 FARM 노드 한 대
/etc/decs-krb/keytabs/<user>.keytab root-only
decs-krb-refresh@<user>.timer
/run/user/<uid>/krb5cc_aialab 사용자 소유
└─ Pod는 ccache와 FARM 홈만 공유

AD 사용자와 Secret은 계정이 있는 동안 유지한다. 노드의 keytab, 환경 파일, timer, ccache는 Pod가 올라간 노드에 서만 쓴다. Pod를 지우면 노드의 파일은 지우지만 AD 사용자와 Secret은 다음 Pod를 위해 남긴다.

3. 계정과 인증 파일

항목	위치	사용하는 곳	권한
사용자·전용 그룹	FARM AD	config-server, winbind, NAS	RFC2307 UID/GID 일치
사용자 keytab	Kubernetes Secret	config-server	관리자만 관리
노드 keytab	선택된 FARM 노드	root, refresh service	root 소유 0400

항목	위치	사용하는 곳	권한
갱신 환경 파일	선택된 FARM 노드	refresh service	root 소유 0400
사용자 ccache	/run/user/<uid>/krb5cc_<ai lab>	Pod, 호스트 NFS 경로	사용자 UID/GID, 0600
머신 ccache	/tmp/krb5cc_0	rpc.gssd	현재 노드 머신 principal 전용

/tmp/krb5cc_0에는 현재 노드의 머신 ticket만 둔다. 여기에 사용자 ticket이나 다른 Realm ticket을 넣으면 NFS 인증이 달라질 수 있다. 사용자 확인과 복구에는 별도 ccache를 사용한다.

4. 계정 생성부터 삭제까지

계정 생성

1. config-server가 UID/GID를 할당하고 계정 파일을 갱신한다.
2. NAS 관리 경로가 같은 숫자 소유권으로 홈 디렉터리를 준비한다.
3. AD 관리 채널이 사용자와 <user>_gid 그룹을 만들고 RFC2307 속성을 설정한다.
4. AD가 사용자 keytab을 발급하고 config-server가 Kubernetes Secret에 저장한다.

마지막 단계가 실패하면 계정 생성도 실패한다. NAS 홈과 계정 파일은 되돌리려고 하지만, AD 사용자·전용 그룹·Secret 일부가 남을 수 있다. 같은 이름으로 다시 만들기 전에는 이 세 곳을 확인한다.

Pod 시작

1. config-server가 대상 FARM 노드를 선택한다.
2. Secret의 keytab을 해당 노드의 제한된 관리 채널로 전달한다.
3. 노드는 root 전용 keytab·환경 파일·decs-krb-refresh@<user>.timer를 준비한다.
4. refresh service가 사용자 TGT를 발급하고 ccache 소유권을 검증한다.
5. 검증에 성공할 때만 Pod가 ccache와 FARM 홈을 hostPath로 공유하며 시작한다.

Pod·계정 삭제

Pod 삭제는 해당 노드의 timer, keytab, 환경 파일과 ccache를 정리한다. 계정 삭제는 AD 사용자와 Secret을 지우고 모든 FARM 노드에 사용자 keytab, timer, ccache 정리를 요청한다. 전용 그룹은 자동으로 지우지 않는다. 지울 필요가 있으면 운영 규칙을 확인한 뒤 따로 처리한다.

5. 접속 경로

접속 경로	대상	할 수 있는 일
AD 관리 채널	AD DC	사용자·전용 그룹 생성/삭제, keytab 발급
노드 관리 채널	FARM 실행 노드	keytab 배포·삭제, timer 제어, ccache 검증

접속 경로	대상	할 수 있는 일
NAS 관리 채널	NAS	홈 생성·소유권 설정·삭제

각 접속 경로는 다른 키와 forced-command를 사용한다. AD를 수정하는 키로 모든 노드에 일반 셸 접속을 할 수 없게 한다.

6. 바꾸면 안 되는 규칙

1. 사용자 keytab을 Pod에 복사하거나 mount하지 않는다.
2. AD의 UID/GID, config-server 계정 파일, NAS 소유권을 따로 바꾸지 않는다.
3. Pod 시작 전에 대상 노드의 TGT와 ccache 소유권을 확인한다.
4. 머신 ccache에는 현재 노드의 머신 principal만 둔다.
5. D-state가 있는 노드에서 강제 unmount·반복 mount·반복 service restart를 하지 않는다.

kdc-setup 운영

이 문서에서 다루는 내용

- AD, Secret, 노드 keytab, ccache, NFS 상태를 어떤 순서로 확인할지
- 계정과 Pod 생성·삭제 뒤에 어떤 흔적이 남아야 하고 무엇이 지워져야 하는지
- 사용자 단위 장애와 노드 전체 장애를 어떻게 구분할지

빠른 이동

필요할 때	문서
전체 구조와 역할 분담을 먼저 본다	개요
파일 위치와 보안 규칙을 본다	설계
Helm, CI, Secret 값의 위치를 본다	설정

1. 작업할 때 지킬 것

1. 계정과 Pod 생성·삭제는 config-server API로 수행한다.
2. keytab, 개인키, Secret `data` 는 일반 운영 문서·로그·티켓에 출력하지 않는다.
3. 사용자 장애는 해당 사용자와 Pod가 배치된 노드 범위에서 먼저 확인한다.
4. `/tmp/krb5cc_0` 에는 현재 노드의 머신 principal 이외의 ticket을 발급하지 않는다.

5. NFS 관련 D-state가 있으면 반복 restart, 강제 unmount, 연속 mount를 중단한다.

2. 정기적으로 확인할 것

Kubernetes

```
kubectl -n ailab-infra get deploy,pod,cronjob
kubectl -n ailab-infra get events --sort-by=.lastTimestamp
kubectl -n ailab-infra logs deploy/containersssh-config-server --since=30m
```

config-server가 Ready인지, AD 계정 생성과 FARM 노드 설정이 반복해서 실패하지 않는지 확인한다. 사용자 keytab Secret은 값이 아니라 메타데이터만 확인한다.

```
target_user='<user>'
kubectl -n ailab-infra get secret "krb5-keytab-$target_user" \
-o custom-columns=NAME:.metadata.name,CREATED:.metadata.creationTimestamp
```

AD와 노드 인증 파일

AD DC에서 사용자·전용 그룹·RFC2307 번호를 확인하고, 선택 노드에서는 timer와 ccache의 권한·유효성을 확인한다.

```
target_user='<user>'
target_uid='<uid>'

sudo samba-tool user show "$target_user"
sudo samba-tool group show "${target_user}_gid"
sudo systemctl is-enabled "decs-krb-refresh@$target_user.timer"
sudo systemctl is-active "decs-krb-refresh@$target_user.timer"
sudo stat -c '%U %G %a %n' \
    "/etc/decs-krb/keytabs/$target_user.keytab" \
    "/etc/decs-krb/refresh.d/$target_user.env" \
    "/run/user/$target_uid/krb5cc_ailab"
sudo env KRB5CCNAME="FILE:/run/user/$target_uid/krb5cc_ailab" klist
```

keytab·환경 파일은 root 소유 0400, ccache는 대상 UID/GID 소유 0600 이어야 한다. klist -k 로 keytab 내용을 출력하지 않는다.

NFS와 노드 Kerberos ticket

```
systemctl is-active rpc-gssd
findmnt -T /home/tako2/share/user -o TARGET,SOURCE,FSTYPE,OPTIONS
sudo klist -c FILE:/tmp/krb5cc_0
ps -eo pid,stat,comm,args | awk '$2 ~ /^D/'
```

홈 경로 mount, rpc.gssd, 머신 principal, D-state를 함께 확인한다. Pod에서만 문제가 나면 Pod UID/GID와 ccache 공유 경로를 먼저 본다. 노드 전체에서 문제가 나면 머신 ticket, rpc.gssd, NFS mount를 먼저 본다.

3. 계정과 Pod를 만든 뒤 확인할 것

계정 생성 후

1. config-server 계정 파일과 AD RFC2307 UID/GID를 대조한다.
2. Secret 메타데이터, NAS 홈의 숫자 소유권·모드를 확인한다.
3. Pod 생성 뒤 대상 노드의 timer·ccache와 Pod 내부 홈 읽기·쓰기를 확인한다.

Pod 삭제 후

대상 노드에서 timer, keytab, 환경 파일이 제거됐는지 확인한다. AD 사용자와 Secret은 다음 Pod 실행을 위해 남아야 한다.

```
target_user='<user>'
sudo systemctl status "decs-krb-refresh@$target_user.timer" --no-pager
sudo test ! -e "/etc/decs-krb/keytabs/$target_user.keytab"
sudo test ! -e "/etc/decs-krb/refresh.d/$target_user.env"
```

계정 삭제 후

AD 사용자와 사용자 keytab Secret이 사라졌는지, 모든 FARM 노드의 사용자 timer·keytab·ccache가 정리됐는지 확인한다. 전용 AD 그룹은 자동 삭제 대상이 아니므로 임의로 지우지 않는다.

4. 문제별 확인 순서

증상	먼저 확인할 것	금지할 것
AD 생성·keytab 발급 실패	config-server stage, 대상 DC, 동일 이름의 사용자·그룹	DC wrapper의 반복 직접 실행
대상 노드 배포 실패	노드 목록, 제한된 SSH 채널, keytab 파일 모드, service journal	keytab을 복사해 우회
ccache 갱신 실패	DNS·시간·Realm·principal·keytab 모드	keytab 내용을 출력

증상	먼저 확인할 것	금지할 것
Pod 홈 접근 실패	Pod UID/GID, ccache, rpc.gssd, 머신 ticket, mount	NAS를 바로 재시작
NFS D-state	stack·kernel log·NFS 응답·네트워크	반복 mount/unmount 또는 service restart

머신 ccache의 principal이 잘못됐거나 ticket이 만료된 경우에만, 정확한 현재 노드 머신 principal을 확인한 후 별도 승인 절차에 따라 복구한다. 사용자 principal이나 다른 Realm을 지정하지 않는다.

5. 노드 추가·변경

신규 AD DC와 FARM 실행 노드는 먼저 제한된 관리 채널, 시간 동기화, rpc.gssd와 FARM 홈 mount를 준비한다. 시험 노드 한 대에서 시험 사용자로 keytab 권한, timer, ccache, TGT, NFS 읽기·쓰기를 검증한 뒤 노드 목록에 추가한다.

공통 refresh 스크립트·systemd 템플릿의 변경은 사용자 배포 중에도 다시 설치될 수 있다. 시험 노드에서 확인한 뒤 기존 노드의 버전·소유권을 비교하고 확대한다.

kdc-setup 설정

이 문서에서 다루는 내용

- FARM AD 관련 설정이 Helm values, CI secret, Kubernetes Secret, 노드 파일 중 어디에 들어가는지
- 값을 직접 노출하지 않고 배포 상태를 확인하는 방법
- 설정을 바꾸기 전에 범위를 어떻게 나눠 확인해야 하는지

빠른 이동

필요할 때	문서
전체 흐름과 문서 경계를 먼저 본다	개요
파일·서비스 위치와 보안 규칙을 본다	설계
운영 중 점검 순서를 본다	운영

설정이 들어가는 곳

config-server의 FARM AD 설정은 Helm values, CI secret, Kubernetes Secret, 노드의 root 전용 파일에 나뉘어 있다. 실제 비밀 값과 변경 이력은 관리자 전용 시크릿과 배포 설정 문서에만 두며, 이 문서에는 값, 개인키, keytab을 적지 않는다.

CI Secret / 관리자 전용 시크릿 문서

- > Helm values 주입
- > config-server Deployment 환경 변수·Secret mount
- > 제한된 AD/노드 관리 채널
- > 노드 root-only keytab과 사용자 ccache

설정별 사용처

구분	대표 항목	사용하는 곳	확인할 것
Realm	KRB5_REALM	config-server, 사용자 Pod	새 설정은 FARM AD Realm을 쓴다.
노드 관리	FARM_SSH_*, 노드 목록	config-server	AD 관리용 키와 계정을 섞지 않는다.
AD 관리	FARM_AD_SSH_*, DC 목록	config-server	AD DC에서 허용한 명령만 실행한다.
홈 경로	FARM_HOME_MOUNT_ROOT	사용자 Pod	FARM 노드에 이미 마운트된 NFS 홈 경로를 가리킨다.
NAS 홈 관리	NAS SSH 설정	config-server	홈 디렉터리 생성과 삭제에만 쓴다.
사용자 keytab	krb5-keytab-<user> Secret	config-server	Pod에는 마운트하지 않는다.

Kubernetes 배포 확인

값을 출력하지 않고 Deployment가 필요한 환경 변수와 Secret mount를 참조하는지 확인한다.

```
kubectl -n ailab-infra get deploy containersssh-config-server \
-o custom-columns=IMAGE:.spec.template.spec.containers[0].image,SA:.spec.template.spec.serviceAccountName
kubectl -n ailab-infra describe deploy containersssh-config-server
kubectl -n ailab-infra get secret farm-ssh-key farm-ad-ssh-key
```

사용자 keytab Secret은 이름·생성 시각만 확인한다. `kubectl get secret -o yaml`, `jsonpath`로 `data`를 출력하거나 terminal history에 남기는 명령은 사용하지 않는다.

노드에서 확인할 파일

노드는 사용자별 keytab을 `/etc/decs-krb/keytabs/` 아래 root 전용으로 보관하고, `decs-krb-refresh@<user>.timer`가 사용자 ccache를 갱신하도록 준비한다. 노드 머신 ccache와 사용자 ccache는 서로 다른 파일·principal로 분리한다.

대상	설정 기준
사용자 keytab·환경 파일	root 소유, 0400

대상	설정 기준
사용자 ccache	대상 UID/GID 소유, 0600
머신 ccache	현재 노드 머신 principal만 사용
rpc.gssd	노드 Kerberos NFS 경로와 함께 점검
forced-command 계정	일반 셀·포괄적인 sudo 없이 허용 동작만 실행

아직 쓰지 않는 ConfigMap

config-server에 마운트되는 `krb5-conf` ConfigMap은 현재 계정과 Pod 생성에 직접 쓰는 Kerberos 설정 파일이 아닙니다. config-server가 KDC 명령을 직접 쓰게 되면 KDC와 admin server 주소를 어디에서 넣을지 먼저 정한다. 그 전에는 비어 있는 endpoint를 정상 설정으로 보거나 복구에 사용하지 않는다.

설정을 바꾸기 전에 확인할 것

1. AD 관리, 노드 관리, NAS 홈 관리 중 무엇을 바꾸는지 먼저 정한다.
2. 실제 비밀 값은 관리자 전용 시크릿 문서에서만 확인하고 일반 문서에 복사하지 않는다.
3. 시험 노드 한 대와 시험 사용자로 ccache·NFS 접근을 확인한다.
4. 기존 Pod가 이전 Realm·홈 경로를 쓰는지 확인하고, 설정값을 덮어쓰지 않는다.
5. 검증 뒤에만 CI/Helm 설정을 확대 적용한다.